

Public SDK Header Documentation Draft

This draft collects the generated documentation for all 19 headers in PUBLIC/include, including the experimental headers and GenTL_v1_5.h. Review each section before using its module in an SDK manual.

Each insertion marker is configured for one header. The documentation build replaces these markers in a generated copy; the source document retains them for subsequent builds. The generated sections follow the marker order below.

KYVPExperimental API

KYVPExperimental.h API reference

Generated from PUBLIC/include/experimental/KYVPExperimental.h.

API documentation generated from PUBLIC/include/experimental/KYVPExperimental.h.

Experimental APIs may change without deprecation. These annotations request compiler diagnostics; they do not enable or disable an API at run time. Place `KYVP_EXPERIMENTAL_PREFIX` before the return type and `KYVP_EXPERIMENTAL_SUFFIX` after the parameter list. Both positions are needed for compiler coverage.

Macros

KYVP_EXPERIMENTAL_PREFIX

Annotation placed before an experimental function return type.

Expands to an MSVC deprecation declaration without prior opt-in; empty in the other branches.

KYVP_EXPERIMENTAL_SUFFIX

Annotation placed after an experimental function parameter list.

Expands to a supported Clang or GCC warning attribute; empty in the other branches.

KYVPLogging API

KYVPLogging.h API reference

Generated from PUBLIC/include/experimental/KYVPLogging.h.

API documentation generated from PUBLIC/include/experimental/KYVPLogging.h.

Use KYVPLOG() or a severity-specific macro to attach the source file, function, and line to a printf-style message. Logger configuration controls which messages are emitted. Applications using these macros must link with the extension library. See Experimental API annotations for compiler diagnostics and the MSVC opt-in include order.

Macros

KYVPLOG_FILE

(kyvp_current_source_filename(__FILE__))

Source basename with its final filename extension removed.

The result uses reusable internal storage and may be truncated to fit. Copy it if it must survive another call. Storage is thread-local only when the active KYVP_TLS branch supplies a thread-local specifier; the fallback uses shared static storage.

KYVP_USE_EXPERIMENTAL_APIS

Legacy flag defined by this logging header.

This definition enables the following logging macro branch. It occurs after the experimental annotations are included, so it does not select MSVC opt-in retroactively. Define the flag before the first include to select that diagnostic mode. This header defines the flag unconditionally; it is not a caller-controlled logging disable switch.

KYVPLOG

KYVPExtension_AppLog_V1(_log_priority, KYVPLOG_FILE, KY_FUNC, __LINE__, _fmt, ##__VA_ARGS__)

Log a formatted message with automatic source location.

Parameter	Description
_log_priority	Message severity from KYVPLOG_PRIORITY.
_fmt	Non-NULL printf-style format string.
...	Optional formatting arguments matching _fmt.

Returns: The status returned by KYVPExtension_AppLog_V1().

KYVPLOG_TRACE

KYVPLOG(KYVPLOG_TRACE_LEVEL, _fmt, ##__VA_ARGS__)

Log a message at TRACE severity.

Parameter	Description
_fmt	Non-NULL printf-style format string.
...	Optional formatting arguments matching _fmt.

Returns: The status returned by KYVPExtension_AppLog_V1().

See: KYVPLOG

KYVPLOG_DEBUG

`KYVPLOG(KYVPLOG_DEBUG_LEVEL, _fmt, ##__VA_ARGS__)`

Log a message at DEBUG severity.

Parameter	Description
<code>_fmt</code>	Non-NULL printf-style format string.
...	Optional formatting arguments matching <code>_fmt</code> .

Returns: The status returned by `KYVPEExtension_AppLog_V1()`.

See: `KYVPLOG`

KYVPLOG_INFO

`KYVPLOG(KYVPLOG_INFO_LEVEL, _fmt, ##__VA_ARGS__)`

Log a message at INFO severity.

Parameter	Description
<code>_fmt</code>	Non-NULL printf-style format string.
...	Optional formatting arguments matching <code>_fmt</code> .

Returns: The status returned by `KYVPEExtension_AppLog_V1()`.

See: `KYVPLOG`

KYVPLOG_NOTICE

`KYVPLOG(KYVPLOG_NOTICE_LEVEL, _fmt, ##__VA_ARGS__)`

Log a message at NOTICE severity.

Parameter	Description
<code>_fmt</code>	Non-NULL printf-style format string.
...	Optional formatting arguments matching <code>_fmt</code> .

Returns: The status returned by `KYVPEExtension_AppLog_V1()`.

See: `KYVPLOG`

KYVPLOG_WARN

`KYVPLOG(KYVPLOG_WARN_LEVEL, _fmt, ##__VA_ARGS__)`

Log a message at WARN severity.

Parameter	Description
<code>_fmt</code>	Non-NULL printf-style format string.
...	Optional formatting arguments matching <code>_fmt</code> .

Returns: The status returned by `KYVPEExtension_AppLog_V1()`.

See: `KYVPLOG`

KYVPLOG_ERROR

`KYVPLOG(KYVPLOG_ERROR_LEVEL, _fmt, ##__VA_ARGS__)`

Log a message at ERROR severity.

Parameter	Description
<code>_fmt</code>	Non-NULL printf-style format string.
...	Optional formatting arguments matching <code>_fmt</code> .

Returns: The status returned by `KYVPExtension_AppLog_V1()`.

See: `KYVPLOG`

KYVPLOG_CRITICAL

```
KYVPLOG(KYVPLOG_CRITICAL_LEVEL, _fmt, ##__VA_ARGS__)
```

Log a message at CRITICAL severity.

Parameter	Description
<code>_fmt</code>	Non-NULL printf-style format string.
...	Optional formatting arguments matching <code>_fmt</code> .

Returns: The status returned by `KYVPExtension_AppLog_V1()`.

See: `KYVPLOG`

KYVPLOG_FATAL

```
KYVPLOG(KYVPLOG_FATAL_LEVEL, _fmt, ##__VA_ARGS__)
```

Log a message at FATAL severity.

Parameter	Description
<code>_fmt</code>	Non-NULL printf-style format string.
...	Optional formatting arguments matching <code>_fmt</code> .

Returns: The status returned by `KYVPExtension_AppLog_V1()`.

See: `KYVPLOG`

KYVPLOG_AND_RETURN_ERROR

```
{
    KYVPLOG_ERROR("returning error %d", _err);
    return _err;
}
```

Log an error and return its value from the enclosing function.

Expands to a statement block. Use braces around surrounding conditional branches. The error expression is evaluated once for logging and again for return; pass a stable value without side effects.

Parameter	Description
<code>_err</code>	Integer error value compatible with <code>d</code> and the enclosing return type.

KYVPLOG_AND_RETURN_ERROR_MSG

```
{
    KYVPLOG_ERROR("returning error %d, " _fmt, _err, ##__VA_ARGS__);
    return _err;
}
```

Log an error with a message and return its value from the enclosing function.

Expands to a statement block. Use braces around surrounding conditional branches. The error expression is evaluated once for logging and again for return; pass a stable value without side effects.

Parameter	Description
<code>_err</code>	Integer error value compatible with <code>d</code> and the enclosing return type.
<code>_fmt</code>	String literal or macro expanding to one, concatenated with the error prefix.
...	Optional formatting arguments matching <code>_fmt</code> .

Enumerations

`_KYVPLOG_PRIORITY`

Message severities accepted by the application logger.

Values

Name	Description
<code>KYVPLOG_TRACE_LEVEL</code>	Detailed execution tracing; least severe priority.
<code>KYVPLOG_DEBUG_LEVEL</code>	Diagnostic details for debugging.
<code>KYVPLOG_INFO_LEVEL</code>	Information about normal application activity.
<code>KYVPLOG_NOTICE_LEVEL</code>	Significant events during normal operation.
<code>KYVPLOG_WARN_LEVEL</code>	A condition that may require attention.
<code>KYVPLOG_ERROR_LEVEL</code>	An operation failed or encountered an error.
<code>KYVPLOG_CRITICAL_LEVEL</code>	A serious condition affecting application operation.
<code>KYVPLOG_FATAL_LEVEL</code>	The highest severity classification; logging alone does not terminate the process.

Type definitions

`KYVPLOG_PRIORITY`

```
typedef enum _KYVPLOG_PRIORITY KYVPLOG_PRIORITY
```

Message severities accepted by the application logger.

Functions

`KYVPExtension_AppLog_V1`

```
KY_RESULT KYVPExtension_AppLog_V1(KYVPLOG_PRIORITY _log_priority, const char *_pszLoggerName, const char *_pszFuncName, int _line, const char *_pszFmt,...)
```

Forward a formatted application message to the SDK logger.

Warning: Experimental backend entry point. Use `KYVPLOG()` or its severity-specific macros; the function signature may change without notice. The logger name is placed under the `KYVPExt` logging namespace. The function prefixes the message with the source line. A success result does not guarantee that logger filters emitted the message.

Parameter	Description
_log_priority	One of the documented KYVPLOG_PRIORITY values.
_pszLoggerName	Non-NULL, null-terminated logger name, normally the source basename.
_pszFuncName	Non-NULL, null-terminated calling-function name.
_line	Calling source line number.
_pszFmt	Non-NULL, null-terminated printf-style format string.
...	Formatting arguments matching the format string.

Returns: The current implementation returns SDK success after submitting the message. Inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

GenTL_v1_5 API

GenTL_v1_5.h API reference

Generated from PUBLIC/include/GenTL_v1_5.h.

API documentation generated from PUBLIC/include/GenTL_v1_5.h.

Macros

GenTLMajorVersion

```
1 /* defines the major version of the GenICam GenTL standard version this header is based on */
```

GenTLMinorVersion

```
5 /* defines the minor version of the GenICam GenTL standard version this header is based on */
```

GenTLSubMinorVersion

```
0 /* defines the sub minor version of the GenICam GenTL standard version this header is based on */
```

GC_GENTL_HEADER_VERSION_CODE

```
((major)<<24)+((minor)<<16)+(subminor))
```

GC_GENTL_HEADER_VERSION

```
GC_GENTL_HEADER_VERSION_CODE(GenTLMajorVersion,GenTLMinorVersion,GenTLSubMinorVersion)
```

TLTypeMixedName

```
"Mixed" /* Type to use for several supported technologies */
```

TLTypeCustomName

```
"Custom" /* Type to use for custom technologies */
```

TLTypeGEVName

```
"GEV" /* Type to use for GigE Vision technology */
```

TLTypeCLName

```
"CL" /* Type to use for Camera Link technology */
```

TLTypeIIDCName

```
"IIDC" /* Type to use for IIDC 1394 technology */
```

TLTypeUVCName

```
"UVC" /* Type to use for USB video class devices */
```

TLTypeCXPName

```
"CXP" /* Type to use for CoaXPress, V1.3 */
```

TLTypeCLHSName

```
"CLHS" /* Type to use for Camera Link HS, V1.3 */
```

TLTypeU3VName

"U3V" /* Type to use for USB3 Vision Standard, V1.4 */

TLTypeETHERNETName

"Ethernet" /* Type to use for Ethernet devices, V1.3 */

TLTypePCIName

"PCI" /* Type to use for PCI/PCIe devices, V1.3 */

TLSystemModuleName

"TSystem" /* Name to identify a system module */

TLInterfaceModuleName

"TLInterface" /* Name to identify a interface module */

TLDeviceModuleName

"TLDevice" /* Name to identify a device module */

TLDataStreamModuleName

"TLDataStream" /* Name to identify a data stream module */

TLBufferModuleName

"TLBuffer" /* Name to identify a buffer module */

TLRemoteDeviceModuleName

"Device" /* Name to identify a remote device module */

GENTL_INVALID_HANDLE

NULL /* Invalid handle value, V1.4 */

GENTL_INFINITE

0xFFFFFFFFFFFFFFFFULL /* Infinite value to be used in various function calls, V1.4 */

GC_API

GC_IMPORT_EXPORT GC_ERROR GC_CALLTYPE

GC_API_P

typedef GC_ERROR(GC_CALLTYPE *function)

Enumerations

GC_ERROR_LIST

Values

Name	Description
GC_ERR_SUCCESS	
GC_ERR_ERROR	

GC_ERR_NOT_INITIALIZED	
GC_ERR_NOT_IMPLEMENTED	
GC_ERR_RESOURCE_IN_USE	
GC_ERR_ACCESS_DENIED	
GC_ERR_INVALID_HANDLE	
GC_ERR_INVALID_ID	
GC_ERR_NO_DATA	
GC_ERR_INVALID_PARAMETER	
GC_ERR_IO	
GC_ERR_TIMEOUT	
GC_ERR_ABORT	
GC_ERR_INVALID_BUFFER	
GC_ERR_NOT_AVAILABLE	
GC_ERR_INVALID_ADDRESS	
GC_ERR_BUFFER_TOO_SMALL	
GC_ERR_INVALID_INDEX	
GC_ERR_PARSING_CHUNK_DATA	
GC_ERR_INVALID_VALUE	
GC_ERR_RESOURCE_EXHAUSTED	
GC_ERR_OUT_OF_MEMORY	
GC_ERR_BUSY	
GC_ERR_CUSTOM_ID	

INFO_DATATYPE_LIST

Values

Name	Description
INFO_DATATYPE_UNKNOWN	
INFO_DATATYPE_STRING	
INFO_DATATYPE_STRINGLIST	
INFO_DATATYPE_INT16	
INFO_DATATYPE_UINT16	
INFO_DATATYPE_INT32	
INFO_DATATYPE_UINT32	
INFO_DATATYPE_INT64	

INFO_DATATYPE_UINT64	
INFO_DATATYPE_FLOAT64	
INFO_DATATYPE_PTR	
INFO_DATATYPE_BOOL8	
INFO_DATATYPE_SIZET	
INFO_DATATYPE_BUFFER	
INFO_DATATYPE_PTRDIFF	
INFO_DATATYPE_CUSTOM_ID	

TL_CHAR_ENCODING_LIST

Values

Name	Description
TL_CHAR_ENCODING_ASCII	
TL_CHAR_ENCODING_UTF8	

TL_INFO_CMD_LIST

Values

Name	Description
TL_INFO_ID	
TL_INFO_VENDOR	
TL_INFO_MODEL	
TL_INFO_VERSION	
TL_INFO_TLTYPE	
TL_INFO_NAME	
TL_INFO_PATHNAME	
TL_INFO_DISPLAYNAME	
TL_INFO_CHAR_ENCODING	
TL_INFO_GENTL_VER_MAJOR	
TL_INFO_GENTL_VER_MINOR	
TL_INFO_CUSTOM_ID	

INTERFACE_INFO_CMD_LIST

Values

Name	Description
------	-------------

INTERFACE_INFO_ID	
INTERFACE_INFO_DISPLAYNAME	
INTERFACE_INFO_TLTYPE	
INTERFACE_INFO_CUSTOM_ID	

DEVICE_ACCESS_FLAGS_LIST

Values

Name	Description
DEVICE_ACCESS_UNKNOWN	
DEVICE_ACCESS_NONE	
DEVICE_ACCESS_READONLY	
DEVICE_ACCESS_CONTROL	
DEVICE_ACCESS_EXCLUSIVE	
DEVICE_ACCESS_CUSTOM_ID	

DEVICE_ACCESS_STATUS_LIST

Values

Name	Description
DEVICE_ACCESS_STATUS_UNKNOWN	
DEVICE_ACCESS_STATUS_READWRITE	
DEVICE_ACCESS_STATUS_READONLY	
DEVICE_ACCESS_STATUS_NOACCESS	
DEVICE_ACCESS_STATUS_BUSY	
DEVICE_ACCESS_STATUS_OPEN_READWRITE	
DEVICE_ACCESS_STATUS_OPEN_READONLY	
DEVICE_ACCESS_STATUS_CUSTOM_ID	

DEVICE_INFO_CMD_LIST

Values

Name	Description
DEVICE_INFO_ID	
DEVICE_INFO_VENDOR	
DEVICE_INFO_MODEL	
DEVICE_INFO_TLTYPE	

DEVICE_INFO_DISPLAYNAME	
DEVICE_INFO_ACCESS_STATUS	
DEVICE_INFO_USER_DEFINED_NAME	
DEVICE_INFO_SERIAL_NUMBER	
DEVICE_INFO_VERSION	
DEVICE_INFO_TIMESTAMP_FREQUENCY	
DEVICE_INFO_CUSTOM_ID	

ACQ_STOP_FLAGS_LIST

Values

Name	Description
ACQ_STOP_FLAGS_DEFAULT	
ACQ_STOP_FLAGS_KILL	
ACQ_STOP_FLAGS_CUSTOM_ID	

ACQ_START_FLAGS_LIST

Values

Name	Description
ACQ_START_FLAGS_DEFAULT	
ACQ_START_FLAGS_CUSTOM_ID	

ACQ_QUEUE_TYPE_LIST

Values

Name	Description
ACQ_QUEUE_INPUT_TO_OUTPUT	
ACQ_QUEUE_OUTPUT_DISCARD	
ACQ_QUEUE_ALL_TO_INPUT	
ACQ_QUEUE_UNQUEUED_TO_INPUT	
ACQ_QUEUE_ALL_DISCARD	
ACQ_QUEUE_CUSTOM_ID	

STREAM_INFO_CMD_LIST

Values

Name	Description
------	-------------

STREAM_INFO_ID	
STREAM_INFO_NUM_DELIVERED	
STREAM_INFO_NUM_UNDERRUN	
STREAM_INFO_NUM_ANNOUNCED	
STREAM_INFO_NUM_QUEUED	
STREAM_INFO_NUM_AWAIT_DELIVERY	
STREAM_INFO_NUM_STARTED	
STREAM_INFO_PAYLOAD_SIZE	
STREAM_INFO_IS_GRABBING	
STREAM_INFO_DEFINES_PAYLOADSIZE	
STREAM_INFO_TLTYPE	
STREAM_INFO_NUM_CHUNKS_MAX	
STREAM_INFO_BUF_ANNOUNCE_MIN	
STREAM_INFO_BUF_ALIGNMENT	
STREAM_INFO_CUSTOM_ID	

BUFFER_INFO_CMD_LIST

Values

Name	Description
BUFFER_INFO_BASE	
BUFFER_INFO_SIZE	
BUFFER_INFO_USER_PTR	
BUFFER_INFO_TIMESTAMP	
BUFFER_INFO_NEW_DATA	
BUFFER_INFO_IS_QUEUED	
BUFFER_INFO_IS_ACQUIRING	
BUFFER_INFO_IS_INCOMPLETE	
BUFFER_INFO_TLTYPE	
BUFFER_INFO_SIZE_FILLED	
BUFFER_INFO_WIDTH	
BUFFER_INFO_HEIGHT	
BUFFER_INFO_XOFFSET	
BUFFER_INFO_YOFFSET	
BUFFER_INFO_XPADDING	

BUFFER_INFO_YPADDING	
BUFFER_INFO_FRAMEID	
BUFFER_INFO_IMAGEPRESENT	
BUFFER_INFO_IMAGEOFFSET	
BUFFER_INFO_PAYLOADTYPE	
BUFFER_INFO_PIXELFORMAT	
BUFFER_INFO_PIXELFORMAT_NAMESPACE	
BUFFER_INFO_DELIVERED_IMAGEHEIGHT	
BUFFER_INFO_DELIVERED_CHUNKPAYLOADSIZE	
BUFFER_INFO_CHUNKLAYOUTID	
BUFFER_INFO_FILENAME	
BUFFER_INFO_PIXEL_ENDIANNESS	
BUFFER_INFO_DATA_SIZE	
BUFFER_INFO_TIMESTAMP_NS	
BUFFER_INFO_DATA_LARGER_THAN_BUFFER	
BUFFER_INFO_CONTAINS_CHUNKDATA	
BUFFER_INFO_CUSTOM_ID	

BUFFER_PART_INFO_CMD_LIST

Values

Name	Description
BUFFER_PART_INFO_BASE	
BUFFER_PART_INFO_DATA_SIZE	
BUFFER_PART_INFO_DATA_TYPE	
BUFFER_PART_INFO_DATA_FORMAT	
BUFFER_PART_INFO_DATA_FORMAT_NAMESPACE	
BUFFER_PART_INFO_WIDTH	
BUFFER_PART_INFO_HEIGHT	
BUFFER_PART_INFO_XOFFSET	
BUFFER_PART_INFO_YOFFSET	
BUFFER_PART_INFO_XPADDING	
BUFFER_PART_INFO_SOURCE_ID	
BUFFER_PART_INFO_DELIVERED_IMAGEHEIGHT	
BUFFER_PART_INFO_CUSTOM_ID	

PAYLOADTYPE_INFO_IDS

Values

Name	Description
PAYLOAD_TYPE_UNKNOWN	
PAYLOAD_TYPE_IMAGE	
PAYLOAD_TYPE_RAW_DATA	
PAYLOAD_TYPE_FILE	
PAYLOAD_TYPE_CHUNK_DATA	
PAYLOAD_TYPE_JPEG	
PAYLOAD_TYPE_JPEG2000	
PAYLOAD_TYPE_H264	
PAYLOAD_TYPE_CHUNK_ONLY	
PAYLOAD_TYPE_DEVICE_SPECIFIC	
PAYLOAD_TYPE_MULTI_PART	
PAYLOAD_TYPE_CUSTOM_ID	

PIXELFORMAT_NAMESPACE_IDS

Values

Name	Description
PIXELFORMAT_NAMESPACE_UNKNOWN	
PIXELFORMAT_NAMESPACE_GEV	
PIXELFORMAT_NAMESPACE_IIDC	
PIXELFORMAT_NAMESPACE_PFNC_16BIT	
PIXELFORMAT_NAMESPACE_PFNC_32BIT	
PIXELFORMAT_NAMESPACE_CUSTOM_ID	

PIXELENDIANNESS_IDS

Values

Name	Description
PIXELENDIANNESS_UNKNOWN	
PIXELENDIANNESS_LITTLE	
PIXELENDIANNESS_BIG	

PARTDATATYPE_IDS

Values

Name	Description
PART_DATATYPE_UNKNOWN	
PART_DATATYPE_2D_IMAGE	
PART_DATATYPE_2D_PLANE_BIPLANAR	
PART_DATATYPE_2D_PLANE_TRIPLANAR	
PART_DATATYPE_2D_PLANE_QUADPLANAR	
PART_DATATYPE_3D_IMAGE	
PART_DATATYPE_3D_PLANE_BIPLANAR	
PART_DATATYPE_3D_PLANE_TRIPLANAR	
PART_DATATYPE_3D_PLANE_QUADPLANAR	
PART_DATATYPE_CONFIDENCE_MAP	
PART_DATATYPE_CUSTOM_ID	

PORT_INFO_CMD_LIST

Values

Name	Description
PORT_INFO_ID	
PORT_INFO_VENDOR	
PORT_INFO_MODEL	
PORT_INFO_TLTYPE	
PORT_INFO_MODULE	
PORT_INFO_LITTLE_ENDIAN	
PORT_INFO_BIG_ENDIAN	
PORT_INFO_ACCESS_READ	
PORT_INFO_ACCESS_WRITE	
PORT_INFO_ACCESS_NA	
PORT_INFO_ACCESS_NI	
PORT_INFO_VERSION	
PORT_INFO_PORTNAME	
PORT_INFO_CUSTOM_ID	

URL_SCHEME_IDS

Values

Name	Description
URL_SCHEME_LOCAL	
URL_SCHEME_HTTP	
URL_SCHEME_FILE	
URL_SCHEME_CUSTOM_ID	

URL_INFO_CMD_LIST

Values

Name	Description
URL_INFO_URL	
URL_INFO_SCHEMA_VER_MAJOR	
URL_INFO_SCHEMA_VER_MINOR	
URL_INFO_FILE_VER_MAJOR	
URL_INFO_FILE_VER_MINOR	
URL_INFO_FILE_VER_SUBMINOR	
URL_INFO_FILE_SHA1_HASH	
URL_INFO_FILE_REGISTER_ADDRESS	
URL_INFO_FILE_SIZE	
URL_INFO_SCHEME	
URL_INFO_FILENAME	
URL_INFO_CUSTOM_ID	

EVENT_TYPE_LIST

Values

Name	Description
EVENT_ERROR	
EVENT_NEW_BUFFER	
EVENT_FEATURE_INVALIDATE	
EVENT_FEATURE_CHANGE	
EVENT_REMOTE_DEVICE	
EVENT_MODULE	
EVENT_CUSTOM_ID	

EVENT_INFO_CMD_LIST

Values

Name	Description
EVENT_EVENT_TYPE	
EVENT_NUM_IN_QUEUE	
EVENT_NUM_FIRED	
EVENT_SIZE_MAX	
EVENT_INFO_DATA_SIZE_MAX	
EVENT_INFO_CUSTOM_ID	

EVENT_DATA_INFO_CMD_LIST

Values

Name	Description
EVENT_DATA_ID	
EVENT_DATA_VALUE	
EVENT_DATA_NUMID	
EVENT_DATA_CUSTOM_ID	

Type definitions

bool8_t

```
typedef uint8_t bool8_t
```

GC_ERROR

```
typedef int32_t GC_ERROR
```

TL_HANDLE

```
typedef void* TL_HANDLE
```

IF_HANDLE

```
typedef void* IF_HANDLE
```

DEV_HANDLE

```
typedef void* DEV_HANDLE
```

DS_HANDLE

```
typedef void* DS_HANDLE
```

PORT_HANDLE

```
typedef void* PORT_HANDLE
```

BUFFER_HANDLE

typedef void* BUFFER_HANDLE

EVENTSRC_HANDLE

typedef void* EVENTSRC_HANDLE

EVENT_HANDLE

typedef void* EVENT_HANDLE

INFO_DATATYPE

typedef int32_t INFO_DATATYPE

TL_CHAR_ENCODING

typedef int32_t TL_CHAR_ENCODING

TL_INFO_CMD

typedef int32_t TL_INFO_CMD

INTERFACE_INFO_CMD

typedef int32_t INTERFACE_INFO_CMD

DEVICE_ACCESS_FLAGS

typedef int32_t DEVICE_ACCESS_FLAGS

DEVICE_ACCESS_STATUS

typedef int32_t DEVICE_ACCESS_STATUS

DEVICE_INFO_CMD

typedef int32_t DEVICE_INFO_CMD

ACQ_STOP_FLAGS

typedef int32_t ACQ_STOP_FLAGS

ACQ_START_FLAGS

typedef int32_t ACQ_START_FLAGS

ACQ_QUEUE_TYPE

typedef int32_t ACQ_QUEUE_TYPE

STREAM_INFO_CMD

typedef int32_t STREAM_INFO_CMD

BUFFER_INFO_CMD

typedef int32_t BUFFER_INFO_CMD

BUFFER_PART_INFO_CMD

typedef int32_t BUFFER_PART_INFO_CMD

PAYLOADTYPE_INFO_ID

typedef int32_t PAYLOADTYPE_INFO_ID

PIXELFORMAT_NAMESPACE_ID

typedef int32_t PIXELFORMAT_NAMESPACE_ID

PIXELENDIANNES_ID

typedef int32_t PIXELENDIANNES_ID

PARTDATATYPE_ID

typedef int32_t PARTDATATYPE_ID

PORT_INFO_CMD

typedef int32_t PORT_INFO_CMD

URL_SCHEME_ID

typedef int32_t URL_SCHEME_ID

URL_INFO_CMD

typedef int32_t URL_INFO_CMD

EVENT_TYPE

typedef int32_t EVENT_TYPE

EVENT_INFO_CMD

typedef int32_t EVENT_INFO_CMD

EVENT_DATA_INFO_CMD

typedef int32_t EVENT_DATA_INFO_CMD

EVENT_NEW_BUFFER_DATA

typedef struct S_EVENT_NEW_BUFFER EVENT_NEW_BUFFER_DATA

PORT_REGISTER_STACK_ENTRY

typedef struct S_PORT_REGISTER_STACK_ENTRY PORT_REGISTER_STACK_ENTRY

SINGLE_CHUNK_DATA

typedef struct S_SINGLE_CHUNK_DATA SINGLE_CHUNK_DATA

Functions

GCGetInfo

GC_API GCGetInfo(TL_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

GCGetLastError

GC_API GCGetLastError(GC_ERROR *piErrorCode, char *sErrText, size_t *piSize)

GCInitLib

GC_API GCInitLib(void)

GCCloseLib

GC_API GCCloseLib(void)

GCReadPort

GC_API GCReadPort(PORT_HANDLE hPort, uint64_t iAddress, void *pBuffer, size_t *piSize)

GCWritePort

GC_API GCWritePort(PORT_HANDLE hPort, uint64_t iAddress, const void *pBuffer, size_t *piSize)

GCGetPortURL

GC_API GCGetPortURL(PORT_HANDLE hPort, char *sURL, size_t *piSize)

GCGetPortInfo

GC_API GCGetPortInfo(PORT_HANDLE hPort, PORT_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

GCRegisterEvent

GC_API GCRegisterEvent(EVENTSRC_HANDLE hEventSrc, EVENT_TYPE iEventID, EVENT_HANDLE *phEvent)

GCUnregisterEvent

GC_API GCUnregisterEvent(EVENTSRC_HANDLE hEventSrc, EVENT_TYPE iEventID)

EventGetData

GC_API EventGetData(EVENT_HANDLE hEvent, void *pBuffer, size_t *piSize, uint64_t iTimeout)

EventGetDataInfo

GC_API EventGetDataInfo(EVENT_HANDLE hEvent, const void *pInBuffer, size_t iInSize, EVENT_DATA_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pOutBuffer, size_t *piOutSize)

EventGetInfo

GC_API EventGetInfo(EVENT_HANDLE hEvent, EVENT_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

EventFlush

GC_API EventFlush(EVENT_HANDLE hEvent)

EventKill

GC_API EventKill(EVENT_HANDLE hEvent)

TLOpen

GC_API TLOpen(TL_HANDLE *phTL)

TLClose

GC_API TLClose(TL_HANDLE hTL)

TLGetInfo

GC_API TLGetInfo(TL_HANDLE hTL, TL_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

TLGetNumInterfaces

GC_API TLGetNumInterfaces(TL_HANDLE hTL, uint32_t *piNumIfaces)

TLGetInterfaceID

GC_API TLGetInterfaceID(TL_HANDLE hTL, uint32_t iIndex, char *sID, size_t *piSize)

TLGetInterfaceInfo

GC_API TLGetInterfaceInfo(TL_HANDLE hTL, const char *sIfaceID, INTERFACE_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

TLOpenInterface

GC_API TLOpenInterface(TL_HANDLE hTL, const char *sIfaceID, IF_HANDLE *phIface)

TLUpdateInterfaceList

GC_API TLUpdateInterfaceList(TL_HANDLE hTL, bool8_t *pbChanged, uint64_t iTimeout)

IFClose

GC_API IFClose(IF_HANDLE hIface)

IFGetInfo

GC_API IFGetInfo(IF_HANDLE hIface, INTERFACE_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

IFGetNumDevices

GC_API IFGetNumDevices(IF_HANDLE hIface, uint32_t *piNumDevices)

IFGetDeviceID

GC_API IFGetDeviceID(IF_HANDLE hIface, uint32_t iIndex, char *sIDeviceID, size_t *piSize)

IFUpdateDeviceList

GC_API IFUpdateDeviceList(IF_HANDLE hIface, bool8_t *pbChanged, uint64_t iTimeout)

IFGetDeviceInfo

GC_API IFGetDeviceInfo(IF_HANDLE hIface, const char *sDeviceID, DEVICE_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

IFOpenDevice

GC_API IFOpenDevice(IF_HANDLE hIface, const char *sDeviceID, DEVICE_ACCESS_FLAGS iOpenFlags, DEV_HANDLE *phDevice)

DevGetPort

GC_API DevGetPort(DEV_HANDLE hDevice, PORT_HANDLE *phRemoteDevice)

DevGetNumDataStreams

GC_API DevGetNumDataStreams(DEV_HANDLE hDevice, uint32_t *piNumDataStreams)

DevGetDataStreamID

GC_API DevGetDataStreamID(DEV_HANDLE hDevice, uint32_t iIndex, char *sDataStreamID, size_t *piSize)

DevOpenDataStream

GC_API DevOpenDataStream(DEV_HANDLE hDevice, const char *sDataStreamID, DS_HANDLE *phDataStream)

DevGetInfo

GC_API DevGetInfo(DEV_HANDLE hDevice, DEVICE_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

DevClose

GC_API DevClose(DEV_HANDLE hDevice)

DSAnnounceBuffer

GC_API DSAnnounceBuffer(DS_HANDLE hDataStream, void *pBuffer, size_t iSize, void *pPrivate, BUFFER_HANDLE *phBuffer)

DSAllocAndAnnounceBuffer

GC_API DSAllocAndAnnounceBuffer(DS_HANDLE hDataStream, size_t iSize, void *pPrivate, BUFFER_HANDLE *phBuffer)

DSFlushQueue

GC_API DSFlushQueue(DS_HANDLE hDataStream, ACQ_QUEUE_TYPE iOperation)

DSStartAcquisition

GC_API DSStartAcquisition(DS_HANDLE hDataStream, ACQ_START_FLAGS iStartFlags, uint64_t iNumToAcquire)

DSStopAcquisition

GC_API DSStopAcquisition(DS_HANDLE hDataStream, ACQ_STOP_FLAGS iStopFlags)

DSGetInfo

GC_API DSGetInfo(DS_HANDLE hDataStream, STREAM_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

DSGetBufferID

GC_API DSGetBufferID(DS_HANDLE hDataStream, uint32_t iIndex, BUFFER_HANDLE *phBuffer)

DSClose

GC_API DSClose(DS_HANDLE hDataStream)

DSRevokeBuffer

GC_API DSRevokeBuffer(DS_HANDLE hDataStream, BUFFER_HANDLE hBuffer, void **pBuffer, void **pPrivate)

DSQueueBuffer

GC_API DSQueueBuffer(DS_HANDLE hDataStream, BUFFER_HANDLE hBuffer)

DSGetBufferInfo

GC_API DSGetBufferInfo(DS_HANDLE hDataStream, BUFFER_HANDLE hBuffer, BUFFER_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

GCGetNumPortURLs

GC_API GCGetNumPortURLs(PORT_HANDLE hPort, uint32_t *piNumURLs)

GCGetPortURLInfo

GC_API GCGetPortURLInfo(PORT_HANDLE hPort, uint32_t iURLIndex, URL_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

GCReadPortStacked

GC_API GCReadPortStacked(PORT_HANDLE hPort, PORT_REGISTER_STACK_ENTRY *pEntries, size_t *piNumEntries)

GCWritePortStacked

GC_API GCWritePortStacked(PORT_HANDLE hPort, PORT_REGISTER_STACK_ENTRY *pEntries, size_t *piNumEntries)

DSGetBufferChunkData

GC_API DSGetBufferChunkData(DS_HANDLE hDataStream, BUFFER_HANDLE hBuffer, SINGLE_CHUNK_DATA *pChunkData, size_t *piNumChunks)

IFGetParentTL

GC_API IFGetParentTL(IF_HANDLE hIface, TL_HANDLE *phSystem)

DevGetParentIF

GC_API DevGetParentIF(DEV_HANDLE hDevice, IF_HANDLE *phIface)

DSGetParentDev

GC_API DSGetParentDev(DS_HANDLE hDataStream, DEV_HANDLE *phDevice)

DSGetNumBufferParts

GC_API DSGetNumBufferParts(DS_HANDLE hDataStream, BUFFER_HANDLE hBuffer, uint32_t *piNumParts)

DSGetBufferPartInfo

GC_API DSGetBufferPartInfo(DS_HANDLE hDataStream, BUFFER_HANDLE hBuffer, uint32_t iPartIndex, BUFFER_PART_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

PGCGetInfo

GC_API_P() PGCGetInfo(TL_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

PGCGetLastError

GC_API_P() PGCGetLastError(GC_ERROR *piErrorCode, char *sErrText, size_t *piSize)

PGCInitLib

GC_API_P() PGCInitLib(void)

PGCCloseLib

GC_API_P() PGCCloseLib(void)

PGCReadPort

GC_API_P() PGCReadPort(PORT_HANDLE hPort, uint64_t iAddress, void *pBuffer, size_t *piSize)

PGCWritePort

GC_API_P() PGCWritePort(PORT_HANDLE hPort, uint64_t iAddress, const void *pBuffer, size_t *piSize)

PGCGetPortURL

GC_API_P() PGCGetPortURL(PORT_HANDLE hPort, char *sURL, size_t *piSize)

PGCGetPortInfo

GC_API_P() PGCGetPortInfo(PORT_HANDLE hPort, PORT_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

PGCRegisterEvent

GC_API_P() PGCRegisterEvent(EVENTSRC_HANDLE hEventSrc, EVENT_TYPE iEventID, EVENT_HANDLE *phEvent)

PGCUnregisterEvent

GC_API_P() PGCUnregisterEvent(EVENTSRC_HANDLE hEventSrc, EVENT_TYPE iEventID)

PEventGetData

GC_API_P() PEventGetData(EVENT_HANDLE hEvent, void *pBuffer, size_t *piSize, uint64_t iTimeout)

PEventGetDataInfo

GC_API_P() PEventGetDataInfo(EVENT_HANDLE hEvent, const void *pInBuffer, size_t iInSize, EVENT_DATA_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pOutBuffer, size_t *piOutSize)

PEventGetInfo

GC_API_P() PEventGetInfo(EVENT_HANDLE hEvent, EVENT_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

PEventFlush

GC_API_P() PEventFlush(EVENT_HANDLE hEvent)

PEventKill

GC_API_P() PEventKill(EVENT_HANDLE hEvent)

PTLOpen

GC_API_P() PTLOpen(TL_HANDLE *phTL)

PTLClose

GC_API_P() PTLClose(TL_HANDLE hTL)

PTLGetInfo

GC_API_P() PTLGetInfo(TL_HANDLE hTL, TL_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

PTLGetNumInterfaces

GC_API_P() PTLGetNumInterfaces(TL_HANDLE hTL, uint32_t *piNumIfaces)

PTLGetInterfaceID

GC_API_P() PTLGetInterfaceID(TL_HANDLE hTL, uint32_t iIndex, char *sID, size_t *piSize)

PTLGetInterfaceInfo

GC_API_P() PTLGetInterfaceInfo(TL_HANDLE hTL, const char *sIfaceID, INTERFACE_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

PTLOpenInterface

GC_API_P() PTLOpenInterface(TL_HANDLE hTL, const char *sIfaceID, IF_HANDLE *phIface)

PTLUpdateInterfaceList

GC_API_P() PTLUpdateInterfaceList(TL_HANDLE hTL, bool8_t *pbChanged, uint64_t iTimeout)

PIFClose

GC_API_P() PIFClose(IF_HANDLE hIface)

PIFGetInfo

GC_API_P() PIFGetInfo(IF_HANDLE hIface, INTERFACE_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

PIFGetNumDevices

GC_API_P() PIFGetNumDevices(IF_HANDLE hIface, uint32_t *piNumDevices)

PIFGetDeviceID

GC_API_P() PIFGetDeviceID(IF_HANDLE hIface, uint32_t iIndex, char *sDeviceID, size_t *piSize)

PIFUpdateDeviceList

GC_API_P() PIFUpdateDeviceList(IF_HANDLE hIface, bool8_t *pbChanged, uint64_t iTimeout)

PIFGetDeviceInfo

GC_API_P() PIFGetDeviceInfo(IF_HANDLE hIface, const char *sDeviceID, DEVICE_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

PIFOpenDevice

GC_API_P() PIFOpenDevice(IF_HANDLE hIface, const char *sDeviceID, DEVICE_ACCESS_FLAGS iOpenFlags, DEV_HANDLE *phDevice)

PDevGetPort

GC_API_P() PDevGetPort(DEV_HANDLE hDevice, PORT_HANDLE *phRemoteDevice)

PDevGetNumDataStreams

GC_API_P() PDevGetNumDataStreams(DEV_HANDLE hDevice, uint32_t *piNumDataStreams)

PDevGetDataStreamID

GC_API_P() PDevGetDataStreamID(DEV_HANDLE hDevice, uint32_t iIndex, char *sDataStreamID, size_t *piSize)

PDevOpenDataStream

GC_API_P() PDevOpenDataStream(DEV_HANDLE hDevice, const char *sDataStreamID, DS_HANDLE *phDataStream)

PDevGetInfo

GC_API_P() PDevGetInfo(DEV_HANDLE hDevice, DEVICE_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

PDevClose

GC_API_P() PDevClose(DEV_HANDLE hDevice)

PDSAnnounceBuffer

GC_API_P() PDSAnnounceBuffer(DS_HANDLE hDataStream, void *pBuffer, size_t iSize, void *pPrivate, BUFFER_HANDLE *phBuffer)

PDSAllocAndAnnounceBuffer

GC_API_P() PDSAllocAndAnnounceBuffer(DS_HANDLE hDataStream, size_t iSize, void *pPrivate, BUFFER_HANDLE *phBuffer)

PDSFlushQueue

GC_API_P() PDSFlushQueue(DS_HANDLE hDataStream, ACQ_QUEUE_TYPE iOperation)

PDSStartAcquisition

GC_API_P() PDSStartAcquisition(DS_HANDLE hDataStream, ACQ_START_FLAGS iStartFlags, uint64_t iNumToAcquire)

PDSStopAcquisition

GC_API_P() PDSStopAcquisition(DS_HANDLE hDataStream, ACQ_STOP_FLAGS iStopFlags)

PDSGetInfo

GC_API_P() PDSGetInfo(DS_HANDLE hDataStream, STREAM_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

PDSGetBufferID

GC_API_P() PDSGetBufferID(DS_HANDLE hDataStream, uint32_t iIndex, BUFFER_HANDLE *phBuffer)

PDSClose

GC_API_P() PDSClose(DS_HANDLE hDataStream)

PDSRevokeBuffer

GC_API_P() PDSRevokeBuffer(DS_HANDLE hDataStream, BUFFER_HANDLE hBuffer, void **pBuffer, void **pPrivate)

PDSQueueBuffer

GC_API_P() PDSQueueBuffer(DS_HANDLE hDataStream, BUFFER_HANDLE hBuffer)

PDSGetBufferInfo

GC_API_P() PDSGetBufferInfo(DS_HANDLE hDataStream, BUFFER_HANDLE hBuffer, BUFFER_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

PGCGetNumPortURLs

GC_API_P() PGCGetNumPortURLs(PORT_HANDLE hPort, uint32_t *iNumURLs)

PGCGetPortURLInfo

GC_API_P() PGCGetPortURLInfo(PORT_HANDLE hPort, uint32_t iURLIndex, URL_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

PGCReadPortStacked

GC_API_P() PGCReadPortStacked(PORT_HANDLE hPort, PORT_REGISTER_STACK_ENTRY *pEntries, size_t *piNumEntries)

PGCWritePortStacked

GC_API_P() PGCWritePortStacked(PORT_HANDLE hPort, PORT_REGISTER_STACK_ENTRY *pEntries, size_t *piNumEntries)

PDSGetBufferChunkData

GC_API_P() PDSGetBufferChunkData(DS_HANDLE hDataStream, BUFFER_HANDLE hBuffer, SINGLE_CHUNK_DATA *pChunkData, size_t *piNumChunks)

PIFGetParentTL

GC_API_P() PIFGetParentTL(IF_HANDLE hIface, TL_HANDLE *phSystem)

PDevGetParentIF

GC_API_P() PDevGetParentIF(DEV_HANDLE hDevice, IF_HANDLE *phIface)

PDSGetParentDev

GC_API_P() PDSGetParentDev(DS_HANDLE hDataStream, DEV_HANDLE *phDevice)

PDSGetNumBufferParts

GC_API_P() PDSGetNumBufferParts(DS_HANDLE hDataStream, BUFFER_HANDLE hBuffer, uint32_t *piNumParts)

PDSGetBufferPartInfo

GC_API_P() PDSGetBufferPartInfo(DS_HANDLE hDataStream, BUFFER_HANDLE hBuffer, uint32_t iPartIndex, BUFFER_PART_INFO_CMD iInfoCmd, INFO_DATATYPE *piType, void *pBuffer, size_t *piSize)

EVENT_NEW_BUFFER_DATA

BufferHandle

BUFFER_HANDLE S_EVENT_NEW_BUFFER::BufferHandle

pUserPointer

void* S_EVENT_NEW_BUFFER::pUserPointer

PORT_REGISTER_STACK_ENTRY

Address

uint64_t S_PORT_REGISTER_STACK_ENTRY::Address

pBuffer

void* S_PORT_REGISTER_STACK_ENTRY::pBuffer

Size

size_t S_PORT_REGISTER_STACK_ENTRY::Size

SINGLE_CHUNK_DATA

ChunkID

uint64_t S_SINGLE_CHUNK_DATA::ChunkID

ChunkOffset

ptrdiff_t S_SINGLE_CHUNK_DATA::ChunkOffset

ChunkLength

size_t S_SINGLE_CHUNK_DATA::ChunkLength

KYFGLib adapter generated API

Generated API reference assembled from the configured public headers.

KYFGLib adapter API

Generated from PUBLIC/include/KYFGLibA.h.

Public adapter functions for accessing Vision Point II handles.

Include this header to combine the legacy KYFGLib API with Vision Point II operations. The accessors return borrowed handles for objects already managed by the adapter. Keep the corresponding grabber or camera open while using them, and close it through the legacy API. C++ applications also receive the enumeration helper declared in KYFGLibAcpp.h.

Functions

KYFG_Grabber_Get_KYVP_PCI_INTERFACE_HANDLE

FGSTATUS KYFG_Grabber_Get_KYVP_PCI_INTERFACE_HANDLE(FGHANDLE handle, KYVP_PCI_INTERFACE_HANDLE *_pKYVP_PCI_INTERFACE_HANDLE)

Retrieve the PCI interface handle associated with a legacy grabber.

Parameter	Description
handle	Open grabber handle returned by KYFG_Open() or KYFG_OpenEx().
_pKYVP_PCI_INTERFACE_HANDLE	Required destination for the borrowed KYVP_PCI_INTERFACE_HANDLE.

Returns: FGSTATUS_OK on success; otherwise an adapter status listed below.

Parameter	Description
FGSTATUS_UNKNOWN_HANDLE	The legacy handle is not registered with the adapter.
FGSTATUS_UNKNOWN_ERROR	The registered handle could not be resolved to a grabber.

Remark: Use the result with interface-level Vision Point II operations while the grabber remains open. It is not a new interface connection and must not be closed separately.

See: KYFG_Grabber_Get_KYVP_COLLECTION_HANDLE()

KYFG_Grabber_Get_KYVP_COLLECTION_HANDLE

FGSTATUS KYFG_Grabber_Get_KYVP_COLLECTION_HANDLE(FGHANDLE handle, KYVP_COLLECTION_HANDLE *_pKYVP_COLLECTION_HANDLE)

Retrieve the parameter collection associated with a legacy grabber.

Parameter	Description
handle	Open grabber handle returned by KYFG_Open() or KYFG_OpenEx().
_pKYVP_COLLECTION_HANDLE	Required destination for the borrowed KYVP_COLLECTION_HANDLE.

Returns: FGSTATUS_OK on success; otherwise an adapter status listed below.

Parameter	Description
FGSTATUS_UNKNOWN_HANDLE	The legacy handle is not registered with the adapter.
FGSTATUS_UNKNOWN_ERROR	The registered handle could not be resolved to a grabber.

Remark: The result refers to the grabber's parameter collection. Use it with the Parameters Handler API while the grabber remains open; do not delete this collection separately.

See: KYFG_Grabber_Get_KYVP_PCI_INTERFACE_HANDLE()

KYFG_Camera_Get_KYVP_DEVICE_HANDLE

FGSTATUS KYFG_Camera_Get_KYVP_DEVICE_HANDLE(CAMHANDLE camHandle, KYVP_DEVICE_HANDLE *_pKYVP_DEVICE_HANDLE)

Retrieve the local-device handle associated with a legacy camera.

Parameter	Description
camHandle	Camera handle successfully opened with KYFG_CameraOpen2() or the legacy KYFG_CameraOpen(); its parent grabber must also remain open.
_pKYVP_DEVICE_HANDLE	Required destination for the borrowed KYVP_DEVICE_HANDLE.

Returns: FGSTATUS_OK on success; otherwise an adapter status listed below.

Parameter	Description
FGSTATUS_UNKNOWN_HANDLE	The camera handle is not registered with the adapter.
FGSTATUS_CAMERA_NOT_OPENED	The registered camera handle could not be resolved to a camera adapter object.

Remark: The result identifies the local device, not a KYVP_REMOTE_DEVICE_HANDLE. Keep the camera and its grabber open while using it, and do not close it separately.

KYFGLibAcpp API

KYFGLibAcpp.h API reference

Generated from PUBLIC/include/KYFGLibAcpp.h.

API documentation generated from PUBLIC/include/KYFGLibAcpp.h.

Include this header directly or through KYFGLibA.h in C++ applications. Definitions use std::map and std::string and own their copied entry data. The C++ API requires a compiler, standard library, and runtime configuration compatible with the SDK binary.

Functions

KY_GetEnumParameterDefinition

FGSTATUS KY_GetEnumParameterDefinition(KYHANDLE handle, const char *paramName, EnumParameterDefinition &enumParameterDefinition)

Add an enumeration's entry definitions to an application-owned result.

Returns entry definitions for the enumeration specified by paramName. With a camera handle, the enumeration may belong to the local or remote device. With a grabber handle, grabber enumerations are queried directly; stream enumerations require an existing stream for the current camera selector. Returned entries may be unavailable or not writable in the current device state.

Parameter	Description
handle	Open FGHANDLE or CAMHANDLE, passed as KYHANDLE.
paramName	Required NUL-terminated enumeration name; matching is case-sensitive.
enumParameterDefinition	Result whose m_Entries map receives copied entry records. Matching numeric keys are replaced; other entries are retained.

Returns: FGSTATUS_OK after inspecting the selected collection, including when no matching entries exist; otherwise an adapter status listed below.

Parameter	Description
FGSTATUS_UNKNOWN_HANDLE	No parameter collection was resolved for the supplied handle and parameter, including a stream parameter with no selected stream available.
FGSTATUS_UNKNOWN_ERROR	A registered grabber or camera adapter object was not found.
FGSTATUS_INVALID_VALUE	The selected collection's node descriptors could not be read.

Remark: The result is not cleared before lookup. Use a fresh result, or call enumParameterDefinition.m_Entries.clear() before querying a different parameter. A reported error leaves the result unchanged. The copied entries belong to the caller and require no SDK release function. Keep the selected grabber, camera, and stream open for the duration of the query.

See: EnumParameterDefinition

See: EnumParameterDefinition_Entry

KYFGLib Adapter C++ Structs

Field reference for the enumeration result and its nested entry type.

These are ordinary C++ objects with automatic storage management. Construct them normally; do not zero their storage with 'memset' or serialize their in-memory representations. Tables list data members in declaration order.

EnumParameterDefinition::Entry

One enumeration entry. Default construction sets 'value' to zero and constructs empty strings. On non-MSVC compilers, the three-argument constructor copies '_value', '_name', and '_displayName'; both string arguments must point to valid NUL-terminated strings. MSVC builds retain aggregate initialization for the same members.

Structure Field	Type	Description	Remarks
value	int64_t	Numeric value of the entry.	Defaults to zero; matches the map key for queried entries.
name	std::string	Symbolic name of the entry.	Owned copy of the descriptor's symbolic value.
displayName	std::string	Display name of the entry.	Owned copy of the descriptor's display name.

EnumParameterDefinition

Application-owned enumeration definition populated by KY_GetEnumParameterDefinition(). The map starts empty and stores entries in ascending numeric-key order. A query replaces existing entries with matching keys and leaves all other entries intact. When descriptors share a numeric value, the last matching descriptor encountered supplies the stored entry.

Structure Field	Type	Description	Remarks
m_Entries	std::map<int64_t, Entry>	Entry records indexed by numeric value.	Entry is EnumParameterDefinition::Entry; one record per distinct value.

KYFoundation API

KYFoundation.h API reference

Generated from PUBLIC/include/KYFoundation.h.

API documentation generated from PUBLIC/include/KYFoundation.h.

Include this header to use KY_RESULT, common result codes, and the result inspection functions. Test completion with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED(); use the accessors for the numeric code, severity, and diagnostic text. A result code or severity alone is not a success test.

Macros

KY_CALLCONV

Foundation calling convention; __cdecl with MSVC and empty otherwise.

KYFOUNDATION_EXTERNAL_C

C linkage in C++; empty in C.

KYFOUNDATION_EXTERNAL_DLL

Foundation DLL import or export attribute on MSVC; empty otherwise.

Application builds use the import declaration. Do not define KY_FOUNDATION_EXPORTS.

KYFOUNDATION_API

KYFOUNDATION_EXTERNAL_C KYFOUNDATION_EXTERNAL_DLL
Linkage and symbol visibility for Foundation API declarations.

KY_PRINTF_PARAM

Annotation for a printf-style format-string parameter.

Place before the parameter type. Enables MSVC format checking; empty otherwise.

KY_PRINTF_ATTR

__attribute__((format(printf, _fmt_index, _args_index)))
Annotation for printf-style argument checking on GNU-compatible compilers.

Place after the function parameter list. Empty on MSVC. For non-static C++ member functions, the implicit this parameter counts as argument one.

Parameter	Description
_fmt_index	One-based index of the format-string parameter.
_args_index	One-based index of the first argument to check; zero for va_list functions.

KY_RESULT_SUCCEEDED

(0 == ((result.uResult >> 31) & 0x01))
Test whether an SDK result reports success.

Parameter	Description
result	Result returned by an SDK operation.

Returns: Nonzero for success; zero for failure.

See: `KY_RESULT_FAILED()`

KY_RESULT_FAILED

```
(!KY_RESULT_SUCCEEDED(result))
```

Test whether an SDK result reports failure.

Parameter	Description
result	Result returned by an SDK operation.

Returns: Nonzero for failure; zero for success.

See: `KY_RESULT_SUCCEEDED()`

Functions

KYFoundation_What

```
const char * KYFoundation_What(KY_RESULT _result)
```

Retrieve diagnostic text for an SDK result.

Parameter	Description
_result	Result returned by an SDK operation.

Returns: Library-owned, null-terminated diagnostic string. Returns "Unknown KY_RESULT" when no description is available.

Remark: Do not modify or free the returned string. Retrieve the description on the thread that performed the operation to obtain any available operation-specific detail. Copy the text before further SDK calls or before retaining it beyond the calling thread's lifetime. Diagnostic wording may vary; do not parse it to determine status.

See: `KYFoundation_GetErrorCode()`

KYFoundation_GetErrorCode

```
uint32_t KYFoundation_GetErrorCode(KY_RESULT _result)
```

Retrieve the numeric code from an SDK result.

Parameter	Description
_result	Result returned by an SDK operation.

Returns: Code component of the result. Compare with `KY_RESULT_CODE_*` constants or the additional result codes documented by the called API.

Remark: The code alone does not identify the complete result or its success state. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` first.

See: Common result codes

KYFoundation_GetSeverity

```
uint32_t KYFoundation_GetSeverity(KY_RESULT _result)
```

Retrieve the severity associated with an SDK result.

Parameter	Description
_result	Result returned by an SDK operation.

Returns: One of the KY_SEVERITY_* values.

Remark: Severity describes the diagnostic category; it is independent of the success test performed by KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: Diagnostic severities

KYFoundation_CompilerWarnings API

KYFoundation_CompilerWarnings.h API reference

Generated from PUBLIC/include/KYFoundation_CompilerWarnings.h.

API documentation generated from PUBLIC/include/KYFoundation_CompilerWarnings.h.

Pair `KY_DISABLE_WARNING_PUSH` with `KY_DISABLE_WARNING_POP` around the smallest region that needs a suppression. The named controls have compiler-specific mappings; an empty mapping has no effect. See Foundation Compiler Warnings for the complete mapping, including names not defined on a given compiler.

Foundation Compiler Warnings

Warning controls for MSVC and GNU-compatible compilers.

This header supports MSVC and GCC/Clang diagnostic syntax. Other compiler families receive no definitions. Named suppressions do not correct a code defect and do not change the generated program's required behavior.

Saving and restoring warning settings

Macro	Usage
<code>KY_DISABLE_WARNING_PUSH</code>	Save the current diagnostic settings before a suppression.
<code>KY_DISABLE_WARNING_POP</code>	Restore the settings saved by the matching push.
<code>KY_DISABLE_WARNING(value)</code>	Pass an MSVC warning number, or an unquoted GCC/Clang warning option supported by the compiler.

`KY_DISABLE_WARNING_PUSH`
`KY_DISABLE_WARNING_UNREFERENCED_FORMAL_PARAMETER`
`KY_DISABLE_WARNING_POP`

Place the declarations or definitions that need the suppression between the named control and `KY_DISABLE_WARNING_POP`.

Direct controls use `KY_DISABLE_WARNING(4100)` on MSVC or `KY_DISABLE_WARNING(-Wunused-parameter)` on GCC/Clang. Prefer the named controls when their mapping suits the application. Diagnostic availability depends on the compiler version and enabled analysis tools.

Named warning controls

Each name below follows the prefix `KY_DISABLE_WARNING_`. "None" means the macro expands to nothing; "Not defined" means the name is unavailable in that compiler branch. Existing macro spellings are retained for source compatibility.

Name suffix	MSVC	GCC/Clang	Diagnostic
<code>UNREFERENCED_FORMAL_PARAMETER</code>	C4100	-Wunused-parameter	Unused function parameter.
<code>UNARY_MINUS_OPERATOR</code>	C4146	None	Unary minus applied to an unsigned value.

CONVERSION_FROM_TYPE2TYPE	C4244	None	Conversion that may lose data.
NEED_DLL_INTERFACE	C4251	None	Member type without the required DLL interface.
CONVERSION_FROM_SIZE_T2TYPE	C4267	None	Conversion from size_t that may lose data.
CAST_GREATER_SIZE	C4312	None	Conversion to a pointer of a larger size.
STRUCT_WAS_PADDED	C4324	None	Structure padding required by an alignment specification.
HIDES_PREVIOUS_DECLARATION	C4456	None	Declaration that hides a previous local declaration.
HIDES_GLOBAL_DECLARATION	C4459	None	Declaration that hides a global declaration.
RELATIVE_INCLUDE_PATH	C4464	None	Relative include path containing a parent-directory component.
UNREFERENCED_FUNCTION	C4505	-Wunused-function	Unused function.
UNREFERENCED_INLINE_FUNCTION	C4514	None	Unused inline function.
SEH_ARE_NO_LONGER_CAUGHT	C4571	None	MSVC diagnostic about catch-all handling of structured exceptions.
NON_MEMBER_NEW_INLINE	C4595	None	Non-member allocation or deallocation function declared inline.
COPY_CTOR_IMPLICITLY_DELETED	C4625	None	Copy constructor that is implicitly deleted.
ASSIGNMENT_OPERATOR_IMPLICITLY_DELETED	C4626	None	Copy assignment operator that is

			implicitly deleted.
FUNCTION_NOT_INLINED	C4710	None	Function that was not inlined.
FMT_STRING_EXPECTED	C4774	None	Format argument that is not a string literal.
BYTES_PADDING_ADDED	C4820	None	Padding bytes added to a structure or class.
NON_PORTABLEUSE_OF_CLASS	C4840	Not defined	Class object passed to a variadic function in a non-portable way.
DEPRECATED	C4996	-Wdeprecated-declarations	Use of a deprecated declaration.
MOVE_CTOR_IMPLICITLY_DELETED	C5026	None	Move constructor that is implicitly deleted.
PTR_OR_REF_TO_THROWING_FUNC	C5039	None	Potentially throwing function passed through a C-linkage interface.
MOVE_ASSIGNMENT_IMPLICITLY_DELETED	C5027	None	Move assignment operator that is implicitly deleted.
UNINITIALIZED_MEMORY	C6001	None	Use of uninitialized memory.
DEREFERENCING_NULL	C6011	None	Possible null-pointer dereference.
INDEX_OUT_OF_RANGE	C6201	None	Index outside the valid buffer range.
HIDES_PREVIOUS_DEFINITION	C6244	None	Declaration that hides a previous definition during code analysis.
SECURITY_DESCRIPTOR_NULL	C6248	None	Security descriptor with a null discretionary access-control list.
INCORRECT_OPERATOR	C6282	None	Potentially incorrect operator in an expression.
ALWAYS_NONZERO_CONSTANT	C6285	None	Logical OR of

			nonzero constants that always evaluates to true.
ILL_DEFINED_FOR_LOOP	C6294	None	Loop whose initial test prevents its body from running.
reserved_not_0	C6388	None	Nonzero argument supplied for a reserved parameter.
AVOID_UNNAMED_OBJECTS	C26444	None	Unnamed temporary object that is immediately destroyed.
MAY_NOT_THROW	C26439	None	Function that can be declared noexcept.
CAST_4BYTES_TO8BYTES	C26451	None	Arithmetic performed in a narrower type before conversion to a wider type.
ALWAYS_INITIALIZE_MEMBER	C26495	-Wmissing-field-initializers	Uninitialized member; missing aggregate field initializer on GCC/Clang.
MARK_CONSTEXPR	C26498	None	Variable that can be declared constexpr.
GetTickCount64	C28159	None	Use of an API for which code analysis recommends a safer alternative.
ASSIGNMENT_IN_CONDITION	C4706	Not defined	Assignment used as a condition.
DECLARATION_HIDES_MEMBER	C4458	None	Declaration that hides a class member.
DECLARATION_HIDES_PREVIOUS	C4456	None	Declaration that hides a previous local declaration.
OVERFLOW	C4463	None	Value outside the representable range of a bit field.
COMPARISON_IS_ALWAYS	None	-Wtype-limits	Comparison whose

			result is fixed by the operand type's limits.
SIGN_COMPARE	None	-Wsign-compare	Comparison between signed and unsigned values.
ARITHMETIC_OVERFLOW	Not defined	None	Compatibility placeholder for arithmetic-overflow diagnostics.

See: <https://learn.microsoft.com/en-us/cpp/preprocessor/warning>

See: <https://gcc.gnu.org/onlinedocs/gcc/Diagnostic-Pragmas.html>

KYFoundation_Errors API

KYFoundation_Errors.h API reference

Generated from PUBLIC/include/KYFoundation_Errors.h.

API documentation generated from PUBLIC/include/KYFoundation_Errors.h.

Include KYFoundation.h to access the result inspection functions. Numeric codes and severities describe components of a KY_RESULT; they are not complete return values and must not be assigned directly to its uResult field.

Macros

KY_RESULT_CODE_GENERIC_SUCCESS

0x0

The operation completed successfully.

KY_RESULT_CODE_GENERIC_ERROR

0x1

The operation failed without a more specific common code.

KY_RESULT_CODE_INTERFACE_NOT_SET

0x2

A required interface has not been configured.

KY_RESULT_CODE_BAD_CONTEXT

0x3

The operation is not valid in the current context.

KY_RESULT_CODE_MEMORY_ERROR

0x4

A memory allocation or memory operation failed.

KY_RESULT_CODE_INVALID_CONFIG

0x5

The supplied configuration is invalid.

KY_RESULT_CODE_INITIALIZATION_FAILED

0x6

Initialization could not be completed.

KY_RESULT_CODE_INVALID_PARAMETER

0x7

An argument has an invalid value or combination of values.

KY_RESULT_CODE_INVALID_CRC_CALCULATION

0x8

A cyclic redundancy check failed.

KY_RESULT_CODE_CANNOT_LOCK_PROCESS_RESOURCE

0x9

A required process resource could not be locked.

KY_RESULT_CODE_FILE_NOT_FOUND

0xA

The requested file could not be found.

KY_RESULT_CODE_FILE_CANNOT_OPEN

0xB

The requested file could not be opened.

KY_RESULT_CODE_FILE_READ_ERROR

0xC

Reading from a file failed.

KY_RESULT_CODE_FILE_WRITE_ERROR

0xD

Writing to a file failed.

KY_RESULT_CODE_OPERATION_BLOCKED

0xE

The operation is blocked by an access restriction or current state.

KY_RESULT_CODE_BUFFER_TOO_SMALL

0xF

The supplied buffer is too small for the requested data.

KY_RESULT_CODE_FUNCTIONALITY_NOT_IMPLEMENTED

0x10

The requested functionality is not implemented.

KY_RESULT_CODE_TIMEOUT

0x11

The operation did not complete within the allowed time.

KY_RESULT_CODE_INVALID_INDEX

0x12

The supplied index is outside the valid range.

KY_RESULT_CODE_INVALID_ID

0x13

The supplied identifier is invalid or unrecognized.

KY_RESULT_CODE_ABORT

0x14

The operation was aborted before completion.

KY_RESULT_CODE_INVALID_HANDLE

0x15

The supplied handle is not valid for the operation.

KY_RESULT_CODE_RESOURCE_IN_USE

0x16

The requested resource is already in use.

KY_RESULT_CODE_INVALID_JSON

0x17

The supplied JSON data is invalid.

KY_RESULT_CODE_JSON_PROPERTY_NOT_FOUND

0x18

A required JSON property is missing.

KY_RESULT_CODE_TARGET_IS_READONLY

0x19

The target does not allow the requested write operation.

KY_RESULT_CODE_STREAM_NOT_READY_FOR_START

0x1A

The stream is not in a state that permits acquisition to start.

KY_RESULT_CODE_DEVICE_NOT_CONNECTED

0x1B

The operation requires a device that is not connected.

KY_SEVERITY_FATAL

0x0

Fatal condition; the affected system or service cannot be used.

KY_SEVERITY_ALERT

0x1

Condition requiring immediate attention.

KY_SEVERITY_CRITICAL

0x2

Critical condition affecting operation.

KY_SEVERITY_ERROR

0x3

Error condition.

KY_SEVERITY_WARNING

0x4

Warning condition requiring attention.

KY_SEVERITY_NOTICE

0x5

Normal but significant condition.

KY_SEVERITY_INFO

0x6

Informational diagnostic.

KY_SEVERITY_DEBUG

0x7

Diagnostic detail intended for troubleshooting.

Type definitions

KY_RESULT

```
typedef struct _KY_RESULT KY_RESULT
```

Encoded SDK operation result.

See: KYFoundation_KY_RESULT

KY_RESULT

Encoded SDK operation result.

See: KYFoundation_KY_RESULT

uResult

```
uint32_t _KY_RESULT::uResult
```

Encoded result value; inspect through the Foundation result helpers.

KYFoundation Errors Structs

KY_RESULT

Result value returned by SDK operations. The declaration uses one-byte packing; preserve the field type and layout when declaring the type in another language.

Structure Field	Type	Description	Remarks
uResult	uint32_t	Encoded SDK result.	Preserve the complete returned value. Use KY_RESULT_SUCCEEDED(), KY_RESULT_FAILED(), and the KYFoundation accessors to inspect it.

KYFoundation_Types API

KYFoundation_Types.h API reference

Generated from PUBLIC/include/KYFoundation_Types.h.

API documentation generated from PUBLIC/include/KYFoundation_Types.h.

The public aliases describe native module, procedure, thread, and process identifiers. Their representation depends on the target platform. KY_BOOL has a fixed eight-bit representation; use KY_TRUE and KY_FALSE for its values. See Foundation Types for type mappings and formatting requirements.

Macros

__STDC_LIMIT_MACROS

Request integer limit macros from compatible C++ standard-library headers.

Defined on non-Windows targets when not already provided.

KY_MEMBER_INIT

Preserve a member-initializer expression in C++; omit it in C.

Parameter	Description
exp	Complete initializer tokens, including the assignment operator when needed.

KY_FUNC

__FUNCTION__

Compiler-provided name of the current function for diagnostics.

Use within a function body.

KY_TRUE

1
SDK Boolean true value, equal to one.

KY_FALSE

0
SDK Boolean false value, equal to zero.

INT64_MAX

0x7fffffffffffffffLL /* maximum signed int64 value */
Fallback maximum value for a signed 64-bit integer.

Used only when the standard integer-limit macro is unavailable.

INT64_MIN

0x8000000000000000LL /* minimum signed int64 value */
Fallback constant representing the minimum signed 64-bit value.

Used only when the standard integer-limit macro is unavailable.

`__PRI64_PREFIX`

"l1"

`__PRIPTR_PREFIX`

`PRId8`

"d"

printf signed decimal fragment for int8_t.

`PRId16`

"d"

printf signed decimal fragment for int16_t.

`PRId32`

"d"

printf signed decimal fragment for int32_t.

`PRId64`

`__PRI64_PREFIX`

"d"

printf signed decimal fragment for int64_t.

`PRIdLEAST8`

"d"

printf signed decimal fragment for int_least8_t.

`PRIdLEAST16`

"d"

printf signed decimal fragment for int_least16_t.

`PRIdLEAST32`

"d"

printf signed decimal fragment for int_least32_t.

`PRIdLEAST64`

`__PRI64_PREFIX`

"d"

printf signed decimal fragment for int_least64_t.

`PRIdFAST8`

"d"

printf signed decimal fragment for int_fast8_t.

`PRIdFAST16`

`__PRIPTR_PREFIX`

"d"

printf signed decimal fragment for int_fast16_t.

`PRIdFAST32`

`__PRIPTR_PREFIX`

"d"

printf signed decimal fragment for int_fast32_t.

PRI64FAST64

__PRI64_PREFIX "d"
printf signed decimal fragment for int_fast64_t.

PRIi8

"i"
printf signed decimal fragment for int8_t.

PRIi16

"i"
printf signed decimal fragment for int16_t.

PRIi32

"i"
printf signed decimal fragment for int32_t.

PRIi64

__PRI64_PREFIX "i"
printf signed decimal fragment for int64_t.

PRIiLEAST8

"i"
printf signed decimal fragment for int_least8_t.

PRIiLEAST16

"i"
printf signed decimal fragment for int_least16_t.

PRIiLEAST32

"i"
printf signed decimal fragment for int_least32_t.

PRIiLEAST64

__PRI64_PREFIX "i"
printf signed decimal fragment for int_least64_t.

PRIiFAST8

"i"
printf signed decimal fragment for int_fast8_t.

PRIiFAST16

__PRIPTR_PREFIX "i"
printf signed decimal fragment for int_fast16_t.

PRIiFAST32

__PRIPTR_PREFIX "i"
printf signed decimal fragment for int_fast32_t.

PRIiFAST64

__PRI64_PREFIX "i"
printf signed decimal fragment for int_fast64_t.

PRio8

"o"
printf octal fragment for uint8_t.

PRio16

"o"
printf octal fragment for uint16_t.

PRio32

"o"
printf octal fragment for uint32_t.

PRio64

__PRI64_PREFIX "o"
printf octal fragment for uint64_t.

PRioLEAST8

"o"
printf octal fragment for uint_least8_t.

PRioLEAST16

"o"
printf octal fragment for uint_least16_t.

PRioLEAST32

"o"
printf octal fragment for uint_least32_t.

PRioLEAST64

__PRI64_PREFIX "o"
printf octal fragment for uint_least64_t.

PRioFAST8

"o"
printf octal fragment for uint_fast8_t.

PRioFAST16

__PRIPTR_PREFIX "o"
printf octal fragment for uint_fast16_t.

PRioFAST32

__PRIPTR_PREFIX "o"
printf octal fragment for uint_fast32_t.

PRIoFAST64

__PRI64_PREFIX "o"
printf octal fragment for uint_fast64_t.

PRlu8

"u"
printf unsigned decimal fragment for uint8_t.

PRlu16

"u"
printf unsigned decimal fragment for uint16_t.

PRlu32

"u"
printf unsigned decimal fragment for uint32_t.

PRlu64

__PRI64_PREFIX "u"
printf unsigned decimal fragment for uint64_t.

PRluLEAST8

"u"
printf unsigned decimal fragment for uint_least8_t.

PRluLEAST16

"u"
printf unsigned decimal fragment for uint_least16_t.

PRluLEAST32

"u"
printf unsigned decimal fragment for uint_least32_t.

PRluLEAST64

__PRI64_PREFIX "u"
printf unsigned decimal fragment for uint_least64_t.

PRluFAST8

"u"
printf unsigned decimal fragment for uint_fast8_t.

PRluFAST16

__PRIPTR_PREFIX "u"
printf unsigned decimal fragment for uint_fast16_t.

PRluFAST32

__PRIPTR_PREFIX "u"
printf unsigned decimal fragment for uint_fast32_t.

PRIuFAST64

__PRI64_PREFIX "u"
printf unsigned decimal fragment for uint_fast64_t.

PRIx8

"x"
printf lowercase hexadecimal fragment for uint8_t.

PRIx16

"x"
printf lowercase hexadecimal fragment for uint16_t.

PRIx32

"x"
printf lowercase hexadecimal fragment for uint32_t.

PRIx64

__PRI64_PREFIX "x"
printf lowercase hexadecimal fragment for uint64_t.

PRIxLEAST8

"x"
printf lowercase hexadecimal fragment for uint_least8_t.

PRIxLEAST16

"x"
printf lowercase hexadecimal fragment for uint_least16_t.

PRIxLEAST32

"x"
printf lowercase hexadecimal fragment for uint_least32_t.

PRIxLEAST64

__PRI64_PREFIX "x"
printf lowercase hexadecimal fragment for uint_least64_t.

PRIxFAST8

"x"
printf lowercase hexadecimal fragment for uint_fast8_t.

PRIxFAST16

__PRIPTR_PREFIX "x"
printf lowercase hexadecimal fragment for uint_fast16_t.

PRIxFAST32

__PRIPTR_PREFIX "x"
printf lowercase hexadecimal fragment for uint_fast32_t.

PRIxFAST64

__PRI64_PREFIX "x"
printf lowercase hexadecimal fragment for uint_fast64_t.

PRIX8

"x"
printf uppercase hexadecimal fragment for uint8_t.

PRIX16

"x"
printf uppercase hexadecimal fragment for uint16_t.

PRIX32

"x"
printf uppercase hexadecimal fragment for uint32_t.

PRIX64

__PRI64_PREFIX "x"
printf uppercase hexadecimal fragment for uint64_t.

PRIXLEAST8

"x"
printf uppercase hexadecimal fragment for uint_least8_t.

PRIXLEAST16

"x"
printf uppercase hexadecimal fragment for uint_least16_t.

PRIXLEAST32

"x"
printf uppercase hexadecimal fragment for uint_least32_t.

PRIXLEAST64

__PRI64_PREFIX "x"
printf uppercase hexadecimal fragment for uint_least64_t.

PRIXFAST8

"x"
printf uppercase hexadecimal fragment for uint_fast8_t.

PRIXFAST16

__PRIPTR_PREFIX "x"
printf uppercase hexadecimal fragment for uint_fast16_t.

PRIXFAST32

__PRIPTR_PREFIX "x"
printf uppercase hexadecimal fragment for uint_fast32_t.

PRIxFAST64

__PRI64_PREFIX "x"
printf uppercase hexadecimal fragment for uint_fast64_t.

PRIdMAX

__PRI64_PREFIX "d"
printf signed decimal fragment for intmax_t.

PRiMAX

__PRI64_PREFIX "i"
printf signed decimal fragment for intmax_t.

PRIoMAX

__PRI64_PREFIX "o"
printf octal fragment for uintmax_t.

PRluMAX

__PRI64_PREFIX "u"
printf unsigned decimal fragment for uintmax_t.

PRIxMAX

__PRI64_PREFIX "x"
printf lowercase hexadecimal fragment for uintmax_t.

PRIxMAX

__PRI64_PREFIX "X"
printf uppercase hexadecimal fragment for uintmax_t.

PRIdPTR

__PRIPTR_PREFIX "d"
printf signed decimal fragment for intptr_t.

PRiPTR

__PRIPTR_PREFIX "i"
printf signed decimal fragment for intptr_t.

PRIoPTR

__PRIPTR_PREFIX "o"
printf octal fragment for uintptr_t.

PRluPTR

__PRIPTR_PREFIX "u"
printf unsigned decimal fragment for uintptr_t.

PRIxPTR

__PRIPTR_PREFIX "x"
printf lowercase hexadecimal fragment for uintptr_t.

PRIXPTR

`__PRIPTR_PREFIX` "x"
printf uppercase hexadecimal fragment for uintptr_t.

SCNd8

"hhd"
scanf signed decimal fragment for a pointer to int8_t.

SCNd16

"hd"
scanf signed decimal fragment for a pointer to int16_t.

SCNd32

"d"
scanf signed decimal fragment for a pointer to int32_t.

SCNd64

`__PRI64_PREFIX` "d"
scanf signed decimal fragment for a pointer to int64_t.

SCNdLEAST8

"hhd"
scanf signed decimal fragment for a pointer to int_least8_t.

SCNdLEAST16

"hd"
scanf signed decimal fragment for a pointer to int_least16_t.

SCNdLEAST32

"d"
scanf signed decimal fragment for a pointer to int_least32_t.

SCNdLEAST64

`__PRI64_PREFIX` "d"
scanf signed decimal fragment for a pointer to int_least64_t.

SCNdFAST8

"hhd"
scanf signed decimal fragment for a pointer to int_fast8_t.

SCNdFAST16

`__PRIPTR_PREFIX` "d"
scanf signed decimal fragment for a pointer to int_fast16_t.

SCNdFAST32

`__PRIPTR_PREFIX` "d"
scanf signed decimal fragment for a pointer to int_fast32_t.

SCNdFAST64

`__PRI64_PREFIX` "d"
scanf signed decimal fragment for a pointer to `int_fast64_t`.

SCNi8

"hhi"
scanf base-detected signed integer fragment for a pointer to `int8_t`.

SCNi16

"hi"
scanf base-detected signed integer fragment for a pointer to `int16_t`.

SCNi32

"i"
scanf base-detected signed integer fragment for a pointer to `int32_t`.

SCNi64

`__PRI64_PREFIX` "i"
scanf base-detected signed integer fragment for a pointer to `int64_t`.

SCNiLEAST8

"hhi"
scanf base-detected signed integer fragment for a pointer to `int_least8_t`.

SCNiLEAST16

"hi"
scanf base-detected signed integer fragment for a pointer to `int_least16_t`.

SCNiLEAST32

"i"
scanf base-detected signed integer fragment for a pointer to `int_least32_t`.

SCNiLEAST64

`__PRI64_PREFIX` "i"
scanf base-detected signed integer fragment for a pointer to `int_least64_t`.

SCNiFAST8

"hhi"
scanf base-detected signed integer fragment for a pointer to `int_fast8_t`.

SCNiFAST16

`__PRIPTR_PREFIX` "i"
scanf base-detected signed integer fragment for a pointer to `int_fast16_t`.

SCNiFAST32

`__PRIPTR_PREFIX` "i"
scanf base-detected signed integer fragment for a pointer to `int_fast32_t`.

SCNiFAST64

`__PRI64_PREFIX` "i"
scanf base-detected signed integer fragment for a pointer to `int_fast64_t`.

SCNu8

"hhu"
scanf unsigned decimal fragment for a pointer to `uint8_t`.

SCNu16

"hu"
scanf unsigned decimal fragment for a pointer to `uint16_t`.

SCNu32

"u"
scanf unsigned decimal fragment for a pointer to `uint32_t`.

SCNu64

`__PRI64_PREFIX` "u"
scanf unsigned decimal fragment for a pointer to `uint64_t`.

SCNuLEAST8

"hhu"
scanf unsigned decimal fragment for a pointer to `uint_least8_t`.

SCNuLEAST16

"hu"
scanf unsigned decimal fragment for a pointer to `uint_least16_t`.

SCNuLEAST32

"u"
scanf unsigned decimal fragment for a pointer to `uint_least32_t`.

SCNuLEAST64

`__PRI64_PREFIX` "u"
scanf unsigned decimal fragment for a pointer to `uint_least64_t`.

SCNuFAST8

"hhu"
scanf unsigned decimal fragment for a pointer to `uint_fast8_t`.

SCNuFAST16

`__PRIPTR_PREFIX` "u"
scanf unsigned decimal fragment for a pointer to `uint_fast16_t`.

SCNuFAST32

`__PRIPTR_PREFIX` "u"
scanf unsigned decimal fragment for a pointer to `uint_fast32_t`.

SCNuFAST64

`__PRI64_PREFIX` "u"
scanf unsigned decimal fragment for a pointer to `uint_fast64_t`.

SCNo8

"hho"
scanf octal fragment for a pointer to `uint8_t`.

SCNo16

"ho"
scanf octal fragment for a pointer to `uint16_t`.

SCNo32

"o"
scanf octal fragment for a pointer to `uint32_t`.

SCNo64

`__PRI64_PREFIX` "o"
scanf octal fragment for a pointer to `uint64_t`.

SCNoLEAST8

"hho"
scanf octal fragment for a pointer to `uint_least8_t`.

SCNoLEAST16

"ho"
scanf octal fragment for a pointer to `uint_least16_t`.

SCNoLEAST32

"o"
scanf octal fragment for a pointer to `uint_least32_t`.

SCNoLEAST64

`__PRI64_PREFIX` "o"
scanf octal fragment for a pointer to `uint_least64_t`.

SCNoFAST8

"hho"
scanf octal fragment for a pointer to `uint_fast8_t`.

SCNoFAST16

`__PRIPTR_PREFIX` "o"
scanf octal fragment for a pointer to `uint_fast16_t`.

SCNoFAST32

`__PRIPTR_PREFIX` "o"
scanf octal fragment for a pointer to `uint_fast32_t`.

SCNoFAST64

`__PRI64_PREFIX` "o"
scanf octal fragment for a pointer to `uint_fast64_t`.

SCNx8

"hhx"
scanf lowercase hexadecimal fragment for a pointer to `uint8_t`.

SCNx16

"hx"
scanf lowercase hexadecimal fragment for a pointer to `uint16_t`.

SCNx32

"x"
scanf lowercase hexadecimal fragment for a pointer to `uint32_t`.

SCNx64

`__PRI64_PREFIX` "x"
scanf lowercase hexadecimal fragment for a pointer to `uint64_t`.

SCNxLEAST8

"hhx"
scanf lowercase hexadecimal fragment for a pointer to `uint_least8_t`.

SCNxLEAST16

"hx"
scanf lowercase hexadecimal fragment for a pointer to `uint_least16_t`.

SCNxLEAST32

"x"
scanf lowercase hexadecimal fragment for a pointer to `uint_least32_t`.

SCNxLEAST64

`__PRI64_PREFIX` "x"
scanf lowercase hexadecimal fragment for a pointer to `uint_least64_t`.

SCNxFAST8

"hhx"
scanf lowercase hexadecimal fragment for a pointer to `uint_fast8_t`.

SCNxFAST16

`__PRIPTR_PREFIX` "x"
scanf lowercase hexadecimal fragment for a pointer to `uint_fast16_t`.

SCNxFAST32

`__PRIPTR_PREFIX` "x"
scanf lowercase hexadecimal fragment for a pointer to `uint_fast32_t`.

SCNxFAST64

__PRI64_PREFIX "x"
scanf lowercase hexadecimal fragment for a pointer to uint_fast64_t.

SCNdMAX

__PRI64_PREFIX "d"
scanf signed decimal fragment for a pointer to intmax_t.

SCNiMAX

__PRI64_PREFIX "i"
scanf base-detected signed integer fragment for a pointer to intmax_t.

SCNoMAX

__PRI64_PREFIX "o"
scanf octal fragment for a pointer to uintmax_t.

SCNuMAX

__PRI64_PREFIX "u"
scanf unsigned decimal fragment for a pointer to uintmax_t.

SCNxMAX

__PRI64_PREFIX "x"
scanf lowercase hexadecimal fragment for a pointer to uintmax_t.

SCNdPTR

__PRIPTR_PREFIX "d"
scanf signed decimal fragment for a pointer to intptr_t.

SCNiPTR

__PRIPTR_PREFIX "i"
scanf base-detected signed integer fragment for a pointer to intptr_t.

SCNoPTR

__PRIPTR_PREFIX "o"
scanf octal fragment for a pointer to uintptr_t.

SCNuPTR

__PRIPTR_PREFIX "u"
scanf unsigned decimal fragment for a pointer to uintptr_t.

SCNxPTR

__PRIPTR_PREFIX "x"
scanf lowercase hexadecimal fragment for a pointer to uintptr_t.

PRISIZET

"zu"
printf decimal format fragment for size_t.

PRIKY_THREADID_X

PRIx64

printf uppercase hexadecimal fragment for a numeric thread identifier.

The argument must match the target's PRIx32 (Windows) or PRIx64 (POSIX) type.

PRIKY_THREADID_U

PRId64

printf decimal fragment for a numeric thread identifier.

The argument must match the target's PRId32 (Windows) or PRId64 (POSIX) type.

PRIKY_PROCID

"d"

printf signed decimal fragment for a process identifier.

The argument must match the int conversion used by this fragment.

Type definitions

KY_HMODULE

typedef void* KY_HMODULE

Native loaded-module handle; ownership follows the returning API.

KY_PROCADDR

typedef void* KY_PROCADDR

Native procedure address; use the referenced function's declared signature.

KY_THREADID

typedef pthread_t KY_THREADID

Native thread identifier with a platform-dependent representation.

KY_PROCID

typedef pid_t KY_PROCID

Native process identifier with a platform-dependent representation.

KY_BOOL

typedef uint8_t KY_BOOL

Eight-bit SDK Boolean; use KY_TRUE or KY_FALSE.

Foundation Types

Platform type aliases, Boolean values, and integer formatting.

Native platform types

Type	Windows	POSIX	Description
KY_HMODULE	HMODULE	void*	Native loaded-module handle.
KY_PROCADDR	FARPROC	void*	Native procedure address. Use the

			declaration and calling convention of the referenced function.
KY_THREADID	DWORD	pthread_t	Native thread identifier; representation is platform-dependent.
KY_PROCID	int	pid_t	Native process identifier.
KY_BOOL	uint8_t	uint8_t	Eight-bit SDK Boolean value.

These aliases do not create resources or transfer ownership. Follow the lifetime and release requirements of the API that returns each value.

KY_BOOL

Defines	Type	Value	Description
KY_TRUE	KY_BOOL in C++; integer macro in C	0x1	Boolean true.
KY_FALSE	KY_BOOL in C++; integer macro in C	0x0	Boolean false.

Integer format strings

Use the PRI* and SCN* format fragments with their matching fixed-width, least-width, fast-width, maximum-width, or pointer-width integer types. Prefix the fragment with a percent sign in a string literal. For scanning, pass a pointer to the exact destination type; printf and scanf fragments are not interchangeable.

Format family	Purpose	Matching types
PRId*, PRIi*	Signed decimal output.	intN_t, int_leastN_t, int_fastN_t, intmax_t, intptr_t
PRId*, PRIo*, PRIx*, PRIx*	Unsigned decimal, octal, or hexadecimal output.	uintN_t, uint_leastN_t, uint_fastN_t, uintmax_t, uintptr_t
SCNd*, SCNi*	Signed input; SCNi* accepts decimal, octal, or hexadecimal prefixes.	Corresponding signed integer pointer.
SCNu*, SCNo*, SCNx*	Unsigned decimal, octal, or hexadecimal input.	Corresponding unsigned integer pointer.
PRISIZET	Decimal output for size_t.	size_t
PRIKY_THREADID_X, PRIKY_THREADID_U	Hexadecimal or decimal output for numeric thread identifiers on supported targets.	KY_THREADID with the target's matching format argument type.
PRIKY_PROCID	Decimal output for a process identifier.	KY_PROCID

Par: For a uint32_t variable named frame_count, use printf("Frames: %" PRIu32 "\n", frame_count);. Include <stdio.h> when using the formatted I/O functions.

- Foundation types and Boolean values
- Integer format fragments
- Compiler compatibility definitions

KYGenTLExtension API

KYGenTLExtension.h API reference

Generated from PUBLIC/include/KYGenTLExtension.h.

API documentation generated from PUBLIC/include/KYGenTLExtension.h.

This header requires C++: command expressions and device handles use the GenTL namespace. Use the constants with the GenTL Producer API. Event records carry GenTL device handles, while pointer-valued information commands return borrowed SDK information structures. Numeric command values, member defaults, field order, and one-byte packing are part of the ABI. See KAYA GenTL Extension for query conventions and event field tables.

Macros

KYGENTLEXTENSION_GC_ERROR_INTERFACE_FW_VERSION_OUTDATED

(GenTL::GC_ERR_CUSTOM_ID - 1)

The interface firmware version is outdated for the SDK.

Custom GenTL::GC_ERROR; firmware maintenance is required for normal operation. An interface handle may still be returned by GenTL::TOpenInterface().

KYGENTLEXTENSION_GC_ERROR_INTERFACE_LIMITED_MODE

(GenTL::GC_ERR_CUSTOM_ID - 2)

The interface is operating with limited functionality.

Custom GenTL::GC_ERROR; query GenTL::GCGetLastError() for the limitation. An interface handle may still be returned by GenTL::TOpenInterface().

KYGENTLEXTENSION_TL_INFO_CMD_API_VER_MAJOR

(GenTL::TL_INFO_CUSTOM_ID + 1)

SDK software API major version.

Returned type: GenTL::INFO_DATATYPE_UINT32; destination: uint32_t. Use with GenTL::GCGetInfo() or GenTL::TLGetInfo().

KYGENTLEXTENSION_TL_INFO_CMD_API_VER_MINOR

(GenTL::TL_INFO_CUSTOM_ID + 2)

SDK software API minor version.

Returned type: GenTL::INFO_DATATYPE_UINT32; destination: uint32_t. Use with GenTL::GCGetInfo() or GenTL::TLGetInfo().

KYGENTLEXTENSION_TL_INFO_CMD_API_VER_PATCH

(GenTL::TL_INFO_CUSTOM_ID + 3)

SDK software API patch version.

Returned type: GenTL::INFO_DATATYPE_UINT32; destination: uint32_t. Use with GenTL::GCGetInfo() or GenTL::TLGetInfo().

KYGENTLEXTENSION_TL_INFO_CMD_API_VER_BUILD

(GenTL::TL_INFO_CUSTOM_ID + 4)

SDK software build revision.

Returned type: GenTL::INFO_DATATYPE_UINT32; destination: uint32_t. Use with GenTL::GCGetInfo() or GenTL::TLGetInfo().

KYGENTLEXTENSION_TL_INFO_CMD_API_VER_ALPHA

(GenTL::TL_INFO_CUSTOM_ID + 5)

Whether the SDK is a alpha build.

Returned type: GenTL::INFO_DATATYPE_BOOL8; destination: uint8_t. Zero means false; a nonzero byte means true. Use with GenTL::GCGetInfo() or GenTL::TLGetInfo().

KYGENTLEXTENSION_TL_INFO_CMD_API_VER_BETA

(GenTL::TL_INFO_CUSTOM_ID + 6)

Whether the SDK is a beta build.

Returned type: GenTL::INFO_DATATYPE_BOOL8; destination: uint8_t. Zero means false; a nonzero byte means true. Use with GenTL::GCGetInfo() or GenTL::TLGetInfo().

KYGENTLEXTENSION_TL_INFO_CMD_API_VER_RC

(GenTL::TL_INFO_CUSTOM_ID + 7)

Whether the SDK is a release-candidate build.

Returned type: GenTL::INFO_DATATYPE_BOOL8; destination: uint8_t. Zero means false; a nonzero byte means true. Use with GenTL::GCGetInfo() or GenTL::TLGetInfo().

KYGENTLEXTENSION_INTERFACE_INFO_CMD_PCI_SLOT

(GenTL::INTERFACE_INFO_CUSTOM_ID + 1)

PCI bus, slot, and function of the interface.

Returned type: GenTL::INFO_DATATYPE_STRING; destination: a NUL-terminated character buffer. The current producer formats the value as {PCI bus:slot:function} with decimal numbers. Use with GenTL::TLGetInterfaceInfo() or GenTL::IFGetInfo().

KYGENTLEXTENSION_INTERFACE_INFO_CMD_KYVP_PCI_INTERFACE_INFO

(GenTL::INTERFACE_INFO_CUSTOM_ID + 2)

Borrow the SDK PCI interface information.

Returned type: GenTL::INFO_DATATYPE_PTR; pointed-to type: KYVP_PCI_INTERFACE_INFO. Use with GenTL::TLGetInterfaceInfo() or GenTL::IFGetInfo() and pass the address of a pointer. The producer owns the record; do not modify or free it. Refresh the pointer after interface-list changes, and discard it when the owning interface is removed or the producer is shut down.

KYGENTLEXTENSION_DEVICE_INFO_CMD_KYVP_DEVICE_INFO

(GenTL::DEVICE_INFO_CUSTOM_ID + 1)

Borrow the SDK device information.

Returned type: GenTL::INFO_DATATYPE_PTR; pointed-to type: KYVP_DEVICE_INFO. Use with GenTL::IFGetDeviceInfo() or GenTL::DevGetInfo() and pass the address of a pointer. The producer owns the record; do not modify or free it. Refresh the pointer after device-list changes, and discard it when the owning device or interface is removed or the producer closes.

KYGENTLEXTENSION_BUFFER_INFO_CMD_BUFFER_ID

(GenTL::BUFFER_INFO_CUSTOM_ID + 1)

SDK buffer identifier in the buffer queue.

Returned type: GenTL::INFO_DATATYPE_UINT64; destination: uint64_t. Use with GenTL::DSGetBufferInfo(). This identifies the buffer, not the acquired frame; query GenTL::BUFFER_INFO_FRAMEID for the frame identifier.

KYGENTLEXTENSION_BUFFER_INFO_CMD_INSTANT_FPS

(GenTL::BUFFER_INFO_CUSTOM_ID + 2)

Instantaneous acquisition rate recorded for the buffer.

Returned type: GenTL::INFO_DATATYPE_FLOAT64; destination: double, in frames per second. Use with GenTL::DSGetBufferInfo(). The value is measured metadata, not a configured rate.

KYGENTLEXTENSION_BUFFER_INFO_CMD_KYVP_BUFFER_INFO

(GenTL::BUFFER_INFO_CUSTOM_ID + 3)

Borrow the SDK buffer information and metadata pointers.

Returned type: GenTL::INFO_DATATYPE_PTR; pointed-to type: KYVP_BUFFER_INFO. Use with GenTL::DSGetBufferInfo() and pass the address of a pointer. The record and its referenced data remain producer-owned. Read frame data before requeuing; do not retain these pointers after buffer revocation or stream closure. Copying the record is a shallow copy.

KYGENTLEXTENSION_BUFFER_INFO_CMD_TIMESTAMP_NS_CHRONO

(GenTL::BUFFER_INFO_CUSTOM_ID + 4)

Host steady-clock timestamp recorded while processing the buffer interrupt.

Returned type: GenTL::INFO_DATATYPE_UINT64; destination: uint64_t, in nanoseconds. Use with GenTL::DSGetBufferInfo(). This is a host std::chrono::steady_clock value, not calendar time, camera exposure time, or the hardware timestamp clock.

KYGENTLEXTENSION_BUFFER_INFO_CMD_TIMESTAMP_NS_HW

((GenTL::BUFFER_INFO_CUSTOM_ID * 10) + 1)

Hardware-clock timestamp sampled while processing the buffer interrupt.

Returned type: GenTL::INFO_DATATYPE_UINT64; destination: uint64_t, in nanoseconds. Use with GenTL::DSGetBufferInfo(). Collection depends on the producer's hardware-timestamp diagnostic setting; a zero/default value need not represent a captured timestamp. The command uses the SDK debug range and a clock distinct from the host steady clock.

KYGENTLEXTENSION_EVENT_TYPE_PCIINTERFACE_EVENT

(GenTL::EVENT_CUSTOM_ID + 1)

Register PCI interface events on a GenTL interface handle.

Event source: GenTL::IF_HANDLE. GenTL::EventGetData() fills KYGENTLEXTENSION_PCIINTERFACE_GENERIC_EVENT. Temperature, heartbeat, and CoaXPress event notifications share this queue and are distinguished by the record's eEventId.

KYGENTLEXTENSION_EVENT_TYPE_DEVICE_EVENT

(GenTL::EVENT_CUSTOM_ID + 2)

Register device events on a GenTL local-device handle.

Event source: GenTL::DEV_HANDLE. GenTL::EventGetData() fills KYGENTLEXTENSION_DEVICE_GENERIC_EVENT. The public payload currently describes connection loss; its handle identifies the affected device and does not imply connectivity.

KYGENTLEXTENSION_PCIINTERFACE_EVENT_TEMPERATURE_VERSION

1

Current layout version of KYGENTLEXTENSION_PCIINTERFACE_EVENT_TEMPERATURE.

KYGENTLEXTENSION_PCIINTERFACE_EVENT_CXP2_HEARTBEAT_VERSION

1

Current layout version of KYGENTLEXTENSION_PCIINTERFACE_EVENT_CXP2_HEARTBEAT.

KYGENTLEXTENSION_PCIINTERFACE_EVENT_CXP2_EVENT_VERSION

1

Current layout version of KYGENTLEXTENSION_PCIINTERFACE_EVENT_CXP2_EVENT.

KYGENTLEXTENSION_PCIINTERFACE_GENERIC_EVENT_VERSION

1

Current layout version of KYGENTLEXTENSION_PCIINTERFACE_GENERIC_EVENT.

KYGENTLEXTENSION_DEVICE_EVENT_CONNECTION_LOST_VERSION

1

Current layout version of KYGENTLEXTENSION_DEVICE_EVENT_CONNECTION_LOST.

KYGENTLEXTENSION_DEVICE_GENERIC_EVENT_VERSION

1

Current layout version of KYGENTLEXTENSION_DEVICE_GENERIC_EVENT.

KAYA GenTL Extension

Custom commands and event payloads provided by the KAYA GenTL Producer.

- Custom interface status codes
- Producer version information
- Interface information
- Device information
- Buffer information
- Custom event registration types
- Event layout versions
- PCI interface events
- Device events

Information queries

Supply a writable GenTL::INFO_DATATYPE variable initialized to zero, a destination, and its capacity in bytes to the appropriate GenTL information function. On success, check the returned type before interpreting the value. A NULL destination queries the required size; string sizes include the NUL terminator. Scalar commands copy values into caller-owned storage.

For GenTL::INFO_DATATYPE_PTR, provide the address of a pointer variable and set the capacity to sizeof(pointer). The returned bytes hold a pointer, not a copy of the pointed-to structure. Treat borrowed

information as read-only and never free it. Copy data that must outlive the corresponding module or buffer; copying a structure containing pointers does not copy their data.

Receiving custom events

Register the appropriate custom event type with `GenTL::GCRegisterEvent()`, using an open interface or device handle. Receive into the matching generic event structure with `GenTL::EventGetData()`. Unregister with `GenTL::GCUnregisterEvent()` before closing its source.

Allocate the entire generic event record and set `piSize` to its `sizeof` before each receive. The current custom-event adapter does not validate the supplied capacity or update it after delivery; its NULL-buffer size query reports the corresponding native SDK event size. Use the GenTL extension record's `sizeof` directly. Treat the destination as valid only on success. Check the outer version, select the union member using `eEventId`, and check its payload version. Other union members and unused payload bytes have no meaning for that event. Probe event IDs are internal tests with no public payload; ignore unsupported IDs and versions.

The producer copies event fields into caller-owned storage. Device handles within those fields are borrowed identifiers and do not extend device lifetime or establish a new connection. All event records are packed to one-byte alignment. Records containing GenTL handles refer to objects in the current process and cannot be used as portable serialized or wire-format data.

PCI interface events

Interface notifications use `KYVP_PCIINTERFACE_EVENT_ID` to select their payload. Any embedded device handle is a `GenTL::DEV_HANDLE`.

KYGENTLEXTENSION_PCIINTERFACE_EVENT_TEMPERATURE

Temperature-threshold notification carried in `data.eventTemperature` when `eEventId` is `KYVP_PCIINTERFACE_EVENT_TEMPERATURE_ID`. The state describes the interface temperature threshold, not a temperature measurement in degrees.

Current layout version: `KYGENTLEXTENSION_PCIINTERFACE_EVENT_TEMPERATURE_VERSION (1)`.

Structure Field	Type	Description	Remarks
<code>version</code>	<code>uint32_t</code>	Event record layout version.	Produce r output; C++ default: 1.
<code>eTemperatureThreshold</code>	<code>KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_ID</code>	Current temperature -threshold state.	Produce r output; C++ default: invalid.

KYGENTLEXTENSION_PCIINTERFACE_EVENT_CXP2_HEARTBEAT

CoaXPress heartbeat notification carried in `data.eventCXP2Heartbeat` when `eEventId` is `KYVP_PCIINTERFACE_EVENT_CXP2_HEARTBEAT_ID`. The heartbeat is embedded by value.

Current layout version: `KYGENTLEXTENSION_PCIINTERFACE_EVENT_CXP2_HEARTBEAT_VERSION (1)`.

Structure Field	Type	Description	Remarks
version	uint32_t	Event record layout version.	Producer output; C++ default: 1.
deviceHandle	GenTL::DEV_HANDLE	GenTL local-device handle associated with the heartbeat.	Borrowed handle; no explicit member default.
heartBeat	KYVP_CXP2_HEARTBEAT	Received heartbeat data, including device time.	Embedded record; device time is in nanoseconds. Check its own version.

KYGENTLEXTENSION_PCIINTERFACE_EVENT_CXP2_EVENT

CoaXPress event notification carried in data.eventCXP2Event when eEventId is KYVP_PCIINTERFACE_EVENT_CXP2_EVENT_ID. The packet data is embedded by value.

Current layout version: KYGENTLEXTENSION_PCIINTERFACE_EVENT_CXP2_EVENT_VERSION (1).

Structure Field	Type	Description	Remarks
version	uint32_t	Event record layout version.	Producer output; C++ default: 1.
deviceHandle	GenTL::DEV_HANDLE	GenTL local-device handle associated with the event packet.	Borrowed handle; no explicit member default.
cxp2Event	KYVP_CXP2_EVENT	Received event packet metadata and data words.	Check its own version; uDataSize counts 32-bit words and must not exceed the array capacity.

KYGENTLEXTENSION_PCIINTERFACE_GENERIC_EVENT

Complete destination record for the KYGENTLEXTENSION_EVENT_TYPE_PCIINTERFACE_EVENT queue. Receive this generic record, then read only the union member selected by eEventId. See Receiving custom events for capacity and version checks.

Current layout version: KYGENTLEXTENSION_PCIINTERFACE_GENERIC_EVENT_VERSION (1).

Structure Field	Type	Description	Remarks
version	uint32_t	Event record layout version.	Producer output; C++ default: 1.
eEventId	KYVP_PCIINTERFACE_EVENT_ID	Identifier selecting the active event payload.	Producer output; C++ default: invalid.
data	union	Storage shared by the interface event payloads.	Read only the selected member listed below.

data union members

Structure Field	Type	Description	Remarks
eventTemperature	KYGENTLEXTENSION_PCIINTERFACE_EVENT_TEMPERATURE	Temperature state.	Event ID 0x02.
eventCXP2Heartbeat	KYGENTLEXTENSION_PCIINTERFACE_EVENT_CXP2_HEARTBEAT	Heartbeat data.	Event ID 0x03.
eventCXP2Event	KYGENTLEXTENSION_PCIINTERFACE_EVENT_CXP2_EVENT	Event packet.	Event ID 0x04.

Device events

Device notifications use KYVP_DEVICE_EVENT_ID to select their payload. The affected device is identified by a borrowed GenTL::DEV_HANDLE.

KYGENTLEXTENSION_DEVICE_EVENT_CONNECTION_LOST

Connection-loss notification carried in data.eventDeviceConnectionLost when eEventId is KYVP_DEVICE_EVENT_CONNECTION_LOST_ID. Receipt does not close the GenTL device handle automatically or establish a replacement connection.

Current layout version: KYGENTLEXTENSION_DEVICE_EVENT_CONNECTION_LOST_VERSION (1).

Structure Field	Type	Description	Remarks
version	uint32_t	Event record layout version.	Producer output; C++ default: 1.
hDevHandle	GenTL::DEV_HANDLE	GenTL local-device handle whose connection was lost.	Borrowed identifier; does not imply that device communication is still available.

KYGENTLEXTENSION_DEVICE_GENERIC_EVENT

Complete destination record for the KYGENTLEXTENSION_EVENT_TYPE_DEVICE_EVENT queue. Receive this generic record, then select its payload using eEventId. See Receiving custom events for capacity and version checks.

Current layout version: KYGENTLEXTENSION_DEVICE_GENERIC_EVENT_VERSION (1).

Structure Field	Type	Description	Remarks
version	uint32_t	Event record layout version.	Producer output; C++ default: 1.
eEventId	KYVP_DEVICE_EVENT_ID	Identifier selecting the active device event payload.	Producer output; C++ default: invalid.
data	union	Storage for the device event payload.	Read only the selected member listed below.

data union members

Structure Field	Type	Description	Remarks
eventDeviceConnectionLost	KYGENTL_EXTENSION_DEVICE_EVENT_CONNECTION_LOST	Connection lost.	Event ID 0x02.

KYIMGPROC_PixelFormatDecoder_Defines API

KYIMGPROC_PixelFormatDecoder_Defines.h API reference

Generated from PUBLIC/include/KYIMGPROC_PixelFormatDecoder_Defines.h.

API documentation generated from PUBLIC/include/KYIMGPROC_PixelFormatDecoder_Defines.h.

The CoaXPress fields describe transport-format components; PFNC values describe named pixel formats. The SDK also retains legacy aliases and sentinels. Use the lookup API to populate application-owned decoder records. Standard encodings were checked against J11A CXP-001-2013 (CoaXPress 1.1), section 9.4.1, tables 24 and 26-34. SDK-specific names are identified separately. See the EMVA pixel-format names and values for PFNC identifiers.

Macros

KYIMGPROC_KYVP_PIXEL_FORMAT_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_PIXEL_DECODER_STRUCT_VERSION

1

Declared structure layout version 1.

Enumerations

_kyvp_video_data_width

Encoded width of one monochrome sample or one color component.

These values are encodings, not literal bit counts. Packing in host memory is described separately by the format and stream packed-data mode.

Values

Name	Description
KYVP_DATA_WIDTH_UNKNOWN	Unspecified sample width; SDK sentinel.
KYVP_DATA_WIDTH_8BIT	8-bit samples; encoded width value 1.
KYVP_DATA_WIDTH_10BIT	10-bit samples; encoded width value 2.
KYVP_DATA_WIDTH_12BIT	12-bit samples; encoded width value 3.
KYVP_DATA_WIDTH_14BIT	14-bit samples; encoded width value 4.
KYVP_DATA_WIDTH_16BIT	16-bit samples; encoded width value 5.

_kyvp_video_data_subtype

Subtype interpreted together with the pixel data type.

Values intentionally overlap across planar, Bayer, YUV, and YCbCr families. Always inspect data_type before interpreting data_subtype. SDK legacy names and invalid sentinels are not additional CoaXPress wire-format definitions.

Values

Name	Description
KYVP_DATA_SUBTYPE_NONE	No subtype; used by formats such as Mono, RGB, and RGBA.
KYVP_DATA_SUBTYPE_PLANAR_RY	Plane 1: normally R for RGB or Y for YUV/YCbCr.
KYVP_DATA_SUBTYPE_PLANAR_GUCB	Plane 2: normally G for RGB, U for YUV, or Cb for YCbCr.
KYVP_DATA_SUBTYPE_PLANAR_BVCR	Plane 3: normally B for RGB, V for YUV, or Cr for YCbCr.
KYVP_DATA_SUBTYPE_PLANAR_RGB	Legacy SDK name for planar RGB; numeric code 0x04 denotes plane 4 in CoaXPress.
KYVP_DATA_SUBTYPE_BAYER_GR	Bayer GR pattern: first row G,R; second row B,G.
KYVP_DATA_SUBTYPE_BAYER_RG	Bayer RG pattern: first row R,G; second row G,B.
KYVP_DATA_SUBTYPE_BAYER_GB	Bayer GB pattern: first row G,B; second row R,G.
KYVP_DATA_SUBTYPE_BAYER_BG	Bayer BG pattern: first row B,G; second row G,R.
KYVP_DATA_SUBTYPE_YUV_411	YUV 4:1:1 with CoaXPress component order Y,Y,U,Y,Y,V.
KYVP_DATA_SUBTYPE_YUV_422	YUV 4:2:2 with CoaXPress component order Y,U,Y,V.
KYVP_DATA_SUBTYPE_YUV_444	YUV 4:4:4 with CoaXPress component order Y,U,V.
KYVP_DATA_SUBTYPE_YCBCR_411	YCbCr 4:1:1 with CoaXPress component order Y,Y,Cb,Y,Y,Cr.
KYVP_DATA_SUBTYPE_YCBCR_422	YCbCr 4:2:2 with CoaXPress component order Y,Cb,Y,Cr.
KYVP_DATA_SUBTYPE_YCBCR_444	YCbCr 4:4:4 with CoaXPress component order Y,Cb,Cr.
KYVP_DATA_SUBTYPE_INVALID	SDK invalid-subtype sentinel; CoaXPress also uses 0x0F for planar plane 15.

_kyvp_video_data_type

Pixel data families represented by the SDK CoaXPress format fields.

Values

Name	Description
KYVP_DATA_TYPE_RAW	Raw or application-specific data; the all-zero CoaXPress format code denotes raw data.
KYVP_DATA_TYPE_MONO	Monochrome luminance samples; no subtype.

KYVP_DATA_TYPE_PLANAR	Separate component planes; the subtype identifies a plane or an SDK planar convention.
KYVP_DATA_TYPE_BAYER	Bayer color-filter samples; the subtype selects the two-row mosaic pattern.
KYVP_DATA_TYPE_RGB	Interleaved red, green, and blue components in that order; no subtype.
KYVP_DATA_TYPE_RGBA	Interleaved red, green, blue, and alpha components in that order; no subtype.
KYVP_DATA_TYPE_YUV	YUV components; the subtype selects 4:1:1, 4:2:2, or 4:4:4 sampling.
KYVP_DATA_TYPE_YCBCR601	YCbCr using ITU-R BT.601 colorimetry; the subtype selects chroma sampling.
KYVP_DATA_TYPE_YCBCR709	YCbCr using ITU-R BT.709 colorimetry; the subtype selects chroma sampling.
KYVP_DATA_TYPE_INVALID	Invalid or unsupported SDK data-type sentinel.

Type definitions

KYVP_VIDEO_DATA_WIDTH

`typedef enum _kyvp_video_data_width KYVP_VIDEO_DATA_WIDTH`
 Encoded width of one monochrome sample or one color component.

These values are encodings, not literal bit counts. Packing in host memory is described separately by the format and stream packed-data mode.

KYVP_VIDEO_DATA_SUBTYPE

`typedef enum _kyvp_video_data_subtype KYVP_VIDEO_DATA_SUBTYPE`
 Subtype interpreted together with the pixel data type.

Values intentionally overlap across planar, Bayer, YUV, and YCbCr families. Always inspect `data_type` before interpreting `data_subtype`. SDK legacy names and invalid sentinels are not additional CoaXPress wire-format definitions.

KYVP_VIDEO_DATA_TYPE

`typedef enum _kyvp_video_data_type KYVP_VIDEO_DATA_TYPE`
 Pixel data families represented by the SDK CoaXPress format fields.

Pixel Format Decoder Structs

KYVP_VIDEO_PIXELIF

Decoded CoaXPress pixel-format fields.

In the numeric CoaXPress code, width occupies bits 3:0, subtype bits 7:4, and type bits 15:8. The all-zero code denotes raw or user-specific data. This C/C++ bit-field structure is an SDK representation. Its allocation and bit ordering depend on the compiler; do not serialize it by copying its object representation.

Structure Field	Type	Description	Remarks
-----------------	------	-------------	---------

data_width	KYVP_VIDEO_DATA_WIDTH : 4	Encoded sample or color-component width.	4-bit field; corresponds to numeric code bits 3:0.
data_subtype	KYVP_VIDEO_DATA_SUBTYPE : 4	Subtype within the selected data family.	4-bit field; corresponds to numeric code bits 7:4.
data_type	KYVP_VIDEO_DATA_TYPE : 8	Pixel data family.	8-bit field; corresponds to numeric code bits 15:8.

KYVP_PIXEL_FORMAT

Pixel-format metadata populated in application-provided storage.

Initialize version to KYIMGPROC_KYVP_PIXEL_FORMAT_STRUCT_VERSION.

The application owns this structure. Returned name pointers refer to library-owned strings; do not modify or free them. A successful lookup does not guarantee that every conversion or saving backend supports the format. uPFNCValue preserves the SDK lookup-table value, which may be a legacy CoaXPress alias. uCxpValue equal to zero may mean raw data or no corresponding CoaXPress mapping.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure layout version.	Input; set to 1.
pszPFNCName	const char*	PFNC-style name of the matched SDK table entry.	Output; library-owned, null-terminated string.
uPFNCValue	uint32_t	Numeric value of the matched SDK table entry.	Output; may be a legacy CoaXPress alias rather than a PFNC identifier.
pszCxpName	const char*	CoaXPress name or alias associated with the matched entry.	Output; library-owned string; may be empty.
uCxpValue	uint32_t	CoaXPress pixel-format code stored in a uint32_t.	Output; low 16 bits encode width, subtype, and type; zero is ambiguous.
blsPacked	KY_BOOL	Packed-data flag recorded in the SDK format table.	Output; does not establish the actual host-buffer layout or conversion support.
bRGBReordered	KY_BOOL	Whether the matched RGB-family entry uses reversed red/blue order.	Output; KY_TRUE identifies BGR-family ordering.
uChannelsPerPixel	uint8_t	Channel count recorded for the matched format.	Output; use with format-specific packing and subsampling rules.
pVPPixelIF	KYVP_VIDEO_PIXELIF	Decoded CoaXPress width, subtype, and data-family fields.	Output; embedded structure, not a pointer.

KYIMGPROC_PIXEL_DECODER

Lookup result and pointer to application-provided pixel-format metadata.

Initialize version to KYIMGPROC_PIXEL_DECODER_STRUCT_VERSION.

Initialize version and attach a writable KYVP_PIXEL_FORMAT through pVP_PIXEL_FORMAT. Pass this decoder through the argument block of KYImgProc_GetPixelFormat() or KYImgProc_GetPixelFormatByName(). On success, the decoder contains the matched table value and a library-owned name. Both records remain application-owned; their returned name strings must not be freed.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure layout version.	Input; set to 1.
uValue	uint32_t	Numeric value of the matched SDK format-table entry.	Output; query input comes from the lookup argument block.
pszPixelFormatName	const char*	Pixel-format name selected by lookup.	Output; library-owned string on success.
pVP_PIXEL_FORMAT	KYVP_PIXEL_FORMAT*	Application-provided storage for decoded pixel metadata.	Input/output; non-NULL; library fills fields without allocating the structure.

KYVPDefines API

KYVPDefines.h API reference

Generated from PUBLIC/include/KYVPDefines.h.

API documentation generated from PUBLIC/include/KYVPDefines.h.

Provides platform macros, event identifiers, CoaXPress link definitions, and structures for auxiliary data, PCI interfaces, devices, and buffers.

Structure packing, member order, enum values, and version constants form part of the SDK ABI. See KYVPDefines Structs for the structure field reference.

Macros

KYVP_CALLCONV

SDK calling convention; `__cdecl` on Microsoft compilers.

KYVP_MEMBER_DEFAULT

Expands to a member initializer in C++ and to nothing in C.

KYVP_EXTERNAL_C

C linkage in C++; empty in C.

KYVP_EXTERNAL

`extern`
External linkage declaration keyword.

KYVP_EXTERNAL_DLL

SDK DLL export or import attribute on Microsoft compilers.

KYVP_API

`KYVP_EXTERNAL_C KYVP_EXTERNAL_DLL`
Public SDK function linkage and visibility attributes.

KYVP_IO_CONTROLLER_MASKED_IO

`((masked_data) & 0xFFFFFFFF)`
Extracts I/O signal bits 0-39 from an auxiliary event mask.

KYVP_IO_CONTROLLER_MASKED_ENCODERS

`((masked_data) >> 40) & 0xF`
Extracts encoder bits 40-43 as bits 0-3.

KYVP_IO_CONTROLLER_MASKED_TIMERS

`((masked_data) >> 44) & 0xFF`
Extracts timer bits 44-51 as bits 0-7.

KYVP_IO_CONTROLLER_MASKED_CAMERA_TRIGGERS

`((masked_data) >> 52) & 0xFF)`

Extracts camera-trigger bits 52-59 as bits 0-7.

KYVP_IO_CONTROLLER_MASKED_TRIGGERS

`((masked_data) >> 60) & 0xF)`

Extracts acquisition-trigger bits 60-63 as bits 0-3.

KYVP_AUX_MESSAGE_ID_IO_CONTROLLER

11

Auxiliary message ID selecting the I/O controller payload.

KYVP_CXP2_HEARTBEAT_VERSION

1

Current version of KYVP_CXP2_HEARTBEAT.

KY_CXP_EVENT_MAX_DATA_SIZE

1024

Maximum CoaXPress 2.0 event data capacity, in bytes.

KYVP_CXP2_EVENT_VERSION

1

Current version of KYVP_CXP2_EVENT.

KYVP_PCI_INTERFACE_STREAM_UNKNOWN

0x0

No stream direction capability flags set.

KYVP_PCI_INTERFACE_STREAM_GRABBER

0x1

Bit flag for acquisition capability (input streams).

KYVP_PCI_INTERFACE_STREAM_GENERATOR

0x2

Bit flag for generation capability (output streams).

KYVP_PCI_CONFIG_VERSION

1

Version identifier associated with KYVP_PCI_CONFIG, which has no version field.

KYVP_MAX_BOARD_INFO_VERSION

1

Maximum declared version of KYVP_BOARD_INFO.

MAX_PCI_INTERFACE_LINKS

8

Maximum number of interface links represented in device topology arrays.

KYVP_PCI_INTERFACE_INFO_VERSION

1

Current version of KYVP_PCI_INTERFACE_INFO.

KY_MAX_PCI_INTERFACE_INFO_STRING_SIZE

256

Maximum interface information string length, excluding the terminator.

KY_MAX_DEVICE_INFO_STRING_SIZE

64

Maximum device information string length, excluding the terminator.

KYVP_DEVICE_INFO_VERSION

1

Declared version of KYVP_DEVICE_INFO.

KYVP_DEVICE_IMAGE_DETAILS_VERSION

1

Current version of KYVP_DEVICE_IMAGE_DETAILS.

KYVP_BUFFER_INFO_VERSION

1

Current version of KYVP_BUFFER_INFO.

Enumerations

_KYVP_PCIINTERFACE_EVENT_ID

PCI interface event identifiers.

Values

Name	Description
KYVP_PCIINTERFACE_EVENT_PROBE_ID	Reserved for internal event delivery tests.
KYVP_PCIINTERFACE_EVENT_TEMPERATURE_ID	The PCI interface temperature threshold state changed.
KYVP_PCIINTERFACE_EVENT_CXP2_HEARTBEAT_ID	A CoaXPress 2.0 heartbeat packet was received from a connected device.
KYVP_PCIINTERFACE_EVENT_CXP2_EVENT_ID	A CoaXPress 2.0 event packet was received from a connected device.
KYVP_PCIINTERFACE_EVENT_INVALID	Invalid or unsupported PCI interface event identifier.

_KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_ID

PCI interface temperature threshold states.

Values

Name	Description
KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_NORMAL	Normal temperature state, outside the warning and critical states.
KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_WARNING	Warning temperature state, as determined by the configured thresholds.
KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_CRITICAL	Critical temperature state, as determined by the configured thresholds.
KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_INVALID	Invalid or undefined temperature threshold state.

_KYVP_DEVICE_EVENT_ID

Device event identifiers.

Values

Name	Description
KYVP_DEVICE_EVENT_PROBE_ID	Reserved for internal event delivery tests.
KYVP_DEVICE_EVENT_CONNECTION_LOST_ID	The connection to the device was lost.
KYVP_DEVICE_EVENT_INVALID	Invalid or unsupported device event identifier.

_KYVP_TAP_GEOMETRY

Supported image sensor tap geometries.

Values

Name	Description
TAP_GEOMETRY_1X_1Y	One horizontal region and one vertical region, read from top to bottom.
TAP_GEOMETRY_1X_2YE	One horizontal region and two vertical regions, read from the top and bottom toward the center.
TAP_GEOMETRY_INVALID	Invalid or unsupported tap geometry.

_KYVP_CXP_LINK_SPEED

CoaXPress link speed encodings; rates are per connection.

Values

Name	Description
KYVP_CXP_LINK_SPEED_CXP0	No link speed selected or known; SDK sentinel, not a CoaXPress transmission rate.

KYVP_CXP_LINK_SPEED_CXP1	CXP-1: 1.25 Gbit/s per connection.
KYVP_CXP_LINK_SPEED_CXP2	CXP-2: 2.5 Gbit/s per connection.
KYVP_CXP_LINK_SPEED_CXP3	CXP-3: 3.125 Gbit/s per connection.
KYVP_CXP_LINK_SPEED_CXP5	CXP-5: 5 Gbit/s per connection.
KYVP_CXP_LINK_SPEED_CXP6	CXP-6: 6.25 Gbit/s per connection.
KYVP_CXP_LINK_SPEED_CXP10	CXP-10: 10 Gbit/s per connection.
KYVP_CXP_LINK_SPEED_CXP12	CXP-12: 12.5 Gbit/s per connection.

_KYVP_PCI_INTERFACE_PROTOCOL

Transport protocol reported by a PCI interface.

Values

Name	Description
KYVP_PCI_INTERFACE_PROTOCOL_CoaXPress	CoaXPress transport.
KYVP_PCI_INTERFACE_PROTOCOL_CLHS	Camera Link HS transport.
KYVP_PCI_INTERFACE_PROTOCOL_PCI	PCI transport.
KYVP_PCI_INTERFACE_PROTOCOL_Mixed	Mixed transport protocol designation.

Type definitions

KYVP_PCIINTERFACE_EVENT_ID

```
typedef enum _KYVP_PCIINTERFACE_EVENT_ID KYVP_PCIINTERFACE_EVENT_ID
```

PCI interface event identifiers.

KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_ID

```
typedef enum _KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_ID KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_ID
```

PCI interface temperature threshold states.

KYVP_DEVICE_EVENT_ID

```
typedef enum _KYVP_DEVICE_EVENT_ID KYVP_DEVICE_EVENT_ID
```

Device event identifiers.

KYVP_TAP_GEOMETRY

```
typedef enum _KYVP_TAP_GEOMETRY KYVP_TAP_GEOMETRY
```

Supported image sensor tap geometries.

KYVP_CXP_LINK_SPEED

```
typedef enum _KYVP_CXP_LINK_SPEED KYVP_CXP_LINK_SPEED
```

CoaXPress link speed encodings; rates are per connection.

KYVP_PCI_INTERFACE_PROTOCOL

```
typedef enum _KYVP_PCI_INTERFACE_PROTOCOL KYVP_PCI_INTERFACE_PROTOCOL
```

Transport protocol reported by a PCI interface.

KYVPDefines Structs

Field reference for auxiliary data, event payloads, and SDK information.

Each table follows declaration order and lists the exact public field types, including array bounds, bit widths, and named union members.

C++ member defaults initialize only the fields shown in the declarations. C callers must initialize input structures explicitly and set their version fields as documented. Library-owned information and callback payloads are populated by the SDK; treat returned information as read-only.

KYVP_IO_AUX_DATA

I/O controller payload carried in KYVP_AUX_DATA. The SDK populates version with 1. Timestamps are in nanoseconds.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Current version: 1.
masked_data	uint64_t	I/O controller event-state mask.	See the bit ranges below.
timestamp	uint64_t	I/O event timestamp.	Nanoseconds.

Within each range, the lowest bit corresponds to signal 0.

Bits	Signals
0-7	Optocoupled inputs 0-7
8-11	LVDS inputs 0-3
12-19	TTL lines 0-7
20-27	LVTTTL lines 0-7
28-35	Optocoupled outputs 0-7
36-39	LVDS outputs 0-3
40-43	Encoders 0-3
44-51	Timers 0-7
52-59	Camera triggers 0-7
60-63	Acquisition triggers 0-3

Use KYVP_IO_CONTROLLER_MASKED_IO, KYVP_IO_CONTROLLER_MASKED_ENCODERS, KYVP_IO_CONTROLLER_MASKED_TIMERS, KYVP_IO_CONTROLLER_MASKED_CAMERA_TRIGGERS, and KYVP_IO_CONTROLLER_MASKED_TRIGGERS to extract the corresponding groups.

KYVP_FRAME_AUX_DATA

Frame-arrival payload reserved for internal use. The fields following version belong to structure version 1.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Version 1 layout.
sequence_number	uint32_t	Sequential frame index within the allocated	

		frame buffer.	
timestamp	uint64_t	Frame-arrival timestamp.	Nanoseconds.
reserved	uint32_t	Reserved for internal use.	

KYVP_AUX_DATA

Auxiliary message delivered by the SDK. The SDK populates version with 1. Access payloads through the named u_data union.

Structure Field	Type	Description	Remarks
version	uint8_t	Structure version.	Current version: 1.
messageID	uint32_t	Identifies the auxiliary message and its payload.	KYVP_AUX_MESSAGE_ID_IO_CONTROLLER for I/O data.
u_data	union	Message payload.	Select the member using messageID.
u_data.io_data	KYVP_IO_AUX_DATA	I/O controller payload.	Used for KYVP_AUX_MESSAGE_ID_IO_CONTROLLER.
u_data.frame_data	KYVP_FRAME_AUX_DATA	Frame-arrival payload.	Reserved for internal use.
m_InterruptTimestampHw	uint64_t	Hardware timestamp captured for the interrupt.	Nanoseconds.
m_InterruptTimestampChronic	uint64_t	Host clock timestamp captured for the interrupt.	Nanoseconds; a separate clock from the hardware timestamp.

KYVP_CXP2_HEARTBEAT

Payload of a received CoaXPress 2.0 heartbeat packet. The SDK sets version to KYVP_CXP2_HEARTBEAT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	
uMasterHostConnectionID	uint32_t	Host connection ID	Value received in the

		associated with the device master connection.	heartbeat packet.
uDeviceTime	uint64_t	Time reported by the remote device in the heartbeat packet.	Nanoseconds.

KYVP_CXP2_EVENT

Payload of a received CoaXPress 2.0 event packet. The SDK sets version to KYVP_CXP2_EVENT_VERSION.

uDataSize counts 32-bit words, while KY_CXP_EVENT_MAX_DATA_SIZE is in bytes. The payload array holds at most 256 words (1024 bytes).

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	
uMasterHostConnectionID	uint32_t	Host connection ID associated with the device master connection.	Value received in the event packet.
uTag	uint8_t	Packet tag incremented for each new event packet.	8-bit value.
uDataSize	uint16_t	Number of 32-bit event data words reported by the packet.	Validate against the array capacity before reading.
uDataWordsArr	uint32_t [KY_CXP_EVENT_MAX_DATA_SIZE / 4]	Event data words containing one or more event messages.	Read at most uDataSize words, bounded by the array capacity.

KYVP_BUFFER_DATA_CHUNK

Describes one memory chunk of a buffer.

Structure Field	Type	Description	Remarks
pBufferChunkMemory	void*	Address of the memory chunk.	
bufferChunkSize	size_t	Size of the memory chunk.	Bytes.

KYVP_PCI_INTERFACE_SLOT

PCI bus, slot, and function identifying an interface.

Structure Field	Type	Description	Remarks
m_uBus	uint8_t : 8	PCI bus number.	8-bit field.

m_uSlot	uint8_t : 5	PCI slot number.	5-bit field.
m_uFunction	uint8_t : 3	PCI function number.	3-bit field; 7 denotes a software-emulated interface in this SDK.

KYVP_PCI_CONFIG

PCI Express generation and lane configuration. This structure has no version member.

Structure Field	Type	Description	Remarks
m_uGeneration	uint8_t	PCI Express generation number.	C++ default: UINT8_MAX.
m_uLanes	uint8_t	Number of PCI Express lanes.	C++ default: UINT8_MAX.

KYVP_BOARD_INFO

Board identity, PCI configuration, and stream capabilities. The current layout version is KYVP_MAX_BOARD_INFO_VERSION.

Structure Field	Type	Description	Remarks
m_uVersion	uint8_t	Structure version.	
m_Slot	KYVP_PCI_INTERFACE_SLOT	PCI bus, slot, and function.	
m_uDevicePID	uint32_t	Board product identifier.	
m_Protocol	KYVP_PCI_INTERFACE_PROTOCOL	Transport protocol supported by the interface.	
m_uDeviceGeneration	uint8_t	Product generation number.	1, 2, 3 correspond to generations I, II, III.
m_uRevision	uint8_t	Board hardware revision number.	
m_uPhysicalLinks	uint32_t	Number of physical interface links.	
m_ActivePCIConfig	KYVP_PCI_CONFIG	Current PCI	

		Express connection configuration.	
m_MinimalPCIConfig	KYVP_PCI_CONFIG	Minimum PCI Express configuration required by the board.	
m_uFlags	uint64_t	Supported stream directions as a bit mask.	KYVP_PCI_INTERFACE_STREAM_GRABBER and KYVP_PCI_INTERFACE_STREAM_GENERATOR.

KYVP_PCI_INTERFACE_INFO

Library-owned information describing a PCI interface. The current layout version is KYVP_PCI_INTERFACE_INFO_VERSION. Treat information returned by the SDK as read-only.

Structure Field	Type	Description	Remarks
m_uVersion	uint8_t	Structure version.	
m_szDeviceDisplayName	char [KY_MAX_PCI_INTERFACE_INFO_STRING_SIZE + 1]	Human-readable interface display name.	Capacity includes one character for the terminator.
m_uSerialNumber	uint32_t	Interface serial number.	
m_KYVP_BoardInfo	KYVP_BOARD_INFO	Board identity, PCI configuration, and stream capabilities.	
m_bIsVirtual	KY_BOOL	Whether the interface is software-emulated.	KY_TRUE for a virtual interface.

KYVP_DEVICE_INFO

Library-owned information describing a detected camera or device. The SDK stores these records in shared memory to coordinate processes using an interface. Treat returned records as read-only. The declared layout version is KYVP_DEVICE_INFO_VERSION.

Structure Field	Type	Description	Remarks
m_uVersion	uint32_t	Structure version supplied by	

		the SDK.	
m_uMasterLink	uint8_t	Host link connected to the device master link.	Zero-based host link index.
m_ProcId	KY_PROCID	ID of the process recorded as owning the device.	
m_uLinkMask	uint8_t	Mask of host links connected to the device.	Bit n corresponds to host link n.
m_nMaxDeviceLinks	int	Maximum number of device links.	
m_vecDeviceConnectionTopology	int [MAX_PCI_INTERFACE_LINKS]	Maps each device link index to its connected host link index.	-1 for an unmapped link in SDK-populated records.
m_linkSpeed	KYVP_CXP_LINK_SPEED	Detected CoaXPress link speed.	
m_uStreamId	uint32_t [256]	Device stream IDs indexed by image stream.	An entry is valid only when its m_uValidStreamId entry is true.
m_uValidStreamId	KY_BOOL [256]	Validity flags for the corresponding m_uStreamId entries.	
m_szDeviceVersion	char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1]	Device version string.	Capacity includes one character for the terminator.
m_szDeviceVendorName	char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1]	Device vendor name.	Capacity includes one character for the terminator.
m_szDeviceManufacturerInfo	char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1]	Additional manufacturer information.	Capacity includes one character for the terminator.

m_szDeviceModelName	char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1]	Device model name.	Capacity includes one character for the terminator.
m_szDeviceID	char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1]	Device identifier string.	Capacity includes one character for the terminator.
m_szDeviceUserID	char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1]	User-defined device identifier.	Capacity includes one character for the terminator.
m_szDeviceFirmwareVersion	char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1]	Device firmware version string.	Capacity includes one character for the terminator.
m_bOutputCamera	KY_BOOL	Whether the device represents an output camera.	KY_TRUE for output-camera mode.
m_bVirtualCamera	KY_BOOL	Whether the camera is virtual.	Used by custom firmware implementations.
m_uLinkConfig	uint32_t	Saved master-link configuration.	Used internally when restoring simulated device detection.

KYVP_DEVICE_IMAGE_DETAILS

Image dimensions, pixel format, and tap geometry of a device. Initialize m_uVersion to KYVP_DEVICE_IMAGE_DETAILS_VERSION for caller-provided instances.

Structure Field	Type	Description	Remarks
m_uVersion	uint32_t	Structure version.	
m_uWidth	uint32_t	Image width.	Pixels.
m_uHeight	uint32_t	Image height.	Pixels.
m_uPixelFormat	uint32_t	Numeric pixel format identifier.	
m_szPixelFormatName	char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1]	Symbolic pixel format name.	Capacity includes one character for the terminator.
m_szTapGeometryType	char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1]	Symbolic tap geometry name.	Capacity includes one character for the terminator.

KYVP_BUFFER_INFO

Library-owned bundle of pointers to buffer data and metadata. The current layout version is KYVP_BUFFER_INFO_VERSION. Metadata pointers refer to SDK-owned values, not copied snapshots. Read them while the buffer is valid and available to the application; acquisition can update the values. Do not free SDK-owned metadata or retain its pointers after the buffer is revoked. m_pBase addresses payload memory; m_pContext is the application context pointer.

Structure Field	Type	Description	Remarks
m_uVersion	uint32_t	Structure version supplied by the SDK.	
m_puFrameId	const uint64_t*	Pointer to the acquired frame identifier.	
m_puBufferId	const uint64_t*	Pointer to the SDK buffer identifier.	
m_puTimestampNS	const uint64_t*	Pointer to the frame timestamp.	Nanoseconds.
m_puInstantFPS	const double*	Pointer to the instantaneous acquisition rate.	Frames per second.
m_pBase	const void*	Base address of the buffer payload memory.	Direct address, not a pointer to a pointer.
m_puSize	const size_t*	Pointer to the allocated or announced buffer size.	Bytes.
m_pContext	const void*	Application context associated with the announced buffer.	Returned as supplied by the application.
m_puWidth	const size_t*	Pointer to the image width.	Pixels.
m_puHeight	const size_t*	Pointer to the image height.	Pixels.
m_puPixelFormat	const uint64_t*	Pointer to the numeric pixel format identifier.	
m_puTimestampNS_HW	const uint64_t*	Pointer to the hardware interrupt timestamp.	Nanoseconds.
m_puTimestampNS_CHRONO	const uint64_t*	Pointer to the host clock interrupt timestamp.	Nanoseconds; a separate clock from the hardware timestamp.

KYVPHWLib_Errors API

KYVPHWLib_Errors.h API reference

Generated from PUBLIC/include/KYVPHWLib_Errors.h.

API documentation generated from PUBLIC/include/KYVPHWLib_Errors.h.

Macros

KY_RESULT_CODE_HWLIB_FW_VERSION_OUTDATED

0x400

KY_RESULT_CODE_HWLIB_LIMITED_MODE

0x401

KYVPImageProcessing API

KYVPImageProcessing.h API reference

Generated from PUBLIC/include/KYVPImageProcessing.h.

API documentation generated from PUBLIC/include/KYVPImageProcessing.h.

See Introduction for operation lifecycles and Initializing image-processing records for C/C++ initialization requirements. Structure versions, declarations, and packed layouts are part of the SDK ABI.

Macros

KYVPIMAGEPROCESSING_EXTERNAL_DLL

Image-processing DLL import/export annotation on MSVC; empty elsewhere.

KYIMGPROC_API

KYVP_EXTERNAL_C KYVPIMAGEPROCESSING_EXTERNAL_DLL
Linkage and DLL visibility for image-processing functions.

KYIMGPROC_VAR

KYVP_EXTERNAL KYVPIMAGEPROCESSING_EXTERNAL_DLL
Linkage and DLL visibility for exported image-processing variables.

DECL_PIXELIF

KYVP_VIDEO_PIXELIF _name;
Declare one CoaXPress pixel-field record.

Parameter	Description
_name	Identifier used for the argument record and generated helper names.

See: kyimgproc_initialization

INIT_PIXEL_IF

_name.data_width = KYVP_DATA_WIDTH_UNKNOWN; \ \n
_name.data_subtype = KYVP_DATA_SUBTYPE_INVALID; \ \n
_name.data_type = KYVP_DATA_TYPE_INVALID; \ \n
Set the pixel-field record to unknown width and invalid type/subtype.

Parameter	Description
_name	Identifier used for the argument record and generated helper names.

See: kyimgproc_initialization

DECL_VP_PIXEL_FORMAT

KYVP_PIXEL_FORMAT _name; \ \n
DECL_PIXELIF(_name##pVPPixelIF) \ \n
Declare a metadata record and its helper pixel-field record.

Parameter	Description
_name	Identifier used for the argument record and generated helper names.

See: `kyimgproc_initialization`

INIT_VP_PIXEL_FORMAT

```
INIT_PIXEL_IF(_name##pVPPixelIF)
    _name.version = KYIMGPROC_KYVP_PIXEL_FORMAT_STRUCT_VERSION;
    _name.pVPPixelIF = _name##pVPPixelIF;
```

\
\

Initialize metadata version and copy the helper pixel fields.

Parameter	Description
_name	Identifier used for the argument record and generated helper names.

See: `kyimgproc_initialization`

DECL_PIXEL_DECODER

```
DECL_VP_PIXEL_FORMAT(_name##pVP_PIXEL_FORMAT)
    KYIMGPROC_PIXEL_DECODER _name;
```

\

Declare a decoder and the metadata storage it will reference.

Parameter	Description
_name	Identifier used for the argument record and generated helper names.

See: `kyimgproc_initialization`

INIT_PIXEL_DECODER

```
INIT_VP_PIXEL_FORMAT(_name##pVP_PIXEL_FORMAT)
    _name.version = KYIMGPROC_PIXEL_DECODER_STRUCT_VERSION;
    _name.pVP_PIXEL_FORMAT = &_name##pVP_PIXEL_FORMAT;
```

\
\

Initialize decoder version and attach the declared metadata storage.

Parameter	Description
_name	Identifier used for the argument record and generated helper names.

See: `kyimgproc_initialization`

DECL_VPIMAGE

```
DECL_PIXEL_DECODER(_name##_pixelFormat)
    VPImage _name;
```

\

Declare an image descriptor and its decoder/metadata storage.

Parameter	Description
_name	Identifier used for the argument record and generated helper names.

See: `kyimgproc_initialization`

INIT_VPIMAGE

```
INIT_PIXEL_DECODER(_name##_pixelFormat)
    _name.uVersion = KYIMGPROC_VPIMAGE_STRUCT_VERSION;
    _name.pixelFormat = &_amp;_name##_pixelFormat;
```

Initialize image version and attach its declared decoder storage.

Parameter	Description
_name	Identifier used for the argument record and generated helper names.

See: [kyimgproc_initialization](#)

DECL_CONVERT_PARAMS

```
DECL_VPIMAGE(_name##_srcImage)
    DECL_VPIMAGE(_name##_dstImage)
    KYIMGPROC_ConvertParam_Args _name;
```

Declare conversion arguments and both nested image descriptors.

Parameter	Description
_name	Identifier used for the argument record and generated helper names.

See: [kyimgproc_initialization](#)

INIT_CONVERT_PARAMS

```
_name.srcImage = &_amp;_name##_srcImage;
_name.dstImage = &_amp;_name##_dstImage;
_name.uVersion = KYIMGPROC_CONVERTPARAM_STRUCT_VERSION;
INIT_VPIMAGE(_name##_srcImage)
INIT_VPIMAGE(_name##_dstImage)
```

Initialize conversion versions and connect source/destination descriptors.

Parameter	Description
_name	Identifier used for the argument record and generated helper names.

See: [kyimgproc_initialization](#)

CONVERT_PARAMS_DECL_AND_INIT

```
DECL_CONVERT_PARAMS(params) INIT_CONVERT_PARAMS(params)
```

Declare and connect storage for a conversion argument block.

Parameter	Description
params	Identifier used for the argument record and generated helper names.

See: [kyimgproc_initialization](#)

DECL_SAVE_IMAGE_PARAMS

```
DECL_VPIMAGE(_name##_srcImage)
    KYIMGPROC_SaveImageArgs _name;
```

Declare image-saving arguments and the source image storage.

Parameter	Description
-----------	-------------

<code>_name</code>	Identifier used for the argument record and generated helper names.
--------------------	---

See: `kyimgproc_initialization`

INIT_SAVE_IMAGE_PARAMS

```
_name.srcImage = &_amp;_name##_srcImage;
_name.uVersion = KYIMGPROC_SAVEIMAGE_STRUCT_VERSION;
INIT_VPIIMAGE(_name##_srcImage)
```

Initialize image-saving versions and connect the source image descriptor.

Parameter	Description
<code>_name</code>	Identifier used for the argument record and generated helper names.

See: `kyimgproc_initialization`

SAVE_IMAGE_PARAMS_DECL_AND_INIT

```
DECL_SAVE_IMAGE_PARAMS(params) INIT_SAVE_IMAGE_PARAMS(params)
```

Declare and connect storage for image-saving arguments.

Parameter	Description
<code>params</code>	Identifier used for the argument record and generated helper names.

See: `kyimgproc_initialization`

DECL_SAVE_VIDEO_PARAMS

```
DECL_PIXEL_DECODER(_name##_pixelFormat)
KYIMGPROC_SaveVideoArgs _name;
```

Declare video-recording arguments and source-format storage.

Parameter	Description
<code>_name</code>	Identifier used for the argument record and generated helper names.

See: `kyimgproc_initialization`

INIT_SAVE_VIDEO_PARAMS

```
_name.uVersion = KYIMGPROC_SAVEVIDEO_STRUCT_VERSION;
INIT_PIXEL_DECODER(_name##_pixelFormat)
_name.srcPixelFormat = &_amp;_name##_pixelFormat;
```

Initialize recording versions and connect the source decoder.

Parameter	Description
<code>_name</code>	Identifier used for the argument record and generated helper names.

See: `kyimgproc_initialization`

SAVE_VIDEO_PARAMS_DECL_AND_INIT

```
DECL_SAVE_VIDEO_PARAMS(params) INIT_SAVE_VIDEO_PARAMS(params)
```

Declare and connect storage for video-recording arguments.

Parameter	Description
params	Identifier used for the argument record and generated helper names.

See: `kyimgproc_initialization`

KYIMGPROC_VIDEO_RECORDER_HANDLE_INVALID_INIT_VALUE

`((void*)-1)`

Native-pointer sentinel for a default invalid handle.

KYIMGPROC_VIDEO_RECORDER_HANDLE_IS_INVALID

`KYImgProc_VideoRecorderHandleIsValid(_hHandle)`

Forward a handle comparison to `KYImgProc_VideoRecorderHandleIsValid()`.

Parameter	Description
<code>_hHandle</code>	Handle value to compare.

Returns: `KY_TRUE` if the comparison succeeds; otherwise `KY_FALSE`. The historical macro spelling is retained for source compatibility.

KYIMGPROC_VIDEO_RECORDER_HANDLE_IS_NULL

`KYImgProc_VideoRecorderHandleIsNull(_hHandle)`

Forward a handle comparison to `KYImgProc_VideoRecorderHandleIsNull()`.

Parameter	Description
<code>_hHandle</code>	Handle value to compare.

Returns: `KY_TRUE` if the comparison succeeds; otherwise `KY_FALSE`.

KYIMGPROC_VIDEO_RECORDER_HANDLE_IS_EQUAL

`KYImgProc_VideoRecorderHandleIsEqual(_hHandleFirst, _hHandleSecond)`

Forward a handle comparison to `KYImgProc_VideoRecorderHandleIsEqual()`.

Parameter	Description
<code>_hHandleFirst</code>	Handle value to compare.
<code>_hHandleSecond</code>	Handle value to compare.

Returns: `KY_TRUE` if the comparison succeeds; otherwise `KY_FALSE`.

KYIMGPROC_IMAGE_CONVERTER_HANDLE_INVALID_INIT_VALUE

`((void*)-1)`

Native-pointer sentinel for a default invalid handle.

KYIMGPROC_IMAGE_CONVERTER_HANDLE_IS_INVALID

`KYImgProc_ImageConverterHandleIsValid(_hHandle)`

Forward a handle comparison to `KYImgProc_ImageConverterHandleIsValid()`.

Parameter	Description
<code>_hHandle</code>	Handle value to compare.

Returns: `KY_TRUE` if the comparison succeeds; otherwise `KY_FALSE`.

KYIMGPROC_IMAGE_CONVERTER_HANDLE_IS_NULL

KYImgProc_ImageConverterHandleIsNull(_hHandle)
Forward a handle comparison to KYImgProc_ImageConverterHandleIsNull().

Parameter	Description
_hHandle	Handle value to compare.

Returns: KY_TRUE if the comparison succeeds; otherwise KY_FALSE.

KYIMGPROC_IMAGE_CONVERTER_HANDLES_IS_EQUAL

KYImgProc_ImageConverterHandlesIsEqual(_hHandleFirst, _hHandleSecond)

KYIMGPROC_VPIMAGE_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_PACKEDDATAINFO_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_INITVIDEORECORDER_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_DELETEVIDEORECORDER_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_INITIMAGECONVERTER_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_DELETEIMAGECONVERTER_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_CONVERTPARAM_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_SAVEIMAGE_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_SAVEIMAGEMULTI_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_SAVERAW_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_SAVERAWMULTI_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_SAVEVIDEO_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_WRITEVIDEOFRAME_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_STOPVIDEORECORD_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_GETPACKEDDATAINFO_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_PREPROCESSPACKEDDATA_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_GETPIXELFORMAT_STRUCT_VERSION

1

Declared structure layout version 1.

KYIMGPROC_GETPIXELFORMATBYNAME_STRUCT_VERSION

1

Declared structure layout version 1.

Enumerations

_BITS_DEPTH_SHIFT_MODE

Alignment requested when saving samples in a 16-bit channel container.

Values

Name	Description
BITS_DEPTH_SHIFT_MODE_UNKNOWN	Unspecified alignment; rejected by the image-saving API.
BITS_DEPTH_SHIFT_MODE_LSB	Keep meaningful bits in the least significant positions of each channel.

BITS_DEPTH_SHIFT_MODE_MSB	Shift meaningful bits to the most significant positions of each 16-bit channel.
----------------------------------	---

_SAVED_IMAGE_FORMAT

Output file formats supported by the image-saving API.

Values

Name	Description
SAVED_IMAGE_FORMAT_UNKNOWN	Unspecified image format; rejected by the image-saving API.
SAVED_IMAGE_FORMAT_BMP	Windows bitmap file; output filename uses the .bmp extension.
SAVED_IMAGE_FORMAT_PNG	PNG file; output filename uses the .png extension.
SAVED_IMAGE_FORMAT_TIF	TIFF file; output filename uses the .tif extension.

Type definitions

BITS_DEPTH_SHIFT_MODE

```
typedef enum _BITS_DEPTH_SHIFT_MODE BITS_DEPTH_SHIFT_MODE
```

Alignment requested when saving samples in a 16-bit channel container.

SAVED_IMAGE_FORMAT

```
typedef enum _SAVED_IMAGE_FORMAT SAVED_IMAGE_FORMAT
```

Output file formats supported by the image-saving API.

Variables

KYIMGPROC_VIDEO_RECORDER_HANDLE_NULL

```
const KYIMGPROC_VIDEO_RECORDER_HANDLE KYIMGPROC_VIDEO_RECORDER_HANDLE_NULL
```

Exported null video-recorder handle value.

KYIMGPROC_VIDEO_RECORDER_HANDLE_INVALID

```
const KYIMGPROC_VIDEO_RECORDER_HANDLE KYIMGPROC_VIDEO_RECORDER_HANDLE_INVALID
```

Exported invalid video-recorder handle value.

KYIMGPROC_IMAGE_CONVERTER_HANDLE_NULL

```
const KYIMGPROC_IMAGE_CONVERTER_HANDLE KYIMGPROC_IMAGE_CONVERTER_HANDLE_NULL
```

Exported null image-converter handle value.

KYIMGPROC_IMAGE_CONVERTER_HANDLE_INVALID

```
const KYIMGPROC_IMAGE_CONVERTER_HANDLE KYIMGPROC_IMAGE_CONVERTER_HANDLE_INVALID
```

Exported invalid image-converter handle value.

Functions

KYImgProc_InitImageConverter

KY_RESULT KYImgProc_InitImageConverter(KYIMGPROC_InitImageConverterArgs *_pArgs)

Create and register an image converter.

Delete the returned handle with KYImgProc_DeleteImageConverter() when finished. The current version-1 conversion API does not use a caller-supplied converter handle.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_InitImageConverterArgs.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_DeleteImageConverter

KYImgProc_InitVideoRecorder

KY_RESULT KYImgProc_InitVideoRecorder(KYIMGPROC_InitVideoRecorderArgs *_pArgs)

Create and register a video recorder.

Configure it with KYImgProc_StartVideoRecord() before submitting frames.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_InitVideoRecorderArgs.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_StartVideoRecord

KYImgProc_DeleteImageConverter

KY_RESULT KYImgProc_DeleteImageConverter(KYIMGPROC_DeleteImageConverterArgs *_pArgs)

Delete an image converter and remove its handle registration.

The existing C++ member default is zero. Set uVersion explicitly to KYIMGPROC_DELETEIMAGECONVERTER_STRUCT_VERSION before use.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_DeleteImageConverterArgs.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

KYImgProc_DeleteVideoRecorder

KY_RESULT KYImgProc_DeleteVideoRecorder(KYIMGPROC_DeleteVideoRecorderArgs *_pArgs)

Remove a video-recorder handle registration.

For an active recorder, stop it first with KYImgProc_StopVideoRecord(). This call removes its handle registration; it does not stop recording or destroy the recorder object.

Parameter	Description
-----------	-------------

_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_DeleteVideoRecorderArgs.
--------	---

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

KYImgProc_GetPixelFormat

KY_RESULT KYImgProc_GetPixelFormat(KYImgProc_GetPixelFormat_Args *_pArgs)

Look up a PFNC or CoaXPress numeric pixel-format code.

Initialize the decoder version and point pVP_PIXEL_FORMAT at a writable KYVP_PIXEL_FORMAT before calling. Zero is rejected by numeric lookup, including the all-zero raw code; lookup by the name Raw is available. Lookup returns the first matching entry in the SDK table. A shared CoaXPress code can map to several PFNC storage formats, so use an exact PFNC numeric value when packed/unpacked identity matters.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYImgProc_GetPixelFormat_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_GetPixelFormatByName

KYImgProc_GetPixelFormatByName

KY_RESULT KYImgProc_GetPixelFormatByName(KYImgProc_GetPixelFormatByName_Args *_pArgs)

Look up a PFNC-style or CoaXPress pixel-format name.

Initialize the decoder version and point pVP_PIXEL_FORMAT at a writable KYVP_PIXEL_FORMAT before calling. Names are matched case-insensitively against both SDK PFNC and CoaXPress names; the first matching table entry is returned.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYImgProc_GetPixelFormatByName_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_GetPixelFormat

KYImgProc_ConvertImage

KY_RESULT KYImgProc_ConvertImage(KYIMGPROC_ConvertParam_Args *_convertStruct)

Convert or resize an image into application-provided destination storage.

The public KYImgProc_ConvertImage() entry point currently accepts version 1 only. Although the layout includes h_ImgProcHandle for version 2, setting uVersion to 2 is currently rejected. Initialize h_ImgProcHandle to KYIMGPROC_IMAGE_CONVERTER_HANDLE_INVALID, including when using version 1. Initialize both image descriptors and their format metadata. Supported conversions depend on the configured image-processing backend.

Parameter	Description
_convertStruct	Non-NULL argument block; initialize its version, required fields, and nested records as described in

	KYIMGPROC_ConvertParam_Args.
--	------------------------------

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: CONVERT_PARAMS_DECL_AND_INIT

KYImgProc_GetPackedDataInfo

KY_RESULT KYImgProc_GetPackedDataInfo(KYImgProc_GetPackedDataInfo_Args *_pArgs)

Calculate a packed-data processing plan and destination allocation size.

Supply a decoded format and positive image dimensions. The result reports the destination allocation size using SDK row-alignment rules; it does not allocate memory.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYImgProc_GetPackedDataInfo_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_PreprocessPackedData

KYImgProc_PreprocessPackedData

KY_RESULT KYImgProc_PreprocessPackedData(KYImgProc_PreprocessPackedData_Args *_pArgs)

Process packed samples according to a prepared processing plan.

Obtain the plan with KYImgProc_GetPackedDataInfo() first. Allocate sufficient non-overlapping source and destination storage. The caller must ensure both buffer capacities are sufficient; buffer-size errors may not be reported. Packed source rows must follow the SDK four-byte row-alignment rule. Provide at least three additional readable padding bytes after the source image data.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYImgProc_PreprocessPackedData_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_GetPackedDataInfo

KYImgProc_SaveImage

KY_RESULT KYImgProc_SaveImage(KYIMGPROC_SaveImageArgs *_pArgs)

Save one image in the selected image-file format.

Select a non-UNKNOWN format and bit alignment. The filename extension is added or replaced to match the selected format. Encoded-image saving requires OpenCV support in the SDK build.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_SaveImageArgs.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_SaveImageMulti

KYImgProc_SaveImageMulti

KY_RESULT KYImgProc_SaveImageMulti(KYIMGPROC_SaveImageMultiArgs *_pArgs)

Save an array of images as one vertically stacked image.

Images are stacked in array order, top to bottom, into a single image. All entries must have matching dimensions, decoded pixel fields, and RGB channel order. This is not a multipage TIFF operation. Select a non-UNKNOWN format and bit alignment. The filename extension is added or replaced. Encoded-image saving requires OpenCV support in the SDK build.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_SaveImageMultiArgs.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

KYImgProc_SaveRawData

KY_RESULT KYImgProc_SaveRawData(KYIMGPROC_SaveRawArgs *_pArgs)

Write one image buffer to a raw binary file.

The output filename uses the .raw extension. The buffer is written unchanged, with no image header or pixel conversion. Raw saving does not require OpenCV.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_SaveRawArgs.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_SaveRawDataMulti

KYImgProc_SaveRawDataMulti

KY_RESULT KYImgProc_SaveRawDataMulti(KYIMGPROC_SaveRawMultiArgs *_pArgs)

Concatenate image buffers into a raw binary file.

The buffers are written unchanged in array order with no headers or separators. Every buffer must have the same nonzero byte count. The output filename uses the .raw extension; raw saving does not require OpenCV.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_SaveRawMultiArgs.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

KYImgProc_StartVideoRecord

KY_RESULT KYImgProc_StartVideoRecord(KYIMGPROC_SaveVideoArgs *_pArgs)

Start an AVI recording with the configured MPEG-4 encoder.

Initialize every field; several members have no C++ default initializer. The recorder retains a pointer to this structure, so keep it and its referenced filename and pixel metadata valid until recording stops. The current backend writes MPEG-4 video in an AVI container.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_SaveVideoArgs.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_WriteVideoFrame

KYImgProc_StopVideoRecord

KY_RESULT KYImgProc_StopVideoRecord(KYImgProc_StopVideoRecordArgs *_pArgs)

Finalize recording and destroy the recorder object.

Stop waits for the frame worker and finalizes the file, then destroys the recorder object. The handle registration remains until KYImgProc_DeleteVideoRecorder() is called. After stop, use the handle only to remove that registration.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYImgProc_StopVideoRecordArgs.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_DeleteVideoRecorder

KYImgProc_WriteVideoFrame

KY_RESULT KYImgProc_WriteVideoFrame(KYIMGPROC_WriteVideoFrameArgs *_pArgs)

Copy a frame into the recorder queue for asynchronous encoding.

Submit only after successful start. The recorder copies image bytes and pixel-format records before queuing the frame. The caller retains ownership of its source storage. A successful return does not guarantee that the frame was queued, encoded, or written to disk. Queueing and asynchronous encoding failures are not reported by this return value.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version, required fields, and nested records as described in KYIMGPROC_WriteVideoFrameArgs.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYImgProc_StopVideoRecord

KYImgProc_ImageConverterHandleIsInvalid

KY_BOOL KYImgProc_ImageConverterHandleIsInvalid(KYIMGPROC_IMAGE_CONVERTER_HANDLE _hHandle)

Test whether a handle equals the invalid sentinel.

This is a stored-value comparison, not a check that the underlying object is live or registered.

Parameter	Description
_hHandle	First handle value.

Returns: KY_TRUE when the handle equals the invalid sentinel; otherwise KY_FALSE.

KYImgProc_ImageConverterHandleIsNull

KY_BOOL KYImgProc_ImageConverterHandleIsNull(KYIMGPROC_IMAGE_CONVERTER_HANDLE _hHandle)

Test whether a handle equals the null sentinel.

This is a stored-value comparison, not a check that the underlying object is live or registered.

Parameter	Description
_hHandle	First handle value.

Returns: KY_TRUE when the handle equals the null sentinel; otherwise KY_FALSE.

KYImgProc_ImageConverterHandlesIsEqual

KY_BOOL KYImgProc_ImageConverterHandlesIsEqual(KYIMGPROC_IMAGE_CONVERTER_HANDLE _hFirstHandle, KYIMGPROC_IMAGE_CONVERTER_HANDLE _hOtherHandle)

Compare two handle values for equality.

This is a stored-value comparison, not a check that the underlying object is live or registered.

Parameter	Description
_hFirstHandle	First handle value.
_hOtherHandle	Second handle value.

Returns: KY_TRUE when the stored handle values are equal; otherwise KY_FALSE.

KYImgProc_VideoRecorderHandleIsInvalid

KY_BOOL KYImgProc_VideoRecorderHandleIsInvalid(KYIMGPROC_VIDEO_RECORDER_HANDLE _hHandle)

Test whether a handle equals the invalid sentinel.

This is a stored-value comparison, not a check that the underlying object is live or registered.

Parameter	Description
_hHandle	First handle value.

Returns: KY_TRUE when the handle equals the invalid sentinel; otherwise KY_FALSE.

KYImgProc_VideoRecorderHandleIsNull

KY_BOOL KYImgProc_VideoRecorderHandleIsNull(KYIMGPROC_VIDEO_RECORDER_HANDLE _hHandle)

Test whether a handle equals the null sentinel.

This is a stored-value comparison, not a check that the underlying object is live or registered.

Parameter	Description
_hHandle	First handle value.

Returns: KY_TRUE when the handle equals the null sentinel; otherwise KY_FALSE.

KYImgProc_VideoRecorderHandlesIsEqual

KY_BOOL KYImgProc_VideoRecorderHandlesIsEqual(KYIMGPROC_VIDEO_RECORDER_HANDLE _hFirstHandle, KYIMGPROC_VIDEO_RECORDER_HANDLE _hOtherHandle)

Compare two handle values for equality.

This is a stored-value comparison, not a check that the underlying object is live or registered.

Parameter	Description
-----------	-------------

_hFirstHandle	First handle value.
_hOtherHandle	Second handle value.

Returns: KY_TRUE when the stored handle values are equal; otherwise KY_FALSE.

Image Processing Library Structs

Field reference for handles, images, and operation arguments.

Tables list fields in declaration order, including exact pointer types. Set versions explicitly in C. C++ defaults do not initialize every member; follow each table's input and ownership requirements.

Pixel-format records are described in Pixel Format Decoder Structs.

KYIMGPROC_VIDEO_RECORDER_HANDLE

Opaque handle identifying a video-recorder registration.

Treat the native pointer as opaque. The C++ default constructor selects the invalid sentinel; initialize C handles explicitly with the exported null or invalid value. Handle comparisons test stored values only; they do not establish that a recorder is still alive.

Structure Field	Type	Description	Remarks
pNativePointer	void*	Native object identity used by the SDK.	Opaque; do not dereference or modify directly.

KYIMGPROC_IMAGE_CONVERTER_HANDLE

Opaque handle identifying an image converter.

Treat the native pointer as opaque. The C++ default constructor selects the invalid sentinel; initialize C handles explicitly with the exported null or invalid value. Do not use the C++ call operator in application code. Use the documented handle functions.

Structure Field	Type	Description	Remarks
pNativePointer	void*	Native object identity used by the SDK.	Opaque; do not dereference or modify directly.

VPIImage

Application-owned image storage and its decoded pixel format.

Initialize uVersion to KYIMGPROC_VPIMAGE_STRUCT_VERSION.

The API does not allocate this image buffer. Supply valid storage and decoded format metadata for conversion, encoded-image saving, and video recording. Raw saving only reads pBuffer and uBufferSize. This structure has no row-stride field. Supply the memory layout required by the operation; image-file saving expects a tightly packed, unpacked-channel image.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
uWidth	uint32_t	Image width in pixels.	Input; greater than zero

			for processing.
uHeight	uint32_t	Image height in pixels.	Input; greater than zero for processing.
pBuffer	void*	Address of the image pixel bytes.	Application-owned; readable for source images, writable for destination images.
uBufferSize	size_t	Number of image-buffer bytes available to the operation.	Input; nonzero; encoded-image saving requires the exact expected image byte count.
pixelFormat	KYIMGPROC_PIXEL_DECODER*	Decoded pixel-format description for this image.	Input; initialized decoder and metadata required for processing.

PackedDataInfo

Packed-data processing plan populated by KYImgProc_GetPackedDataInfo().

Initialize uVersion to KYIMGPROC_PACKEDDATAINFO_STRUCT_VERSION.

Initialize uVersion before the query; KYImgProc_GetPackedDataInfo() fills the other fields without writing the version. The result borrows the source decoder pointer, which must remain valid while this plan is used.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
ePackedModeType	DS_PACKED_DATA_MODE	Packing mode of the source stream data.	Output; copied from the query inputs.
eProcessingModeType	DS_PACKED_DATA_PROCESSING_MODE	Requested processing: extract 8-bit samples or expand to 16-bit samples.	Output; copied from the query inputs.
pPixelFormat	const KYIMGPROC_PIXEL_DECODER*	Decoded source pixel format used for size and sample calculations.	Output; copied from the query inputs; borrowed pointer.
uWidth	size_t	Image width in pixels.	Output; copied from the query inputs.
uHeight	size_t	Image height in pixels.	Output; copied from the query inputs.

uUnpackedBufferSize	size_t	Destination allocation size reported by the SDK, in bytes.	Output; includes the size calculation's row-alignment allowance.
---------------------	--------	--	--

KYIMGPROC_InitVideoRecorderArgs

Arguments for creating a video-recorder handle.

Initialize uVersion to KYIMGPROC_INITVIDEORECORDER_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
phRecordHandle	KYIMGPROC_VIDEO_RECORDER_HANDLE*	Receives the newly created video-recorder handle.	Output; non-NULL.

KYIMGPROC_DeleteVideoRecorderArgs

Arguments for removing a video-recorder handle registration.

Initialize uVersion to KYIMGPROC_DELETEVIDEORECORDER_STRUCT_VERSION.

For an active recorder, stop it first with KYImgProc_StopVideoRecord(). This call removes its handle registration; it does not stop recording or destroy the recorder object.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
phRecordHandle	KYIMGPROC_VIDEO_RECORDER_HANDLE	Recorder handle whose registration is to be removed.	Input; handle value, despite the ph prefix; stop active recording first.

KYIMGPROC_InitImageConverterArgs

Arguments for creating an image-converter handle.

Initialize uVersion to KYIMGPROC_INITIMAGECONVERTER_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
phImageProcHandle	KYIMGPROC_IMAGE_CONVERTER_HANDLE*	Receives the newly created image-converter handle.	Output; non-NULL.

KYIMGPROC_DeleteImageConverterArgs

Arguments for deleting an image converter.

Initialize uVersion to KYIMGPROC_DELETEIMAGECONVERTER_STRUCT_VERSION.

The existing C++ member default is zero. Set uVersion explicitly to KYIMGPROC_DELETEIMAGECONVERTER_STRUCT_VERSION before use.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1. Existing C++ default: 0.
hHandle	KYIMGPROC_IMAGE_CONVERTER_HANDLE	Image-converter handle to delete.	Input; becomes unusable after deletion.

KYIMGPROC_ConvertParam_Args

Source and destination images for conversion.

Initialize uVersion to KYIMGPROC_CONVERTPARAM_STRUCT_VERSION.

The public KYImgProc_ConvertImage() entry point currently accepts version 1 only. Although the layout includes h_ImgProcHandle for version 2, setting uVersion to 2 is currently rejected. Initialize h_ImgProcHandle to KYIMGPROC_IMAGE_CONVERTER_HANDLE_INVALID, including when using version 1. Initialize both image descriptors and their format metadata.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
srcImage	VPIImage*	Source image descriptor and readable pixel buffer.	Input; non-NULL; application-owned.
dstImage	VPIImage*	Destination dimensions, decoded format, and writable pixel buffer.	Input/output; non-NULL; buffer allocated by the caller.
h_ImgProcHandle	KYIMGPROC_IMAGE_CONVERTER_HANDLE	Converter handle field reserved for the version-2 conversion path.	Initialize to the invalid sentinel; public conversion currently accepts version 1 only.

KYIMGPROC_SaveImageArgs

Arguments for saving one image in a selected file format.

Initialize uVersion to KYIMGPROC_SAVEIMAGE_STRUCT_VERSION.

Select a non-UNKNOWN format and bit alignment. The filename extension is added or replaced to match the selected format. Encoded-image saving requires OpenCV support in the SDK build.

Structure Field	Type	Description	Remarks
-----------------	------	-------------	---------

uVersion	uint32_t	Structure layout version.	Input; set to 1.
eImageFormat	SAVED_IMAGE_FORMAT	Output image-file format.	Input; UNKNOWN is not accepted.
eBitsDepthShiftMode	BITS_DEPTH_SHIFT_MODE	Bit alignment of saved channel samples.	Input; use LSB or MSB, including for 8-bit images.
srcImage	const VImage*	Source image descriptor and readable pixel buffer.	Input; non-NULL; application-owned.
szFileName	const char*	Output file path.	Input; non-NULL, null-terminated string; existing output may be overwritten.

KYIMGPROC_SaveImageMultiArgs

Arguments for saving equally sized images as one vertically stacked image.

Initialize uVersion to KYIMGPROC_SAVEIMAGEMULTI_STRUCT_VERSION.

Images are stacked in array order, top to bottom, into a single image. All entries must have matching dimensions, decoded pixel fields, and RGB channel order. This is not a multipage TIFF operation. Select a non-UNKNOWN format and bit alignment. The filename extension is added or replaced. Encoded-image saving requires OpenCV support in the SDK build.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
eImageFormat	SAVED_IMAGE_FORMAT	Output image-file format.	Input; UNKNOWN is not accepted.
eBitsDepthShiftMode	BITS_DEPTH_SHIFT_MODE	Bit alignment of saved channel samples.	Input; use LSB or MSB, including for 8-bit images.
ppSrcImages	const VImage	Array of pointers to source image descriptors.	Input; non-NULL array with no NULL entries.
uSrcImageCount	size_t	Number of image pointers in ppSrcImages.	Input; greater than zero.
szFileName	const char*	Output file path.	Input; non-NULL, null-terminated string; existing output may be overwritten.

KYIMGPROC_SaveRawArgs

Arguments for writing one image buffer as raw bytes.

Initialize uVersion to KYIMGPROC_SAVERAW_STRUCT_VERSION.

The output filename uses the .raw extension. The buffer is written unchanged, with no image header or pixel conversion. Raw saving does not require OpenCV.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
srcImage	const VPIImage*	Source image descriptor and readable pixel buffer.	Input; non-NULL; application-owned.
szFileName	const char*	Output file path.	Input; non-NULL, null-terminated string; existing output may be overwritten.

KYIMGPROC_SaveRawMultiArgs

Arguments for concatenating image buffers into one raw file.

Initialize uVersion to KYIMGPROC_SAVERAWMULTI_STRUCT_VERSION.

The buffers are written unchanged in array order with no headers or separators. Every buffer must have the same nonzero byte count. The output filename uses the .raw extension; raw saving does not require OpenCV.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
ppSrcImages	const VPIImage	Array of pointers to source image descriptors.	Input; non-NULL array with no NULL entries.
uSrcImageCount	size_t	Number of image pointers in ppSrcImages.	Input; greater than zero.
szFileName	const char*	Output file path.	Input; non-NULL, null-terminated string; existing output may be overwritten.

KYIMGPROC_SaveVideoArgs

Video-recorder configuration retained by KYImgProc_StartVideoRecord().

Initialize uVersion to KYIMGPROC_SAVEVIDEO_STRUCT_VERSION.

Initialize every field; several members have no C++ default initializer. The recorder retains a pointer to this structure, so keep it and its referenced filename and pixel metadata valid until recording stops. The current backend writes MPEG-4 video in an AVI container.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
vHandler	KYIMGPROC_VIDEO_RECORDER_HANDLE	Video-recorder handle for the operation.	Input; valid registration; observe the recording lifecycle.
szFileName	const char*	Output file path.	Input; non-NULL, null-terminated string; existing

			output may be overwritten.
srcPixelFormat	KYIMGPROC_PIXEL_DECODER*	Decoded pixel format expected for submitted frames.	Input; non-NULL; keep metadata valid while recording.
bitRate	uint32_t	Target encoder bit rate in bits per second.	Input; explicitly configure before starting.
dFps	double	Recording frame rate in frames per second.	Input; greater than zero.
uWidth	uint32_t	Image width in pixels.	Input; greater than zero for processing.
uHeight	uint32_t	Image height in pixels.	Input; greater than zero for processing.

KYIMGPROC_WriteVideoFrameArgs

Arguments for submitting one frame to the recording queue.

Initialize uVersion to KYIMGPROC_WRITEVIDEOFRAME_STRUCT_VERSION.

Submit only after successful start. The recorder copies image bytes and pixel-format records before queuing the frame. The caller retains ownership of its source storage. A successful return does not guarantee that the frame was queued, encoded, or written to disk. Queueing and asynchronous encoding failures are not reported by this return value.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
vHandler	KYIMGPROC_VIDEO_RECORDER_HANDLE	Video-recorder handle for the operation.	Input; valid registration; observe the recording lifecycle.
pSrcImage	VPIImage*	Image frame to copy into the recording queue.	Input; non-NULL; dimensions and format must match the recording configuration.

KYImgProc_StopVideoRecordArgs

Arguments for finalizing a recording and destroying its recorder object.

Initialize uVersion to KYIMGPROC_STOPVIDEORECORD_STRUCT_VERSION.

Stop waits for the frame worker and finalizes the file, then destroys the recorder object. The handle registration remains until KYImgProc_DeleteVideoRecorder() is called. After stop, use the handle only to remove that registration.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
vHandler	KYIMGPROC_VIDEO_RECORDER_HANDLE	Video-recorder handle for the operation.	Input; valid registration; observe the recording lifecycle.

KYImgProc_GetPackedDataInfo_Args

Arguments for calculating packed-data processing information.

Initialize uVersion to KYIMGPROC_GETPACKEDDATAINFO_STRUCT_VERSION.

Supply a decoded format and positive image dimensions. The result reports the destination allocation size using SDK row-alignment rules; it does not allocate memory.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
pPixelFormat	const KYIMGPROC_PIXEL_DECODER*	Decoded source pixel format used for size and sample calculations.	Input; non-NULL decoder with valid metadata.
ePackedModeType	DS_PACKED_DATA_MODE	Packing mode of the source stream data.	Input; select the mode matching the actual buffer layout.
eProcessingModeType	DS_PACKED_DATA_PROCESSING_MODE	Requested processing: extract 8-bit samples or expand to 16-bit samples.	Input; use MSB8 or EXPAND16.
uWidth	size_t	Image width in pixels.	Input; greater than zero for processing.
uHeight	size_t	Image height in pixels.	Input; greater than zero for processing.
pPackedDataInfoStruct	PackedDataInfo*	Processing plan to populate.	Input/output; non-NULL; initialize its version first.

KYImgProc_PreprocessPackedData_Args

Source and destination buffers for packed-data processing.

Initialize uVersion to KYIMGPROC_PREPROCESSPACKEDDATA_STRUCT_VERSION.

Obtain the plan with KYImgProc_GetPackedDataInfo() first. Allocate sufficient non-overlapping source and destination storage. The caller must ensure both buffer capacities are sufficient; buffer-size errors may not be

reported. Packed source rows must follow the SDK four-byte row-alignment rule. Provide at least three additional readable padding bytes after the source image data.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
pPackedDataInfoStruct	const PackedDataInfo*	Processing plan previously populated by the query API.	Input; non-NULL; referenced decoder must remain valid.
pSourceData	const void*	Readable source buffer containing packed image bytes.	Input; non-NULL; include row alignment and readable end padding.
uSourceDataSize	size_t	Source data byte count supplied to the processing operation.	Input; nonzero; must cover the image data.
pDestinationData	void*	Writable buffer receiving processed samples.	Output; non-NULL; must not overlap the source buffer.
uDestinationDataSize	size_t	Destination capacity in bytes.	Input; allocate at least uUnpackedBufferSize from the processing plan.

KYImgProc_GetPixelFormat_Args

Arguments for looking up a numeric pixel-format code.

Initialize uVersion to KYIMGPROC_GETPIXELFORMAT_STRUCT_VERSION.

Initialize the decoder version and point pVP_PIXEL_FORMAT at a writable KYVP_PIXEL_FORMAT before calling. Zero is rejected by numeric lookup, including the all-zero raw code; lookup by the name Raw is available. Lookup returns the first matching entry in the SDK table. A shared CoaXPress code can map to several PFNC storage formats, so use an exact PFNC numeric value when packed/unpacked identity matters.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
uPixelFormatValue	uint32_t	PFNC 32-bit value or CoaXPress 16-bit code carried in a uint32_t.	Input; nonzero; exact numeric PFNC codes distinguish storage variants.
pPixelFormatDecoder	KYIMGPROC_PIXEL_DECODER*	Decoder and attached metadata storage to populate.	Input/output; non-NULL; initialize version and pVP_PIXEL_FORMAT first.

KYImgProc_GetPixelFormatByName_Args

Arguments for looking up a pixel format by name.

Initialize uVersion to KYIMGPROC_GETPIXELFORMATBYNAME_STRUCT_VERSION.

Initialize the decoder version and point pVP_PIXEL_FORMAT at a writable KYVP_PIXEL_FORMAT before calling. Names are matched case-insensitively against both SDK PFNC and CoaXPress names; the first matching table entry is returned.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Structure layout version.	Input; set to 1.
pszPixelFormatName	const char*	PFNC-style or CoaXPress name to look up.	Input; non-NULL, nonempty, null-terminated string; case-insensitive match.
pPixelFormatDecoder	KYIMGPROC_PIXEL_DECODER*	Decoder and attached metadata storage to populate.	Input/output; non-NULL; initialize version and pVP_PIXEL_FORMAT first.

KYVPLibExtension API

KYVPLibExtension.h API reference

Generated from PUBLIC/include/KYVPLibExtension.h.

API documentation generated from PUBLIC/include/KYVPLibExtension.h.

Use the argument-structure entry points or their versioned convenience functions. Initialize each argument version explicitly in C; C++ member defaults supply the declared versions. Structure field tables preserve the packed public layouts and document input, output, and ownership requirements. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to inspect status values.

Macros

KYVPEXTENSION_PCIINTERFACE_FIRMWAREUPDATE_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_PCIINTERFACE_CHECKUPDATEFILE_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_PCIINTERFACE_EVENTCALLBACKREGISTER_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_PCIINTERFACE_EVENTCALLBACKUNREGISTER_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_DEVICE_EVENTCALLBACKREGISTER_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_DEVICE_EVENTCALLBACKUNREGISTER_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_REMOTEDEVICEDIRECTHWREADREG_STRUCT_VERSION

1

Structure layout version 1.

KYVPEXTENSION_REMOTEDEVICEDIRECTHWWRITE_REG_STRUCT_VERSION

1

Structure layout version 1.

KYVPEXTENSION_REMOTEDEVICE_SETIMAGEDetails_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_PCIINTERFACEDIRECTHWREADREG_STRUCT_VERSION

1

Structure layout version 1.

KYVPEXTENSION_PCIINTERFACEDIRECTHWWRITEREG_STRUCT_VERSION

1

Structure layout version 1.

KYVPEXTENSION_PCIINTERFACELOCKLINK_STRUCT_VERSION

1

Structure layout version 1.

KYVPEXTENSION_PCIINTERFACEUNLOCKLINK_STRUCT_VERSION

1

Structure layout version 1.

KYVPEXTENSION_PCIINTERFACEREADLINK_STRUCT_VERSION

1

Structure layout version 1.

KYVPEXTENSION_PCIINTERFACEWRITELINK_STRUCT_VERSION

1

Structure layout version 1.

KYVPEXTENSION_PCIINTERFACESENDLINKEVENT_STRUCT_VERSION

1

Structure layout version 1.

KYVPEXTENSION_PCIINTERFACELINKGETSTATUS_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_AUX_DATA_CALLBACK_REGISTER_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_AUX_DATA_CALLBACK_UNREGISTER_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_DATASTREAMBUFFERCHUNK_STRUCT_VERSION

1

Structure layout version 1.

KYVPEXTENSION_DATASTREAMCREATE_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_DATASTREAMANNOUNCEBUFFERCHUNKS_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_DATASTREAM_CLOSE_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_DATASTREAMCALLBACK_REGISTER_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_DATASTREAMCALLBACKUNREGISTER_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_SERIALCALLBACK_REGISTER_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_SERIALCALLBACKUNREGISTER_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_SERIAL_IPCSERVER_START_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_SERIAL_IPCSERVER_STOP_ARGS_VERSION

1

Structure layout version 1.

KYVPEXTENSION_REMOTEDEVICE_GETPARAMSCOLLECTIONHANDLE_ARGS

1

Structure layout version 1.

KYVPEXTENSION_DEVICELINKDEFINITION_VERSION

1

Structure layout version 1.

KYVPEXTENSION_DEVICEDEFINITION_VERSION

1

Structure layout version 1.

KYVPEXTENSION_DETECTDEVICESMANUALLY_ARGS

3

Structure layout version 3.

Type definitions

KYVP_UPDATE_CALLBACK

```
typedef KY_BOOL( * KYVP_UPDATE_CALLBACK) (const KYVP_UPDATE_STATUS *UpdateStatus, void *context))(const KYVP_UPDATE_STATUS *UpdateStatus, void *context)
```

Receive firmware-update progress.

Parameter	Description
UpdateStatus	Non-NULL progress snapshot; valid for the callback duration.
context	Application context supplied with the update request; may be NULL.

Returns: Reserved Boolean result. Current firmware backends ignore this value; it does not cancel the update.

See: KYVP_UPDATE_STATUS KYVPExtension_PCIInterface_FirmwareUpdate

KYVP_DATA_STREAM_CALLBACK

```
typedef void( * KYVP_DATA_STREAM_CALLBACK) (KYVP_BUFFER_HANDLE _phBufferHandle, void *pContext))(KYVP_BUFFER_HANDLE _phBufferHandle, void *_pContext)
```

Receive a data-stream buffer notification.

Parameter	Description
_phBufferHandle	Handle of the buffer associated with the notification.
_pContext	Application context supplied at registration; may be NULL.

See: KYVPExtension_DataStream_Callback_Register

KYVP_AUX_DATA_CALLBACK

```
typedef void(* KYVP_AUX_DATA_CALLBACK) (const KYVP_AUX_DATA *_pData, void *_pContext))(const KYVP_AUX_DATA *_pData, void *_pContext)
```

Receive auxiliary data generated by a PCI interface.

Interpret the data according to its context and message identifier. Copy any payload needed after the callback returns.

Parameter	Description
_pData	Auxiliary-data record; valid for the callback duration.
_pContext	Application context supplied at registration; may be NULL.

See: KYVPExtension_PCIInterface_AuxDataCallback_Register

KYVP_SERIAL_CALLBACK

```
typedef void(* KYVP_SERIAL_CALLBACK) (uint32_t _uChannel, uint16_t _uEventId, const void *_pData, uint32_t _uSize, void *_pContext))(uint32_t _uChannel, uint16_t _uEventId, const void *_pData, uint32_t _uSize, void *_pContext)
```

Receive a serial-port event and its payload.

Parameter	Description
_uChannel	Serial channel reporting the event.

<code>_uEventId</code>	Transport-specific serial event identifier.
<code>_pData</code>	Event payload; interpret according to the event identifier. Valid during the callback.
<code>_uSize</code>	Payload size in bytes.
<code>_pContext</code>	Application context supplied at registration; may be NULL.

See: `KYVPExtension_Serial_Callback_Register`

KYVP_PCI_INTERFACE_EVENT_CALLBACK

```
typedef void(* KYVP_PCI_INTERFACE_EVENT_CALLBACK) (KYVP_PCIINTERFACE_GENERIC_EVENT *_pGenericEventData, void
*_pUserContext))(KYVP_PCIINTERFACE_GENERIC_EVENT *_pGenericEventData, void *_pUserContext)
```

Receive a PCI interface event.

Parameter	Description
<code>_pGenericEventData</code>	Event record; use its event identifier to interpret the payload. Valid during the callback.
<code>_pUserContext</code>	Application context supplied at registration; may be NULL.

See: `KYVPExtension_PCIIInterface_EventCallBackRegister`

KYVP_DEVICE_EVENT_CALLBACK

```
typedef void(* KYVP_DEVICE_EVENT_CALLBACK) (KYVP_DEVICE_GENERIC_EVENT *_pGenericEventData, void
*_pUserContext))(KYVP_DEVICE_GENERIC_EVENT *_pGenericEventData, void *_pUserContext)
```

Receive a device event.

Parameter	Description
<code>_pGenericEventData</code>	Event record; use its event identifier to interpret the payload. Valid during the callback.
<code>_pUserContext</code>	Application context supplied at registration; may be NULL.

See: `KYVPExtension_Device_EventCallBackRegister`

GENTL_DS_HANDLE

```
typedef DS_HANDLE GENTL_DS_HANDLE
```

GenTL data-stream handle used by the experimental diagnostics API.

GENTL_BUFFER_HANDLE

```
typedef BUFFER_HANDLE GENTL_BUFFER_HANDLE
```

GenTL buffer handle used by the experimental diagnostics API.

Functions

KYVPExtension_PCIIInterface_FirmwareUpdate

```
KY_RESULT KYVPExtension_PCIIInterface_FirmwareUpdate(KYVPExtension_PCIIInterface_FirmwareUpdate_Args *_pArgs)
```

Update PCI interface firmware from a file.

The optional callback reports write and validation progress. Its return value is ignored by the current firmware backends.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_PCIIInterface_FirmwareUpdate_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

KYVPEExtension_PCIIInterface_CheckUpdateFile

KY_RESULT KYVPEExtension_PCIIInterface_CheckUpdateFile(KYVPEExtension_PCIIInterface_CheckUpdateFile_Args *_pArgs)
Check a firmware update file and return file and installed-flash metadata.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_PCIIInterface_CheckUpdateFile_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

KYVPEExtension_PCIIInterface_EventCallBackRegister

KY_RESULT KYVPEExtension_PCIIInterface_EventCallBackRegister(KYVPEExtension_PCIIInterface_EventCallBackRegister_Args *_pArgs)
Register a callback for PCI interface events.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_PCIIInterface_EventCallBackRegister_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_PCIIInterface_EventCallBackRegister_V1

KYVPEExtension_PCIIInterface_EventCallBackUnregister

KY_RESULT
KYVPEExtension_PCIIInterface_EventCallBackUnregister(KYVPEExtension_PCIIInterface_EventCallBackUnregister_Args *_pArgs)
Remove a PCI interface event callback registration.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_PCIIInterface_EventCallBackUnregister_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_PCIIInterface_EventCallBackUnregister_V1

KYVPEExtension_PCIIInterface_DirectHWReadReg

KY_RESULT KYVPEExtension_PCIIInterface_DirectHWReadReg(KYVPEExtension_PCIIInterface_DirectHWReadReg_Args *_pArgs)
Read PCI interface registers through direct hardware access.

BAR 0 is the normal access target. Access to other BARs is denied unless enabled by the SDK configuration.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_PCIIInterface_DirectHWReadReg_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_PCIIInterface_DirectHWReadReg_V1

KYVPEExtension_PCIIInterface_DirectHWWriteReg

KY_RESULT KYVPEExtension_PCIIInterface_DirectHWWriteReg(KYVPEExtension_PCIIInterface_DirectHWWriteReg_Args *_pArgs)
Write PCI interface registers through direct hardware access.

BAR 0 is the normal access target. Access to other BARs is denied unless enabled by the SDK configuration.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_PCIIInterface_DirectHWWriteReg_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_PCIIInterface_DirectHWWriteReg_V1

KYVPEExtension_PCIIInterface_LockLink

KY_RESULT KYVPEExtension_PCIIInterface_LockLink(KYVPEExtension_PCIIInterface_LockLink_Args *_pArgs)
Lock a PCI interface link for exclusive access.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_PCIIInterface_LockLink_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_PCIIInterface_LockLink_V1

KYVPEExtension_PCIIInterface_UnlockLink

KY_RESULT KYVPEExtension_PCIIInterface_UnlockLink(KYVPEExtension_PCIIInterface_UnlockLink_Args *_pArgs)
Release a PCI interface link lock.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_PCIIInterface_UnlockLink_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_PCIIInterface_UnlockLink_V1

KYVPEExtension_PCIIInterface_ReadLink

KY_RESULT KYVPEExtension_PCIIInterface_ReadLink(KYVPEExtension_PCIIInterface_ReadLink_Args *_pArgs)
Read registers through the selected PCI interface link.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_PCIIInterface_ReadLink_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_PCIIInterface_ReadLink_V1

KYVPEExtension_PCIIInterface_WriteLink

KY_RESULT KYVPEExtension_PCIIInterface_WriteLink(KYVPEExtension_PCIIInterface_WriteLink_Args *_pArgs)
Write registers through the selected PCI interface link.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_PCIIInterface_WriteLink_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_PCIIInterface_WriteLink_V1

KYVPEExtension_PCIIInterface_SendLinkEvent

KY_RESULT KYVPEExtension_PCIIInterface_SendLinkEvent(KYVPEExtension_PCIIInterface_SendLinkEvent_Args *_pArgs)
Send an event through the selected PCI interface link.

Support depends on the link transport; an unsupported transport returns an error.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_PCIIInterface_SendLinkEvent_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

KYVPEExtension_PCIIInterface_GetLinkStatus

KY_RESULT KYVPEExtension_PCIIInterface_GetLinkStatus(KYVPEExtension_PCIIInterface_LinkGetStatus_Args *_pArgs)
Query connection or lock status for a PCI interface link.

Connection status is returned as int32_t; lock status is returned as KY_BOOL. Set the buffer to NULL to query the required size.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_PCIIInterface_LinkGetStatus_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_PCIIInterface_GetLinkStatus_V1

KYVPEExtension_PCIIInterface_AuxDataCallback_Register

KY_RESULT KYVPEExtension_PCIIInterface_AuxDataCallback_Register(KYVPEExtension_AuxDataCallback_Register_Args *_pArgs)
Register a callback for auxiliary data from a PCI interface.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_AuxDataCallback_Register_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_PCIIInterface_AuxDataCallback_Register_V1

KYVPEExtension_PCIIInterface_AuxDataCallback_Unregister

KY_RESULT KYVPEExtension_PCIIInterface_AuxDataCallback_Unregister(KYVPEExtension_AuxDataCallback_Unregister_Args *_pArgs)

Remove a PCI interface auxiliary-data callback registration.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_AuxDataCallback_Unregister_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_PCIIInterface_AuxDataCallback_Unregister_V1

KYVPEExtension_Device_EventCallBackRegister

KY_RESULT KYVPEExtension_Device_EventCallBackRegister(KYVPEExtension_Device_EventCallBackRegister_Args *_pArgs)

Register a callback for device events.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_Device_EventCallBackRegister_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_Device_EventCallBackRegister_V1

KYVPEExtension_Device_EventCallBackUnregister

KY_RESULT KYVPEExtension_Device_EventCallBackUnregister(KYVPEExtension_Device_EventCallBackUnregister_Args *_pArgs)

Remove a device event callback registration.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPEExtension_Device_EventCallBackUnregister_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_Device_EventCallBackUnregister_V1

KYVPEExtension_RemoteDevice_DirectHWReadReg

KY_RESULT KYVPEExtension_RemoteDevice_DirectHWReadReg(KYVPEExtension_RemoteDevice_DirectHWReadReg_Args *_pArgs)

Read remote-device registers through direct hardware access.

Parameter	Description
-----------	-------------

_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPExtension_RemoteDevice_DirectHWReadReg_Args.
--------	--

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_DirectHWReadReg_V1

KYVPExtension_RemoteDevice_DirectHWWriteReg

KY_RESULT KYVPExtension_RemoteDevice_DirectHWWriteReg(KYVPExtension_RemoteDevice_DirectHWWriteReg_Args *_pArgs)

Write remote-device registers through direct hardware access.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPExtension_RemoteDevice_DirectHWWriteReg_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_DirectHWWriteReg_V1

KYVPExtension_RemoteDevice_SetImageDetails

KY_RESULT KYVPExtension_RemoteDevice_SetImageDetails(KYVPExtension_RemoteDevice_SetImageDetails_Args *_pArgs)

Set the remote-device image geometry and pixel format used by the SDK.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPExtension_RemoteDevice_SetImageDetails_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_SetImageDetails_V1

KYVPExtension_RemoteDevice_GetParamsCollectionHandle

KY_RESULT

KYVPExtension_RemoteDevice_GetParamsCollectionHandle(KYVPExtension_RemoteDevice_GetParamsCollectionHandle_Args *_pArgs)

Retrieve the parameter collection associated with a remote device.

The collection belongs to the remote device. Use the parameter-handler API to access it; do not delete it as an application-created collection.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPExtension_RemoteDevice_GetParamsCollectionHandle_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_GetParamsCollectionHandle_V1

KYVPExtension_DataStream_Create

KY_RESULT KYVPExtension_DataStream_Create(KYVPExtension_DataStream_Create_Args *_pArgs)

Open a device data stream with the selected buffer-management mode.

The stream identifier must identify a device stream known to the initialized SDK. Close the returned handle with `KYVPExtension_DataStream_Close()`.

Parameter	Description
<code>_pArgs</code>	Non-NULL argument block; initialize its version and fields as described in <code>KYVPExtension_DataStream_Create_Args</code> .

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_DataStream_Create_V1`

KYVPExtension_DataStream_Close

`KY_RESULT KYVPExtension_DataStream_Close(KYVPExtension_DataStream_Close_Args *_pArgs)`

Close and release a data stream.

The stream handle becomes invalid on success.

Parameter	Description
<code>_pArgs</code>	Non-NULL argument block; initialize its version and fields as described in <code>KYVPExtension_DataStream_Close_Args</code> .

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_DataStream_Close_V1`

KYVPExtension_DSAnnounceBufferChunks

`KY_RESULT KYVPExtension_DSAnnounceBufferChunks(KYVPExtension_DSAnnounceBufferChunks_Args *_pArgs)`

Announce a buffer backed by multiple application-allocated memory chunks.

The chunk memory belongs to the application and must remain valid while the announced buffer is in use. Revoke the buffer before releasing its memory.

Parameter	Description
<code>_pArgs</code>	Non-NULL argument block; initialize its version and fields as described in <code>KYVPExtension_DSAnnounceBufferChunks_Args</code> .

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_DSAnnounceBufferChunks_V1`

KYVPExtension_DataStream_Callback_Register

`KY_RESULT KYVPExtension_DataStream_Callback_Register(KYVPExtension_DataStream_Callback_Register_Args *_pArgs)`

Register a callback for data-stream buffer notifications.

Parameter	Description
<code>_pArgs</code>	Non-NULL argument block; initialize its version and fields as described in <code>KYVPExtension_DataStream_Callback_Register_Args</code> .

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_DataStream_Callback_Register_V1`

KYVPExtension_DataStream_Callback_Unregister

KY_RESULT KYVPExtension_DataStream_Callback_Unregister(KYVPExtension_DataStream_Callback_Unregister_Args *_pArgs)
Remove a data-stream callback registration.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPExtension_DataStream_Callback_Unregister_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DataStream_Callback_Unregister_V1

KYVPExtension_Serial_Callback_Register

KY_RESULT KYVPExtension_Serial_Callback_Register(KYVPExtension_Serial_Callback_Register_Args *_pArgs)
Register a serial-event callback for a PCI interface link.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPExtension_Serial_Callback_Register_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_Callback_Register_V1

KYVPExtension_Serial_Callback_Unregister

KY_RESULT KYVPExtension_Serial_Callback_Unregister(KYVPExtension_Serial_Callback_Unregister_Args *_pArgs)
Request removal of a serial-event callback registration.

Warning: The current implementation returns success without removing the registered callback. Do not use this call as confirmation that callback delivery has stopped.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPExtension_Serial_Callback_Unregister_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_Callback_Unregister_V1

KYVPExtension_Serial_IPCServer_Start

KY_RESULT KYVPExtension_Serial_IPCServer_Start(KYVPExtension_Serial_IPCServer_Start_Args *_pArgs)
Start the serial-port IPC server in the calling process.

Starting an already running server succeeds without creating another instance.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPExtension_Serial_IPCServer_Start_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_IPCServer_Start_V1

KYVPExtension_Serial_IPCServer_Stop

KY_RESULT KYVPExtension_Serial_IPCServer_Stop(KYVPExtension_Serial_IPCServer_Stop_Args *_pArgs)

Stop the serial-port IPC server in the calling process.

Stopping an already stopped server succeeds.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPExtension_Serial_IPCServer_Stop_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_IPCServer_Stop_V1

KYVPExtension_DetectDevicesManually

KY_RESULT KYVPExtension_DetectDevicesManually(KYVPExtension_DetectDevicesManually_Args *_pArgs)

Detect devices using an application-supplied link topology.

Each device definition supplies its expected speed and a device-link-to-host-link mapping. Detection writes the found flag and device ID into each definition. Version 1 verifies link IDs and retains open devices. Version 2 adds optional link-ID verification suppression. Version 3 adds control over retaining open devices. With bRetainOpenDevices set to KY_FALSE, this process closes its devices and clears its device lists before fresh detection; existing handles and streams must be discarded. Devices opened by other processes are preserved in both modes.

Parameter	Description
_pArgs	Non-NULL argument block; initialize its version and fields as described in KYVPExtension_DetectDevicesManually_Args.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DetectDevicesManually_V3

KYVPExtension_PCIIInterface_DirectHWReadReg_V1

KY_RESULT KYVPExtension_PCIIInterface_DirectHWReadReg_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iBar, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)

Read PCI interface registers through direct hardware access.

BAR 0 is the normal access target. Access to other BARs is denied unless enabled by the SDK configuration. Convenience entry point for KYVPExtension_PCIIInterface_DirectHWReadReg() with explicit parameters.

Parameter	Description
_hIFHandle	PCI interface to operate on. Valid interface handle.
_iBar	PCI Base Address Register index. Normally 0.
_iAddress	Starting register address in the selected address space.
_pBuffer	Destination buffer for register data. Non-NULL for a nonzero transfer.

_piSize	Requested transfer size in bytes; may be updated by the transport. Non-NULL; buffer must cover the requested size.
---------	--

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIIInterface_DirectHWReadReg

KYVPExtension_PCIIInterface_DirectHWWriteReg_V1

KY_RESULT KYVPExtension_PCIIInterface_DirectHWWriteReg_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iBar, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)

Write PCI interface registers through direct hardware access.

BAR 0 is the normal access target. Access to other BARs is denied unless enabled by the SDK configuration.
Convenience entry point for KYVPExtension_PCIIInterface_DirectHWWriteReg() with explicit parameters.

Parameter	Description
_hIFHandle	PCI interface to operate on. Valid interface handle.
_iBar	PCI Base Address Register index. Normally 0.
_iAddress	Starting register address in the selected address space.
_pBuffer	Source buffer containing bytes to write or send. Non-NULL for a nonzero transfer.
_piSize	Requested transfer size in bytes; may be updated by the transport. Non-NULL; buffer must cover the requested size.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIIInterface_DirectHWWriteReg

KYVPExtension_PCIIInterface_LockLink_V1

KY_RESULT KYVPExtension_PCIIInterface_LockLink_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink)

Lock a PCI interface link for exclusive access.

Convenience entry point for KYVPExtension_PCIIInterface_LockLink() with explicit parameters.

Parameter	Description
_hIFHandle	PCI interface to operate on. Valid interface handle.
_iLink	Zero-based host link index on the PCI interface.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIIInterface_LockLink

KYVPExtension_PCIIInterface_UnlockLink_V1

KY_RESULT KYVPExtension_PCIIInterface_UnlockLink_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink)

Release a PCI interface link lock.

Convenience entry point for KYVPExtension_PCIIInterface_UnlockLink() with explicit parameters.

Parameter	Description
-----------	-------------

_hIFHandle	PCI interface to operate on. Valid interface handle.
_iLink	Zero-based host link index on the PCI interface.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIIInterface_UnlockLink

KYVPExtension_PCIIInterface_ReadLink_V1

KY_RESULT KYVPExtension_PCIIInterface_ReadLink_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)

Read registers through the selected PCI interface link.

Convenience entry point for KYVPExtension_PCIIInterface_ReadLink() with explicit parameters.

Parameter	Description
_hIFHandle	PCI interface to operate on. Valid interface handle.
_iLink	Zero-based host link index on the PCI interface.
_iAddress	Starting register address in the selected address space.
_pBuffer	Destination buffer for register data. Non-NULL for a nonzero transfer.
_piSize	Requested transfer size in bytes; may be updated by the transport. Non-NULL; buffer must cover the requested size.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIIInterface_ReadLink

KYVPExtension_PCIIInterface_WriteLink_V1

KY_RESULT KYVPExtension_PCIIInterface_WriteLink_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)

Write registers through the selected PCI interface link.

Convenience entry point for KYVPExtension_PCIIInterface_WriteLink() with explicit parameters.

Parameter	Description
_hIFHandle	PCI interface to operate on. Valid interface handle.
_iLink	Zero-based host link index on the PCI interface.
_iAddress	Starting register address in the selected address space.
_pBuffer	Source buffer containing bytes to write or send. Non-NULL for a nonzero transfer.
_piSize	Requested transfer size in bytes; may be updated by the transport. Non-NULL; buffer must cover the requested size.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIIInterface_WriteLink

KYVPExtension_PCIInterface_GetLinkStatus_V1

KY_RESULT KYVPExtension_PCIInterface_GetLinkStatus_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink, KYVP_LINK_STATUS_CMD _iInfoCmd, KYVP_INFO_DATATYPE *_piType, void *_pBuffer, size_t *_piSize)

Query connection or lock status for a PCI interface link.

Connection status is returned as int32_t; lock status is returned as KY_BOOL. Set the buffer to NULL to query the required size. Convenience entry point for KYVPExtension_PCIInterface_GetLinkStatus() with explicit parameters.

Parameter	Description
_hIFHandle	PCI interface to operate on. Valid interface handle.
_iLink	Zero-based host link index on the PCI interface.
_iInfoCmd	Link-status property to query.
_piType	Receives the data type of the returned status value. Non-NULL, including size-only queries.
_pBuffer	Receives the selected link-status value. NULL queries the required size without copying data.
_piSize	Buffer capacity on input; required value size on output, in bytes. Non-NULL.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_GetLinkStatus

KYVPExtension_PCIInterface_AuxDataCallback_Register_V1

KY_RESULT KYVPExtension_PCIInterface_AuxDataCallback_Register_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KYVP_AUX_DATA_CALLBACK _pCallbackFunction, void *_pUserContext)

Register a callback for auxiliary data from a PCI interface.

Convenience entry point for KYVPExtension_PCIInterface_AuxDataCallback_Register() with explicit parameters.

Parameter	Description
_hIFHandle	PCI interface to operate on. Valid interface handle.
_pCallbackFunction	Callback function to invoke. Non-NULL.
_pUserContext	Application context passed back to the callback. Optional, may be NULL.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_PCIInterface_AuxDataCallback_Register

KYVPExtension_PCIInterface_AuxDataCallback_Unregister_V1

KY_RESULT KYVPExtension_PCIInterface_AuxDataCallback_Unregister_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KYVP_AUX_DATA_CALLBACK _pCallbackFunction)

Remove a PCI interface auxiliary-data callback registration.

Convenience entry point for KYVPExtension_PCIInterface_AuxDataCallback_Unregister() with explicit parameters.

Parameter	Description
_hIFHandle	PCI interface to operate on. Valid interface handle.

<code>_pCallbackFunction</code>	Callback function to remove. Non-NULL; pass the registered function.
---------------------------------	--

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_PCIIInterface_AuxDataCallback_Unregister`

KYVPExtension_PCIIInterface_EventCallbackRegister_V1

`KY_RESULT KYVPExtension_PCIIInterface_EventCallbackRegister_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KYVP_PCI_INTERFACE_EVENT_CALLBACK _pUserFunc, void *_pUserContext)`

Register a callback for PCI interface events.

Convenience entry point for `KYVPExtension_PCIIInterface_EventCallbackRegister()` with explicit parameters.

Parameter	Description
<code>_hIFHandle</code>	PCI interface to operate on. Valid interface handle.
<code>_pUserFunc</code>	Callback function to invoke. Non-NULL.
<code>_pUserContext</code>	Application context passed back to the callback. Optional, may be NULL.

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_PCIIInterface_EventCallbackRegister`

KYVPExtension_PCIIInterface_EventCallbackUnregister_V1

`KY_RESULT KYVPExtension_PCIIInterface_EventCallbackUnregister_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KYVP_PCI_INTERFACE_EVENT_CALLBACK _pUserFunc)`

Remove a PCI interface event callback registration.

Convenience entry point for `KYVPExtension_PCIIInterface_EventCallbackUnregister()` with explicit parameters.

Parameter	Description
<code>_hIFHandle</code>	PCI interface to operate on. Valid interface handle.
<code>_pUserFunc</code>	Callback function to remove. Non-NULL; pass the registered function.

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_PCIIInterface_EventCallbackUnregister`

KYVPExtension_Device_EventCallbackRegister_V1

`KY_RESULT KYVPExtension_Device_EventCallbackRegister_V1(KYVP_DEVICE_HANDLE _hDevHandle, KYVP_DEVICE_EVENT_CALLBACK _pUserFunc, void *_pUserContext)`

Register a callback for device events.

Convenience entry point for `KYVPExtension_Device_EventCallbackRegister()` with explicit parameters.

Parameter	Description
<code>_hDevHandle</code>	Device to receive event notifications from. Valid device handle.
<code>_pUserFunc</code>	Callback function to invoke. Non-NULL.
<code>_pUserContext</code>	Application context passed back to the callback. Optional, may be NULL.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Device_EventCallbackRegister

KYVPExtension_Device_EventCallbackUnregister_V1

KY_RESULT KYVPExtension_Device_EventCallbackUnregister_V1(KYVP_DEVICE_HANDLE _hDevHandle, KYVP_DEVICE_EVENT_CALLBACK _pUserFunc)

Remove a device event callback registration.

Convenience entry point for KYVPExtension_Device_EventCallbackUnregister() with explicit parameters.

Parameter	Description
_hDevHandle	Device to receive event notifications from. Valid device handle.
_pUserFunc	Callback function to remove. Non-NULL; pass the registered function.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Device_EventCallbackUnregister

KYVPExtension_RemoteDevice_DirectHWReadReg_V1

KY_RESULT KYVPExtension_RemoteDevice_DirectHWReadReg_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)

Read remote-device registers through direct hardware access.

Convenience entry point for KYVPExtension_RemoteDevice_DirectHWReadReg() with explicit parameters.

Parameter	Description
_hRemoteDeviceHandle	Remote device to operate on. Valid remote-device handle.
_iAddress	Starting register address in the selected address space.
_pBuffer	Destination buffer for register data. Non-NULL for a nonzero transfer.
_piSize	Requested transfer size in bytes; may be updated by the transport. Non-NULL; buffer must cover the requested size.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_RemoteDevice_DirectHWReadReg

KYVPExtension_RemoteDevice_DirectHWWriteReg_V1

KY_RESULT KYVPExtension_RemoteDevice_DirectHWWriteReg_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)

Write remote-device registers through direct hardware access.

Convenience entry point for KYVPExtension_RemoteDevice_DirectHWWriteReg() with explicit parameters.

Parameter	Description
_hRemoteDeviceHandle	Remote device to operate on. Valid remote-device handle.

<code>_iAddress</code>	Starting register address in the selected address space.
<code>_pBuffer</code>	Source buffer containing bytes to write or send. Non-NULL for a nonzero transfer.
<code>_piSize</code>	Requested transfer size in bytes; may be updated by the transport. Non-NULL; buffer must cover the requested size.

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_RemoteDevice_DirectHWWriteReg`

KYVPExtension_RemoteDevice_SetImageDetails_V1

`KY_RESULT KYVPExtension_RemoteDevice_SetImageDetails_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle, KYVP_DEVICE_IMAGE_DETAILS _imageDetails)`

Set the remote-device image geometry and pixel format used by the SDK.

Convenience entry point for `KYVPExtension_RemoteDevice_SetImageDetails()` with explicit parameters.

Parameter	Description
<code>_hRemoteDeviceHandle</code>	Remote device to operate on. Valid remote-device handle.
<code>_imageDetails</code>	Image dimensions and pixel format to apply. Initialize the nested structure version and fields.

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_RemoteDevice_SetImageDetails`

KYVPExtension_RemoteDevice_GetParamsCollectionHandle_V1

`KY_RESULT KYVPExtension_RemoteDevice_GetParamsCollectionHandle_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle, KYVP_COLLECTION_HANDLE *_phParamsCollectionHandle)`

Retrieve the parameter collection associated with a remote device.

The collection belongs to the remote device. Use the parameter-handler API to access it; do not delete it as an application-created collection. Convenience entry point for

`KYVPExtension_RemoteDevice_GetParamsCollectionHandle()` with explicit parameters.

Parameter	Description
<code>_hRemoteDeviceHandle</code>	Remote device to operate on. Valid remote-device handle.
<code>_phParamsCollectionHandle</code>	Receives the remote-device parameter collection handle. Non-NULL; owned by the remote device.

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_RemoteDevice_GetParamsCollectionHandle`

KYVPExtension_DataStream_Create_V1

`KY_RESULT KYVPExtension_DataStream_Create_V1(const char *_pszDevDataStreamID, KYVP_STREAM_BUFFERS_TYPE _eStreamBufferType, KYVP_STREAM_HANDLE *_phDSHandle)`

Open a device data stream with the selected buffer-management mode.

The stream identifier must identify a device stream known to the initialized SDK. Close the returned handle with `KYVPExtension_DataStream_Close()`. Convenience entry point for `KYVPExtension_DataStream_Create()` with explicit parameters.

Parameter	Description
<code>_pszDevDataStreamID</code>	Identifier of the device data stream to open. Non-NULL, null-terminated string.
<code>_eStreamBufferType</code>	Buffer-management mode for the stream.
<code>_phDSHandle</code>	Receives the newly opened data-stream handle. Non-NULL.

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_DataStream_Create`

KYVPExtension_DataStream_Close_V1

`KY_RESULT KYVPExtension_DataStream_Close_V1(KYVP_STREAM_HANDLE _hDSHandle)`

Close and release a data stream.

The stream handle becomes invalid on success. Convenience entry point for `KYVPExtension_DataStream_Close()` with explicit parameters.

Parameter	Description
<code>_hDSHandle</code>	Data stream to operate on. Valid stream handle.

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_DataStream_Close`

KYVPExtension_DSAnnounceBufferChunks_V1

`KY_RESULT KYVPExtension_DSAnnounceBufferChunks_V1(KYVP_STREAM_HANDLE _hDSHandle, const KYVPExtension_DataStreamBufferChunk *_pChunks, size_t _iNumChunks, void *_pPrivate, KYVP_BUFFER_HANDLE *_phBuffer)`
Announce a buffer backed by multiple application-allocated memory chunks.

The chunk memory belongs to the application and must remain valid while the announced buffer is in use. Revoke the buffer before releasing its memory. Convenience entry point for `KYVPExtension_DSAnnounceBufferChunks()` with explicit parameters.

Parameter	Description
<code>_hDSHandle</code>	Data stream to operate on. Valid stream handle.
<code>_pChunks</code>	Array describing the memory chunks that make up one buffer. Non-NULL, with <code>_iNumChunks</code> entries.
<code>_iNumChunks</code>	Number of entries in <code>_pChunks</code> . Greater than zero.
<code>_pPrivate</code>	Application data associated with the announced buffer. Optional, may be NULL.
<code>_phBuffer</code>	Receives the announced buffer handle. Optional, may be NULL.

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPExtension_DSAnnounceBufferChunks`

KYVPExtension_DataStream_Callback_Register_V1

KY_RESULT KYVPExtension_DataStream_Callback_Register_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_DATA_STREAM_CALLBACK _pCallbackFunction, void *_pContext)

Register a callback for data-stream buffer notifications.

Convenience entry point for KYVPExtension_DataStream_Callback_Register() with explicit parameters.

Parameter	Description
_hDSHandle	Data stream to operate on. Valid stream handle.
_pCallbackFunction	Callback function to invoke. Non-NULL.
_pContext	Application context passed back to the callback. Optional, may be NULL.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DataStream_Callback_Register

KYVPExtension_DataStream_Callback_Unregister_V1

KY_RESULT KYVPExtension_DataStream_Callback_Unregister_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_DATA_STREAM_CALLBACK _pCallbackFunction)

Remove a data-stream callback registration.

Convenience entry point for KYVPExtension_DataStream_Callback_Unregister() with explicit parameters.

Parameter	Description
_hDSHandle	Data stream to operate on. Valid stream handle.
_pCallbackFunction	Callback function to remove. Non-NULL; pass the registered function.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DataStream_Callback_Unregister

KYVPExtension_Serial_Callback_Register_V1

KY_RESULT KYVPExtension_Serial_Callback_Register_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink, KYVP_SERIAL_CALLBACK _pCallbackFunction, void *_pContext)

Register a serial-event callback for a PCI interface link.

Convenience entry point for KYVPExtension_Serial_Callback_Register() with explicit parameters.

Parameter	Description
_hIFHandle	PCI interface to operate on. Valid interface handle.
_iLink	Zero-based host link index on the PCI interface.
_pCallbackFunction	Callback function to invoke. Non-NULL.
_pContext	Application context passed back to the callback. Optional, may be NULL.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_Serial_Callback_Register

KYVPEExtension_Serial_Callback_Unregister_V1

KY_RESULT KYVPEExtension_Serial_Callback_Unregister_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iLink, KYVP_SERIAL_CALLBACK _pCallbackFunction)

Request removal of a serial-event callback registration.

Warning: The current implementation returns success without removing the registered callback. Do not use this call as confirmation that callback delivery has stopped. Convenience entry point for KYVPEExtension_Serial_Callback_Unregister() with explicit parameters.

Parameter	Description
_hIFHandle	PCI interface to operate on. Valid interface handle.
_iLink	Zero-based host link index on the PCI interface.
_pCallbackFunction	Callback function to remove. Non-NULL; pass the registered function.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_Serial_Callback_Unregister

KYVPEExtension_Serial_IPCServer_Start_V1

KY_RESULT KYVPEExtension_Serial_IPCServer_Start_V1()

Start the serial-port IPC server in the calling process.

Starting an already running server succeeds without creating another instance. Convenience entry point for KYVPEExtension_Serial_IPCServer_Start(). The argument version is supplied internally.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_Serial_IPCServer_Start

KYVPEExtension_Serial_IPCServer_Stop_V1

KY_RESULT KYVPEExtension_Serial_IPCServer_Stop_V1()

Stop the serial-port IPC server in the calling process.

Stopping an already stopped server succeeds. Convenience entry point for KYVPEExtension_Serial_IPCServer_Stop(). The argument version is supplied internally.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPEExtension_Serial_IPCServer_Stop

KYVPEExtension_DetectDevicesManually_V1

KY_RESULT KYVPEExtension_DetectDevicesManually_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KY_BOOL _bNoDevicesAccess, KY_BOOL *_pbChanged, DeviceDefinition *_pDeviceDefinitionArray, size_t _uDeviceDefinitionArrayCount)

Detect devices using an application-supplied link topology.

Each device definition supplies its expected speed and a device-link-to-host-link mapping. Detection writes the found flag and device ID into each definition. Convenience entry point for KYVPEExtension_DetectDevicesManually() with explicit parameters. This entry point verifies link IDs and retains open devices.

Parameter	Description
_hIFHandle	PCI interface to operate on. Valid interface handle.
_bNoDevicesAccess	Skip detection operations that require remote-device

	access. KY_TRUE enables this mode; SDK configuration can override it.
_pbChanged	Receives whether the device list changed. Optional, may be NULL.
_pDeviceDefinitionArray	Device topologies to detect, with detection results written back. Non-NULL array.
_uDeviceDefinitionArrayCount	Number of entries in _pDeviceDefinitionArray. Greater than zero.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DetectDevicesManually

KYVPExtension_DetectDevicesManually_V2

KY_RESULT KYVPExtension_DetectDevicesManually_V2(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KY_BOOL _bNoDevicesAccess, KY_BOOL _bSuppressLinkIDVerification, KY_BOOL *_pbChanged, DeviceDefinition *_pDeviceDefinitionArray, size_t _uDeviceDefinitionArrayCount)

Detect devices using an application-supplied link topology.

Each device definition supplies its expected speed and a device-link-to-host-link mapping. Detection writes the found flag and device ID into each definition. Convenience entry point for KYVPExtension_DetectDevicesManually() with explicit parameters. This entry point retains open devices.

Parameter	Description
_hIFHandle	PCI interface to operate on. Valid interface handle.
_bNoDevicesAccess	Skip detection operations that require remote-device access. KY_TRUE enables this mode; SDK configuration can override it.
_bSuppressLinkIDVerification	Allow detection without verifying the configured device link-ID order. Since version 2.
_pbChanged	Receives whether the device list changed. Optional, may be NULL.
_pDeviceDefinitionArray	Device topologies to detect, with detection results written back. Non-NULL array.
_uDeviceDefinitionArrayCount	Number of entries in _pDeviceDefinitionArray. Greater than zero.

Returns: SDK status; inspect with KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED().

See: KYVPExtension_DetectDevicesManually

KYVPExtension_DetectDevicesManually_V3

KY_RESULT KYVPExtension_DetectDevicesManually_V3(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KY_BOOL _bNoDevicesAccess, KY_BOOL _bSuppressLinkIDVerification, KY_BOOL *_pbChanged, DeviceDefinition *_pDeviceDefinitionArray, size_t _uDeviceDefinitionArrayCount, KY_BOOL _bRetainOpenDevices)

Detect devices using an application-supplied link topology.

Each device definition supplies its expected speed and a device-link-to-host-link mapping. Detection writes the found flag and device ID into each definition. With _bRetainOpenDevices set to KY_FALSE, this process closes its devices and clears its device lists before fresh detection; existing handles and streams must be discarded.

Devices opened by other processes are preserved in both modes. Convenience entry point for `KYVPEExtension_DetectDevicesManually()` with explicit parameters.

Parameter	Description
<code>_hIFHandle</code>	PCI interface to operate on. Valid interface handle.
<code>_bNoDevicesAccess</code>	Skip detection operations that require remote-device access. <code>KY_TRUE</code> enables this mode; SDK configuration can override it.
<code>_bSuppressLinkIDVerification</code>	Allow detection without verifying the configured device link-ID order. Since version 2.
<code>_pbChanged</code>	Receives whether the device list changed. Optional, may be NULL.
<code>_pDeviceDefinitionArray</code>	Device topologies to detect, with detection results written back. Non-NULL array.
<code>_uDeviceDefinitionArrayCount</code>	Number of entries in <code>_pDeviceDefinitionArray</code> . Greater than zero.
<code>_bRetainOpenDevices</code>	Preserve open devices; <code>KY_FALSE</code> closes this process's devices before fresh detection. Since version 3. Other processes' devices are preserved.

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

See: `KYVPEExtension_DetectDevicesManually`

KYVPEExtension_KYVPLibTL_DebugBuffer_V1

`KY_RESULT KYVPEExtension_KYVPLibTL_DebugBuffer_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_BUFFER_HANDLE _hBuffer)`
Log diagnostic details for a KYVPLibTL stream buffer.

Warning: Experimental API. Its interface and behavior may change without deprecation.

Parameter	Description
<code>_hDSHandle</code>	Data stream to operate on. Valid stream handle.
<code>_hBuffer</code>	Buffer handle belonging to the selected stream.

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

KYVPEExtension_GentTL_DebugBuffer_V1

`KY_RESULT KYVPEExtension_GentTL_DebugBuffer_V1(GENTL_DS_HANDLE _hDataStream, GENTL_BUFFER_HANDLE _hBuffer)`
Log diagnostic details for a buffer from the KAYA GenTL producer.

Warning: Experimental API. Its interface and behavior may change without deprecation.

Parameter	Description
<code>_hDataStream</code>	Data-stream handle from the KAYA GenTL producer.
<code>_hBuffer</code>	Buffer handle belonging to the selected stream.

Returns: SDK status; inspect with `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()`.

KYVPExtension Structs

Field reference for extension API records and argument structures.

Tables list fields in declaration order. Set each version to the documented constant before calling the API. C++ default initialization does not replace required handles, buffers, and callback pointers.

KYVP_UPDATE_STATUS

Progress snapshot supplied to the firmware-update callback.

The library supplies this structure. Read fields only for a supported struct_version; copy any data needed after the callback returns.

Structure Field	Type	Description	Remarks
struct_version	int	Firmware-progress structure layout version.	Output; currently 1.
bytes_sent	uint64_t	Progress byte count reported for the current firmware operation.	Output; interpret together with total_size and is_writing.
total_size	uint64_t	Total byte count used for progress reporting.	Output; firmware payload or phase size reported by the backend.
is_writing	KY_BOOL	Firmware operation phase.	Output; KY_TRUE for writing, KY_FALSE for validation.

KYVPExtension_PCInterface_FirmwareUpdate_Args

Arguments: Update PCI interface firmware from a file.

Initialize version to KYVPEXTENSION_PCINTERFACE_FIRMWAREUPDATE_ARGS_VERSION. The optional callback reports write and validation progress. Its return value is ignored by the current firmware backends.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
pszFilePath	char*	Path to the firmware update file.	Input; non-NULL, null-terminated string.
pCallbackFunction	KYVP_UPDATE_CALLBACK	Callback function to invoke.	Input; optional, may be NULL.
pUserContext	void*	Application context passed back to the callback.	Input; optional, may be NULL.

KYVPExtension_PCInterface_CheckUpdateFile_Args

Arguments: Check a firmware update file and return file and installed-flash metadata.

Initialize version to KYVPEXTENSION_PCIINTERFACE_CHECKUPDATEFILE_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
fileMinorRev	uint16_t	Minor revision from the update file.	Output; valid on success.
fileMajorRev	uint16_t	Major revision from the update file.	Output; valid on success.
fileVendorId	uint16_t	Vendor identifier from the update file.	Output; valid on success.
fileBoardId	uint16_t	Board identifier from the update file.	Output; valid on success.
fileVersion	uint32_t	Firmware version from the update file.	Output; valid on success.
fileTimeStamp	uint32_t	Firmware timestamp value from the update file.	Output; valid on success.
flashMinorRev	uint16_t	Minor revision from the installed flash.	Output; valid on success.
flashMajorRev	uint16_t	Major revision from the installed flash.	Output; valid on success.
flashVendorId	uint32_t	Vendor identifier from the installed flash.	Output; valid on success.
flashBoardId	uint32_t	Board identifier from the installed flash.	Output; valid on success.
flashVersion	uint32_t	Firmware version from the installed flash.	Output; valid on success.
flashTimeStamp	uint32_t	Firmware timestamp value from the installed flash.	Output; valid on success.
pszFilePath	char*	Path to the firmware update file.	Input; non-NULL, null-terminated string.

KYVPExtension_PCIInterface_EventCallbackRegister_Args

Arguments: Register a callback for PCI interface events.

Initialize version to KYVPEXTENSION_PCIINTERFACE_EVENTCALLBACKREGISTER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure	Input; set to 1.

		layout version.	
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
pUserFunc	KYVP_PCI_INTERFACE_EVENT_CALLBACK	Callback function to invoke.	Input; non-NULL.
pUserContext	void*	Application context passed back to the callback.	Input; optional, may be NULL.

KYVPExtension_PCIInterface_EventCallbackUnregister_Args

Arguments: Remove a PCI interface event callback registration.

Initialize version to KYVPEXTENSION_PCIINTERFACE_EVENTCALLBACKUNREGISTER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
pUserFunc	KYVP_PCI_INTERFACE_EVENT_CALLBACK	Callback function to remove.	Input; non-NULL; pass the registered function.

KYVPExtension_Device_EventCallbackRegister_Args

Arguments: Register a callback for device events.

Initialize version to KYVPEXTENSION_DEVICE_EVENTCALLBACKREGISTER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hDevHandle	KYVP_DEVICE_HANDLE	Device to receive event notifications from.	Input; valid device handle.
pUserFunc	KYVP_DEVICE_EVENT_CALLBACK	Callback function to invoke.	Input; non-NULL.
pUserContext	void*	Application context passed back to the callback.	Input; optional, may be NULL.

KYVPExtension_Device_EventCallbackUnregister_Args

Arguments: Remove a device event callback registration.

Initialize version to KYVPEXTENSION_DEVICE_EVENTCALLBACKUNREGISTER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
-----------------	------	-------------	---------

version	uint32_t	Argument structure layout version.	Input; set to 1.
hDevHandle	KYVP_DEVICE_HANDLE	Device to receive event notifications from.	Input; valid device handle.
pUserFunc	KYVP_DEVICE_EVENT_CALLBACK	Callback function to remove.	Input; non-NULL; pass the registered function.

KYVPExtension_RemoteDevice_DirectHWReadReg_Args

Arguments: Read remote-device registers through direct hardware access.

Initialize version to KYVPEXTENSION_REMOTEDIVIDIRECTHWREADREG_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hRemoteDeviceHandle	KYVP_REMOTE_DEVICE_HANDLE	Remote device to operate on.	Input; valid remote-device handle.
iAddress	uint64_t	Starting register address in the selected address space.	Input.
pBuffer	void*	Destination buffer for register data.	Output; non-NULL for a nonzero transfer.
piSize	size_t*	Requested transfer size in bytes; may be updated by the transport.	Input/output; non-NULL; buffer must cover the requested size.

KYVPExtension_RemoteDevice_DirectHWWriteReg_Args

Arguments: Write remote-device registers through direct hardware access.

Initialize version to KYVPEXTENSION_REMOTEDIVIDIRECTHWWRITEREG_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hRemoteDeviceHandle	KYVP_REMOTE_DEVICE_HANDLE	Remote device to operate on.	Input; valid remote-device handle.
iAddress	uint64_t	Starting register address in the selected address space.	Input.
pBuffer	const void*	Source buffer containing bytes to write or send.	Input; non-NULL for a nonzero transfer.
piSize	size_t*	Requested transfer size in bytes; may be	Input/output; non-NULL; buffer must

		updated by the transport.	cover the requested size.
--	--	---------------------------	---------------------------

KYVPExtension_RemoteDevice_SetImageDetails_Args

Arguments: Set the remote-device image geometry and pixel format used by the SDK.

Initialize version to KYVPEXTENSION_REMOTEDEVICE_SETIMAGEDetails_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hRemoteDeviceHandle	KYVP_REMOTE_DEVICE_HANDLE	Remote device to operate on.	Input; valid remote-device handle.
imageDetails	KYVP_DEVICE_IMAGE_DETAILS	Image dimensions and pixel format to apply.	Input; initialize the nested structure version and fields.

KYVPExtension_PCInterface_DirectHWReadReg_Args

Arguments: Read PCI interface registers through direct hardware access.

Initialize version to KYVPEXTENSION_PCIINTERFACEDIRECTHWREADREG_STRUCT_VERSION. BAR 0 is the normal access target. Access to other BARs is denied unless enabled by the SDK configuration.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
iBar	uint64_t	PCI Base Address Register index.	Input; normally 0.
iAddress	uint64_t	Starting register address in the selected address space.	Input.
pBuffer	void*	Destination buffer for register data.	Output; non-NULL for a nonzero transfer.
piSize	size_t*	Requested transfer size in bytes; may be updated by the transport.	Input/output; non-NULL; buffer must cover the requested size.

KYVPExtension_PCInterface_DirectHWWriteReg_Args

Arguments: Write PCI interface registers through direct hardware access.

Initialize version to KYVPEXTENSION_PCIINTERFACEDIRECTHWWRITEREG_STRUCT_VERSION. BAR 0 is the normal access target. Access to other BARs is denied unless enabled by the SDK configuration.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
iBar	uint64_t	PCI Base Address Register index.	Input; normally 0.
iAddress	uint64_t	Starting register address in the selected address space.	Input.
pBuffer	const void*	Source buffer containing bytes to write or send.	Input; non-NULL for a nonzero transfer.
piSize	size_t*	Requested transfer size in bytes; may be updated by the transport.	Input/output; non-NULL; buffer must cover the requested size.

KYVPExtension_PCIInterface_LockLink_Args

Arguments: Lock a PCI interface link for exclusive access.

Initialize version to KYVPEXTENSION_PCIINTERFACELOCKLINK_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
iLink	uint64_t	Zero-based host link index on the PCI interface.	Input.

KYVPExtension_PCIInterface_UnlockLink_Args

Arguments: Release a PCI interface link lock.

Initialize version to KYVPEXTENSION_PCIINTERFACEUNLOCKLINK_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
iLink	uint64_t	Zero-based host link index on the PCI interface.	Input.

KYVPExtension_PCIInterface_ReadLink_Args

Arguments: Read registers through the selected PCI interface link.

Initialize version to KYVPEXTENSION_PCIINTERFACEREADLINK_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
iLink	uint64_t	Zero-based host link index on the PCI interface.	Input.
iAddress	uint64_t	Starting register address in the selected address space.	Input.
pBuffer	void*	Destination buffer for register data.	Output; non-NULL for a nonzero transfer.
piSize	size_t*	Requested transfer size in bytes; may be updated by the transport.	Input/output; non-NULL; buffer must cover the requested size.

KYVPExtension_PCIInterface_WriteLink_Args

Arguments: Write registers through the selected PCI interface link.

Initialize version to KYVPEXTENSION_PCIINTERFACEWRITELINK_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
iLink	uint64_t	Zero-based host link index on the PCI interface.	Input.
iAddress	uint64_t	Starting register address in the selected address space.	Input.
pBuffer	const void*	Source buffer containing bytes to write or send.	Input; non-NULL for a nonzero transfer.
piSize	size_t*	Requested transfer size in bytes; may be updated by the transport.	Input/output; non-NULL; buffer must cover the requested size.

KYVPExtension_PCIInterface_SendLinkEvent_Args

Arguments: Send an event through the selected PCI interface link.

Initialize version to KYVPEXTENSION_PCIINTERFACEWRITELINK_STRUCT_VERSION. Support depends on the link transport; an unsupported transport returns an error.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
iLink	uint64_t	Zero-based host link index on the PCI interface.	Input.
iEventId	uint16_t	Transport-specific event identifier.	Input.
pBuffer	const void*	Source buffer containing bytes to write or send.	Input; non-NULL for a nonzero transfer.
piSize	size_t*	Requested transfer size in bytes; may be updated by the transport.	Input/output; non-NULL; buffer must cover the requested size.

KYVPExtension_PCIInterface_LinkGetStatus_Args

Arguments: Query connection or lock status for a PCI interface link.

Initialize version to KYVPEXTENSION_PCIINTERFACELINKGETSTATUS_ARGS_VERSION. Connection status is returned as int32_t; lock status is returned as KY_BOOL. Set the buffer to NULL to query the required size.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
iLink	uint64_t	Zero-based host link index on the PCI interface.	Input.
iInfoCmd	KYVP_LINK_STATUS_CMD	Link-status property to query.	Input.
piType	KYVP_INFO_DATATYPE*	Receives the data type of the returned status value.	Output; non-NULL, including size-only queries.
pBuffer	void*	Receives the selected link-status value.	Output; NULL queries the required size without copying data.

piSize	size_t*	Buffer capacity on input; required value size on output, in bytes.	Input/output; non-NULL.
--------	---------	--	-------------------------

KYVPExtension_AuxDataCallback_Register_Args

Arguments: Register a callback for auxiliary data from a PCI interface.

Initialize version to KYVPEXTENSION_AUX_DATA_CALLBACK_REGISTER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hiFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
pCallbackFunction	KYVP_AUX_DATA_CALLBACK	Callback function to invoke.	Input; non-NULL.
pUserContext	void*	Application context passed back to the callback.	Input; optional, may be NULL.

KYVPExtension_AuxDataCallback_Unregister_Args

Arguments: Remove a PCI interface auxiliary-data callback registration.

Initialize version to KYVPEXTENSION_AUX_DATA_CALLBACK_UNREGISTER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hiFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
pCallbackFunction	KYVP_AUX_DATA_CALLBACK	Callback function to remove.	Input; non-NULL; pass the registered function.

KYVPExtension_DataStreamBufferChunk

One memory segment of a buffer announced with KYVPExtension_DSAnnounceBufferChunks().

Initialize version to KYVPEXTENSION_DATASTREAMBUFFERCHUNK_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
pBuffer	void*	Virtual address of application-allocated chunk memory.	Input; retain the memory while the buffer is announced.
iSize	size_t	Size of this memory chunk in bytes.	Input.

KYVPExtension_DataStream_Create_Args

Arguments: Open a device data stream with the selected buffer-management mode.

Initialize version to KYVPEXTENSION_DATASTREAMCREATE_ARGS_VERSION. The stream identifier must identify a device stream known to the initialized SDK. Close the returned handle with KYVPExtension_DataStream_Close().

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
pszDevDataStreamID	const char*	Identifier of the device data stream to open.	Input; non-NULL, null-terminated string.
eStreamBufferType	KYVP_STREAM_BUFFERS_TYPE	Buffer-management mode for the stream.	Input.
phDSHandle	KYVP_STREAM_HANDLE*	Receives the newly opened data-stream handle.	Output; non-NULL.

KYVPExtension_DSAnnounceBufferChunks_Args

Arguments: Announce a buffer backed by multiple application-allocated memory chunks.

Initialize version to KYVPEXTENSION_DATASTREAMANNOUNCEBUFFERCHUNKS_ARGS_VERSION. The chunk memory belongs to the application and must remain valid while the announced buffer is in use. Revoke the buffer before releasing its memory.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hDSHandle	KYVP_STREAM_HANDLE	Data stream to operate on.	Input; valid stream handle.
pChunks	const KYVPExtension_DataStreamBufferChunk*	Array describing the memory chunks that make up one buffer.	Input; non-NULL, with iNumChunks entries.
iNumChunks	size_t	Number of entries in pChunks.	Input; greater than zero.
pPrivate	void*	Application data associated with the announced buffer.	Input; optional, may be NULL.
phBuffer	KYVP_BUFFER_HANDLE*	Receives the announced buffer handle.	Output; optional, may be NULL.

KYVPExtension_DataStream_Close_Args

Arguments: Close and release a data stream.

Initialize version to KYVPEXTENSION_DATASTREAM_CLOSE_ARGS_VERSION. The stream handle becomes invalid on success.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hDSHandle	KYVP_STREAM_HANDLE	Data stream to operate on.	Input; valid stream handle.

KYVPExtension_DataStream_Callback_Register_Args

Arguments: Register a callback for data-stream buffer notifications.

Initialize version to 1. The C constant is KYVPEXTENSION_DATASTREAMCALLBACK_REGISTER_ARGS_VERSION; the C++ constant is KYVPEXTENSION_DATASTREAMCALLBACKREGISTER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hDSHandle	KYVP_STREAM_HANDLE	Data stream to operate on.	Input; valid stream handle.
pCallbackFunction	KYVP_DATA_STREAM_CALLBACK	Callback function to invoke.	Input; non-NULL.
pContext	void*	Application context passed back to the callback.	Input; optional, may be NULL.

KYVPExtension_DataStream_Callback_Unregister_Args

Arguments: Remove a data-stream callback registration.

Initialize version to KYVPEXTENSION_DATASTREAMCALLBACKUNREGISTER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hDSHandle	KYVP_STREAM_HANDLE	Data stream to operate on.	Input; valid stream handle.
pCallbackFunction	KYVP_DATA_STREAM_CALLBACK	Callback function to remove.	Input; non-NULL; pass the registered function.

KYVPExtension_Serial_Callback_Register_Args

Arguments: Register a serial-event callback for a PCI interface link.

Initialize version to 1. The C constant is KYVPEXTENSION_SERIALCALLBACK_REGISTER_ARGS_VERSION; the C++ constant is KYVPEXTENSION_SERIALCALLBACKREGISTER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
-----------------	------	-------------	---------

version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
iLink	uint64_t	Zero-based host link index on the PCI interface.	Input.
pCallbackFunction	KYVP_SERIAL_CALLBACK	Callback function to invoke.	Input; non-NULL.
pContext	void*	Application context passed back to the callback.	Input; optional, may be NULL.

KYVPExtension_Serial_Callback_Unregister_Args

Arguments: Request removal of a serial-event callback registration.

Initialize version to KYVP_EXTENSION_SERIAL_CALLBACK_UNREGISTER_ARGS_VERSION.

Warning: The current implementation returns success without removing the registered callback. Do not use this call as confirmation that callback delivery has stopped.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
iLink	uint64_t	Zero-based host link index on the PCI interface.	Input.
pCallbackFunction	KYVP_SERIAL_CALLBACK	Callback function to remove.	Input; non-NULL; pass the registered function.

KYVPExtension_Serial_IPCServer_Start_Args

Arguments: Start the serial-port IPC server in the calling process.

Initialize version to KYVP_EXTENSION_SERIAL_IPC_SERVER_START_ARGS_VERSION. Starting an already running server succeeds without creating another instance.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.

KYVPExtension_Serial_IPCServer_Stop_Args

Arguments: Stop the serial-port IPC server in the calling process.

Initialize version to KYVP_EXTENSION_SERIAL_IPC_SERVER_STOP_ARGS_VERSION. Stopping an already stopped server succeeds.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.

KYVPExtension_RemoteDevice_GetParamsCollectionHandle_Args

Arguments: Retrieve the parameter collection associated with a remote device.

Initialize version to KYVPEXTENSION_REMOTEDevice_GETPARAMSCOLLECTIONHANDLE_ARGS. The collection belongs to the remote device. Use the parameter-handler API to access it; do not delete it as an application-created collection.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 1.
hRemoteDeviceHandle	KYVP_REMOTE_DEVICE_HANDLE	Remote device to operate on.	Input; valid remote-device handle.
phParamsCollectionHandle	KYVP_COLLECTION_HANDLE*	Receives the remote-device parameter collection handle.	Output; non-NULL; owned by the remote device.

DeviceLinkDefinition

Mapping for one device link in a manually configured device.

Initialize uVersion to KYVPEXTENSION_DEVICELINKDEFINITION_VERSION.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Argument structure layout version.	Input; set to 1.
uHostLink	uint32_t	Zero-based host link connected to this device link.	Input; replace the unassigned default with a valid index.

DeviceDefinition

Expected device topology and the result of manual detection.

Initialize uVersion to KYVPEXTENSION_DEVICEDEFINITION_VERSION.

Structure Field	Type	Description	Remarks
uVersion	uint32_t	Argument structure layout version.	Input; set to 1.
eConnectionSpeed	KYVP_CXP_LINK_SPEED	Expected current CoaXPress link speed of the device.	Input.
pDeviceLinkDefinitionArray	DeviceLinkDefinition*	Array mapping device link indices to host link indices.	Input; non-NULL; array

			position identifies the device link.
uDeviceLinkDefinitionArrayCount	size_t	Number of entries in pDeviceLinkDefinitionArray.	Input.
bFound	KY_BOOL	Whether the configured device was found.	Output; KY_TRUE when found.
szID	char [KY_MAX_DEVICE_INFO_STRING_SIZE + 1]	Transport-layer device identifier returned by detection.	Output; null-terminated when the device is found.

KYVPExtension_DetectDevicesManually_Args

Arguments: Detect devices using an application-supplied link topology.

Initialize version to KYVPEXTENSION_DETECTDEVICESMANUALLY_ARGS. Each device definition supplies its expected speed and a device-link-to-host-link mapping. Detection writes the found flag and device ID into each definition. Version 1 verifies link IDs and retains open devices. Version 2 adds optional link-ID verification suppression. Version 3 adds control over retaining open devices. With bRetainOpenDevices set to KY_FALSE, this process closes its devices and clears its device lists before fresh detection; existing handles and streams must be discarded. Devices opened by other processes are preserved in both modes.

Structure Field	Type	Description	Remarks
version	uint32_t	Argument structure layout version.	Input; set to 3.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface to operate on.	Input; valid interface handle.
pbChanged	KY_BOOL*	Receives whether the device list changed.	Output; optional, may be NULL.
pDeviceDefinitionArray	DeviceDefinition*	Device topologies to detect, with detection results written back.	Input/output; non-NULL array.
uDeviceDefinitionArrayCount	size_t	Number of entries in pDeviceDefinitionArray.	Input; greater than zero.
bNoDevicesAccess	KY_BOOL	Skip detection operations that require remote-device access.	Input; KY_TRUE enables this mode; SDK configuration can override it.
bSuppressLinkIDVerification	KY_BOOL	Allow detection without verifying the configured	Input; since version 2; defaults to

		device link-ID order.	KY_FALSE.
bRetainOpenDevices	KY_BOOL	Preserve open devices; KY_FALSE closes this process's devices before fresh detection.	Input; since version 3; defaults to KY_TRUE. Other processes' devices are preserved.

KYVPLibExtensionDefines API

KYVPLibExtensionDefines.h API reference

Generated from PUBLIC/include/KYVPLibExtensionDefines.h.

API documentation generated from PUBLIC/include/KYVPLibExtensionDefines.h.

Macros

KYVP_EXTENSION_EXTERNAL_DLL

KYVP_EXTENSION_API

KYVP_EXTERNAL_C KYVP_EXTENSION_EXTERNAL_DLL

KYVPLibTL API

KYVPLibTL.h API reference

Generated from PUBLIC/include/KYVPLibTL.h.

API documentation generated from PUBLIC/include/KYVPLibTL.h.

Initialize the library with KYVPLibTL_InitLib() before using the API. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to inspect KY_RESULT values. Use KYFoundation_What() to obtain a result description.

Structure packing, member order, and version constants form part of the SDK ABI.

Macros

KYVPTL_EXTERNAL_DLL

KYVPTL_API

KYVP_EXTERNAL_C KYVPTL_EXTERNAL_DLL

KYVPTL_VAR

KYVP_EXTERNAL KYVPTL_EXTERNAL_DLL

KYVP_CUSTOM_ID_DEBUG

10000

GENTL_NS

entry

INVALID_HANDLE_INIT_VALUE

((void*)-1)

TL_HANDLE_IS_INVALID

KYVPLibTL_TLHandleIsValid(_hHandle)

TL_HANDLE_IS_NULL

KYVPLibTL_TLHandleIsNull(_hHandle)

TL_HANDLES_IS_EQUAL

KYVPLibTL_TLHandlesIsEqual(_hHandleFirst, _hHandleSecond)

PCI_INTERFACE_HANDLE_IS_INVALID

KYVPLibTL_IFHandleIsValid(_hHandle)

PCI_INTERFACE_HANDLE_IS_NULL

KYVPLibTL_IFHandleIsNull(_hHandle)

PCI_INTERFACE_HANDLES_IS_EQUAL

KYVPLibTL_IFHandlesAreEqual(_hHandleFirst, _hHandleSecond)

DEVICE_HANDLE_IS_INVALID

KYVPLibTL_DevHandleIsValid(_hHandle)

DEVICE_HANDLE_IS_NULL

KYVPLibTL_DevHandleIsNull(_hHandle)

DEVICE_HANDLES_IS_EQUAL

KYVPLibTL_DevHandlesIsEqual(_hHandleFirst, _hHandleSecond)

REMOTE_DEVICE_HANDLE_IS_INVALID

KYVPLibTL_RemoteDeviceHandleIsValid(_hHandle)

REMOTE_DEVICE_HANDLE_IS_NULL

KYVPLibTL_RemoteDeviceHandleIsNull(_hHandle)

REMOTE_DEVICE_HANDLES_IS_EQUAL

KYVPLibTL_RemoteDeviceHandlesIsEqual(_hHandleFirst, _hHandleSecond)

STREAM_HANDLE_IS_INVALID

KYVPLibTL_DSHandleIsValid(_hHandle)

STREAM_HANDLE_IS_NULL

KYVPLibTL_DSHandleIsNull(_hHandle)

STREAM_HANDLES_IS_EQUAL

KYVPLibTL_DSHandlesIsEqual(_hHandleFirst, _hHandleSecond)

BUFFER_HANDLE_IS_INVALID

KYVPLibTL_BufferHandleIsValid(_hHandle)

BUFFER_HANDLE_IS_NULL

KYVPLibTL_BufferHandleIsNull(_hHandle)

BUFFER_HANDLES_IS_EQUAL

KYVPLibTL_BufferHandlesIsEqual(_hHandleFirst, _hHandleSecond)

EVENT_HANDLE_IS_INVALID

KYVPLibTL_EventHandleIsValid(_hHandle)

EVENT_HANDLE_IS_NULL

KYVPLibTL_EventHandleIsNull(_hHandle)

EVENT_HANDLES_IS_EQUAL

KYVPLibTL_EventHandlesIsEqual(_hHandleFirst, _hHandleSecond)

KYVP_PCIINTERFACE_EVENT_TEMPERATURE_VERSION

1

KYVP_PCIINTERFACE_EVENT_CXP2_HEARTBEAT_VERSION

1

KYVP_PCIINTERFACE_EVENT_CXP2_EVENT_VERSION

1

KYVP_PCIINTERFACE_GENERIC_EVENT_VERSION

1

KYVP_DEVICE_EVENT_CONNECTION_LOST_VERSION

1

KYVP_DEVICE_GENERIC_EVENT_VERSION

1

KYVPLIBTL_GETPORTURL_STRUCT_VERSION

1

KYVPLIBTL_READPORT_STRUCT_VERSION

1

KYVPLIBTL_WRITEPORT_STRUCT_VERSION

1

KYVPLIBTL_PCIINTERFACEGETPORTURL_STRUCT_VERSION

1

KYVPLIBTL_PCIINTERFACEREADPORT_STRUCT_VERSION

1

KYVPLIBTL_PCIINTERFACEWRITEPORT_STRUCT_VERSION

1

KYVPLIBTL_DEVICEGETPORTURL_STRUCT_VERSION

1

KYVPLIBTL_DEVICEREADPORT_STRUCT_VERSION

1

KYVPLIBTL_DEVICEWRITEPORT_STRUCT_VERSION

1

KYVPLIBTL_REMOTEDeviceGETPORTURL_STRUCT_VERSION

1

KYVPLIBTL_REMOTEDevicEREADPORT_STRUCT_VERSION

1

KYVPLIBTL_REMOTEDEVICEWRITEPORT_STRUCT_VERSION

1

KYVPLIBTL_STREAMGETPORTURL_STRUCT_VERSION

1

KYVPLIBTL_STREAMREADPORT_STRUCT_VERSION

1

KYVPLIBTL_STREAMWRITEPORT_STRUCT_VERSION

1

KYVPLIBTL_TLOPEN_ARGS_VERSION

1

KYVPLIBTL_TLCLOSE_ARGS_VERSION

1

KYVPLIBTL_TLGETINFO_ARGS_VERSION

1

KYVPLIBTL_TLGETNUMINTERFACES_ARGS_VERSION

1

KYVPLIBTL_TLGETINTERFACEID_ARGS_VERSION

1

KYVPLIBTL_TLGETPCIINTERFACEINFO_ARGS_VERSION

1

KYVPLIBTL_TLOPENPCIINTERFACE_STRUCT_VERSION

1

KYVPLIBTL_TLUPDATEPCIINTERFACELIST_ARGS_VERSION

1

KYVPLIBTL_IFCLOSE_ARGS_VERSION

1

KYVPLIBTL_IFGETINFO_ARGS_VERSION

1

KYVPLIBTL_IFGETNUMDEVICES_ARGS_VERSION

1

KYVPLIBTL_IFGETDEVICEID_ARGS_VERSION

1

KYVPLIBTL_IFUPDATEDEVICELIST_ARGS_VERSION

2

KYVPLIBTL_IFGETDEVICEINFO_ARGS_VERSION

1

KYVPLIBTL_IFOPENDEVICE_ARGS_VERSION

1

KYVPLIBTL_IFREGISTEREVENT_ARGS_VERSION

1

KYVPLIBTL_IFUNREGISTEREVENT_ARGS_VERSION

1

KYVPLIBTL_DEVGETPORT_ARGS_VERSION

1

KYVPLIBTL_DEVGETNUMDATASTREAMS_ARGS_VERSION

1

KYVPLIBTL_DEVGETDATASTREAMID_ARGS_VERSION

1

KYVPLIBTL_DEVOPENDATASTREAM_ARGS_VERSION

1

KYVPLIBTL_DEVGETINFO_ARGS_VERSION

1

KYVPLIBTL_DEVCLOSE_ARGS_VERSION

1

KYVPLIBTL_DEVREGISTEREVENT_ARGS_VERSION

1

KYVPLIBTL_DEVUNREGISTEREVENT_ARGS_VERSION

1

KYVPLIBTL_DSANNOUNCEBUFFER_ARGS_VERSION

1

KYVPLIBTL_DSALLOCANDANNOUNCEBUFFER_ARGS_VERSION

1

KYVPLIBTL_DSSTARTACQUISITION_ARGS_VERSION

1

KYVPLIBTL_DSSTOPACQUISITION_ARGS_VERSION

1

KYVPLIBTL_DSGETINFO_ARGS_VERSION

1

KYVPLIBTL_DSGETBUFFEREID_STRUCT_VERSION

1

KYVPLIBTL_DSCLOSE_ARGS_VERSION

1

KYVPLIBTL_DSREVOKEBUFFER_ARGS_VERSION

1

KYVPLIBTL_DSQUEUEBUFFER_ARGS_VERSION

1

KYVPLIBTL_DSFLUSHQUEUE_ARGS_VERSION

1

KYVPLIBTL_DSGETBUFFERINFO_ARGS_VERSION

1

KYVPLIBTL_DSREGISTEREVENT_ARGS_VERSION

1

KYVPLIBTL_DSUNREGISTEREVENT_ARGS_VERSION

1

KYVPLIBTL_EVENTGETDATA_ARGS_VERSION

1

KYVPLIBTL_EVENTGETDATAINFO_ARGS_VERSION

1

KYVPLIBTL_EVENTGETINFO_ARGS_VERSION

1

KYVPLIBTL_EVENTFLUSH_ARGS_VERSION

1

KYVPLIBTL_EVENTKILL_ARGS_VERSION

1

Enumerations

_KYVP_INFO_DATATYPE

Data types returned by information queries.

Defines the data type possible for the various Info functions. The data type itself may define its size. For buffer or string types the piSize parameter must be used to query the actual amount of data being written.

Values

Name	Description
KYVP_INFO_DATATYPE_UNKNOWN	Unknown data type. This value is never returned from a function but can be used to initialize the variable to inquire the type.
KYVP_INFO_DATATYPE_STRING	Null-terminated C string (encoding according to the TL_INFO_CHAR_ENCODING info command).
KYVP_INFO_DATATYPE_STRINGLIST	Concatenated INFO_DATATYPE_STRING list. End of list is signaled with an additional 0.
KYVP_INFO_DATATYPE_INT16	Signed 16-bit integer.
KYVP_INFO_DATATYPE_UINT16	Unsigned 16-bit integer.
KYVP_INFO_DATATYPE_INT32	Signed 32-bit integer.
KYVP_INFO_DATATYPE_UINT32	Unsigned 32-bit integer.
KYVP_INFO_DATATYPE_INT64	Signed 64-bit integer.
KYVP_INFO_DATATYPE_UINT64	Unsigned 64-bit integer.
KYVP_INFO_DATATYPE_FLOAT64	Signed 64-bit floating point number.
KYVP_INFO_DATATYPE_PTR	Pointer type (void*). Size is platform dependent (32-bit on 32-bit platforms).
KYVP_INFO_DATATYPE_BOOL8	Boolean value occupying 8-bit. 0 for false and nonzero for true.
KYVP_INFO_DATATYPE_SIZET	Platform dependent unsigned integer (32-bit on 32-bit platforms).
KYVP_INFO_DATATYPE_BUFFER	Like a INFO_DATATYPE_STRING but with arbitrary data and no null termination.
KYVP_INFO_DATATYPE_PTRDIFF	The type ptrdiff_t is a type that can hold the result of subtracting two pointers.
KYVP_INFO_DATATYPE_INVALID	SDK-specific sentinel for an invalid information data type.

_KYVP_TL_CHAR_ENCODING

Character encodings used by the transport layer.

Values

Name	Description
KYVP_TL_CHAR_ENCODING_ASCII	The producer uses ASCII for character strings.
KYVP_TL_CHAR_ENCODING_UTF8	The producer uses UTF-8 for character strings.
KYVP_TL_CHAR_ENCODING_INVALID	SDK-specific sentinel for an invalid character encoding.

_KYVP_TL_INFO_CMD

Transport layer information commands.

Values

Name	Description
KYVP_TL_INFO_CMD_ID	Unique ID identifying a GenTL. Returned data type: STRING.
KYVP_TL_INFO_CMD_VENDOR	Vendor name. Returned data type: STRING.
KYVP_TL_INFO_CMD_MODEL	Model name. Returned data type: STRING.
KYVP_TL_INFO_CMD_VERSION	Producer version. Returned data type: STRING.
KYVP_TL_INFO_CMD_TLTYPE	Transport layer technology that is supported. Returned data type: STRING.
KYVP_TL_INFO_CMD_NAME	File name including extension of the library. Returned data type: STRING.
KYVP_TL_INFO_CMD_PATHNAME	Full path including file name and extension of the library. Returned data type: STRING.
KYVP_TL_INFO_CMD_DISPLAYNAME	User readable name. Returned data type: STRING.
KYVP_TL_INFO_CMD_CHAR_ENCODING	The char encoding. Returned data type: INT32.
KYVP_TL_INFO_CMD_GENTL_VER_MAJOR	Major version number of GenTL Standard Version this Producer complies with. Returned data type: INT32.
KYVP_TL_INFO_CMD_GENTL_VER_MINOR	Minor version number of GenTL Standard Version this Producer complies with. Returned data type: INT32.
KYVP_TL_INFO_CMD_API_VER_MAJOR	Software API major version. Returned data type: INT32.
KYVP_TL_INFO_CMD_API_VER_MINOR	Software API minor version. Returned data type: INT32.
KYVP_TL_INFO_CMD_API_VER_PATCH	Software API patch version. Returned data type: INT32.
KYVP_TL_INFO_CMD_API_VER_BUILD	Software API build (revision) version. Returned data type: INT32.
KYVP_TL_INFO_CMD_API_VER_ALPHA	True if alpha build. Returned data type: BOOL.

KYVP_TL_INFO_CMD_API_VER_BETA	True if beta build. Returned data type: BOOL.
KYVP_TL_INFO_CMD_API_VER_RC	True if release candidate build. Returned data type: BOOL.
KYVP_TL_INFO_CMD_INVALID	SDK-specific sentinel for an invalid system information command.

_KYVP_INTERFACE_INFO_CMD

Interface information commands.

Values

Name	Description
KYVP_INTERFACE_INFO_CMD_ID	Unique ID of the interface. Returned data type: STRING.
KYVP_INTERFACE_INFO_CMD_DISPLAYNAME	User readable name of the interface. Returned data type: STRING.
KYVP_INTERFACE_INFO_CMD_TLTYPE	Transport layer technology that is supported. Returned data type: STRING.
KYVP_INTERFACE_INFO_CMD_PCI_SLOT	Info about interface PCI slot. Returned data type: STRING.
KYVP_INTERFACE_INFO_CMD_KYVP_PCI_INTERFACE_INFO	Pointer to KYVP_PCI_INTERFACE_INFO struct. Returned data type: PTR.
KYVP_INTERFACE_INFO_CMD_INVALID	SDK-specific sentinel for an invalid interface information command.

_KYVP_DEVICE_ACCESS_FLAGS

Device access modes requested when opening a device.

Values

Name	Description
KYVP_DEVICE_ACCESS_FLAGS_UNKNOWN	Unknown access mode, suitable for initializing a query result; not an open-device command.
KYVP_DEVICE_ACCESS_FLAGS_NONE	No device access: the device has not been opened, or access was denied.
KYVP_DEVICE_ACCESS_FLAGS_READONLY	Open the device for reading only; port writes are not permitted.
KYVP_DEVICE_ACCESS_FLAGS_CONTROL	Open with read/write control while allowing other processes or hosts to read the device.
KYVP_DEVICE_ACCESS_FLAGS_EXCLUSIVE	Open with exclusive read/write access; other processes and hosts cannot access the device.

_KYVP_DEVICE_ACCESS_STATUS

Device access status reported by the producer.

Values

Name	Description
KYVP_DEVICE_ACCESS_STATUS_UNKNOWN	The producer cannot determine whether the device is accessible.
KYVP_DEVICE_ACCESS_STATUS_READWRITE	The device is not open and can be opened for read/write access. After opening, the status becomes KYVP_DEVICE_ACCESS_STATUS_OPEN_READWRITE.
KYVP_DEVICE_ACCESS_STATUS_READONLY	The device is not open and can be opened for reading only. Read/write availability takes precedence when both modes are possible.
KYVP_DEVICE_ACCESS_STATUS_NOACCESS	The producer can identify the device, but the device is unreachable and cannot be accessed.
KYVP_DEVICE_ACCESS_STATUS_BUSY	Another process or host already owns or has opened the device.
KYVP_DEVICE_ACCESS_STATUS_OPEN_READWRITE	This producer has already opened the device with read/write access.
KYVP_DEVICE_ACCESS_STATUS_OPEN_READONLY	This producer has already opened the device with read-only access.

_KYVP_DEVICE_INFO_CMD

Device information commands.

Values

Name	Description
KYVP_DEVICE_INFO_CMD_ID	Unique ID of the device. Returned data type: STRING.
KYVP_DEVICE_INFO_CMD_VENDOR	Device vendor name. Returned data type: STRING.
KYVP_DEVICE_INFO_CMD_MODEL	Device model name. Returned data type: STRING.
KYVP_DEVICE_INFO_CMD_TLTYPE	Transport layer technology that is supported. Returned data type: STRING.
KYVP_DEVICE_INFO_CMD_DISPLAYNAME	User readable name of the device. Returned data type: STRING.
KYVP_DEVICE_INFO_CMD_ACCESS_STATUS	Gets the access status the Producer has on the device. Returned data type: INT32.
KYVP_DEVICE_INFO_CMD_USER_DEFINED_NAME	String containing the user defined name of the device. Returned data type: STRING.
KYVP_DEVICE_INFO_CMD_SERIAL_NUMBER	Serial number of the device in string format. Returned data type: STRING.

KYVP_DEVICE_INFO_CMD_VERSION	Device version in string format. Returned data type: STRING.
KYVP_DEVICE_INFO_CMD_TIMESTAMP_FREQUENCY	Tick frequency of the device's timestamp counter in ticks per second. Returned data type: UINT64.
KYVP_DEVICE_INFO_CMD_KYVP_DEVICE_INFO	Pointer to KYVP_DEVICE_INFO struct. The pointed data must be treated as read-only: modifying its contents is undefined behavior. Returned data type: PTR.
KYVP_DEVICE_INFO_CMD_INVALID	SDK-specific sentinel for an invalid device information command.

_KYVP_ACQ_START_FLAGS

Acquisition start flags.

Values

Name	Description
KYVP_ACQ_START_FLAGS_DEFAULT	Default behavior.

_KYVP_ACQ_STOP_FLAGS

Acquisition stop flags.

Values

Name	Description
KYVP_ACQ_STOP_FLAGS_DEFAULT	Stops the acquisition engine when the currently running tasks like filling a buffer are completed.
KYVP_ACQ_STOP_FLAGS_KILL	Stop the acquisition engine immediately. In case this results in a partially filled buffer the Producer will return the buffer through the regular mechanism to the user, indicating through the info function of that buffer that this buffer is not complete.

_KYVP_ACQ_QUEUE_TYPE

Buffer queue operations.

Values

Name	Description
KYVP_ACQ_QUEUE_TYPE_INPUT_TO_OUTPUT	Move input-pool buffers to the output queue and generate new-buffer events as needed. Buffers currently being filled remain in place. This describes the default buffer handling mode.
KYVP_ACQ_QUEUE_OUTPUT_DISCARD	Remove buffers from the output queue and remove their pending new-buffer events as needed. The

	buffers remain announced.
KYVP_ACQ_QUEUE_ALL_TO_INPUT	Return every announced buffer to the input pool, including output-queue buffers and buffers being filled, and discard pending new-buffer events.
KYVP_ACQ_QUEUE_UNQUEUED_TO_INPUT	Queue announced buffers that are neither in the input pool nor in the output queue nor currently being filled.
KYVP_ACQ_QUEUE_ALL_DISCARD	Remove every buffer from the input pool, output queue, and active filling operation so it can be revoked or queued again. This does not itself revoke the buffers.
KYVP_ACQ_QUEUE_INVALID	SDK-specific sentinel for an invalid queue operation.

_KYVP_STREAM_INFO_CMD

Data stream information commands.

Defines the data type possible for the various Info functions. The data type itself may define its size. For buffer or string types the piSize parameter must be used to query the actual amount of data being written.

Values

Name	Description
KYVP_STREAM_INFO_CMD_ID	Unique ID of the data stream.
KYVP_STREAM_INFO_CMD_NUM_DELIVERED	Number of delivered buffers since last acquisition start.
KYVP_STREAM_INFO_CMD_NUM_UNDERRUN	Number of lost frames due to queue underrun.
KYVP_STREAM_INFO_CMD_NUM_ANNOUNCED	Number of announced buffers.
KYVP_STREAM_INFO_CMD_NUM_QUEUED	Number of buffers in the input pool plus the buffer(s) currently being filled.
KYVP_STREAM_INFO_CMD_NUM_AWAIT_DELIVERY	Number of buffers in the output buffer queue.
KYVP_STREAM_INFO_CMD_NUM_STARTED	Number of frames started in the acquisition engine.
KYVP_STREAM_INFO_CMD_PAYLOAD_SIZE	Size of the expected data in bytes.
KYVP_STREAM_INFO_CMD_IS_GRABBING	Flag indicating whether the acquisition engine is started or not.
KYVP_STREAM_INFO_CMD_DEFINES_PAYLOADSIZE	Flag indicating that this data stream defines a payload size independent from the remote device.
KYVP_STREAM_INFO_CMD_TLTYPE	Transport layer technology that is supported.
KYVP_STREAM_INFO_CMD_NUM_CHUNKS_MAX	Maximum number of chunks to be expected in a buffer.
KYVP_STREAM_INFO_CMD_BUF_ANNOUNCE_MIN	Minimum number of buffers to announce.
KYVP_STREAM_INFO_CMD_BUF_ALIGNMENT	Alignment size in bytes of the buffer passed to

	DSAnnounceBuffer.
KYVP_STREAM_INFO_CMD_INVALID	SDK-specific sentinel for an invalid stream information command.

_KYVP_BUFFER_INFO_CMD

Buffer information commands.

Values

Name	Description
KYVP_BUFFER_INFO_CMD_BASE	Base address of the buffer memory as passed to the KYVPLibTL_DSAnnounceBuffer function.
KYVP_BUFFER_INFO_CMD_SIZE	Size of the buffer in bytes.
KYVP_BUFFER_INFO_CMD_USER_PTR	Private data pointer casted to an integer provided at buffer announcement using KYVPLibTL_DSAnnounceBuffer or KYVPLibTL_DSAllocAndAnnounceBuffer.
KYVP_BUFFER_INFO_CMD_TIMESTAMP	Timestamp the buffer was acquired.
KYVP_BUFFER_INFO_CMD_NEW_DATA	Flag to indicate that the buffer contains new data since the last delivery.
KYVP_BUFFER_INFO_CMD_IS_QUEUED	If this flag is set to true the buffer is in the input pool, the buffer is currently being filled or the buffer is in the output buffer queue. In case this value is true the buffer is owned by the GenTL Producer and it can not be revoked.
KYVP_BUFFER_INFO_CMD_IS_ACQUIRING	Flag to indicate that the buffer is currently being filled with data.
KYVP_BUFFER_INFO_CMD_IS_INCOMPLETE	Flag to indicate that a buffer was filled but an error occurred during that process.
KYVP_BUFFER_INFO_CMD_TLTYPE	Transport layer technology that is supported.
KYVP_BUFFER_INFO_CMD_SIZE_FILLED	Number of bytes written into the buffer the last time it has been filled. This value is reset to 0 when the buffer is placed into the Input Buffer Pool. If the buffer is incomplete (such as if there are missing packets), only the number of bytes successfully written to the buffer are reported. If the buffer is complete, the number equals to the size reported through KYVP_BUFFER_INFO_DATA_SIZE.
KYVP_BUFFER_INFO_CMD_WIDTH	Width of the data in the buffer in number of pixels. This information refers for example to the width entry in the GigE Vision image stream data leader. For other technologies this is to be implemented accordingly.

KYVP_BUFFER_INFO_CMD_HEIGHT	Height of the data in the buffer in number of pixels as configured. For variable size images this is the maximum height of the buffer. For example this information refers to the height entry in the GigE Vision image stream data leader. For other technologies this is to be implemented accordingly.
KYVP_BUFFER_INFO_CMD_XOFFSET	XOffset of the data in the buffer in number of pixels from the image origin to handle areas of interest. This information refers for example to the information provided in the GigE Vision image stream data leader. For other technologies this is to be implemented accordingly.
KYVP_BUFFER_INFO_CMD_YOFFSET	YOffset of the data in the buffer in number of lines from the image origin to handle areas of interest. This information refers for example to the information provided in the GigE Vision image stream data leader. For other technologies this is to be implemented accordingly.
KYVP_BUFFER_INFO_CMD_XPADDING	XPadding of the data in the buffer in number of bytes. This information refers for example to the information provided in the GigE Vision image stream data leader. For other technologies this is may be implemented accordingly.
KYVP_BUFFER_INFO_CMD_YPADDING	YPadding of the data in the buffer in number of bytes. This information refers for example to the information provided in the GigE Vision image stream data leader. For other technologies this may be implemented accordingly.
KYVP_BUFFER_INFO_CMD_FRAMEID	A sequentially incremented number of the frame. This information refers for example to the information provided in the GigE Vision image stream block id. For other technologies this is to be implemented accordingly. The wrap around of this number is transportation technology dependent.
KYVP_BUFFER_INFO_CMD_IMAGEPRESENT	Flag to indicate if the current data in the buffer contains image data.
KYVP_BUFFER_INFO_CMD_IMAGEOFFSET	Offset of the image data from the beginning of the delivered buffer in bytes.
KYVP_BUFFER_INFO_CMD_PAYLOADTYPE	Payload type of the data.
KYVP_BUFFER_INFO_CMD_PIXELFORMAT	Pixel format of the data.
KYVP_BUFFER_INFO_CMD_PIXELFORMAT_NAMESPACE	Namespace used to interpret KYVP_BUFFER_INFO_CMD_PIXELFORMAT, expressed as a KYVP_PIXELFORMAT_NAMESPACE_IDS value.

	Returned data type: UINT64.
KYVP_BUFFER_INFO_CMD_DELIVERED_IMAGEHEIGHT	The number of lines in the current buffer as delivered by the transport mechanism.
KYVP_BUFFER_INFO_CMD_DELIVERED_CHUNKPAYLOADSIZE	This information refers for example to the information provided in the GigE Vision image stream data trailer.
KYVP_BUFFER_INFO_CMD_CHUNKLAYOUTID	This information refers for example to the information provided in the GigE Vision image stream data leader. The chunk layout id serves as an indicator that the chunk layout has changed and the application should reparse the chunk layout in the buffer. When a chunk layout (availability or position of individual chunks) changes since the last buffer delivered by the device through the same stream, the device must change the chunk layout id. As long as the chunk layout remains stable, the camera must keep the chunk layout id intact. When switching back to a layout, which was already used before, the camera can use the same id again or use a new id. A chunk layout id value of 0 is invalid. It is reserved for use by cameras not supporting the layout id functionality. The algorithm used to compute the chunk layout id is left as quality of implementation. For other technologies this is to be implemented accordingly.
KYVP_BUFFER_INFO_CMD_FILENAME	Filename in case the payload contains a file.
KYVP_BUFFER_INFO_CMD_PIXEL_ENDIANNES	Endianness of the multi-byte pixel data in the buffer.
KYVP_BUFFER_INFO_CMD_DATA_SIZE	Size of the data intended to be written to the buffer last time it has been filled.
KYVP_BUFFER_INFO_CMD_TIMESTAMP_NS	Timestamp the buffer was acquired, in units of 1 ns (1 000 000 000 ticks per second).
KYVP_BUFFER_INFO_CMD_DATA_LARGER_THAN_BUFFER	If this values is set to true it indicates that the payload transferred would not fit into the announced buffer and that therefore only parts of the payload or no payload.
KYVP_BUFFER_INFO_CMD_CONTAINS_CHUNKDATA	If this values is set to true it indicates that the payload transferred contains chunk data.
KYVP_BUFFER_INFO_CMD_BUFFERID	Buffer ID in buffers queue.
KYVP_BUFFER_INFO_CMD_INSTANT_FPS	Instant FPS counter.
KYVP_BUFFER_INFO_CMD_KYVP_BUFFER_INFO	Pointer to KYVP_BUFFER_INFO struct.
KYVP_BUFFER_INFO_CMD_TIMESTAMP_NS_CHRONO	Timestamp the interrupt for this buffer was received (std::chrono::steady_clock), in units of 1

	ns (1 000 000 000 ticks per second).
KYVP_BUFFER_INFO_CMD_TIMESTAMP_NS_HW	Timestamp the hw interrupt for this buffer was received (hardware register), in units of 1 ns (1 000 000 000 ticks per second). Used for debugging purposes; activated when the corresponding INTERNAL setting is enabled.
KYVP_BUFFER_INFO_CMD_INVALID	SDK-specific sentinel for an invalid buffer information command.

_KYVP_BUFFER_PART_INFO_CMD

Multipart buffer information commands.

Values

Name	Description
KYVP_BUFFER_PART_INFO_CMD_BASE	Address of the first valid byte in this part, excluding padding before the part. Returned data type: PTR.
KYVP_BUFFER_PART_INFO_CMD_DATA_SIZE	Number of valid data bytes in this part, excluding padding between parts. For variable-size data, reports the actual valid size. Returned data type: SIZET.
KYVP_BUFFER_PART_INFO_CMD_DATA_TYPE	Kind of data in this part, expressed as a KYVP_PARTDATATYPE_IDS value. Returned data type: SIZET.
KYVP_BUFFER_PART_INFO_CMD_DATA_FORMAT	Format identifier for items such as pixels in this part. Interpret it using KYVP_BUFFER_PART_INFO_CMD_DATA_FORMAT_NAME_SPACE and the part data type. Returned data type: UINT64.
KYVP_BUFFER_PART_INFO_CMD_DATA_FORMAT_NAME_SPACE	Namespace of the data-format identifier, expressed as a KYVP_PIXELFORMAT_NAME_SPACE_IDS value. Returned data type: UINT64.
KYVP_BUFFER_PART_INFO_CMD_WIDTH	Width of the part image in pixels, when applicable to its data type. Returned data type: SIZET.
KYVP_BUFFER_PART_INFO_CMD_HEIGHT	Expected height of the part image in pixels, when applicable to its data type. Returned data type: SIZET.
KYVP_BUFFER_PART_INFO_CMD_XOFFSET	Horizontal offset of this part image from the image origin, in pixels. Returned data type: SIZET.
KYVP_BUFFER_PART_INFO_CMD_YOFFSET	Vertical offset of this part image from the image origin, in pixels. Returned data type: SIZET.
KYVP_BUFFER_PART_INFO_CMD_XPADDING	Horizontal padding of the part image, measured in pixels. Returned data type: SIZET.
KYVP_BUFFER_PART_INFO_CMD_SOURCE_ID	Identifier used to associate parts from the same source, such as an image and its confidence map. Source IDs need not form a contiguous sequence.

	Returned data type: UINT64.
KYVP_BUFFER_PART_INFO_CMD_DELIVERED_IMAGEHEIGHT	Number of image lines actually delivered in this part. Variable-height line-scan data can contain fewer lines than the configured height. Returned data type: SIZET.
KYVP_BUFFER_PART_INFO_CMD_INVALID	SDK-specific sentinel for an invalid buffer-part information command.

_KYVP_PAYLOADTYPE_INFO_IDS

Buffer payload types.

Values

Name	Description
KYVP_PAYLOADTYPE_INFO_IDS_UNKNOWN	The producer does not recognize the payload type; the consumer can treat its contents as raw data.
KYVP_PAYLOADTYPE_INFO_IDS_IMAGE	Image payload, which can also contain chunk data. Query KYVP_BUFFER_INFO_CMD_CONTAINS_CHUNKDATA to check for chunks.
KYVP_PAYLOADTYPE_INFO_IDS_RAW_DATA	Unspecified raw bytes, for example acquisition statistics, without an implied image format.
KYVP_PAYLOADTYPE_INFO_IDS_FILE	File contents carried in the buffer. KYVP_BUFFER_INFO_CMD_FILENAME provides a filename that can help identify the format.
KYVP_PAYLOADTYPE_INFO_IDS_CHUNK_DATA	DeprecatedSince GenTL 1.5, chunk data may accompany other payload types. Query KYVP_BUFFER_INFO_CMD_CONTAINS_CHUNKDATA instead.
KYVP_PAYLOADTYPE_INFO_IDS_JPEG	JPEG-encoded payload using the GigE Vision 2.0 representation; buffer information commands provide additional metadata.
KYVP_PAYLOADTYPE_INFO_IDS_JPEG2000	JPEG 2000-encoded payload using the GigE Vision 2.0 representation; buffer information commands provide additional metadata.
KYVP_PAYLOADTYPE_INFO_IDS_H264	H.264-encoded payload using the GigE Vision 2.0 representation; buffer information commands provide additional metadata.
KYVP_PAYLOADTYPE_INFO_IDS_CHUNK_ONLY	A payload consisting entirely of chunk data, without an accompanying image or other payload.
KYVP_PAYLOADTYPE_INFO_IDS_DEVICE_SPECIFIC	Payload whose format is defined by the device; buffer information commands can provide additional metadata.
KYVP_PAYLOADTYPE_INFO_IDS_MULTI_PART	A payload containing multiple data parts, each with its own data type and format. See

	KYVP_BUFFER_PART_INFO_CMD for part metadata.
KYVP_PAYLOADTYPE_INFO_IDS_INVALID	SDK-specific sentinel for an invalid payload type.

_KYVP_PIXELFORMAT_NAMESPACE_IDS

Pixel format namespaces.

Values

Name	Description
KYVP_PIXELFORMAT_NAMESPACE_IDS_UNKNOWN	The producer does not know how to interpret the pixel-format identifier.
KYVP_PIXELFORMAT_NAMESPACE_IDS_GEV	Pixel-format identifiers follow the GigE Vision 1.x definitions.
KYVP_PIXELFORMAT_NAMESPACE_IDS_IIDC	Pixel-format identifiers follow the IIDC 1.x definitions.
KYVP_PIXELFORMAT_NAMESPACE_IDS_PFNC_16BIT	Pixel-format identifiers use the 16-bit PFNC namespace. GenTL recommends the 32-bit PFNC namespace when possible for broader consumer support.
KYVP_PIXELFORMAT_NAMESPACE_IDS_PFNC_32BIT	Pixel-format identifiers use the 32-bit PFNC namespace.
KYVP_PIXELFORMAT_NAMESPACE_IDS_INVALID	SDK-specific sentinel for an invalid pixel-format namespace.

_KYVP_PXELENDIANNES_IDS

Pixel data byte orders.

Values

Name	Description
KYVP_PXELENDIANNES_IDS_UNKNOWN	The producer does not know the byte order of multibyte pixel data.
KYVP_PXELENDIANNES_IDS_LITTLE	Multibyte pixel values store the least significant byte first.
KYVP_PXELENDIANNES_IDS_BIG	Multibyte pixel values store the most significant byte first.
KYVP_PXELENDIANNES_IDS_INVALID	SDK-specific sentinel for an invalid pixel byte order.

_KYVP_PARTDATATYPE_IDS

Multipart buffer data types.

Values

Name	Description
------	-------------

KYVP_PARTDATATYPE_IDS_UNKNOWN	The producer does not recognize the part data type; the consumer can treat the part as raw data.
KYVP_PARTDATATYPE_IDS_2D_IMAGE	A complete monochrome or color 2D image, including all of its pixel data.
KYVP_PARTDATATYPE_IDS_2D_PLANE_BIPLANAR	One color plane of a 2D image made of two planes. The planes occupy consecutive buffer parts.
KYVP_PARTDATATYPE_IDS_2D_PLANE_TRIPLANAR	One color plane of a 2D image made of three planes. The planes occupy consecutive buffer parts.
KYVP_PARTDATATYPE_IDS_2D_PLANE_QUADPLANAR	One color plane of a 2D image made of four planes. The planes occupy consecutive buffer parts.
KYVP_PARTDATATYPE_IDS_3D_IMAGE	Complete 3D image data, such as pixel coordinates or a depth map, contained in one part.
KYVP_PARTDATATYPE_IDS_3D_PLANE_BIPLANAR	One coordinate plane of a 3D image made of two planes. The planes occupy consecutive buffer parts.
KYVP_PARTDATATYPE_IDS_3D_PLANE_TRIPLANAR	One coordinate plane of a 3D image made of three planes. The planes occupy consecutive buffer parts.
KYVP_PARTDATATYPE_IDS_3D_PLANE_QUADPLANAR	One coordinate plane of a 3D image made of four planes. The planes occupy consecutive buffer parts.
KYVP_PARTDATATYPE_IDS_CONFIDENCE_MAP	Per-pixel validity or confidence values paired with image parts of matching dimensions. Each value describes the corresponding image pixel; use a PFNC Confidence format.
KYVP_PARTDATATYPE_IDS_INVALID	SDK-specific sentinel for an invalid buffer-part data type.

_KYVP_PORT_INFO_CMD

Port information commands.

Values

Name	Description
KYVP_PORT_INFO_CMD_ID	Identifier of the module associated with the port. A remote device uses its local device ID; a buffer uses its address in hexadecimal without a 0x prefix. Returned data type: STRING.
KYVP_PORT_INFO_CMD_VENDOR	Vendor of the underlying module, or the producer vendor when the module has no separate vendor. Returned data type: STRING.
KYVP_PORT_INFO_CMD_MODEL	Model of the underlying module, or the producer model when the module has no separate model. Returned data type: STRING.
KYVP_PORT_INFO_CMD_TLTYPE	Name of the transport technology supported by the port. Returned data type: STRING.

KYVP_PORT_INFO_CMD_MODULE	Module category: TLSystem, TLInterface, TLDevice, TLDataStream, TLBuffer, or Device for a remote device. Returned data type: STRING.
KYVP_PORT_INFO_CMD_LITTLE_ENDIAN	Whether the port data uses little-endian byte order. Returned data type: BOOL8.
KYVP_PORT_INFO_CMD_BIG_ENDIAN	Whether the port data uses big-endian byte order. Returned data type: BOOL8.
KYVP_PORT_INFO_CMD_ACCESS_READ	Whether reading from the port is permitted. Returned data type: BOOL8.
KYVP_PORT_INFO_CMD_ACCESS_WRITE	Whether writing to the port is permitted. Returned data type: BOOL8.
KYVP_PORT_INFO_CMD_ACCESS_NA	Whether the port is currently unavailable for access. Returned data type: BOOL8.
KYVP_PORT_INFO_CMD_ACCESS_NI	Whether no port is implemented. GenTL permits this condition only for buffer modules. Returned data type: BOOL8.
KYVP_PORT_INFO_CMD_VERSION	Version identifier of the port. Returned data type: STRING.
KYVP_PORT_INFO_CMD_PORTNAME	Port name used in the XML description to connect the port to the module nodemap. Returned data type: STRING.
KYVP_PORT_INFO_CMD_INVALID	SDK-specific sentinel for an invalid port information command.

_KYVP_URL_SCHEME_IDS

Port URL schemes.

Values

Name	Description
KYVP_URL_SCHEME_IDS_LOCAL	The XML file resides in the port register map. Query KYVP_URL_INFO_CMD_FILE_REGISTER_ADDRESS and KYVP_URL_INFO_CMD_FILE_SIZE to locate it.
KYVP_URL_SCHEME_IDS_HTTP	Retrieve the XML file from a web server using HTTP; KYVP_URL_INFO_CMD_URL supplies the address.
KYVP_URL_SCHEME_IDS_FILE	The XML file resides on the local filesystem; KYVP_URL_INFO_CMD_FILENAME supplies its filename.
KYVP_URL_SCHEME_IDS_INVALID	SDK-specific sentinel for an invalid URL scheme.

_KYVP_URL_INFO_CMD

Port URL information commands.

Values

Name	Description
KYVP_URL_INFO_CMD_URL	URL identifying the XML description associated with the port. Returned data type: STRING.
KYVP_URL_INFO_CMD_SCHEMA_VER_MAJOR	Major version of the XML schema used by the referenced description. Returned data type: INT32.
KYVP_URL_INFO_CMD_SCHEMA_VER_MINOR	Minor version of the XML schema used by the referenced description. Returned data type: INT32.
KYVP_URL_INFO_CMD_FILE_VER_MAJOR	Major version component of the referenced XML file. Returned data type: INT32.
KYVP_URL_INFO_CMD_FILE_VER_MINOR	Minor version component of the referenced XML file. Returned data type: INT32.
KYVP_URL_INFO_CMD_FILE_VER_SUBMINOR	Subminor version component of the referenced XML file. Returned data type: INT32.
KYVP_URL_INFO_CMD_FILE_SHA1_HASH	SHA-1 digest of the referenced XML file, provided as 20 bytes (160 bits). Returned data type: BUFFER.
KYVP_URL_INFO_CMD_FILE_REGISTER_ADDRESS	Register-map address of the XML file. Available only when the file is stored in the device register map. Returned data type: UINT64.
KYVP_URL_INFO_CMD_FILE_SIZE	Size of the referenced XML file in bytes. Returned data type: UINT64.
KYVP_URL_INFO_CMD_SCHEME	Location scheme expressed as a KYVP_URL_SCHEME_IDS value. Returned data type: INT32.
KYVP_URL_INFO_CMD_FILENAME	Filename for the file scheme, or a suggested filename for XML held in the local register map. Returned data type: STRING.
KYVP_URL_INFO_CMD_INVALID	SDK-specific sentinel for an invalid URL information command.

_KYVP_EVENT_TYPE

Event types available for registration.

Values

Name	Description
KYVP_EVENT_TYPE_ERROR_	Notification on module errors.
KYVP_EVENT_TYPE_NEW_BUFFER	Notification on newly filled buffers.
KYVP_EVENT_TYPE_FEATURE_INVALIDATE	Notification if a feature was changed by the Producer driver and thus needs to be invalidated.
KYVP_EVENT_TYPE_FEATURE_CHANGE	Notification if the Producer driver wants to manually

	set a feature in the instance.
KYVP_EVENT_TYPE_REMOTE_DEVICE	Notification if the Producer wants to inform the instance of the remote device that a GenApi compatible event was fired.
KYVP_EVENT_TYPE_MODULE	Notification that one GenTL Producer module wants to inform the GenICam GenApi instance of this module that a GenApi compatible event was fired. This Event is to be registered on any module handle except on the Remote Device.
KYVP_EVENT_TYPE_PCIINTERFACE_EVENT	KAYA-specific PCI interface notification. KYVPLibTL_EventGetData() returns a KYVP_PCIINTERFACE_GENERIC_EVENT payload.
KYVP_EVENT_TYPE_DEVICE_EVENT	KAYA-specific device notification. KYVPLibTL_EventGetData() returns a KYVP_DEVICE_GENERIC_EVENT payload.
KYVP_EVENT_TYPE_INVALID	SDK-specific sentinel for an invalid event type.

_KYVP_EVENT_INFO_CMD

Event information commands.

Values

Name	Description
KYVP_EVENT_INFO_CMD_TYPE	The event type of the event handle.
KYVP_EVENT_INFO_CMD_NUM_IN_QUEUE	Number of events in the event data queue.
KYVP_EVENT_INFO_CMD_NUM_FIRED	Number of events that were fired since the registration of the event.
KYVP_EVENT_INFO_CMD_EVENT_SIZE_MAX	Maximum size in bytes of the event data provided by the event.
KYVP_EVENT_INFO_CMD_INFO_DATA_SIZE_MAX	Maximum size in bytes of the information output buffer.
KYVP_EVENT_INFO_CMD_INVALID	SDK-specific sentinel for an invalid event information command.

_KYVP_EVENT_DATA_INFO_CMD

Event data information commands.

Values

Name	Description
KYVP_EVENT_DATA_INFO_CMD_ID	Attribute in the event data to identify the object or feature the event refers to.
KYVP_EVENT_DATA_INFO_CMD_VALUE	Defines additional data to an ID.

KYVP_EVENT_DATA_INFO_CMD_NUMID	Attribute in the event data to identify the object or feature the event refers to.
KYVP_EVENT_DATA_INFO_CMD_INVALID	SDK-specific sentinel for an invalid event-data information command.

_KYVP_LINK_STATUS

Link connection states.

Values

Name	Description
KYVP_KYVP_LINK_STATUS_DISCONNECTED	The link is disconnected, no link has been established.
KYVP_KYVP_LINK_STATUS_SYNCHRONIZED	The link is synchronized and awaiting connection.
KYVP_KYVP_LINK_STATUS_CONNECTING	A connection is trying to be established on link.
KYVP_KYVP_LINK_STATUS_CONNECTED	The link is connected and assigned to device.

_KYVP_LINK_STATUS_CMD

Link status commands.

Values

Name	Description
KYVP_LINK_STATUS_CMD_LINK_CONNECTION_STATUS	Return the link connection status.
KYVP_LINK_STATUS_CMD_LINK_LOCK_STATUS	Return the link lock status.
KYVP_LINK_STATUS_CMD_INVALID	SDK-specific sentinel for an invalid link-status command.

_KYVP_STREAM_BUFFERS_TYPE

Data stream buffer modes.

Values

Name	Description
KYVP_STREAM_BUFFERS_TYPE_QUEUED_BUFFERS	Data stream buffers type queued.
KYVP_STREAM_BUFFERS_TYPE_CYCLIC_BUFFERS	Data stream buffers type cyclic.
KYVP_STREAM_BUFFERS_TYPE_INVALID	SDK-specific sentinel for an invalid stream buffer mode.

_DS_PACKED_DATA_MODE

Packed data layouts.

Values

Name	Description
DS_PACKED_DATA_MODE_UNPACKED	Data unpacked.
DS_PACKED_DATA_MODE_LINE32ALIGNED	Data packed row aligned 32.
DS_PACKED_DATA_MODE_LINE32ALIGNED_REVERSE	Data packed row aligned 32 reverse.
DS_PACKED_DATA_MODE_INVALID	SDK-specific sentinel for an invalid packed data layout.

_DS_PACKED_DATA_PROCESSING_MODE

Packed data processing modes.

Values

Name	Description
DS_PACKED_DATA_PROCESSING_MODE_MSB8	Trim packed data bits to most significant 8-bits.
DS_PACKED_DATA_PROCESSING_MODE_EXPAND16	Data packed bits expand to 16-bits.
DS_PACKED_DATA_PROCESSING_MODE_INVALID	SDK-specific sentinel for an invalid packed data processing mode.

Type definitions

KYVP_INFO_DATATYPE

```
typedef enum _KYVP_INFO_DATATYPE KYVP_INFO_DATATYPE
```

Data types returned by information queries.

Defines the data type possible for the various Info functions. The data type itself may define its size. For buffer or string types the piSize parameter must be used to query the actual amount of data being written.

KYVP_TL_CHAR_ENCODING

```
typedef enum _KYVP_TL_CHAR_ENCODING KYVP_TL_CHAR_ENCODING
```

Character encodings used by the transport layer.

KYVP_TL_INFO_CMD

```
typedef enum _KYVP_TL_INFO_CMD KYVP_TL_INFO_CMD
```

Transport layer information commands.

KYVP_INTERFACE_INFO_CMD

```
typedef enum _KYVP_INTERFACE_INFO_CMD KYVP_INTERFACE_INFO_CMD
```

Interface information commands.

KYVP_DEVICE_ACCESS_FLAGS

```
typedef enum _KYVP_DEVICE_ACCESS_FLAGS KYVP_DEVICE_ACCESS_FLAGS
```

Device access modes requested when opening a device.

KYVP_DEVICE_ACCESS_STATUS

typedef enum _KYVP_DEVICE_ACCESS_STATUS KYVP_DEVICE_ACCESS_STATUS
Device access status reported by the producer.

KYVP_DEVICE_INFO_CMD

typedef enum _KYVP_DEVICE_INFO_CMD KYVP_DEVICE_INFO_CMD
Device information commands.

KYVP_ACQ_START_FLAGS

typedef enum _KYVP_ACQ_START_FLAGS KYVP_ACQ_START_FLAGS
Acquisition start flags.

KYVP_ACQ_STOP_FLAGS

typedef enum _KYVP_ACQ_STOP_FLAGS KYVP_ACQ_STOP_FLAGS
Acquisition stop flags.

KYVP_ACQ_QUEUE_TYPE

typedef enum _KYVP_ACQ_QUEUE_TYPE KYVP_ACQ_QUEUE_TYPE
Buffer queue operations.

KYVP_STREAM_INFO_CMD

typedef enum _KYVP_STREAM_INFO_CMD KYVP_STREAM_INFO_CMD
Data stream information commands.

Defines the data type possible for the various Info functions. The data type itself may define its size. For buffer or string types the piSize parameter must be used to query the actual amount of data being written.

KYVP_BUFFER_INFO_CMD

typedef enum _KYVP_BUFFER_INFO_CMD KYVP_BUFFER_INFO_CMD
Buffer information commands.

KYVP_BUFFER_PART_INFO_CMD

typedef enum _KYVP_BUFFER_PART_INFO_CMD KYVP_BUFFER_PART_INFO_CMD
Multipart buffer information commands.

KYVP_PAYLOADTYPE_INFO_IDS

typedef enum _KYVP_PAYLOADTYPE_INFO_IDS KYVP_PAYLOADTYPE_INFO_IDS
Buffer payload types.

KYVP_PIXELFORMAT_NAMESPACE_IDS

typedef enum _KYVP_PIXELFORMAT_NAMESPACE_IDS KYVP_PIXELFORMAT_NAMESPACE_IDS
Pixel format namespaces.

KYVP_PIXELENDIANNESS_IDS

typedef enum _KYVP_PIXELENDIANNESS_IDS KYVP_PIXELENDIANNESS_IDS
Pixel data byte orders.

KYVP_PARTDATATYPE_IDS

typedef enum _KYVP_PARTDATATYPE_IDS KYVP_PARTDATATYPE_IDS

Multipart buffer data types.

KYVP_PORT_INFO_CMD

typedef enum _KYVP_PORT_INFO_CMD KYVP_PORT_INFO_CMD
Port information commands.

KYVP_URL_SCHEME_IDS

typedef enum _KYVP_URL_SCHEME_IDS KYVP_URL_SCHEME_IDS
Port URL schemes.

KYVP_URL_INFO_CMD

typedef enum _KYVP_URL_INFO_CMD KYVP_URL_INFO_CMD
Port URL information commands.

KYVP_EVENT_TYPE

typedef enum _KYVP_EVENT_TYPE KYVP_EVENT_TYPE
Event types available for registration.

KYVP_EVENT_INFO_CMD

typedef enum _KYVP_EVENT_INFO_CMD KYVP_EVENT_INFO_CMD
Event information commands.

KYVP_EVENT_DATA_INFO_CMD

typedef enum _KYVP_EVENT_DATA_INFO_CMD KYVP_EVENT_DATA_INFO_CMD
Event data information commands.

KYVP_LINK_STATUS

typedef enum _KYVP_LINK_STATUS KYVP_LINK_STATUS
Link connection states.

KYVP_LINK_STATUS_CMD

typedef enum _KYVP_LINK_STATUS_CMD KYVP_LINK_STATUS_CMD
Link status commands.

KYVP_STREAM_BUFFERS_TYPE

typedef enum _KYVP_STREAM_BUFFERS_TYPE KYVP_STREAM_BUFFERS_TYPE
Data stream buffer modes.

DS_PACKED_DATA_MODE

typedef enum _DS_PACKED_DATA_MODE DS_PACKED_DATA_MODE
Packed data layouts.

DS_PACKED_DATA_PROCESSING_MODE

typedef enum _DS_PACKED_DATA_PROCESSING_MODE DS_PACKED_DATA_PROCESSING_MODE
Packed data processing modes.

KYVP_TL_HANDLE

typedef struct _KYVP_TL_HANDLE KYVP_TL_HANDLE
Transport layer system handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVP_PCI_INTERFACE_HANDLE

```
typedef struct _KYVP_PCI_INTERFACE_HANDLE KYVP_PCI_INTERFACE_HANDLE
```

PCI interface handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVP_DEVICE_HANDLE

```
typedef struct _KYVP_DEVICE_HANDLE KYVP_DEVICE_HANDLE
```

Local device handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVP_REMOTE_DEVICE_HANDLE

```
typedef struct _KYVP_REMOTE_DEVICE_HANDLE KYVP_REMOTE_DEVICE_HANDLE
```

Remote device handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVP_STREAM_HANDLE

```
typedef struct _KYVP_STREAM_HANDLE KYVP_STREAM_HANDLE
```

Data stream handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVP_BUFFER_HANDLE

```
typedef struct _KYVP_BUFFER_HANDLE KYVP_BUFFER_HANDLE
```

Buffer handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVP_EVENT_HANDLE

```
typedef struct _KYVP_EVENT_HANDLE KYVP_EVENT_HANDLE
```

Event handle.

Treat this type as opaque. Use the handle comparison functions or macros.

Variables

KYVP_TL_HANDLE_NULL

```
const KYVP_TL_HANDLE KYVP_TL_HANDLE_NULL
```

KYVP_TL_HANDLE_INVALID

```
const KYVP_TL_HANDLE KYVP_TL_HANDLE_INVALID
```

KYVP_PCI_INTERFACE_HANDLE_NULL

```
const KYVP_PCI_INTERFACE_HANDLE KYVP_PCI_INTERFACE_HANDLE_NULL
```

KYVP_PCI_INTERFACE_HANDLE_INVALID

```
const KYVP_PCI_INTERFACE_HANDLE KYVP_PCI_INTERFACE_HANDLE_INVALID
```

KYVP_DEVICE_HANDLE_NULL

const KYVP_DEVICE_HANDLE KYVP_DEVICE_HANDLE_NULL

KYVP_DEVICE_HANDLE_INVALID

const KYVP_DEVICE_HANDLE KYVP_DEVICE_HANDLE_INVALID

KYVP_REMOTE_DEVICE_HANDLE_NULL

const KYVP_REMOTE_DEVICE_HANDLE KYVP_REMOTE_DEVICE_HANDLE_NULL

KYVP_REMOTE_DEVICE_HANDLE_INVALID

const KYVP_REMOTE_DEVICE_HANDLE KYVP_REMOTE_DEVICE_HANDLE_INVALID

KYVP_STREAM_HANDLE_NULL

const KYVP_STREAM_HANDLE KYVP_STREAM_HANDLE_NULL

KYVP_STREAM_HANDLE_INVALID

const KYVP_STREAM_HANDLE KYVP_STREAM_HANDLE_INVALID

KYVP_BUFFER_HANDLE_NULL

const KYVP_BUFFER_HANDLE KYVP_BUFFER_HANDLE_NULL

KYVP_BUFFER_HANDLE_INVALID

const KYVP_BUFFER_HANDLE KYVP_BUFFER_HANDLE_INVALID

KYVP_EVENT_HANDLE_NULL

const KYVP_EVENT_HANDLE KYVP_EVENT_HANDLE_NULL

KYVP_EVENT_HANDLE_INVALID

const KYVP_EVENT_HANDLE KYVP_EVENT_HANDLE_INVALID

Functions

KYVPLibTL_InitLib

KY_RESULT KYVPLibTL_InitLib(void)

Initializes the library and the GenTL producer driver.

Call this function before using other library functions.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_CloseLib

KY_RESULT KYVPLibTL_CloseLib(void)

Closes the library and releases the GenTL producer resources.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_GetPortURL

KY_RESULT KYVPLibTL_GetPortURL(KYVPLibTL_GetPortUr1_Args *_pArgs)

Retrieves port URL information for the system module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_GetPortUrl_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_ReadPort

KY_RESULT KYVPLibTL_ReadPort(KYVPLibTL_ReadPort_Args *_pArgs)

Reads bytes from the specified address on the system port.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_ReadPort_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_WritePort

KY_RESULT KYVPLibTL_WritePort(KYVPLibTL_WritePort_Args *_pArgs)

Writes bytes to the specified address on the system port.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_WritePort_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_PCIInterface_GetPortURL

KY_RESULT KYVPLibTL_PCIInterface_GetPortURL(KYVPLibTL_PCIInterface_GetPortUrl_Args *_pArgs)

Retrieves port URL information for the PCI interface module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_PCIInterface_GetPortUrl_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_PCIInterface_ReadPort

KY_RESULT KYVPLibTL_PCIInterface_ReadPort(KYVPLibTL_PCIInterface_ReadPort_Args *_pArgs)

Reads bytes from the specified address on the PCI interface port.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_PCIInterface_ReadPort_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_PCIInterface_WritePort

KY_RESULT KYVPLibTL_PCIInterface_WritePort(KYVPLibTL_PCIInterface_WritePort_Args *_pArgs)

Writes bytes to the specified address on the PCI interface port.

Parameter	Description
_pArgs	Pointer to the initialized

	KYVPLibTL_PCInterface_WritePort_Args arguments.
--	---

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_Device_GetPortURL

KY_RESULT KYVPLibTL_Device_GetPortURL(KYVPLibTL_Device_GetPortUrl_Args *_pArgs)

Retrieves port URL information for the device module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_Device_GetPortUrl_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_Device_ReadPort

KY_RESULT KYVPLibTL_Device_ReadPort(KYVPLibTL_Device_ReadPort_Args *_pArgs)

Reads bytes from the specified address on the device port.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_Device_ReadPort_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_Device_WritePort

KY_RESULT KYVPLibTL_Device_WritePort(KYVPLibTL_Device_WritePort_Args *_pArgs)

Writes bytes to the specified address on the device port.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_Device_WritePort_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_RemoteDevice_GetPortURL

KY_RESULT KYVPLibTL_RemoteDevice_GetPortURL(KYVPLibTL_RemoteDevice_GetPortUrl_Args *_pArgs)

Retrieves port URL information for the remote device module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_RemoteDevice_GetPortUrl_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_RemoteDevice_ReadPort

KY_RESULT KYVPLibTL_RemoteDevice_ReadPort(KYVPLibTL_RemoteDevice_ReadPort_Args *_pArgs)

Reads bytes from the specified address on the remote device port.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_RemoteDevice_ReadPort_Args

	arguments.
--	------------

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_RemoteDevice_WritePort

KY_RESULT KYVPLibTL_RemoteDevice_WritePort(KYVPLibTL_RemoteDevice_WritePort_Args *_pArgs)

Writes bytes to the specified address on the remote device port.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_RemoteDevice_WritePort_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_Stream_GetPortURL

KY_RESULT KYVPLibTL_Stream_GetPortURL(KYVPLibTL_Stream_GetPortUrl_Args *_pArgs)

Retrieves port URL information for the data stream module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_Stream_GetPortUrl_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_Stream_ReadPort

KY_RESULT KYVPLibTL_Stream_ReadPort(KYVPLibTL_Stream_ReadPort_Args *_pArgs)

Reads bytes from the specified address on the data stream port.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_Stream_ReadPort_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_Stream_WritePort

KY_RESULT KYVPLibTL_Stream_WritePort(KYVPLibTL_Stream_WritePort_Args *_pArgs)

Writes bytes to the specified address on the data stream port.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_Stream_WritePort_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_TLOpen

KY_RESULT KYVPLibTL_TLOpen(KYVPLibTL_TLOpen_Args *_pArgs)

Opens the transport layer system module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_TLOpen_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

Pre: KYVPLibTL_InitLib() has completed successfully.

See: KYVPLibTL_TLClose

KYVPLibTL_TLClose

KY_RESULT KYVPLibTL_TLClose(KYVPLibTL_TLClose_Args *_pArgs)

Closes the transport layer system module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_TLClose_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_TLGetInfo

KY_RESULT KYVPLibTL_TLGetInfo(KYVPLibTL_TLGetInfo_Args *_pArgs)

Queries information about the system module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_TLGetInfo_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_TLGetNumInterfaces

KY_RESULT KYVPLibTL_TLGetNumInterfaces(KYVPLibTL_TLGetNumInterfaces_Args *_pArgs)

Queries the number of available interfaces on this System module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_TLGetNumInterfaces_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_TLGetInterfaceID

KY_RESULT KYVPLibTL_TLGetInterfaceID(KYVPLibTL_TLGetInterfaceID_Args *_pArgs)

Queries the unique ID of the interface at iIndex in the internal interface list.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_TLGetInterfaceID_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_TLGetInterfaceInfo

KY_RESULT KYVPLibTL_TLGetInterfaceInfo(KYVPLibTL_TLGetPCIInterfaceInfo_Args *_pArgs)

Queries information about an interface on the system module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_TLGetPCIInterfaceInfo_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_TLOpenInterface

KY_RESULT KYVPLibTL_TLOpenInterface(KYVPLibTL_TLOpenInterface_Args *_pArgs)

Opens the given interface ID.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_TLOpenInterface_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_TLUpdateInterfaceList

KY_RESULT KYVPLibTL_TLUpdateInterfaceList(KYVPLibTL_TLUpdatePCIInterfaceList_Args *_pArgs)

Updates the internal list of available interfaces.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_TLUpdatePCIInterfaceList_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_IFClose

KY_RESULT KYVPLibTL_IFClose(KYVPLibTL_IFClose_Args *_pArgs)

Closes the Interface module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_IFClose_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_IFGetInfo

KY_RESULT KYVPLibTL_IFGetInfo(KYVPLibTL_IFGetInfo_Args *_pArgs)

Inquires information about the Interface module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_IFGetInfo_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_IFGetNumDevices

KY_RESULT KYVPLibTL_IFGetNumDevices(KYVPLibTL_IFGetNumDevices_Args *_pArgs)

Queries the number of available devices on this Interface module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_IFGetNumDevices_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_IFGetDeviceID

KY_RESULT KYVPLibTL_IFGetDeviceID(KYVPLibTL_IFGetDeviceID_Args *_pArgs)

Queries the unique ID of the device at iIndex in the internal device list.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_IFGetDeviceID_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_IFUpdateDeviceList

KY_RESULT KYVPLibTL_IFUpdateDeviceList(KYVPLibTL_IFUpdateDeviceList_Args *_pArgs)

Updates the internal list of available devices.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_IFUpdateDeviceList_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_IFGetDeviceInfo

KY_RESULT KYVPLibTL_IFGetDeviceInfo(KYVPLibTL_IFGetDeviceInfo_Args *_pArgs)

Inquires information about a device.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_IFGetDeviceInfo_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_IFOpenDevice

KY_RESULT KYVPLibTL_IFOpenDevice(KYVPLibTL_IFOpenDevice_Args *_pArgs)

Opens the specified device with the requested access flags.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_IFOpenDevice_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_IFRegisterEvent

KY_RESULT KYVPLibTL_IFRegisterEvent(KYVPLibTL_IFRegisterEvent_Args *_pArgs)

Registers an event on a PCI interface.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_IFRegisterEvent_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_IFUnregisterEvent

KY_RESULT KYVPLibTL_IFUnregisterEvent(KYVPLibTL_IFUnregisterEvent_Args *_pArgs)

Unregisters an event from a PCI interface.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_IFUnregisterEvent_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DevGetPort

KY_RESULT KYVPLibTL_DevGetPort(KYVPLibTL_DevGetPort_Args *_pArgs)

Retrieves the port handle for the associated remote device.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DevGetPort_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DevGetNumDataStreams

KY_RESULT KYVPLibTL_DevGetNumDataStreams(KYVPLibTL_DevGetNumDataStreams_Args *_pArgs)

Queries the number of available data streams on this Device module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DevGetNumDataStreams_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DevGetDataStreamID

KY_RESULT KYVPLibTL_DevGetDataStreamID(KYVPLibTL_DevGetDataStreamID_Args *_pArgs)

Queries the unique ID of the data stream at iIndex in the internal data stream list.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DevGetDataStreamID_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DevOpenDataStream

KY_RESULT KYVPLibTL_DevOpenDataStream(KYVPLibTL_DevOpenDataStream_Args *_pArgs)

Opens the specified data stream on a device.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DevOpenDataStream_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DevGetInfo

KY_RESULT KYVPLibTL_DevGetInfo(KYVPLibTL_DevGetInfo_Args *_pArgs)

Inquires information about a device.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DevGetInfo_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DevClose

KY_RESULT KYVPLibTL_DevClose(KYVPLibTL_DevClose_Args *_pArgs)

Closes the device module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DevClose_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DevRegisterEvent

KY_RESULT KYVPLibTL_DevRegisterEvent(KYVPLibTL_DevRegisterEvent_Args *_pArgs)

Registers an event on a device.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DevRegisterEvent_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DevUnregisterEvent

KY_RESULT KYVPLibTL_DevUnregisterEvent(KYVPLibTL_DevUnregisterEvent_Args *_pArgs)

Unregisters an event from a device.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DevUnregisterEvent_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSAnnounceBuffer

KY_RESULT KYVPLibTL_DSAnnounceBuffer(KYVPLibTL_DSAnnounceBuffer_Args *_pArgs)

Announces caller-allocated memory to a data stream.

The returned buffer handle references this memory until the buffer is revoked.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSAnnounceBuffer_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSAllocAndAnnounceBuffer

KY_RESULT KYVPLibTL_DSAllocAndAnnounceBuffer(KYVPLibTL_DSAllocAndAnnounceBuffer_Args *_pArgs)

Allocates memory for a buffer and announces it to a data stream.

The returned buffer handle references this memory until the buffer is revoked.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSAllocAndAnnounceBuffer_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSStartAcquisition

KY_RESULT KYVPLibTL_DSStartAcquisition(KYVPLibTL_DSStartAcquisition_Args *_pArgs)

Starts acquisition on the host.

Start acquisition on the remote device separately after this call.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSStartAcquisition_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSStopAcquisition

KY_RESULT KYVPLibTL_DSStopAcquisition(KYVPLibTL_DSStopAcquisition_Args *_pArgs)

Stops acquisition on the host.

Stop acquisition on the remote device separately after this call.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSStopAcquisition_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSGetInfo

KY_RESULT KYVPLibTL_DSGetInfo(KYVPLibTL_DSGetInfo_Args *_pArgs)

Queries information about the data stream module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSGetInfo_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSGetBufferID

KY_RESULT KYVPLibTL_DSGetBufferID(KYVPLibTL_DSGetBufferID_Args *_pArgs)

Queries a buffer handle by its index in the data stream.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSGetBufferID_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSClose

KY_RESULT KYVPLibTL_DSClose(KYVPLibTL_DSClose_Args *_pArgs)

Closes the data stream module.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSCLose_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSRevokeBuffer

KY_RESULT KYVPLibTL_DSRevokeBuffer(KYVPLibTL_DSRevokeBuffer_Args *_pArgs)

Removes an announced buffer from the acquisition engine.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSRevokeBuffer_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSQueueBuffer

KY_RESULT KYVPLibTL_DSQueueBuffer(KYVPLibTL_DSQueueBuffer_Args *_pArgs)

Queues a buffer for acquisition.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSQueueBuffer_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSFlushQueue

KY_RESULT KYVPLibTL_DSFlushQueue(KYVPLibTL_DSFlushQueue_Args *_pArgs)

Moves or discards queued buffers according to the requested queue operation.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSFlushQueue_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSGetBufferInfo

KY_RESULT KYVPLibTL_DSGetBufferInfo(KYVPLibTL_DSGetBufferInfo_Args *_pArgs)

Queries information about a buffer on a data stream.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSGetBufferInfo_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSRegisterEvent

KY_RESULT KYVPLibTL_DSRegisterEvent(KYVPLibTL_DSRegisterEvent_Args *_pArgs)

Registers an event on a data stream.

The implementation may differ between platforms.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSRegisterEvent_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_DSUnregisterEvent

KY_RESULT KYVPLibTL_DSUnregisterEvent(KYVPLibTL_DSUnregisterEvent_Args *_pArgs)

Unregisters the specified event from a data stream.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_DSUnregisterEvent_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_EventGetData

KY_RESULT KYVPLibTL_EventGetData(KYVPLibTL_EventGetData_Args *_pArgs)

Retrieves the next data entry from the event queue.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_EventGetData_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_EventGetDataInfo

KY_RESULT KYVPLibTL_EventGetDataInfo(KYVPLibTL_EventGetDataInfo_Args *_pArgs)

Queries information about an event data buffer.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_EventGetDataInfo_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_EventGetInfo

KY_RESULT KYVPLibTL_EventGetInfo(KYVPLibTL_EventGetInfo_Args *_pArgs)

Queries information about an event object.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_EventGetInfo_Args arguments.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYVPLibTL_EventFlush

KY_RESULT KYVPLibTL_EventFlush(KYVPLibTL_EventFlush_Args *_pArgs)

Flushes the queue of an event object.

Parameter	Description
_pArgs	Pointer to the initialized KYVPLibTL_EventFlush_Args

	arguments.
--	------------

Returns: Status and error information. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYVPLibTL_EventKill

`KY_RESULT KYVPLibTL_EventKill(KYVPLibTL_EventKill_Args *_pArgs)`

Terminates a waiting operation on a previously registered event object.

Parameter	Description
<code>_pArgs</code>	Pointer to the initialized <code>KYVPLibTL_EventKill_Args</code> arguments.

Returns: Status and error information. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYVPLibTL_GetPortURL_V1

`KY_RESULT KYVPLibTL_GetPortURL_V1(KYVP_TL_HANDLE _hTLHandle, char *_pszURL, size_t *_piSize)`

Retrieves port URL information for the system module.

Parameter	Description
<code>_hTLHandle</code>	TL Handle.
<code>_pszURL</code>	Pointer to caller-allocated buffer.
<code>_piSize</code>	Pointer to caller-allocated <code>size_t</code> buffer.

Returns: Status and error information. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

See: `KYVPLibTL_GetPortURL`

KYVPLibTL_ReadPort_V1

`KY_RESULT KYVPLibTL_ReadPort_V1(KYVP_TL_HANDLE _hTLHandle, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)`

Reads bytes from the specified address on the system port.

Parameter	Description
<code>_hTLHandle</code>	TL Handle.
<code>_iAddress</code>	Port address.
<code>_pBuffer</code>	Pointer to caller-allocated buffer.
<code>_piSize</code>	Pointer to caller-allocated <code>size_t</code> buffer.

Returns: Status and error information. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

See: `KYVPLibTL_ReadPort`

KYVPLibTL_WritePort_V1

`KY_RESULT KYVPLibTL_WritePort_V1(KYVP_TL_HANDLE _hTLHandle, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)`

Writes bytes to the specified address on the system port.

Parameter	Description
<code>_hTLHandle</code>	TL Handle.
<code>_iAddress</code>	Port address.
<code>_pBuffer</code>	Pointer to caller-allocated const buffer.

_piSize	Pointer to caller-allocated size_t buffer.
---------	--

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_WritePort

KYVPLibTL_PCIInterface_GetPortURL_V1

KY_RESULT KYVPLibTL_PCIInterface_GetPortURL_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, char *_pszURL, size_t *_piSize)

Retrieves port URL information for the PCI interface module.

Parameter	Description
_hIFHandle	Interface Handle.
_pszURL	Pointer to caller-allocated buffer.
_piSize	Pointer to caller-allocated size_t buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_PCIInterface_GetPortURL

KYVPLibTL_PCIInterface_ReadPort_V1

KY_RESULT KYVPLibTL_PCIInterface_ReadPort_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)

Reads bytes from the specified address on the PCI interface port.

Parameter	Description
_hIFHandle	Interface Handle.
_iAddress	Port address.
_pBuffer	Pointer to caller-allocated buffer.
_piSize	Pointer to caller-allocated size_t buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_PCIInterface_ReadPort

KYVPLibTL_PCIInterface_WritePort_V1

KY_RESULT KYVPLibTL_PCIInterface_WritePort_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)

Writes bytes to the specified address on the PCI interface port.

Parameter	Description
_hIFHandle	Interface Handle.
_iAddress	Port address.
_pBuffer	Pointer to caller-allocated const buffer.
_piSize	Pointer to caller-allocated size_t buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_PCIInterface_WritePort

KYVPLibTL_Device_GetPortURL_V1

KY_RESULT KYVPLibTL_Device_GetPortURL_V1(KYVP_DEVICE_HANDLE _hDevHandle, char *_pszURL, size_t *_piSize)
Retrieves port URL information for the device module.

Parameter	Description
_hDevHandle	Local device handle.
_pszURL	Pointer to caller-allocated buffer.
_piSize	Pointer to caller-allocated size_t buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_Device_GetPortURL

KYVPLibTL_Device_ReadPort_V1

KY_RESULT KYVPLibTL_Device_ReadPort_V1(KYVP_DEVICE_HANDLE _hDevHandle, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)
Reads bytes from the specified address on the device port.

Parameter	Description
_hDevHandle	Local device handle.
_iAddress	Port address.
_pBuffer	Pointer to caller-allocated buffer.
_piSize	Pointer to caller-allocated size_t buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_Device_ReadPort

KYVPLibTL_Device_WritePort_V1

KY_RESULT KYVPLibTL_Device_WritePort_V1(KYVP_DEVICE_HANDLE _hDevHandle, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)
Writes bytes to the specified address on the device port.

Parameter	Description
_hDevHandle	Local device handle.
_iAddress	Port address.
_pBuffer	Pointer to caller-allocated const buffer.
_piSize	Pointer to caller-allocated size_t buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_Device_WritePort

KYVPLibTL_RemoteDevice_GetPortURL_V1

KY_RESULT KYVPLibTL_RemoteDevice_GetPortURL_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle, char *_pszURL, size_t *_piSize)
Retrieves port URL information for the remote device module.

Parameter	Description
_hRemoteDeviceHandle	Remote device handle.

_pszURL	Pointer to caller-allocated buffer.
_piSize	Pointer to caller-allocated size_t buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_RemoteDevice_GetPortURL

KYVPLibTL_RemoteDevice_ReadPort_V1

KY_RESULT KYVPLibTL_RemoteDevice_ReadPort_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)

Reads bytes from the specified address on the remote device port.

Parameter	Description
_hRemoteDeviceHandle	Remote device handle.
_iAddress	Port address.
_pBuffer	Pointer to caller-allocated buffer.
_piSize	Pointer to caller-allocated size_t buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_RemoteDevice_ReadPort

KYVPLibTL_RemoteDevice_WritePort_V1

KY_RESULT KYVPLibTL_RemoteDevice_WritePort_V1(KYVP_REMOTE_DEVICE_HANDLE _hRemoteDeviceHandle, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)

Writes bytes to the specified address on the remote device port.

Parameter	Description
_hRemoteDeviceHandle	Remote device handle.
_iAddress	Port Address.
_pBuffer	Pointer to caller-allocated const buffer.
_piSize	Pointer to caller-allocated size_t buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_RemoteDevice_WritePort

KYVPLibTL_Stream_GetPortURL_V1

KY_RESULT KYVPLibTL_Stream_GetPortURL_V1(KYVP_STREAM_HANDLE _hStreamHandle, char *_pszURL, size_t *_piSize)

Retrieves port URL information for the data stream module.

Parameter	Description
_hStreamHandle	Stream handle.
_pszURL	Pointer to caller-allocated buffer.
_piSize	Pointer to caller-allocated size_t buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_Stream_GetPortURL

KYVPLibTL_Stream_ReadPort_V1

KY_RESULT KYVPLibTL_Stream_ReadPort_V1(KYVP_STREAM_HANDLE _hStreamHandle, uint64_t _iAddress, void *_pBuffer, size_t *_piSize)

Reads bytes from the specified address on the data stream port.

Parameter	Description
_hStreamHandle	Stream handle.
_iAddress	Port address.
_pBuffer	Pointer to caller-allocated buffer.
_piSize	Pointer to caller-allocated size_t buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_Stream_ReadPort

KYVPLibTL_Stream_WritePort_V1

KY_RESULT KYVPLibTL_Stream_WritePort_V1(KYVP_STREAM_HANDLE _hStreamHandle, uint64_t _iAddress, const void *_pBuffer, size_t *_piSize)

Writes bytes to the specified address on the data stream port.

Parameter	Description
_hStreamHandle	Stream handle.
_iAddress	Port address.
_pBuffer	Pointer to caller-allocated const buffer.
_piSize	Pointer to caller-allocated size_t buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_Stream_WritePort

KYVPLibTL_TLOpen_V1

KY_RESULT KYVPLibTL_TLOpen_V1(KYVP_TL_HANDLE *_phTLHandle)

Opens the transport layer system module.

Parameter	Description
_phTLHandle	Pointer to KYVP_TL_HANDLE.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLOpen

KYVPLibTL_TLClose_V1

KY_RESULT KYVPLibTL_TLClose_V1(KYVP_TL_HANDLE _hTLHandle)

Closes the transport layer system module.

Parameter	Description
_hTLHandle	KYVP_TL HANDLE.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLClose

KYVPLibTL_TLGetInfo_V1

KY_RESULT KYVPLibTL_TLGetInfo_V1(KYVP_TL_HANDLE _hTLHandle, KYVP_TL_INFO_CMD _eInfoCmd, KYVP_INFO_DATATYPE *_pType, void *_pBuffer, size_t *_piSize)

Queries information about the system module.

Parameter	Description
_hTLHandle	KYVP_TL_HANDLE.
_eInfoCmd	Information to be retrieved as defined in KYVP_TL_INFO_CMD.
_pType	Data type of the pBuffer content as defined in the KYVP_INFO_DATATYPE.
_pBuffer	Pointer to a caller-allocated buffer to receive the requested information. If this parameter is NULL, piSize will contain the minimal size of pBuffer in bytes. If the iType is a string the size includes the terminating 0.
_piSize	Number of bytes filled by the function.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLGetInfo

KYVPLibTL_TLGetNumInterfaces_V1

KY_RESULT KYVPLibTL_TLGetNumInterfaces_V1(KYVP_TL_HANDLE _hTLHandle, uint32_t *_piNumIfaces)

Queries the number of available interfaces on this System module.

Parameter	Description
_hTLHandle	KYVP_TL_HANDLE.
_piNumIfaces	Pointer to caller-allocated uint32_t.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLGetNumInterfaces

KYVPLibTL_TLGetInterfaceID_V1

KY_RESULT KYVPLibTL_TLGetInterfaceID_V1(KYVP_TL_HANDLE _hTLHandle, uint32_t _iIndex, char *_pszID, size_t *_piSize)

Queries the unique ID of the interface at iIndex in the internal interface list.

Parameter	Description
_hTLHandle	KYVP_TL_HANDLE.
_iIndex	Interface index.
_pszID	Pointer to caller-allocated char* buffer.
_piSize	Pointer to caller-allocated size_t* buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLGetInterfaceID

KYVPLibTL_TLGetInterfaceInfo_V1

KY_RESULT KYVPLibTL_TLGetInterfaceInfo_V1(KYVP_TL_HANDLE _hTLHandle, const char *_pszIfaceID, KYVP_INTERFACE_INFO_CMD _eInfoCmd, KYVP_INFO_DATATYPE *_pType, void *_pBuffer, size_t *_piSize)
Queries information about an interface on the system module.

Parameter	Description
_hTLHandle	KYVP_TL_HANDLE.
_pszIfaceID	Interface ID from KYVPLibTL_TLGetInterfaceID func.
_eInfoCmd	Information to be retrieved.
_pType	Data type of the pBuffer content.
_pBuffer	Pointer to a caller-allocated buffer.
_piSize	Out: minimal size of pBuffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLGetInterfaceInfo

KYVPLibTL_TLOpenInterface_V1

KY_RESULT KYVPLibTL_TLOpenInterface_V1(KYVP_TL_HANDLE _hTLHandle, const char *_pszIfaceID, KYVP_PCI_INTERFACE_HANDLE *_phPCIInterfaceHandle)
Opens the given interface ID.

Parameter	Description
_hTLHandle	KYVP_TL_HANDLE.
_pszIfaceID	Unique interface ID to open as a null-terminated C string.
_phPCIInterfaceHandle	Pointer to KYVP_PCI_INTERFACE_HANDLE.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLOpenInterface

KYVPLibTL_TLUpdateInterfaceList_V1

KY_RESULT KYVPLibTL_TLUpdateInterfaceList_V1(KYVP_TL_HANDLE _hTLHandle, KY_BOOL *_pbChanged, uint64_t _iTimeout)
Updates the internal list of available interfaces.

Parameter	Description
_hTLHandle	KYVP_TL_HANDLE.
_pbChanged	Contains KY_TRUE if the internal list was changed and KY_FALSE otherwise. If set to NULL nothing is written to this parameter.
_iTimeout	Timeout in ms.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_TLUpdateInterfaceList

KYVPLibTL_IFClose_V1

KY_RESULT KYVPLibTL_IFClose_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle)

Closes the Interface module.

Parameter	Description
_hIFHandle	System module handle to close.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFClose

KYVPLibTL_IFGetInfo_V1

KY_RESULT KYVPLibTL_IFGetInfo_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KYVP_INTERFACE_INFO_CMD _eInfoCmd, KYVP_INFO_DATATYPE *_pType, void *_pBuffer, size_t *_piSize)

Inquires information about the Interface module.

Parameter	Description
_hIFHandle	Interface module to work on.
_eInfoCmd	Information to be retrieved.
_pType	Pointer to data type of the pBuffer content.
_pBuffer	Pointer to a caller-allocated buffer.
_piSize	Number of bytes filled by the function.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFGetInfo

KYVPLibTL_IFGetNumDevices_V1

KY_RESULT KYVPLibTL_IFGetNumDevices_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint32_t *_pNumDevices)

Queries the number of available devices on this Interface module.

Parameter	Description
_hIFHandle	Interface module to work on.
_pNumDevices	Number of devices.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFGetNumDevices

KYVPLibTL_IFGetDeviceID_V1

KY_RESULT KYVPLibTL_IFGetDeviceID_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, uint32_t _iIndex, char *_pszID, size_t *_piSize)

Queries the unique ID of the device at iIndex in the internal device list.

Parameter	Description
_hIFHandle	Interface module to work on.
_iIndex	Device index.
_pszID	Pointer to a caller-allocated C string buffer to receive the Device module ID at the given iIndex.
_piSize	Minimal size of pBuffer in bytes to hold all information.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFGetDeviceID

KYVPLibTL_IFUpdateDeviceList_V1

KY_RESULT KYVPLibTL_IFUpdateDeviceList_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KY_BOOL *_pbChanged, uint64_t _uTimeout)

Updates the internal list of available devices.

Parameter	Description
_hIFHandle	Interface module to work on.
_pbChanged	Contains KY_TRUE if the internal list was changed and KY_FALSE otherwise.
_uTimeout	Timeout in ms.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFUpdateDeviceList

KYVPLibTL_IFUpdateDeviceList_V2

KY_RESULT KYVPLibTL_IFUpdateDeviceList_V2(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KY_BOOL *_pbChanged, uint64_t _uTimeout, KY_BOOL _bRetainOpenDevices)

Updates the internal list of available devices.

Parameter	Description
_hIFHandle	Interface module to work on.
_pbChanged	Contains KY_TRUE if the internal list was changed and KY_FALSE otherwise.
_uTimeout	Timeout in ms.
_bRetainOpenDevices	KY_TRUE preserves open devices in all processes. KY_FALSE closes this process's devices, clears its device lists and performs fresh detection. Devices opened by other processes are preserved in both modes. Since version 2; defaults to KY_TRUE.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFUpdateDeviceList

KYVPLibTL_IFGetDeviceInfo_V1

KY_RESULT KYVPLibTL_IFGetDeviceInfo_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, const char *_pszIfaceID, KYVP_DEVICE_INFO_CMD _eInfoCmd, KYVP_INFO_DATATYPE *_pType, void *_pBuffer, size_t *_piSize)

Inquires information about a device.

Parameter	Description
_hIFHandle	Interface module to work on.
_pszIfaceID	Unique ID of the device to inquire information about.
_eInfoCmd	Information to be retrieved.
_pType	Data type of the pBuffer content.
_pBuffer	Pointer to a caller-allocated buffer.

_piSize	Number of bytes filled by the function.
---------	---

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFGetDeviceInfo

KYVPLibTL_IFOpenDevice_V1

KY_RESULT KYVPLibTL_IFOpenDevice_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, const char *_pszDeviceID, KYVP_DEVICE_ACCESS_FLAGS _iOpenFlags, KYVP_DEVICE_HANDLE *_phDeviceHandle)

Opens the specified device with the requested access flags.

Parameter	Description
_hIFHandle	Interface module to work on.
_pszDeviceID	Unique device ID to open as a null-terminated C string.
_iOpenFlags	Configures the open process.
_phDeviceHandle	Device handle of the newly created Device module.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFOpenDevice

KYVPLibTL_IFRegisterEvent_V1

KY_RESULT KYVPLibTL_IFRegisterEvent_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KYVP_EVENT_TYPE _iEventID, KYVP_EVENT_HANDLE *_phEvent)

Registers an event on a PCI interface.

Parameter	Description
_hIFHandle	Module handle to access to register event to.
_iEventID	Event type to register.
_phEvent	New handle to an event object to work with.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFRegisterEvent

KYVPLibTL_IFUnregisterEvent_V1

KY_RESULT KYVPLibTL_IFUnregisterEvent_V1(KYVP_PCI_INTERFACE_HANDLE _hIFHandle, KYVP_EVENT_TYPE _iEventID)

Unregisters an event from a PCI interface.

Parameter	Description
_hIFHandle	Module handle to access to unregister event from.
_iEventID	Event type to register.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_IFUnregisterEvent

KYVPLibTL_DevGetPort_V1

KY_RESULT KYVPLibTL_DevGetPort_V1(KYVP_DEVICE_HANDLE _hDevHandle, KYVP_REMOTE_DEVICE_HANDLE *_phRemoteDeviceHandle)

Retrieves the port handle for the associated remote device.

Parameter	Description
_hDevHandle	Device module to work on.
_phRemoteDeviceHandle	Port handle for the remote device.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevGetPort

KYVPLibTL_DevGetNumDataStreams_V1

KY_RESULT KYVPLibTL_DevGetNumDataStreams_V1(KYVP_DEVICE_HANDLE _hDevHandle, uint32_t *_piNumDataStreams)

Queries the number of available data streams on this Device module.

Parameter	Description
_hDevHandle	Device module to work on.
_piNumDataStreams	Number of data stream on this Device module.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevGetNumDataStreams

KYVPLibTL_DevGetDataStreamID_V1

KY_RESULT KYVPLibTL_DevGetDataStreamID_V1(KYVP_DEVICE_HANDLE _hDevHandle, uint32_t _iIndex, char *_sDataStreamID, size_t *_piSize)

Queries the unique ID of the data stream at iIndex in the internal data stream list.

Parameter	Description
_hDevHandle	Device module to work on.
_iIndex	Zero-based index of the data stream on this device.
_sDataStreamID	Pointer to a caller-allocated C string buffer.
_piSize	Number of bytes filled by the function.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevGetDataStreamID

KYVPLibTL_DevOpenDataStream_V1

KY_RESULT KYVPLibTL_DevOpenDataStream_V1(KYVP_DEVICE_HANDLE _hDevHandle, const char *_pszDataStreamID, KYVP_STREAM_HANDLE *_phDataStream)

Opens the specified data stream on a device.

Parameter	Description
_hDevHandle	Device module to work on.
_pszDataStreamID	Unique data stream ID to open as a null-terminated C string.
_phDataStream	Data Stream module handle of the newly created stream.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevOpenDataStream

KYVPLibTL_DevGetInfo_V1

KY_RESULT KYVPLibTL_DevGetInfo_V1(KYVP_DEVICE_HANDLE _hDevHandle, KYVP_DEVICE_INFO_CMD _iInfoCmd, KYVP_INFO_DATATYPE *_piType, void *_pBuffer, size_t *_piSize)
Inquires information about a device.

Parameter	Description
_hDevHandle	Device module to work on.
_iInfoCmd	Information to be retrieved.
_piType	Data type of the pBuffer content.
_pBuffer	Pointer to a caller-allocated buffer.
_piSize	Number of bytes filled by the function.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevGetInfo

KYVPLibTL_DevClose_V1

KY_RESULT KYVPLibTL_DevClose_V1(KYVP_DEVICE_HANDLE _hDevHandle)
Closes the device module.

Parameter	Description
_hDevHandle	Device module handle to close.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevClose

KYVPLibTL_DevRegisterEvent_V1

KY_RESULT KYVPLibTL_DevRegisterEvent_V1(KYVP_DEVICE_HANDLE _hDevHandle, KYVP_EVENT_TYPE _iEventID, KYVP_EVENT_HANDLE *_phEvent)
Registers an event on a device.

Parameter	Description
_hDevHandle	Device handle to access to register event to.
_iEventID	Event type to register.
_phEvent	New handle to an event object to work with.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevRegisterEvent

KYVPLibTL_DevUnregisterEvent_V1

KY_RESULT KYVPLibTL_DevUnregisterEvent_V1(KYVP_DEVICE_HANDLE _hDevHandle, KYVP_EVENT_TYPE _iEventID)
Unregisters an event from a device.

Parameter	Description
_hDevHandle	Device handle to access to unregister event from.
_iEventID	Event type to unregister.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DevUnregisterEvent

KYVPLibTL_DSAnnounceBuffer_V1

KY_RESULT KYVPLibTL_DSAnnounceBuffer_V1(KYVP_STREAM_HANDLE _hDSHandle, void *_pBuffer, size_t _iSize, void *_pPrivate, KYVP_BUFFER_HANDLE *_phBuffer)

Announces caller-allocated memory to a data stream.

The returned buffer handle references this memory until the buffer is revoked.

Parameter	Description
_hDSHandle	Data Stream module to work on.
_pBuffer	Pointer to buffer memory to announce.
_iSize	Size of the pBuffer in bytes.
_pPrivate	Pointer to private data which will be passed to the GenTL Consumer on New Buffer events. This parameter may be NULL.
_phBuffer	Buffer module handle of the newly announced buffer. I.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSAnnounceBuffer

KYVPLibTL_DSAllocAndAnnounceBuffer_V1

KY_RESULT KYVPLibTL_DSAllocAndAnnounceBuffer_V1(KYVP_STREAM_HANDLE _hDSHandle, size_t _iSize, void *_pPrivate, KYVP_BUFFER_HANDLE *_phBuffer)

Allocates memory for a buffer and announces it to a data stream.

The returned buffer handle references this memory until the buffer is revoked.

Parameter	Description
_hDSHandle	Data Stream module to work on.
_iSize	Size of the buffer in bytes.
_pPrivate	Pointer to private data which will be passed to the Consumer on New Buffer events. This parameter may be NULL.
_phBuffer	Buffer module handle of the newly announced buffer.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSAllocAndAnnounceBuffer

KYVPLibTL_DSStartAcquisition_V1

KY_RESULT KYVPLibTL_DSStartAcquisition_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_ACQ_START_FLAGS _iStartFlags, uint64_t _iNumToAcquire)

Starts acquisition on the host.

Start acquisition on the remote device separately after this call.

Parameter	Description
_hDSHandle	Data Stream module to work on.

<code>_iStartFlags</code>	As defined in <code>KYVP_ACQ_START_FLAGS</code> .
<code>_iNumToAcquire</code>	Sets the number of filled/delivered buffers after which the acquisition engine stops automatically. Buffers that are internally discarded or missed are not counted. If set to 0, acquisition continues until a call to <code>KYVPLibTL_DSStopAcquisition</code> . To achieve precise accuracy and fine control over the number of acquired frames, the use of triggers is recommended.

Returns: Status and error information. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

See: `KYVPLibTL_DSStartAcquisition`

KYVPLibTL_DSStopAcquisition_V1

`KY_RESULT KYVPLibTL_DSStopAcquisition_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_ACQ_STOP_FLAGS _iStopFlags)`

Stops acquisition on the host.

Stop acquisition on the remote device separately after this call.

Parameter	Description
<code>_hDSHandle</code>	Data Stream module to work on.
<code>_iStopFlags</code>	Stops the acquisition as defined in <code>KYVP_ACQ_STOP_FLAGS</code> .

Returns: Status and error information. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

See: `KYVPLibTL_DSStopAcquisition`

KYVPLibTL_DSGetInfo_V1

`KY_RESULT KYVPLibTL_DSGetInfo_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_STREAM_INFO_CMD _eInfoCmd, KYVP_INFO_DATATYPE *_pType, void *_pBuffer, size_t *_piSize)`

Queries information about the data stream module.

Parameter	Description
<code>_hDSHandle</code>	Data Stream module to work on.
<code>_eInfoCmd</code>	Information to be retrieved.
<code>_pType</code>	Data type of the pBuffer content.
<code>_pBuffer</code>	Pointer to a caller-allocated buffer to receive the requested information.
<code>_piSize</code>	Number of bytes filled by the function.

Returns: Status and error information. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

See: `KYVPLibTL_DSGetInfo`

KYVPLibTL_DSGetBufferID_V1

`KY_RESULT KYVPLibTL_DSGetBufferID_V1(KYVP_STREAM_HANDLE _hDSHandle, uint32_t _iIndex, KYVP_BUFFER_HANDLE *_ppBufferHandle)`

Queries a buffer handle by its index in the data stream.

Parameter	Description
-----------	-------------

_hDSHandle	Data Stream module to work on.
_iIndex	Zero-based index of the buffer on this data stream.
_ppBufferHandle	Buffer module handle of the given iIndex.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSGetBufferID

KYVPLibTL_DSClose_V1

KY_RESULT KYVPLibTL_DSClose_V1(KYVP_STREAM_HANDLE _hDSHandle)

Closes the data stream module.

Parameter	Description
_hDSHandle	Data Stream module to work on.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSClose

KYVPLibTL_DSRevokeBuffer_V1

KY_RESULT KYVPLibTL_DSRevokeBuffer_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_BUFFER_HANDLE _hBufferHandle, void **_ppBuffer, void **_ppPrivate)

Removes an announced buffer from the acquisition engine.

Parameter	Description
_hDSHandle	Data Stream module to work on.
_hBufferHandle	Buffer handle to revoke.
_ppBuffer	Pointer to the buffer memory This is for convenience if consumer allocated memory is used which is to be freed. If the buffer was allocated by the GenTL Producer NULL is to be returned. If the parameter is set to NULL it is ignored.
_ppPrivate	Pointer to the user data pointer given in the announce function. If the parameter is set to NULL it is ignored.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSRevokeBuffer

KYVPLibTL_DSQueueBuffer_V1

KY_RESULT KYVPLibTL_DSQueueBuffer_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_BUFFER_HANDLE _hBufferHandle)

Queues a buffer for acquisition.

Parameter	Description
_hDSHandle	Data Stream module to work on.
_hBufferHandle	Buffer handle to queue.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSQueueBuffer

KYVPLibTL_DSFlushQueue_V1

KY_RESULT KYVPLibTL_DSFlushQueue_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_ACQ_QUEUE_TYPE _iOperation)

Moves or discards queued buffers according to the requested queue operation.

Parameter	Description
_hDSHandle	Data Stream module to work on.
_iOperation	Flush operation type.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSFlushQueue

KYVPLibTL_DSGetBufferInfo_V1

KY_RESULT KYVPLibTL_DSGetBufferInfo_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_BUFFER_HANDLE _hBufferHandle, KYVP_BUFFER_INFO_CMD _iInfoCmd, KYVP_INFO_DATATYPE *_piType, void *_pBuffer, size_t *_piSize)

Queries information about a buffer on a data stream.

Parameter	Description
_hDSHandle	Data Stream module to work on.
_hBufferHandle	Buffer handle to retrieve information about.
_iInfoCmd	Information to be retrieved.
_piType	Data type of the pBuffer content.
_pBuffer	Pointer to a caller-allocated buffer to receive the requested information.
_piSize	Number of bytes filled by the function.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSGetBufferInfo

KYVPLibTL_DSRegisterEvent_V1

KY_RESULT KYVPLibTL_DSRegisterEvent_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_EVENT_TYPE _iEventID, KYVP_EVENT_HANDLE *_phEvent)

Registers an event on a data stream.

The implementation may differ between platforms.

Parameter	Description
_hDSHandle	Module handle to access to register event to.
_iEventID	Event type to register.
_phEvent	New handle to an event object to work with.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSRegisterEvent

KYVPLibTL_DSUnregisterEvent_V1

KY_RESULT KYVPLibTL_DSUnregisterEvent_V1(KYVP_STREAM_HANDLE _hDSHandle, KYVP_EVENT_TYPE _iEventID)

Unregisters the specified event from a data stream.

Parameter	Description
_hDSHandle	Module handle to access to unregister event from.
_iEventID	Event type to register.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_DSUnregisterEvent

KYVPLibTL_EventGetData_V1

KY_RESULT KYVPLibTL_EventGetData_V1(KYVP_EVENT_HANDLE _hEventHandle, void *_pBuffer, size_t *_piSize, uint64_t _iTimeout)

Retrieves the next data entry from the event queue.

Parameter	Description
_hEventHandle	Event handle to wait for.
_pBuffer	Pointer to a caller-allocated buffer.
_piSize	Size of the provided pBuffer in bytes.
_iTimeout	Timeout for the wait in ms.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_EventGetData

KYVPLibTL_EventGetDataInfo_V1

KY_RESULT KYVPLibTL_EventGetDataInfo_V1(KYVP_EVENT_HANDLE _hEventHandle, const void *pInBuffer, size_t _iInSize, KYVP_EVENT_DATA_INFO_CMD _iInfoCmd, KYVP_INFO_DATATYPE *_piType, void *_pOutBuffer, size_t *_piOutSize)

Queries information about an event data buffer.

Parameter	Description
_hEventHandle	Event handle to parse data from.
pInBuffer	Pointer to a buffer containing event data.
_iInSize	Size of the provided pInBuffer in bytes.
_iInfoCmd	Information to be retrieved.
_piType	Data type of the pOutBuffer content.
_pOutBuffer	Pointer to a caller-allocated buffer.
_piOutSize	Number of bytes filled by the function.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_EventGetDataInfo

KYVPLibTL_EventGetInfo_V1

KY_RESULT KYVPLibTL_EventGetInfo_V1(KYVP_EVENT_HANDLE _hEventHandle, KYVP_EVENT_INFO_CMD _iInfoCmd, KYVP_INFO_DATATYPE *_piType, void *_pBuffer, size_t *_piSize)

Queries information about an event object.

Parameter	Description
_hEventHandle	Event handle to parse data from.

_iInfoCmd	Information to be retrieved.
_piType	Data type of the pBuffer content.
_pBuffer	Pointer to a caller-allocated buffer.
piSize	Number of bytes filled by the function.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_EventGetInfo

KYVPLibTL_EventFlush_V1

KY_RESULT KYVPLibTL_EventFlush_V1(KYVP_EVENT_HANDLE _hEventHandle)

Flushes the queue of an event object.

Parameter	Description
_hEventHandle	Event handle.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_EventFlush

KYVPLibTL_EventKill_V1

KY_RESULT KYVPLibTL_EventKill_V1(KYVP_EVENT_HANDLE _hEventHandle)

Terminates a waiting operation on a previously registered event object.

Parameter	Description
_hEventHandle	Event handle.

Returns: Status and error information. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

See: KYVPLibTL_EventKill

KYVPLibTL_TLHandleIsInvalid

KY_BOOL KYVPLibTL_TLHandleIsInvalid(KYVP_TL_HANDLE _hHandle)

Reports whether the handle is invalid.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

KYVPLibTL_TLHandleIsNull

KY_BOOL KYVPLibTL_TLHandleIsNull(KYVP_TL_HANDLE _hHandle)

Reports whether the handle is null.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

KYVPLibTL_TLHandlesIsEqual

KY_BOOL KYVPLibTL_TLHandlesIsEqual(KYVP_TL_HANDLE _hHandleFirst, KYVP_TL_HANDLE _hHandleSecond)

Reports whether the handles are equal.

Parameter	Description
_hHandleFirst	First handle to compare.
_hHandleSecond	Second handle to compare.

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

KYVPLibTL_IFHandleIsInvalid

KY_BOOL KYVPLibTL_IFHandleIsInvalid(KYVP_PCI_INTERFACE_HANDLE _hHandle)

Reports whether the handle is invalid.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

KYVPLibTL_IFHandleIsNull

KY_BOOL KYVPLibTL_IFHandleIsNull(KYVP_PCI_INTERFACE_HANDLE _hHandle)

Reports whether the handle is null.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

KYVPLibTL_IFHandlesAreEqual

KY_BOOL KYVPLibTL_IFHandlesAreEqual(KYVP_PCI_INTERFACE_HANDLE _hHandleFirst, KYVP_PCI_INTERFACE_HANDLE _hHandleSecond)

Reports whether the handles are equal.

Parameter	Description
_hHandleFirst	First handle to compare.
_hHandleSecond	Second handle to compare.

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

KYVPLibTL_DevHandleIsInvalid

KY_BOOL KYVPLibTL_DevHandleIsInvalid(KYVP_DEVICE_HANDLE _hHandle)

Reports whether the handle is invalid.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

KYVPLibTL_DevHandleIsNull

KY_BOOL KYVPLibTL_DevHandleIsNull(KYVP_DEVICE_HANDLE _hHandle)

Reports whether the handle is null.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

KYVPLibTL_DevHandlesIsEqual

KY_BOOL KYVPLibTL_DevHandlesIsEqual(KYVP_DEVICE_HANDLE _hHandleFirst, KYVP_DEVICE_HANDLE _hHandleSecond)

Reports whether the handles are equal.

Parameter	Description
_hHandleFirst	First handle to compare.
_hHandleSecond	Second handle to compare.

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

KYVPLibTL_RemoteDeviceHandleIsInvalid

KY_BOOL KYVPLibTL_RemoteDeviceHandleIsInvalid(KYVP_REMOTE_DEVICE_HANDLE _hHandle)

Reports whether the handle is invalid.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

KYVPLibTL_RemoteDeviceHandleIsNull

KY_BOOL KYVPLibTL_RemoteDeviceHandleIsNull(KYVP_REMOTE_DEVICE_HANDLE _hHandle)

Reports whether the handle is null.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

KYVPLibTL_RemoteDeviceHandlesIsEqual

KY_BOOL KYVPLibTL_RemoteDeviceHandlesIsEqual(KYVP_REMOTE_DEVICE_HANDLE _hHandleFirst, KYVP_REMOTE_DEVICE_HANDLE _hHandleSecond)

Reports whether the handles are equal.

Parameter	Description
_hHandleFirst	First handle to compare.
_hHandleSecond	Second handle to compare.

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

KYVPLibTL_DSHandleIsInvalid

KY_BOOL KYVPLibTL_DSHandleIsInvalid(KYVP_STREAM_HANDLE _hHandle)

Reports whether the handle is invalid.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

KYVPLibTL_DSHandleIsNull

KY_BOOL KYVPLibTL_DSHandleIsNull(KYVP_STREAM_HANDLE _hHandle)

Reports whether the handle is null.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

KYVPLibTL_DSHandlesIsEqual

KY_BOOL KYVPLibTL_DSHandlesIsEqual(KYVP_STREAM_HANDLE _hHandleFirst, KYVP_STREAM_HANDLE _hHandleSecond)

Reports whether the handles are equal.

Parameter	Description
_hHandleFirst	First handle to compare.
_hHandleSecond	Second handle to compare.

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

KYVPLibTL_BufferHandleIsInvalid

KY_BOOL KYVPLibTL_BufferHandleIsInvalid(KYVP_BUFFER_HANDLE _hHandle)

Reports whether the handle is invalid.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

KYVPLibTL_BufferHandleIsNull

KY_BOOL KYVPLibTL_BufferHandleIsNull(KYVP_BUFFER_HANDLE _hHandle)

Reports whether the handle is null.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

KYVPLibTL_BufferHandlesIsEqual

KY_BOOL KYVPLibTL_BufferHandlesIsEqual(KYVP_BUFFER_HANDLE _hHandleFirst, KYVP_BUFFER_HANDLE _hHandleSecond)

Reports whether the handles are equal.

Parameter	Description
_hHandleFirst	First handle to compare.
_hHandleSecond	Second handle to compare.

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

KYVPLibTL_EventHandleIsInvalid

KY_BOOL KYVPLibTL_EventHandleIsInvalid(KYVP_EVENT_HANDLE _hHandle)

Reports whether the handle is invalid.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is invalid; otherwise KY_FALSE.

KYVPLibTL_EventHandleIsNull

KY_BOOL KYVPLibTL_EventHandleIsNull(KYVP_EVENT_HANDLE _hHandle)

Reports whether the handle is null.

Parameter	Description
_hHandle	Handle to inspect.

Returns: KY_TRUE if the handle is null; otherwise KY_FALSE.

KYVPLibTL_EventHandlesEqual

KY_BOOL KYVPLibTL_EventHandlesEqual(KYVP_EVENT_HANDLE _hHandleFirst, KYVP_EVENT_HANDLE _hHandleSecond)

Reports whether the handles are equal.

Parameter	Description
_hHandleFirst	First handle to compare.
_hHandleSecond	Second handle to compare.

Returns: KY_TRUE if the handles are equal; otherwise KY_FALSE.

KYVP_TL_HANDLE

Transport layer system handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVP_PCI_INTERFACE_HANDLE

PCI interface handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVP_DEVICE_HANDLE

Local device handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVP_REMOTE_DEVICE_HANDLE

Remote device handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVP_STREAM_HANDLE

Data stream handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVP_BUFFER_HANDLE

Buffer handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVP_EVENT_HANDLE

Event handle.

Treat this type as opaque. Use the handle comparison functions or macros.

KYVPLibTL Structs

Field reference for function arguments and event payloads.

C++ member defaults initialize structure versions and default values. C callers must set version to the constant shown above each table and initialize the fields required by the operation explicitly. Zero initialization alone does not select a valid structure version.

Each table follows the declaration order and lists the exact public field types. Handle types are opaque. Use the documented API to work with them.

See: KYVPLibTL API

KYVPLIBTL_EVENT_NEW_BUFFER_DATA

Event data for a newly available buffer.

Structure Field	Type	Description	Remarks
hBufferHandle	KYVP_BUFFER_HANDLE	Buffer handle which contains new data.	
pUserPointer	void*	User pointer provided at announcement of the buffer.	

KYVP_PCIINTERFACE_EVENT_TEMPERATURE

PCI interface temperature event data.

Initialize version to KYVP_PCIINTERFACE_EVENT_TEMPERATURE_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required .
eTemperatureThresholdId	KYVP_PCIINTERFACE_EVENT_TEMPERATURE_THRESHOLD_ID	Indicates the current temperature threshold state of the PCI interface.	

KYVP_PCIINTERFACE_EVENT_CXP2_HEARTBEAT

PCI interface CXP 2.0 heartbeat event data.

Initialize version to KYVP_PCIINTERFACE_EVENT_CXP2_HEARTBEAT_VERSION.

Structure Field	Type	Description	Remarks
-----------------	------	-------------	---------

version	uint32_t	Structure version.	Required.
deviceHandle	KYVP_DEVICE_HANDLE	Handle of the device associated with the event.	
heartBeat	KYVP_CXP2_HEARTBEAT	CXP 2.0 heartbeat data structure.	

KYVP_PCIINTERFACE_EVENT_CXP2_EVENT

PCI interface CXP 2.0 event data.

Initialize version to KYVP_PCIINTERFACE_EVENT_CXP2_EVENT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
deviceHandle	KYVP_DEVICE_HANDLE	Handle of the device associated with the event.	
cxp2Event	KYVP_CXP2_EVENT	CXP 2.0 event data structure containing event messages.	

KYVP_PCIINTERFACE_GENERIC_EVENT

PCI interface event identifier and payload.

Initialize version to KYVP_PCIINTERFACE_GENERIC_EVENT_VERSION.

Note: The data union contains one of the following structures depending on eEventId:

- KYVP_PCIINTERFACE_EVENT_TEMPERATURE
- KYVP_PCIINTERFACE_EVENT_CXP2_HEARTBEAT
- KYVP_PCIINTERFACE_EVENT_CXP2_EVENT

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
eEventId	KYVP_PCIINTERFACE_EVENT_ID	Identifier of the event type.	
data	union	Event-specific data. The active member is determined by eEventId.	

KYVP_DEVICE_EVENT_CONNECTION_LOST

Device connection lost event data.

Initialize version to KYVP_DEVICE_EVENT_CONNECTION_LOST_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.

hDevHandle	KYVP_DEVICE_HANDLE	Handle of the device associated with the event.	
------------	--------------------	---	--

KYVP_DEVICE_GENERIC_EVENT

Device event identifier and payload.

Initialize version to KYVP_DEVICE_GENERIC_EVENT_VERSION.

Note: The data union contains one of the following structures depending on eEventId:

- KYVP_DEVICE_EVENT_CONNECTION_LOST

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
eEventId	KYVP_DEVICE_EVENT_ID	Identifier of the event type.	
data	union	Event-specific data. The active member is determined by eEventId.	

KYVPLibTL_GetPortUrl_Args

Arguments for KYVPLibTL_GetPortURL().

Initialize version to KYVPLIBTL_GETPORTURL_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hTLHandle	KYVP_TL_HANDLE	Transport layer system handle.	
pszURL	char*	Pointer to caller-allocated buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_ReadPort_Args

Arguments for KYVPLibTL_ReadPort().

Initialize version to KYVPLIBTL_READPORT_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hTLHandle	KYVP_TL_HANDLE	Transport layer system handle.	
iAddress	uint64_t	Port address.	
pBuffer	void*	Pointer to caller-	

		allocated buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_WritePort_Args

Arguments for KYVPLibTL_WritePort().

Initialize version to KYVPLIBTL_WRITEPORT_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hTLHandle	KYVP_TL_HANDLE	Transport layer system handle.	
iAddress	uint64_t	Port address.	
pBuffer	const void*	Pointer to caller-allocated const buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_PCIInterface_GetPortUrl_Args

Arguments for KYVPLibTL_PCIInterface_GetPortURL().

Initialize version to KYVPLIBTL_PCIINTERFACEGETPORTURL_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface handle.	
pszURL	char*	Pointer to caller-allocated buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_PCIInterface_ReadPort_Args

Arguments for KYVPLibTL_PCIInterface_ReadPort().

Initialize version to KYVPLIBTL_PCIINTERFACEREADPORT_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface handle.	
iAddress	uint64_t	Port address.	
pBuffer	void*	Pointer to caller-allocated buffer.	

piSize	size_t*	Pointer to caller-allocated size_t buffer.	
--------	---------	--	--

KYVPLibTL_PCIInterface_WritePort_Args

Arguments for KYVPLibTL_PCIInterface_WritePort().

Initialize version to KYVPLIBTL_PCIINTERFACEWRITEPORT_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface handle.	
iAddress	uint64_t	Port address.	
pBuffer	const void*	Pointer to caller-allocated const buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_Device_GetPortUrl_Args

Arguments for KYVPLibTL_Device_GetPortURL().

Initialize version to KYVPLIBTL_DEVICEGETPORTURL_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDevHandle	KYVP_DEVICE_HANDLE	Local device handle.	
pszURL	char*	Pointer to caller-allocated buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_Device_ReadPort_Args

Arguments for KYVPLibTL_Device_ReadPort().

Initialize version to KYVPLIBTL_DEVICEREADPORT_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDevHandle	KYVP_DEVICE_HANDLE	Local device handle.	
iAddress	uint64_t	Port address.	
pBuffer	void*	Pointer to caller-allocated buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_Device_WritePort_Args

Arguments for KYVPLibTL_Device_WritePort().

Initialize version to KYVPLIBTL_DEVICEWRITEPORT_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDevHandle	KYVP_DEVICE_HANDLE	Local device handle.	
iAddress	uint64_t	Port address.	
pBuffer	const void*	Pointer to caller-allocated const buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_RemoteDevice_GetPortUrl_Args

Arguments for KYVPLibTL_RemoteDevice_GetPortURL().

Initialize version to KYVPLIBTL_REMOTEDEVICEGETPORTURL_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hRemoteDeviceHandle	KYVP_REMOTE_DEVICE_HANDLE	Remote device handle.	
pszURL	char*	Pointer to caller-allocated buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_RemoteDevice_ReadPort_Args

Arguments for KYVPLibTL_RemoteDevice_ReadPort().

Initialize version to KYVPLIBTL_REMOTEDEVICEREADPORT_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hRemoteDeviceHandle	KYVP_REMOTE_DEVICE_HANDLE	Remote device handle.	
iAddress	uint64_t	Port address.	
pBuffer	void*	Pointer to caller-allocated buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_RemoteDevice_WritePort_Args

Arguments for KYVPLibTL_RemoteDevice_WritePort().

Initialize version to KYVPLIBTL_REMOTEDEVICEWRITEPORT_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hRemoteDeviceHandle	KYVP_REMOTE_DEVICE_HANDLE	Remote device handle.	
iAddress	uint64_t	Port Address.	
pBuffer	const void*	Pointer to caller-allocated const buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_Stream_GetPortUrl_Args

Arguments for KYVPLibTL_Stream_GetPortURL().

Initialize version to KYVPLIBTL_STREAMGETPORTURL_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hStreamHandle	KYVP_STREAM_HANDLE	Stream handle.	
pszURL	char*	Pointer to caller-allocated buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_Stream_ReadPort_Args

Arguments for KYVPLibTL_Stream_ReadPort().

Initialize version to KYVPLIBTL_STREAMREADPORT_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hStreamHandle	KYVP_STREAM_HANDLE	Stream handle.	
iAddress	uint64_t	Port address.	
pBuffer	void*	Pointer to caller-allocated buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_Stream_WritePort_Args

Arguments for KYVPLibTL_Stream_WritePort().

Initialize version to KYVPLIBTL_STREAMWRITEPORT_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
-----------------	------	-------------	---------

version	uint32_t	Structure version.	Required.
hStreamHandle	KYVP_STREAM_HANDLE	Stream handle.	
iAddress	uint64_t	Port address.	
pBuffer	const void*	Pointer to caller-allocated const buffer.	
piSize	size_t*	Pointer to caller-allocated size_t buffer.	

KYVPLibTL_TLOpen_Args

Arguments for KYVPLibTL_TLOpen().

Initialize version to KYVPLIBTL_TLOPEN_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
phTLHandle	KYVP_TL_HANDLE *	Pointer to KYVP_TL_HANDLE.	

KYVPLibTL_TLClose_Args

Arguments for KYVPLibTL_TLClose().

Initialize version to KYVPLIBTL_TLCLOSE_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hTLHandle	KYVP_TL_HANDLE	Transport layer system handle.	

KYVPLibTL_TLGetInfo_Args

Arguments for KYVPLibTL_TLGetInfo().

Initialize version to KYVPLIBTL_TLGETINFO_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hTLHandle	KYVP_TL_HANDLE	Transport layer system handle.	
eInfoCmd	KYVP_TL_INFO_CMD	Information to be retrieved as defined in KYVP_TL_INFO_CMD.	
pType	KYVP_INFO_DATATYPE *	Data type of the pBuffer content as defined in the KYVP_INFO_DATATYPE.	
pBuffer	void*	Pointer to a caller-allocated buffer to	

		receive the requested information. If this parameter is NULL, piSize will contain the minimal size of pBuffer in bytes. If the iType is a string the size includes the terminating 0.	
piSize	size_t*	Number of bytes filled by the function.	

KYVPLibTL_TLGetNumInterfaces_Args

Arguments for KYVPLibTL_TLGetNumInterfaces().

Initialize version to KYVPLIBTL_TLGETNUMINTERFACES_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hTLHandle	KYVP_TL_HANDLE	Transport layer system handle.	
piNumIfaces	uint32_t*	Pointer to caller-allocated uint32_t.	

KYVPLibTL_TLGetInterfaceID_Args

Arguments for KYVPLibTL_TLGetInterfaceID().

Initialize version to KYVPLIBTL_TLGETINTERFACEID_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hTLHandle	KYVP_TL_HANDLE	Transport layer system handle.	
iIndex	uint32_t	Interface index.	
pszID	char*	Pointer to caller-allocated char* buffer.	
piSize	size_t*	Pointer to caller-allocated size_t* buffer.	

KYVPLibTL_TLGetPCIInterfaceInfo_Args

Arguments for KYVPLibTL_TLGetInterfaceInfo().

Initialize version to KYVPLIBTL_TLGETPCIINTERFACEINFO_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.

hTLHandle	KYVP_TL_HANDLE	Transport layer system handle.	
pszIfacelD	const char*	Interface ID from KYVPLibTL_TLGetInterfaceID func.	
eInfoCmd	KYVP_INTERFACE_INFO_CMD	Information to be retrieved.	
pType	KYVP_INFO_DATATYPE *	Data type of the pBuffer content.	
pBuffer	void*	Pointer to a caller-allocated buffer.	
piSize	size_t*	Out: minimal size of pBuffer.	

KYVPLibTL_TLOpenInterface_Args

Arguments for KYVPLibTL_TLOpenInterface().

Initialize version to KYVPLIBTL_TLOPENPCIINTERFACE_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hTLHandle	KYVP_TL_HANDLE	Transport layer system handle.	
pszIfacelD	const char*	Unique interface ID to open as a null-terminated C string.	
phPCIInterfaceHandle	KYVP_PCI_INTERFACE_HANDLE *	Pointer to KYVP_PCI_INTERFACE_HANDLE.	

KYVPLibTL_TLUpdatePCIInterfaceList_Args

Arguments for KYVPLibTL_TLUpdateInterfaceList().

Initialize version to KYVPLIBTL_TLUPDATEPCIINTERFACELIST_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hTLHandle	KYVP_TL_HANDLE	Transport layer system handle.	
pbChanged	KY_BOOL *	Contains KY_TRUE if the internal list was changed and KY_FALSE otherwise. If set to NULL nothing is written to this parameter.	
iTimeout	uint64_t	Timeout in ms.	

KYVPLibTL_IFClose_Args

Arguments for KYVPLibTL_IFClose().

Initialize version to KYVPLIBTL_IFCLOSE_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface handle to close.	

KYVPLibTL_IFGetInfo_Args

Arguments for KYVPLibTL_IFGetInfo().

Initialize version to KYVPLIBTL_IFGETINFO_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface handle.	
eInfoCmd	KYVP_INTERFACE_INFO_CMD	Information to be retrieved.	
pType	KYVP_INFO_DATATYPE *	Pointer to data type of the pBuffer content.	
pBuffer	void*	Pointer to a caller-allocated buffer.	
piSize	size_t*	Number of bytes filled by the function.	

KYVPLibTL_IFGetNumDevices_Args

Arguments for KYVPLibTL_IFGetNumDevices().

Initialize version to KYVPLIBTL_IFGETNUMDEVICES_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface handle.	
pNumDevices	uint32_t*	Number of devices.	

KYVPLibTL_IFGetDeviceID_Args

Arguments for KYVPLibTL_IFGetDeviceID().

Initialize version to KYVPLIBTL_IFGETDEVICEID_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface handle.	

iIndex	uint32_t	Device index.	
pszID	char*	Pointer to a caller-allocated C string buffer to receive the Device module ID at the given iIndex.	
piSize	size_t*	Minimum size of pszID in bytes to hold the device ID.	

KYVPLibTL_IFUpdateDeviceList_Args

Arguments for KYVPLibTL_IFUpdateDeviceList().

Initialize version to KYVPLIBTL_IFUPDETEDEVICELIST_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface handle.	
pbChanged	KY_BOOL *	Contains KY_TRUE if the internal list was changed and KY_FALSE otherwise.	
uTimeout	uint64_t	Timeout in ms.	
bRetainOpenDevices	KY_BOOL	KY_TRUE preserves open devices in all processes. KY_FALSE closes this process's devices, clears its device lists and performs fresh detection. Devices opened by other processes are preserved in both modes.	Since version 2; defaults to KY_TRUE.

KYVPLibTL_IFGetDeviceInfo_Args

Arguments for KYVPLibTL_IFGetDeviceInfo().

Initialize version to KYVPLIBTL_IFGETDEVICEINFO_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface handle.	
pszIfacelD	const char*	Unique ID of the device to inquire information about.	

eInfoCmd	KYVP_DEVICE_INFO_CMD	Information to be retrieved.	
pType	KYVP_INFO_DATATYPE *	Data type of the pBuffer content.	
pBuffer	void*	Pointer to a caller-allocated buffer.	
piSize	size_t*	Number of bytes filled by the function.	

KYVPLibTL_IFOpenDevice_Args

Arguments for KYVPLibTL_IFOpenDevice().

Initialize version to KYVPLIBTL_IFOPENDEVICE_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	PCI interface handle.	
pszDeviceID	const char*	Unique device ID to open as a null-terminated C string.	
iOpenFlags	KYVP_DEVICE_ACCESS_FLAGS	Configures the open process.	
phDeviceHandle	KYVP_DEVICE_HANDLE *	Device handle of the newly created Device module.	

KYVPLibTL_IFRegisterEvent_Args

Arguments for KYVPLibTL_IFRegisterEvent().

Initialize version to KYVPLIBTL_IFREGISTEREVENT_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	Handle of the module on which to register the event.	
iEventID	KYVP_EVENT_TYPE	Event type to register.	
phEvent	KYVP_EVENT_HANDLE *	Pointer that receives the new event handle.	

KYVPLibTL_IFUnregisterEvent_Args

Arguments for KYVPLibTL_IFUnregisterEvent().

Initialize version to KYVPLIBTL_IFUNREGISTEREVENT_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hIFHandle	KYVP_PCI_INTERFACE_HANDLE	Handle of the module from which to unregister the event.	
iEventID	KYVP_EVENT_TYPE	Event type to unregister.	

KYVPLibTL_DevGetPort_Args

Arguments for KYVPLibTL_DevGetPort().

Initialize version to KYVPLIBTL_DEVGETPORT_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDevHandle	KYVP_DEVICE_HANDLE	Device handle.	
phRemoteDeviceHandle	KYVP_REMOTE_DEVICE_HANDLE *	Port handle for the remote device.	

KYVPLibTL_DevGetNumDataStreams_Args

Arguments for KYVPLibTL_DevGetNumDataStreams().

Initialize version to KYVPLIBTL_DEVGETNUMDATASTREAMS_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDevHandle	KYVP_DEVICE_HANDLE	Device handle.	
piNumDataStreams	uint32_t*	Pointer that receives the number of data streams on this device.	

KYVPLibTL_DevGetDataStreamID_Args

Arguments for KYVPLibTL_DevGetDataStreamID().

Initialize version to KYVPLIBTL_DEVGETDATASTREAMID_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDevHandle	KYVP_DEVICE_HANDLE	Device handle.	
iIndex	uint32_t	Zero-based index of the data stream on this device.	
sDataStreamID	char*	Pointer to a caller-allocated C string buffer.	
piSize	size_t*	Number of bytes filled by	

		the function.	
--	--	---------------	--

KYVPLibTL_DevOpenDataStream_Args

Arguments for KYVPLibTL_DevOpenDataStream().

Initialize version to KYVPLIBTL_DEVOPENDATASTREAM_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDevHandle	KYVP_DEVICE_HANDLE	Device handle.	
pszDataStreamID	const char*	Unique data stream ID to open as a null-terminated C string.	
phDataStream	KYVP_STREAM_HANDLE *	Data Stream module handle of the newly created stream.	

KYVPLibTL_DevGetInfo_Args

Arguments for KYVPLibTL_DevGetInfo().

Initialize version to KYVPLIBTL_DEVGETINFO_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDevHandle	KYVP_DEVICE_HANDLE	Device handle.	
iInfoCmd	KYVP_DEVICE_INFO_CMD	Information to be retrieved.	
piType	KYVP_INFO_DATATYPE *	Data type of the pBuffer content.	
pBuffer	void*	Pointer to a caller-allocated buffer.	
piSize	size_t*	Number of bytes filled by the function.	

KYVPLibTL_DevClose_Args

Arguments for KYVPLibTL_DevClose().

Initialize version to KYVPLIBTL_DEVCLOSE_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDevHandle	KYVP_DEVICE_HANDLE	Device module handle to close.	

KYVPLibTL_DevRegisterEvent_Args

Arguments for KYVPLibTL_DevRegisterEvent().

Initialize version to KYVPLIBTL_DEVREGISTEREVENT_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDevHandle	KYVP_DEVICE_HANDLE	Handle of the device on which to register the event.	
iEventID	KYVP_EVENT_TYPE	Event type to register.	
phEvent	KYVP_EVENT_HANDLE *	Pointer that receives the new event handle.	

KYVPLibTL_DevUnregisterEvent_Args

Arguments for KYVPLibTL_DevUnregisterEvent().

Initialize version to KYVPLIBTL_DEVUNREGISTEREVENT_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDevHandle	KYVP_DEVICE_HANDLE	Handle of the device from which to unregister the event.	
iEventID	KYVP_EVENT_TYPE	Event type to unregister.	

KYVPLibTL_DSAnnounceBuffer_Args

Arguments for KYVPLibTL_DSAnnounceBuffer().

Initialize version to KYVPLIBTL_DSANNOUNCEBUFFER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Data stream handle.	
pBuffer	void*	Pointer to buffer memory to announce.	
iSize	size_t	Size of the pBuffer in bytes.	
pPrivate	void*	Pointer to private data which will be passed to the GenTL Consumer on New Buffer events. This parameter may be NULL.	
phBuffer	KYVP_BUFFER_HANDLE *	Buffer module handle of the newly announced	

		buffer.	
--	--	---------	--

KYVPLibTL_DSAllocAndAnnounceBuffer_Args

Arguments for KYVPLibTL_DSAllocAndAnnounceBuffer().

Initialize version to KYVPLIBTL_DSALLOCANDANNOUNCEBUFFER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Data stream handle.	
iSize	size_t	Size of the buffer in bytes.	
pPrivate	void*	Pointer to private data which will be passed to the Consumer on New Buffer events. This parameter may be NULL.	
phBuffer	KYVP_BUFFER_HANDLE *	Buffer module handle of the newly announced buffer.	

KYVPLibTL_DSStartAcquisition_Args

Arguments for KYVPLibTL_DSStartAcquisition().

Initialize version to KYVPLIBTL_DSSTARTACQUISITION_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Data stream handle.	
iStartFlags	KYVP_ACQ_START_FLAGS	As defined in KYVP_ACQ_START_FLAGS.	
iNumToAcquire	uint64_t	Sets the number of filled/delivered buffers after which the acquisition engine stops automatically. Buffers that are internally discarded or missed are not counted. If set to 0, acquisition continues until a call to KYVPLibTL_DSStopAcquisition. To achieve precise accuracy and fine control over the number of acquired frames, the use of triggers is recommended.	

KYVPLibTL_DSStopAcquisition_Args

Arguments for KYVPLibTL_DSStopAcquisition().

Initialize version to KYVPLIBTL_DSSTOPACQUISITION_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Data stream handle.	
iStopFlags	KYVP_ACQ_STOP_FLAGS	Stops the acquisition as defined in KYVP_ACQ_STOP_FLAGS.	

KYVPLibTL_DSGetInfo_Args

Arguments for KYVPLibTL_DSGetInfo().

Initialize version to KYVPLIBTL_DSGETINFO_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Data stream handle.	
eInfoCmd	KYVP_STREAM_INFO_CMD	Information to be retrieved.	
pType	KYVP_INFO_DATATYPE *	Data type of the pBuffer content.	
pBuffer	void*	Pointer to a caller-allocated buffer to receive the requested information.	
piSize	size_t*	Number of bytes filled by the function.	

KYVPLibTL_DSGetBufferID_Args

Arguments for KYVPLibTL_DSGetBufferID().

Initialize version to KYVPLIBTL_DSGETBUFFEREID_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Data stream handle.	
iIndex	uint32_t	Zero-based index of the buffer on this data stream.	
ppBufferHandle	KYVP_BUFFER_HANDLE *	Buffer module handle of the given iIndex.	

KYVPLibTL_DSCLose_Args

Arguments for KYVPLibTL_DSCLose().

Initialize version to KYVPLIBTL_DSCLose_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Data stream handle.	

KYVPLibTL_DSRevokeBuffer_Args

Arguments for KYVPLibTL_DSRevokeBuffer().

Initialize version to KYVPLIBTL_DSREVOKEBUFFER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Data stream handle.	
hBufferHandle	KYVP_BUFFER_HANDLE	Buffer handle to revoke.	
ppBuffer	void	Pointer to the buffer memory This is for convenience if consumer allocated memory is used which is to be freed. If the buffer was allocated by the GenTL Producer NULL is to be returned. If the parameter is set to NULL it is ignored.	
ppPrivate	void	Pointer to the user data pointer given in the announce function. If the parameter is set to NULL it is ignored.	

KYVPLibTL_DSQueueBuffer_Args

Arguments for KYVPLibTL_DSQueueBuffer().

Initialize version to KYVPLIBTL_DSQUEUEBUFFER_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Data stream handle.	
hBufferHandle	KYVP_BUFFER_HANDLE	Buffer handle to queue.	

KYVPLibTL_DSFlushQueue_Args

Arguments for KYVPLibTL_DSFlushQueue().

Initialize version to KYVPLIBTL_DSFLUSHQUEUE_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Data stream handle.	
iOperation	KYVP_ACQ_QUEUE_TYPE	Flush operation type.	

KYVPLibTL_DSGetBufferInfo_Args

Arguments for KYVPLibTL_DSGetBufferInfo().

Initialize version to KYVPLIBTL_DSGETBUFFERINFO_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Data stream handle.	
hBufferHandle	KYVP_BUFFER_HANDLE	Buffer handle to retrieve information about.	
iInfoCmd	KYVP_BUFFER_INFO_CMD	Information to be retrieved.	
piType	KYVP_INFO_DATATYPE *	Data type of the pBuffer content.	
pBuffer	void*	Pointer to a caller-allocated buffer to receive the requested information.	
piSize	size_t*	Number of bytes filled by the function.	

KYVPLibTL_DSRegisterEvent_Args

Arguments for KYVPLibTL_DSRegisterEvent().

Initialize version to KYVPLIBTL_DSREGISTEREVENT_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Handle of the module on which to register the event.	
iEventID	KYVP_EVENT_TYPE	Event type to register.	
phEvent	KYVP_EVENT_HANDLE *	Pointer that receives the new event handle.	

KYVPLibTL_DSUnregisterEvent_Args

Arguments for KYVPLibTL_DSUnregisterEvent().

Initialize version to KYVPLIBTL_DSUNREGISTEREVENT_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hDSHandle	KYVP_STREAM_HANDLE	Handle of the module from which to unregister the event.	
iEventID	KYVP_EVENT_TYPE	Event type to unregister.	

KYVPLibTL_EventGetData_Args

Arguments for KYVPLibTL_EventGetData().

Initialize version to KYVPLIBTL_EVENTGETDATA_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hEventHandle	KYVP_EVENT_HANDLE	Event handle to wait for.	
pBuffer	void*	Pointer to a caller-allocated buffer.	
piSize	size_t*	Size of the provided pBuffer in bytes.	
iTimeout	uint64_t	Timeout for the wait in ms.	

KYVPLibTL_EventGetDataInfo_Args

Arguments for KYVPLibTL_EventGetDataInfo().

Initialize version to KYVPLIBTL_EVENTGETDATAINFO_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hEventHandle	KYVP_EVENT_HANDLE	Event handle to parse data from.	
pInBuffer	const void*	Pointer to a buffer containing event data.	
iInSize	size_t	Size of the provided pInBuffer in bytes.	
iInfoCmd	KYVP_EVENT_DATA_INFO_CMD	Information to be retrieved.	
piType	KYVP_INFO_DATATYPE *	Data type of the pOutBuffer content.	
pOutBuffer	void*	Pointer to a caller-allocated buffer.	

piOutSize	size_t*	Number of bytes filled by the function.	
-----------	---------	---	--

KYVPLibTL_EventGetInfo_Args

Arguments for KYVPLibTL_EventGetInfo().

Initialize version to KYVPLIBTL_EVENTGETINFO_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hEventHandle	KYVP_EVENT_HANDLE	Event handle to parse data from.	
iInfoCmd	KYVP_EVENT_INFO_CMD	Information to be retrieved.	
piType	KYVP_INFO_DATATYPE *	Data type of the pBuffer content.	
pBuffer	void*	Pointer to a caller-allocated buffer.	
piSize	size_t*	Number of bytes filled by the function.	

KYVPLibTL_EventFlush_Args

Arguments for KYVPLibTL_EventFlush().

Initialize version to KYVPLIBTL_EVENTFLUSH_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hEventHandle	KYVP_EVENT_HANDLE	Event handle.	

KYVPLibTL_EventKill_Args

Arguments for KYVPLibTL_EventKill().

Initialize version to KYVPLIBTL_EVENTKILL_ARGS_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	Required.
hEventHandle	KYVP_EVENT_HANDLE	Event handle.	

KYVPNodeDescriptor API

KYVPNodeDescriptor.h API reference

Generated from PUBLIC/include/KYVPNodeDescriptor.h.

API documentation generated from PUBLIC/include/KYVPNodeDescriptor.h.

Public enum values, structure packing, member order, and C++ special members form part of the SDK ABI.

Enumerations

_KYVP_NodeDescriptorType

Descriptor notification classifications.

Values

Name	Description
KYVP_NodeDescriptorType_Invalid	No valid descriptor notification classification.
KYVP_NodeDescriptorType_NewNode	Descriptor classification for a newly introduced parameter node.
KYVP_NodeDescriptorType_NewEnumEntry	Descriptor classification for a newly introduced enumeration entry.
KYVP_NodeDescriptorType_UpdateNode	Descriptor classification for an update to an existing node.

_KYVP_ParameterInterfaceType

GenApi principal interface types exposed by a parameter.

Values

Name	Description
KYVP_ParameterInterfaceType_IValue	Generic value interface with textual conversion.
KYVP_ParameterInterfaceType_IBase	Base interface exposing access mode.
KYVP_ParameterInterfaceType_IInteger	Integer value interface.
KYVP_ParameterInterfaceType_IBoolean	Boolean value interface.
KYVP_ParameterInterfaceType_ICommand	Command execution and completion interface.
KYVP_ParameterInterfaceType_IFloat	Floating-point value interface.
KYVP_ParameterInterfaceType_IString	String value interface.
KYVP_ParameterInterfaceType_IRegister	Raw register data interface.
KYVP_ParameterInterfaceType_ICategory	Category containing related feature nodes.
KYVP_ParameterInterfaceType_IEnumeration	Enumeration with named entries.

KYVP_ParameterInterfaceType_IEnumEntry	One named entry of an enumeration.
KYVP_ParameterInterfaceType_IPort	Addressed device-port access interface.
KYVP_ParameterInterfaceType_IUnknown	Unknown or unrecognized principal interface.

_KYVP_ParameterRepresentation

Hints for displaying numeric parameter values.

Values

Name	Description
KYVP_ParameterRepresentation_Linear	Display with a linear scale.
KYVP_ParameterRepresentation_Logarithmic	Display with a logarithmic scale.
KYVP_ParameterRepresentation_Boolean	Display as a Boolean control.
KYVP_ParameterRepresentation_PureNumber	Display as a decimal number.
KYVP_ParameterRepresentation_HexNumber	Display as a hexadecimal number.
KYVP_ParameterRepresentation_IPv4Address	Display in IPv4 address notation.
KYVP_ParameterRepresentation_MACAddress	Display in MAC address notation.
KYVP_ParameterRepresentation_UndefinedRepresentation	No defined display representation.

_KYVP_ParameterVisibility

User-interface visibility levels; these do not grant access rights.

Values

Name	Description
KYVP_ParameterVisibility_Beginner	Visible at the beginner level and above.
KYVP_ParameterVisibility_Expert	Visible at the expert level and above.
KYVP_ParameterVisibility_Guru	Visible at the guru level.
KYVP_ParameterVisibility_Invisible	Hidden from the normal feature view.
KYVP_ParameterVisibility_Undefined	Undefined visibility; SDK value 99 (decimal).
KYVP_ParameterVisibility_Developer	KAYA developer visibility extension; SDK value 200 (decimal).

_KYVP_ParameterAttributeType

Parameter properties available through the attribute query API.

Values

Name	Description
KYVP_ParameterAttributeType_Invalidator	The pInvalidator property naming a node whose changes invalidate this node cache.

KYVP_ParameterAttributeType_Invalid	Invalid or unsupported attribute selector.
-------------------------------------	--

Type definitions

KYVP_NodeDescriptorType

typedef enum _KYVP_NodeDescriptorType KYVP_NodeDescriptorType
Descriptor notification classifications.

KYVP_ParameterInterfaceType

typedef enum _KYVP_ParameterInterfaceType KYVP_ParameterInterfaceType
GenApi principal interface types exposed by a parameter.

KYVP_ParameterRepresentation

typedef enum _KYVP_ParameterRepresentation KYVP_ParameterRepresentation
Hints for displaying numeric parameter values.

KYVP_ParameterVisibility

typedef enum _KYVP_ParameterVisibility KYVP_ParameterVisibility
User-interface visibility levels; these do not grant access rights.

KYVP_ParameterAttributeType

typedef enum _KYVP_ParameterAttributeType KYVP_ParameterAttributeType
Parameter properties available through the attribute query API.

PARAMETER_CALLBACK_FUNC

typedef void(* PARAMETER_CALLBACK_FUNC) (const KYVP_NodeDescriptor *_pNodeDescriptor, void *_pContext))(const KYVP_NodeDescriptor *_pNodeDescriptor, void *_pContext)

Receives a notification with the current parameter descriptor.

The callback runs synchronously in the node-notification path. Keep it short and do not let C++ exceptions cross the callback boundary.

Parameter	Description
_pNodeDescriptor	Library-owned descriptor; inspect or copy the required data during the callback.
_pContext	Application context supplied when the callback was registered.

KYVP Node Descriptor

KYVP_NodeDescriptor

Library-owned metadata and cached values for a parameter node. Interpret value and range fields using m_eInterfaceType; unrelated fields can retain defaults. The descriptor and its referenced strings belong to the collection. Treat returned data as read-only. Values and string addresses can change after node updates. Copy any information you need to retain and do not use these pointers after the collection is deleted or reinitialized.

The C++ constructor stores the supplied string pointers and implementation flag; it does not copy strings. C++ supports move construction. Copy construction and both assignment operators are deleted.

Structure Field	Type	Description	Remarks
m_nVersion	int	Descriptor layout version.	SDK default: 0; populated by the library.
m_szParamName	const char*	Node name used by parameter APIs.	Library-owned string.
m_szParamDisplayName	const char*	Human-readable feature name.	Falls back to the node name when no display name is supplied.
m_szToolTip	const char*	Short feature help text.	Library-owned string.
m_szDescription	const char*	Detailed feature description.	Library-owned string.
m_szPhysicalUnitName	const char*	Physical unit label for a numeric feature.	May be empty.
m_eDescriptorType	KYVP_NodeDescriptorType	Descriptor notification classification.	May retain the Invalid default.
m_eInterfaceType	KYVP_ParameterInterfaceType	Principal interface determining how to interpret value fields.	
m_eRepresentation	KYVP_ParameterRepresentation	Suggested numeric display format.	Presentation hint.
m_eVisibility	KYVP_ParameterVisibility	Suggested feature visibility level.	Independent of read/write access.
m_nLogicalIndex	int64_t	Logical index reserved for enumeration-entry presentation.	SDK default: 0.
m_pParentNode	const struct _KYVP_NodeDescriptor*	Parent category or enumeration descriptor.	Library-owned; NULL when no parent is assigned.
m_bIsSelector	KY_BOOL	Whether this feature selects other features.	
m_szSelectorName	const char*	Name of a feature selecting this node.	May be empty; represents one selector.
m_nCurSelectorValue	int64_t	Cached selector value slot.	May retain its default when not populated.
m_bIsImplemented	const KY_BOOL	Implementation flag fixed when the descriptor is constructed.	Read-only member.
m_bIsAvailable	KY_BOOL	Cached availability of	May change with

		the node.	device state.
m_bIsWritable	KY_BOOL	Cached write-access flag.	
m_bIsReadable	KY_BOOL	Cached read-access flag.	
m_nMinIntValue	int64_t	Integer minimum or enumeration entry index.	For an enumeration, the selected entry index; for an entry, its index.
m_nMaxIntValue	int64_t	Integer maximum or enumeration entry count.	Interpret according to m_eInterfaceType.
m_nIncIntValue	int64_t	Integer value increment.	Applies to integer nodes.
m_nCurIntValue	int64_t	Current integer value or numeric enumeration value.	For an enum entry, its assigned value.
m_fMinFloatValue	double	Minimum floating-point value.	Applies to float nodes.
m_fMaxFloatValue	double	Maximum floating-point value.	Applies to float nodes.
m_fIncFloatValue	double	Floating-point increment when supplied by the node.	A default value does not establish a supported increment.
m_nFloatDisplayPrecision	int64_t	Suggested floating-point display precision.	Formatting hint supplied by GenApi.
m_fCurFloatValue	double	Current floating-point value.	Applies to float nodes.
m_bCurBoolValue	KY_BOOL	Current Boolean value.	Applies to Boolean nodes.
m_szCurStringValue	const char*	Current string, enum-entry symbolic name, or register data address.	Register data is binary; use the value API to obtain its length.
m_bTracing	KY_BOOL	Tracing flag reserved for diagnostics.	SDK default: KY_FALSE.

KYVPPParametersHandler API

KYVPPParametersHandler.h API reference

Generated from PUBLIC/include/KYVPPParametersHandler.h.

API documentation generated from PUBLIC/include/KYVPPParametersHandler.h.

Initialize the library, create a collection, register its transport, and initialize the collection before reading or writing parameter nodes. Delete each collection when finished. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to inspect returned status values. Public declarations, version defaults, and packed layouts form part of the SDK ABI.

Macros

KYVPPARAMLIB_EXTERNAL_DLL

Parameters Handler DLL import/export attribute on Microsoft compilers.

KYVPPARAMLIB_API

`KYVP_EXTERNAL_C KYVPPARAMLIB_EXTERNAL_DLL`

Public Parameters Handler function linkage and visibility.

KYVPPARAMLIB_VAR

`KYVP_EXTERNAL KYVPPARAMLIB_EXTERNAL_DLL`

Public Parameters Handler variable linkage and visibility.

KYVP_MAX_ERROR_STRING_BUFFER_SIZE

`1024`

Convenience capacity in bytes for an error-string buffer.

COLLECTION_INVALID_HANDLE_INIT_VALUE

`((void*)-1)`

Internal invalid-handle initializer; applications should use the public sentinel.

COLLECTION_HANDLE_IS_INVALID

`KYParametersHandler_CollectionHandleIsValid(_hHandle)`

Test a collection handle against the invalid sentinel.

COLLECTION_HANDLE_IS_NULL

`KYParametersHandler_CollectionHandleIsNull(_hHandle)`

Test a collection handle against the null sentinel.

COLLECTION_HANDLES_IS_EQUAL

`KYParametersHandler_CollectionHandlesIsEqual(_hHandleFirst, _hHandleSecond)`

Compare two collection handle identities.

KYPARAMETERSHANDLER_TRANSPORTCALLPARAMS_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_INITPARAMETERS_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_INITIALIZE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_CREATEPARAMETERCOLLECTION_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_DELETEPARAMETERCOLLECTION_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_REGISTERPARAMETERCOLLECTIONTRANSPORT_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_UNREGISTERPARAMETERCOLLECTIONTRANSPORT_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_REGISTERPARAMETERCOLLECTIONCALLBACK_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_UNREGISTERPARAMETERCOLLECTIONCALLBACK_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_INITIALIZEPARAMETERCOLLECTION_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_GETNODEDESCRIPTOR_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_GETNODEDESCRIPTORSARRAY_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_GETVALUE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_SETVALUE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_GETVALUEENUMASSTRING_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_SETVALUEENUMBYVALUENAME_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_INVALIDATEPARAMETERCOLLECTION_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_INVALIDATEPARAMETER_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_SAVETOFILE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYVP_PARAMETER_NAME_REMAP_MAX_CHARS

128

Capacity of each remap-name array, including the NUL terminator.

KYPARAMETERSHANDLER_SETPARAMETERNAME_REMAPTABLE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_CLEARPARAMETERNAME_REMAPTABLE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_GETPARAMETERNAME_REMAPTABLESIZE_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_ADDPARAMETERNAME_REMAPENTRY_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_REMOVEPARAMETERNAME_REMAPENTRY_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

KYPARAMETERSHANDLER_GETPARAMETERNAME_REMAPENTRYBYINDEX_STRUCT_VERSION

1

Version 1 layout identifier for the corresponding Parameters Handler arguments.

Enumerations

_KYParametersHandler_TransportOperationType

Operations requested from the transport callback.

Values

Name	Description
KYParametersHandler_TransportOperationType_Read	Read iSize bytes at iAddress into the supplied pData buffer.
KYParametersHandler_TransportOperationType_Write	Write iSize bytes from the supplied pData buffer to iAddress.
KYParametersHandler_TransportOperationType_GetResourceData	Supply the GenICam XML resource through pData, iSize, and bResourceDataZipped.

_KYParametersHandler_TransportOperationResult

Result supplied by the transport callback.

Values

Name	Description
KYParametersHandler_TransportOperationResult_Success	The requested transport operation completed successfully.
KYParametersHandler_TransportOperationResult_Failed	The requested transport operation failed.
KYParametersHandler_TransportOperationResult_Timeout	The requested transport operation timed out.

Type definitions

KYParametersHandler_TransportOperationType

```
typedef enum _KYParametersHandler_TransportOperationType KYParametersHandler_TransportOperationType
```

Operations requested from the transport callback.

KYParametersHandler_TransportOperationResult

```
typedef enum _KYParametersHandler_TransportOperationResult KYParametersHandler_TransportOperationResult
```

Result supplied by the transport callback.

TRANSPORT_FUNC

```
typedef void(* TRANSPORT_FUNC) (KYParametersHandler_TransportCallParams_Args  
*_pArgs)(KYParametersHandler_TransportCallParams_Args *_pArgs)
```

Implements synchronous register access and XML-resource retrieval for a collection.

Read eOperation and fill the outputs documented in KYParametersHandler_TransportCallParams_Args. Set eResult before returning. Do not let C++ exceptions cross the callback boundary.

Parameter	Description
_pArgs	Transport request and callback outputs.

Variables

KYVP_COLLECTION_HANDLE_NULL

const KYVP_COLLECTION_HANDLE KYVP_COLLECTION_HANDLE_NULL
Null collection-handle sentinel.

KYVP_COLLECTION_HANDLE_INVALID

const KYVP_COLLECTION_HANDLE KYVP_COLLECTION_HANDLE_INVALID
Invalid collection-handle sentinel.

Functions

KYParametersHandler_Initialize

KY_RESULT KYParametersHandler_Initialize(KYParametersHandler_Initialize_Args *_pArgs)
Initialize the Parameters Handler library.

Call before creating a parameter collection.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_Initialize_Args; initialize its version and required input fields.

See: KYParametersHandler_Initialize_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_CreateParameterCollection

KY_RESULT KYParametersHandler_CreateParameterCollection(KYParametersHandler_CreateParameterCollection_Args *_pArgs)

Create an empty parameter collection.

Register its transport and initialize the collection before accessing nodes.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_CreateParameterCollection_Args; initialize its version and required input fields.

See: KYParametersHandler_CreateParameterCollection_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_RegisterParameterCollectionTransport

KY_RESULT
KYParametersHandler_RegisterParameterCollectionTransport(KYParametersHandler_RegisterParameterCollectionTransport_Args *_pArgs)

Register the transport callback and its application context.

Registration replaces the previously registered transport callback.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_RegisterParameterCollectionTransport_Args; initialize its version and required input fields.

See: KYParametersHandler_RegisterParameterCollectionTransport_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_RegisterParameterCollectionCallback

KY_RESULT

KYParametersHandler_RegisterParameterCollectionCallback(KYParametersHandler_RegisterParameterCollectionCallback_Args *_pArgs)

Register a callback for parameter-node updates.

Registration replaces the previously registered node callback.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_RegisterParameterCollectionCallback_Args; initialize its version and required input fields.

See: KYParametersHandler_RegisterParameterCollectionCallback_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_InitializeParameterCollection

KY_RESULT KYParametersHandler_InitializeParameterCollection(KYParametersHandler_InitializeParameterCollection_Args *_pArgs)

Load the transport-supplied GenICam XML and construct the node collection.

A transport callback must be registered. It supplies the XML resource using GetResourceData.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_InitializeParameterCollection_Args; initialize its version and required input fields.

See: KYParametersHandler_InitializeParameterCollection_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_DeleteParameterCollection

KY_RESULT KYParametersHandler_DeleteParameterCollection(KYParametersHandler_DeleteParameterCollection_Args *_pArgs)

Delete a collection and release its nodes and cached resource.

All descriptors and other library-owned pointers from this collection become invalid.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_DeleteParameterCollection_Args; initialize its version and required input fields.

See: KYParametersHandler_DeleteParameterCollection_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_UnregisterParameterCollectionTransport

KY_RESULT

KYParametersHandler_UnregisterParameterCollectionTransport(KYParametersHandler_UnregisterParameterCollectionTransport_Args *_pArgs)

Remove the transport callback and its context.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_UnregisterParameterCollectionTransport_Args; initialize its version and required input fields.

See: KYParametersHandler_UnregisterParameterCollectionTransport_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_UnregisterParameterCollectionCallback

KY_RESULT

KYParametersHandler_UnregisterParameterCollectionCallback(KYParametersHandler_UnregisterParameterCollectionCallback_Args *_pArgs)

Remove the parameter-node callback and its context.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_UnregisterParameterCollectionCallback_Args; initialize its version and required input fields.

See: KYParametersHandler_UnregisterParameterCollectionCallback_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_GetNodeDescriptor

KY_RESULT KYParametersHandler_GetNodeDescriptor(KYParametersHandler_GetNodeDescriptor_Args *_pArgs)

Return the library-owned descriptor for a named node.

Treat the returned descriptor as read-only. Its lifetime is tied to the initialized collection.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_GetNodeDescriptor_Args; initialize its version and required input fields.

See: KYParametersHandler_GetNodeDescriptor_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_GetNodeDescriptorsArray

KY_RESULT KYParametersHandler_GetNodeDescriptorsArray(KYParametersHandler_GetNodeDescriptorsArray_Args *_pArgs)

Return the library-owned contiguous descriptor array and its element count.

This returns an array of descriptors, not an array of descriptor pointers. Do not free or modify the array; its lifetime is tied to the initialized collection.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_GetNodeDescriptorsArray_Args;

	initialize its version and required input fields.
--	---

See: `KYParametersHandler_GetNodeDescriptorsArray_V1()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_GetValue

`KY_RESULT KYParametersHandler_GetValue(KYParametersHandler_GetValue_Args *_pArgs)`

Read a parameter value or query its required storage size.

Pass a NULL value buffer to query the required byte count for a supported value node. Use storage of the type and size described in Parameter value storage.

Parameter	Description
<code>_pArgs</code>	Required pointer to <code>KYParametersHandler_GetValue_Args</code> ; initialize its version and required input fields.

See: `KYParametersHandler_GetValue_V1()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_SetValue

`KY_RESULT KYParametersHandler_SetValue(KYParametersHandler_SetValue_Args *_pArgs)`

Write a parameter value, or execute a command node.

Supply a non-NULL value pointer and byte-count pointer. See Parameter value storage.

Parameter	Description
<code>_pArgs</code>	Required pointer to <code>KYParametersHandler_SetValue_Args</code> ; initialize its version and required input fields.

See: `KYParametersHandler_SetValue_V1()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_GetValueEnumAsString

`KY_RESULT KYParametersHandler_GetValueEnumAsString(KYParametersHandler_GetValueEnumAsString_Args *_pArgs)`

Read the symbolic name of the current enumeration entry.

Pass a NULL destination to query the byte count including the terminator. An undersized destination reports the required count without copying the string.

Parameter	Description
<code>_pArgs</code>	Required pointer to <code>KYParametersHandler_GetValueEnumAsString_Args</code> ; initialize its version and required input fields.

See: `KYParametersHandler_GetValueEnumAsString_V1()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_SetValueEnumByValueName

`KY_RESULT KYParametersHandler_SetValueEnumByValueName(KYParametersHandler_SetValueEnumByValueName_Args *_pArgs)`

Select an enumeration entry by its symbolic name.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_SetValueEnumByValueName_Args; initialize its version and required input fields.

See: KYParametersHandler_SetValueEnumByValueName_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_InvalidateParameterCollection

KY_RESULT KYParametersHandler_InvalidateParameterCollection(KYParametersHandler_InvalidateParameterCollection_Args *_pArgs)

Invalidate cached values throughout a collection.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_InvalidateParameterCollection_Args; initialize its version and required input fields.

See: KYParametersHandler_InvalidateParameterCollection_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_InvalidateParameter

KY_RESULT KYParametersHandler_InvalidateParameter(KYParametersHandler_InvalidateParameter_Args *_pArgs)

Invalidate the cached value of a named parameter.

A valid, non-NULL parameter name is required. Use KYParametersHandler_InvalidateParameterCollection() to invalidate all nodes.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_InvalidateParameter_Args; initialize its version and required input fields.

See: KYParametersHandler_InvalidateParameter_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_SaveToFile

KY_RESULT KYParametersHandler_SaveToFile(KYParametersHandler_SaveToFile_Args *_pArgs)

Write the collection's original XML or ZIP resource to a file.

The original resource bytes are saved, preserving their compressed or uncompressed form. This does not serialize current parameter values. An existing destination file is overwritten.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_SaveToFile_Args; initialize its version and required input fields.

See: KYParametersHandler_SaveToFile_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_GetParameterAttributeValue

KY_RESULT KYParametersHandler_GetParameterAttributeValue(KYParametersHandler_GetParameterAttributeValueArgs *_pArgs)

Read a supported GenApi property as text.

Pass a NULL property buffer first to obtain its required byte count, including the terminator. Allocate at least that many bytes before retrieving the text; the caller is responsible for sufficient capacity.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_GetParameterAttributeValueArgs; initialize its version and required input fields.

See: KYParametersHandler_GetParameterAttributeValue_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_IsParameterAvailable

KY_RESULT KYParametersHandler_IsParameterAvailable(KYParametersHandler_IsParameterAvailableArgs *_pArgs)

Check whether a named node exists in the collection.

This checks collection membership. Inspect the descriptor access flags for current device availability and read/write access.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_IsParameterAvailableArgs; initialize its version and required input fields.

See: KYParametersHandler_IsParameterAvailable_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_SetParameterNameRemapTable

KY_RESULT KYParametersHandler_SetParameterNameRemapTable(KYParametersHandler_SetParameterNameRemapTable_Args *_pArgs)

Replace the complete parameter-name remapping table.

Entries are copied. A zero count clears the table. Validate every entry before calling: replacement clears the old table before processing the new entries, and an invalid entry can leave a partial table.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_SetParameterNameRemapTable_Args; initialize its version and required input fields.

See: KYParametersHandler_SetParameterNameRemapTable_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_AddParameterNameRemapEntry

KY_RESULT KYParametersHandler_AddParameterNameRemapEntry(KYParametersHandler_AddParameterNameRemapEntry_Args *_pArgs)

Add a parameter-name mapping or replace the first matching source name.

The entry is copied into the collection.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_AddParameterNameRemapEntry_Args; initialize its version and required input fields.

See: KYParametersHandler_AddParameterNameRemapEntry_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_RemoveParameterNameRemapEntry

KY_RESULT KYParametersHandler_RemoveParameterNameRemapEntry(KYParametersHandler_RemoveParameterNameRemapEntry_Args *_pArgs)

Remove the first mapping for a source parameter name.

A missing source name is not an error. The optional output reports whether an entry was removed.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_RemoveParameterNameRemapEntry_Args; initialize its version and required input fields.

See: KYParametersHandler_RemoveParameterNameRemapEntry_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_GetParameterNameRemapEntryByIndex

KY_RESULT
KYParametersHandler_GetParameterNameRemapEntryByIndex(KYParametersHandler_GetParameterNameRemapEntryByIndex_Args *_pArgs)

Copy one parameter-name mapping by its zero-based index.

Query the table size first; the index must be less than the current entry count.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_GetParameterNameRemapEntryByIndex_Args; initialize its version and required input fields.

See: KYParametersHandler_GetParameterNameRemapEntryByIndex_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_GetParameterNameRemapTableSize

KY_RESULT
KYParametersHandler_GetParameterNameRemapTableSize(KYParametersHandler_GetParameterNameRemapTableSize_Args *_pArgs)

Return the number of parameter-name mappings.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_GetParameterNameRemapTableSize_Args; initialize its version and required input fields.

See: KYParametersHandler_GetParameterNameRemapTableSize_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_ClearParameterNameRemapTable

KY_RESULT KYParametersHandler_ClearParameterNameRemapTable(KYParametersHandler_ClearParameterNameRemapTable_Args *_pArgs)

Remove all parameter-name mappings from a collection.

Parameter	Description
_pArgs	Required pointer to KYParametersHandler_ClearParameterNameRemapTable_Args; initialize its version and required input fields.

See: KYParametersHandler_ClearParameterNameRemapTable_V1()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_CollectionHandleIsInvalid

KY_BOOL KYParametersHandler_CollectionHandleIsInvalid(KYVP_COLLECTION_HANDLE _hCollectionHandle)

Test whether a handle equals the invalid sentinel.

These comparisons do not validate whether a collection is still alive.

Parameter	Description
_hCollectionHandle	Collection handle to compare.

Returns: KY_TRUE when the comparison matches; KY_FALSE otherwise.

KYParametersHandler_CollectionHandleIsNull

KY_BOOL KYParametersHandler_CollectionHandleIsNull(KYVP_COLLECTION_HANDLE _hCollectionHandle)

Test whether a handle equals the null sentinel.

These comparisons do not validate whether a collection is still alive.

Parameter	Description
_hCollectionHandle	Collection handle to compare.

Returns: KY_TRUE when the comparison matches; KY_FALSE otherwise.

KYParametersHandler_CollectionHandlesIsEqual

KY_BOOL KYParametersHandler_CollectionHandlesIsEqual(KYVP_COLLECTION_HANDLE _hCollectionHandleFirst, KYVP_COLLECTION_HANDLE _hCollectionHandleSecond)

Test whether two collection handles have the same native identity.

These comparisons do not validate whether a collection is still alive.

Parameter	Description
_hCollectionHandleFirst	Collection handle to compare.
_hCollectionHandleSecond	Collection handle to compare.

Returns: KY_TRUE when the comparison matches; KY_FALSE otherwise.

KYParametersHandler_Initialize_V1

KY_RESULT KYParametersHandler_Initialize_V1(KYParametersHandler_InitParameters *_pKYParametersHandler_InitParameters, char *_pszErrorString, size_t *_piErrorStringSize)

Initialize the Parameters Handler library.

Call before creating a parameter collection.

Parameter	Description
_pKYParametersHandler_InitParameters	Required initialization options; initialize the nested version.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_Initialize()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_CreateParameterCollection_V1

KY_RESULT KYParametersHandler_CreateParameterCollection_V1(KYVP_COLLECTION_HANDLE *_phCollectionHandle, char *_pszErrorString, size_t *_piErrorStringSize)

Create an empty parameter collection.

Register its transport and initialize the collection before accessing nodes.

Parameter	Description
_phCollectionHandle	Receives the new collection handle. Required.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_CreateParameterCollection()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_DeleteParameterCollection_V1

KY_RESULT KYParametersHandler_DeleteParameterCollection_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, char *_pszErrorString, size_t *_piErrorStringSize)

Delete a collection and release its nodes and cached resource.

All descriptors and other library-owned pointers from this collection become invalid.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_DeleteParameterCollection()

Note: The handle is passed by value; discard the caller's copy after successful deletion.

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_RegisterParameterCollectionTransport_V1

KY_RESULT KYParametersHandler_RegisterParameterCollectionTransport_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, TRANSPORT_FUNC _pTRANSPORT_FUNC, void *_pContext, char *_pszErrorString, size_t *_piErrorStringSize)

Register the transport callback and its application context.

Registration replaces the previously registered transport callback.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pTRANSPORT_FUNC	Callback implementing register and XML-resource transport. Required.
_pContext	Application context passed to the registered callback. May be NULL.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_RegisterParameterCollectionTransport()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_UnregisterParameterCollectionTransport_V1

KY_RESULT KYParametersHandler_UnregisterParameterCollectionTransport_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, char *_pszErrorString, size_t *_piErrorStringSize)

Remove the transport callback and its context.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_UnregisterParameterCollectionTransport()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_RegisterParameterCollectionCallback_V1

KY_RESULT KYParametersHandler_RegisterParameterCollectionCallback_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, PARAMETER_CALLBACK_FUNC _pPARAMETER_CALLBACK_FUNC, void *_pContext, char *_pszErrorString, size_t *_piErrorStringSize)

Register a callback for parameter-node updates.

Registration replaces the previously registered node callback.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.

<code>_pPARAMETER_CALLBACK_FUNC</code>	Callback receiving node updates. Required.
<code>_pContext</code>	Application context passed to the registered callback. May be NULL.
<code>_pszErrorString</code>	Optional diagnostic text buffer. See Optional diagnostic buffers.
<code>_piErrorStringSize</code>	Pointer to diagnostic buffer capacity in bytes. Optional; required when <code>pszErrorString</code> is supplied.

See: `KYParametersHandler_RegisterParameterCollectionCallback()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_UnregisterParameterCollectionCallback_V1

`KY_RESULT KYParametersHandler_UnregisterParameterCollectionCallback_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, char *_pszErrorString, size_t *_piErrorStringSize)`

Remove the parameter-node callback and its context.

Parameter	Description
<code>_hCollectionHandle</code>	Collection to operate on. Valid collection handle required.
<code>_pszErrorString</code>	Optional diagnostic text buffer. See Optional diagnostic buffers.
<code>_piErrorStringSize</code>	Pointer to diagnostic buffer capacity in bytes. Optional; required when <code>pszErrorString</code> is supplied.

See: `KYParametersHandler_UnregisterParameterCollectionCallback()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_InitializeParameterCollection_V1

`KY_RESULT KYParametersHandler_InitializeParameterCollection_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, char *_pszErrorString, size_t *_piErrorStringSize)`

Load the transport-supplied GenICam XML and construct the node collection.

A transport callback must be registered. It supplies the XML resource using `GetResourceData`.

Parameter	Description
<code>_hCollectionHandle</code>	Collection to operate on. Valid collection handle required.
<code>_pszErrorString</code>	Optional diagnostic text buffer. See Optional diagnostic buffers.
<code>_piErrorStringSize</code>	Pointer to diagnostic buffer capacity in bytes. Optional; required when <code>pszErrorString</code> is supplied.

See: `KYParametersHandler_InitializeParameterCollection()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_GetNodeDescriptor_V1

`KY_RESULT KYParametersHandler_GetNodeDescriptor_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, KYVP_NodeDescriptor **_ppNodeDescriptor, char *_pszErrorString, size_t *_piErrorStringSize)`

Return the library-owned descriptor for a named node.

Treat the returned descriptor as read-only. Its lifetime is tied to the initialized collection.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszParamName	Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies.
_ppNodeDescriptor	Receives a pointer to the library-owned node descriptor. Required; treat the descriptor as read-only.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: `KYParametersHandler_GetNodeDescriptor()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_GetNodeDescriptorsArray_V1

`KY_RESULT KYParametersHandler_GetNodeDescriptorsArray_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const KYVP_NodeDescriptor **_ppArray, size_t *_piSize, char *_pszErrorString, size_t *_piErrorStringSize)`

Return the library-owned contiguous descriptor array and its element count.

This returns an array of descriptors, not an array of descriptor pointers. Do not free or modify the array; its lifetime is tied to the initialized collection.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_ppArray	Receives the base address of the library-owned descriptor array. Required; elements are descriptors, not pointers.
_piSize	Required pointer receiving the number of descriptors.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: `KYParametersHandler_GetNodeDescriptorsArray()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_GetValue_V1

`KY_RESULT KYParametersHandler_GetValue_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, void *_pParamValue, size_t *_piParamValueSize, char *_pszErrorString, size_t *_piErrorStringSize)`

Read a parameter value or query its required storage size.

Pass a NULL value buffer to query the required byte count for a supported value node. Use storage of the type and size described in Parameter value storage.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszParamName	Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies.
_pParamValue	Value buffer; NULL queries required size when reading. See Parameter value storage.
_piParamValueSize	Required pointer to value-buffer size in bytes; receives size on a NULL-buffer query.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_GetValue()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_SetValue_V1

KY_RESULT KYParametersHandler_SetValue_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, const void *_pParamValue, size_t *_piParamValueSize, char *_pszErrorString, size_t *_piErrorStringSize)

Write a parameter value, or execute a command node.

Supply a non-NULL value pointer and byte-count pointer. See Parameter value storage.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszParamName	Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies.
_pParamValue	Required value buffer. See Parameter value storage.
_piParamValueSize	Required pointer to input value size in bytes; the count is not updated.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_SetValue()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_GetValueEnumAsString_V1

KY_RESULT KYParametersHandler_GetValueEnumAsString_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszEnumerationName, char *_pszParamValueString, size_t *_piParamValueStringSize, char *_pszErrorString, size_t *_piErrorStringSize)

Read the symbolic name of the current enumeration entry.

Pass a NULL destination to query the byte count including the terminator. An undersized destination reports the required count without copying the string.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszEnumerationName	Name of the enumeration to read. Non-NULL, NUL-terminated string; remapping applies.
_pszParamValueString	Buffer receiving the current enumeration entry symbolic name. NULL queries the required size.
_piParamValueStringSize	Pointer to destination capacity in bytes, including the terminator. Required; updated on a size query or insufficient capacity.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: `KYParametersHandler_GetValueEnumAsString()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_SetValueEnumByValueName_V1

`KY_RESULT KYParametersHandler_SetValueEnumByValueName_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, const char *_pszParamValue, char *_pszErrorString, size_t *_piErrorStringSize)`

Select an enumeration entry by its symbolic name.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszParamName	Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies.
_pszParamValue	Symbolic name of the enumeration entry to select. Non-NULL, NUL-terminated string.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: `KYParametersHandler_SetValueEnumByValueName()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_SetValueEnum_V1

`KY_RESULT KYParametersHandler_SetValueEnum_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, int64_t _Value, char *_pszErrorString, size_t *_piErrorStringSize)`

Set an enumeration to its numeric entry value.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszParamName	Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies.
_Value	Value to write.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_SetValue()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_SetValueInt_V1

KY_RESULT KYParametersHandler_SetValueInt_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, int64_t _Value, char *_pszErrorString, size_t *_piErrorStringSize)

Write a signed 64-bit integer parameter value.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszParamName	Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies.
_Value	Value to write.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_SetValue()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_SetValueBool_V1

KY_RESULT KYParametersHandler_SetValueBool_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, KY_BOOL _Value, char *_pszErrorString, size_t *_piErrorStringSize)

Write a Boolean parameter value.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszParamName	Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies.
_Value	Value to write.
_pszErrorString	Optional diagnostic text buffer. See Optional

	diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_SetValue()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_InvalidateParameterCollection_V1

KY_RESULT KYParametersHandler_InvalidateParameterCollection_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, char *_pszErrorString, size_t *_piErrorStringSize)

Invalidate cached values throughout a collection.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_InvalidateParameterCollection()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_InvalidateParameter_V1

KY_RESULT KYParametersHandler_InvalidateParameter_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, char *_pszErrorString, size_t *_piErrorStringSize)

Invalidate the cached value of a named parameter.

A valid, non-NULL parameter name is required. Use KYParametersHandler_InvalidateParameterCollection() to invalidate all nodes.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszParamName	Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_InvalidateParameter()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_SaveToFile_V1

KY_RESULT KYParametersHandler_SaveToFile_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszFilePath, char *_pszErrorString, size_t *_piErrorStringSize)

Write the collection's original XML or ZIP resource to a file.

The original resource bytes are saved, preserving their compressed or uncompressed form. This does not serialize current parameter values. An existing destination file is overwritten.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszFilePath	Destination path for the original XML or ZIP resource. Non-NULL, NUL-terminated string.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: `KYParametersHandler_SaveToFile()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_GetParameterAttributeValue_V1

`KY_RESULT KYParametersHandler_GetParameterAttributeValue_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszParamName, KYVP_ParameterAttributeType _eAttributeType, char *_pszPropertyValue, size_t *_piPropertyValueSize, char *_pszErrorString, size_t *_piErrorStringSize)`

Read a supported GenApi property as text.

Pass a NULL property buffer first to obtain its required byte count, including the terminator. Allocate at least that many bytes before retrieving the text; the caller is responsible for sufficient capacity.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszParamName	Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies.
_eAttributeType	GenApi property to query. Currently supports Invalidator.
_pszPropertyValue	Buffer receiving the property text. NULL queries the required byte count.
_piPropertyValueSize	Receives the property text size, including its terminator. Required; query before allocating the text buffer.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: `KYParametersHandler_GetParameterAttributeValue()`

Returns: Operation status. Use `KY_RESULT_SUCCEEDED()` or `KY_RESULT_FAILED()` to test the result.

KYParametersHandler_IsParameterAvailable_V1

KY_RESULT KYParametersHandler_IsParameterAvailable_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *pszParamName, KY_BOOL *_pbIsAvailable, char *pszErrorString, size_t *_piErrorStringSize)

Check whether a named node exists in the collection.

This checks collection membership. Inspect the descriptor access flags for current device availability and read/write access.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
pszParamName	Name of the parameter to access. Non-NULL, NUL-terminated string; remapping applies.
_pbIsAvailable	Required pointer receiving whether the node exists in the collection.
_pszErrorString	Optional diagnostic text buffer. See Optional diagnostic buffers.
_piErrorStringSize	Pointer to diagnostic buffer capacity in bytes. Optional; required when pszErrorString is supplied.

See: KYParametersHandler_IsParameterAvailable()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_SetParameterNameRemapTable_V1

KY_RESULT KYParametersHandler_SetParameterNameRemapTable_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const KYVP_ParameterNameRemapEntry *_pEntries, size_t _iNumEntries)

Replace the complete parameter-name remapping table.

Entries are copied. A zero count clears the table. Validate every entry before calling: replacement clears the old table before processing the new entries, and an invalid entry can leave a partial table.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pEntries	Array of mappings to copy into the collection. Required when iNumEntries is nonzero.
_iNumEntries	Number of mappings in pEntries. Zero clears the table.

See: KYParametersHandler_SetParameterNameRemapTable()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_AddParameterNameRemapEntry_V1

KY_RESULT KYParametersHandler_AddParameterNameRemapEntry_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const KYVP_ParameterNameRemapEntry *_pEntry, KY_BOOL *_pbReplaced)

Add a parameter-name mapping or replace the first matching source name.

The entry is copied into the collection.

Parameter	Description
-----------	-------------

_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pEntry	Required pointer to the mapping to copy.
_pbReplaced	Receives KY_TRUE when an existing source mapping was replaced. Optional.

See: KYParametersHandler_AddParameterNameRemapEntry()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_RemoveParameterNameRemapEntry_V1

KY_RESULT KYParametersHandler_RemoveParameterNameRemapEntry_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, const char *_pszFromName, KY_BOOL *_pbRemoved)

Remove the first mapping for a source parameter name.

A missing source name is not an error. The optional output reports whether an entry was removed.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_pszFromName	Source name whose mapping should be removed. Non-NULL, NUL-terminated string; case-sensitive.
_pbRemoved	Receives KY_TRUE when a mapping was removed. Optional.

See: KYParametersHandler_RemoveParameterNameRemapEntry()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_GetParameterNameRemapEntryByIndex_V1

KY_RESULT KYParametersHandler_GetParameterNameRemapEntryByIndex_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, size_t _iIndex, KYVP_ParameterNameRemapEntry *_pEntry)

Copy one parameter-name mapping by its zero-based index.

Query the table size first; the index must be less than the current entry count.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_iIndex	Zero-based index of the mapping to retrieve. Must be smaller than the table size.
_pEntry	Required pointer receiving a copy of the mapping.

See: KYParametersHandler_GetParameterNameRemapEntryByIndex()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_GetParameterNameRemapTableSize_V1

KY_RESULT KYParametersHandler_GetParameterNameRemapTableSize_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle, size_t *_piSize)

Return the number of parameter-name mappings.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.
_piSize	Required pointer receiving the number of remap entries.

See: KYParametersHandler_GetParameterNameRemapTableSize()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

KYParametersHandler_ClearParameterNameRemapTable_V1

KY_RESULT KYParametersHandler_ClearParameterNameRemapTable_V1(KYVP_COLLECTION_HANDLE _hCollectionHandle)

Remove all parameter-name mappings from a collection.

Parameter	Description
_hCollectionHandle	Collection to operate on. Valid collection handle required.

See: KYParametersHandler_ClearParameterNameRemapTable()

Returns: Operation status. Use KY_RESULT_SUCCEEDED() or KY_RESULT_FAILED() to test the result.

Parameters Handler Library Structs

Field reference for collection handles, transport callbacks, and function arguments.

Initialize each version field to the constant shown above its table. C++ defaults provide these values; C callers must initialize them explicitly. Version zero is not the declared argument layout. Tables list fields in declaration order and distinguish required inputs, outputs, and optional pointers. Descriptor pointers and arrays returned by the library remain owned by their collection. See Optional diagnostic buffers for optional diagnostic buffers and Parameter value storage for value storage.

KYVP_COLLECTION_HANDLE

Opaque collection handle. Use the create/delete functions and public comparison helpers. The C++ default constructor selects the invalid sentinel. In C, use the exported KYVP_COLLECTION_HANDLE_INVALID or KYVP_COLLECTION_HANDLE_NULL constants. The native pointer is an implementation detail; do not dereference or synthesize it.

Structure Field	Type	Description	Remarks
pNativePointer	void*	Opaque native collection identity.	Managed by the SDK; protected in C++.

KYParametersHandler_TransportCallParams_Args

KYParametersHandler_TransportCallParams_Args

Arguments passed to TRANSPORT_FUNC. Complete the requested operation before returning. For Read, fill the supplied buffer. For Write, consume the supplied bytes without modifying them. For GetResourceData, return a pointer to the XML or ZIP resource and its size. The resource must remain valid until collection initialization returns; the library copies the bytes.

The library sets version to KYPARAMETERSHANDLER_TRANSPORTCALLPARAMS_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Transport argument layout version.	IN. Supplied by the library.
eOperation	KYParametersHandler_TransportOperationType	Requested transport operation.	IN.
pData	void*	Transfer buffer or returned XML/ZIP resource address.	Read: destination; Write: source; GetResourceData: OUT pointer.
pContext	void*	Application context registered with the transport callback.	IN.
iAddress	uint64_t	Starting device register address.	IN. Used for Read and Write.
iSize	size_t	Number of transfer or resource bytes.	Read/Write: IN; GetResourceData: OUT.
bResourceDataZipped	KY_BOOL	Whether the returned XML resource is ZIP-compressed.	OUT for GetResourceData; ignored for Read/Write.
eResult	KYParametersHandler_TransportOperationResult	Result of the requested transport operation.	OUT. Set before returning; defaults to Failed.

KYParametersHandler_InitParameters

Library initialization options. The current layout contains only its version field.

Initialize version to KYPARAMETERSHANDLER_INITPARAMETERS_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.

KYParametersHandler_Initialize_Args

Arguments for KYParametersHandler_Initialize(). Call before creating a parameter collection.

Initialize version to KYPARAMETERSHANDLER_INITIALIZE_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
pInitParameters	KYParametersHandler_InitParameters*	Library initialization parameters.	IN. Required; initialize the nested version.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_CreateParameterCollection_Args

KYParametersHandler_CreateParameterCollection_Args

Arguments for KYParametersHandler_CreateParameterCollection(). Register its transport and initialize the collection before accessing nodes.

Initialize version to KYPARAMETERSHANDLER_CREATEPARAMETERCOLLECTION_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
phCollectionHandle	KYVP_COLLECTION_HANDLE*	Receives the new collection handle.	OUT. Required.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_DeleteParameterCollection_Args

KYParametersHandler_DeleteParameterCollection_Args

Arguments for KYParametersHandler_DeleteParameterCollection(). All descriptors and other library-owned pointers from this collection become invalid.

Initialize version to KYPARAMETERSHANDLER_DELETEPARAMETERCOLLECTION_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to delete.	IN/OUT. Set to the invalid sentinel on success.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.

piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.
-------------------	---------	---	---

KYParametersHandler_RegisterParameterCollectionTransport_Args

KYParametersHandler_RegisterParameterCollectionTransport_Args

Arguments for KYParametersHandler_RegisterParameterCollectionTransport(). Registration replaces the previously registered transport callback.

Initialize version to

KYPARAMETERSHANDLER_REGISTERPARAMETERCOLLECTIONTRANSPORT_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pTRANSPORT_FUNC	TRANSPORT_FUNC	Callback implementing register and XML-resource transport.	IN. Required.
pContext	void*	Application context passed to the registered callback.	IN. May be NULL.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_UnregisterParameterCollectionTransport_Args

KYParametersHandler_UnregisterParameterCollectionTransport_Args

Arguments for KYParametersHandler_UnregisterParameterCollectionTransport().

Initialize version to

KYPARAMETERSHANDLER_UNREGISTERPARAMETERCOLLECTIONTRANSPORT_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_RegisterParameterCollectionCallback_Args

KYParametersHandler_RegisterParameterCollectionCallback_Args

Arguments for KYParametersHandler_RegisterParameterCollectionCallback(). Registration replaces the previously registered node callback.

Initialize version to KYPARAMETERSHANDLER_REGISTERPARAMETERCOLLECTIONCALLBACK_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pPARAMETER_CALLBACK_FUNC	PARAMETER_CALLBACK_FUNC	Callback receiving node updates.	IN. Required.
pContext	void*	Application context passed to the registered callback.	IN. May be NULL.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_UnregisterParameterCollectionCallback_Args

KYParametersHandler_UnregisterParameterCollectionCallback_Args

Arguments for KYParametersHandler_UnregisterParameterCollectionCallback().

Initialize version to

KYPARAMETERSHANDLER_UNREGISTERPARAMETERCOLLECTIONCALLBACK_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_InitializeParameterCollection_Args

KYParametersHandler_InitializeParameterCollection_Args

Arguments for KYParametersHandler_InitializeParameterCollection(). A transport callback must be registered. It supplies the XML resource using GetResourceData.

Initialize version to KYPARAMETERSHANDLER_INITIALIZEPARAMETERCOLLECTION_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_GetNodeDescriptor_Args

Arguments for KYParametersHandler_GetNodeDescriptor(). Treat the returned descriptor as read-only. Its lifetime is tied to the initialized collection.

Initialize version to KYPARAMETERSHANDLER_GETNODEDESCRIPTOR_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszParamName	const char*	Name of the parameter to access.	IN. Non-NULL, NUL-terminated string; remapping applies.
ppNodeDescriptor	KYVP_NodeDescriptor	Receives a pointer to the library-owned node descriptor.	OUT. Required; treat the descriptor as read-only.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_GetNodeDescriptorsArray_Args

KYParametersHandler_GetNodeDescriptorsArray_Args

Arguments for KYParametersHandler_GetNodeDescriptorsArray(). This returns an array of descriptors, not an array of descriptor pointers. Do not free or modify the array; its lifetime is tied to the initialized collection.

Initialize version to KYPARAMETERSHANDLER_GETNODEDESCRIPTORSARRAY_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.

ppArray	const KYVP_NodeDescriptor	Receives the base address of the library-owned descriptor array.	OUT. Required; elements are descriptors, not pointers.
piSize	size_t*	Receives the number of descriptors.	OUT. Required; count is in elements, not bytes.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_GetValue_Args

Arguments for KYParametersHandler_GetValue(). Pass a NULL value buffer to query the required byte count for a supported value node. Use storage of the type and size described in Parameter value storage.

Initialize version to KYPARAMETERSHANDLER_GETVALUE_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszParamName	const char*	Name of the parameter to access.	IN. Non-NULL, NUL-terminated string; remapping applies.
pParamValue	void*	Destination for the parameter value.	OUT. NULL queries size; see Parameter value storage.
piParamValueSize	size_t*	Pointer to value-buffer size in bytes.	IN/OUT. Required; receives the required size on a NULL-buffer query.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_SetValue_Args

Arguments for KYParametersHandler_SetValue(). Supply a non-NULL value pointer and byte-count pointer. See Parameter value storage.

Initialize version to KYPARAMETERSHANDLER_SETVALUE_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.

hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszParamName	const char*	Name of the parameter to access.	IN. Non-NULL, NUL-terminated string; remapping applies.
pParamValue	const void*	Value to write, interpreted by node interface type.	IN. Required; see Parameter value storage.
piParamValueSize	size_t*	Pointer to the input value size in bytes.	IN. Required; the pointed count is not updated.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_GetValueEnumAsString_Args

KYParametersHandler_GetValueEnumAsString_Args

Arguments for KYParametersHandler_GetValueEnumAsString(). Pass a NULL destination to query the byte count including the terminator. An undersized destination reports the required count without copying the string.

Initialize version to KYPARAMETERSHANDLER_GETVALUEENUMASSTRING_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszEnumerationName	const char*	Name of the enumeration to read.	IN. Non-NULL, NUL-terminated string; remapping applies.
pszParamValueString	char*	Buffer receiving the current enumeration entry symbolic name.	OUT. NULL queries the required size.
piParamValueStringSize	size_t*	Pointer to destination capacity in bytes, including the terminator.	IN/OUT. Required; updated on a size query or insufficient capacity.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_SetValueEnumByValueName_Args

KYParametersHandler_SetValueEnumByValueName_Args

Arguments for KYParametersHandler_SetValueEnumByValueName().

Initialize version to KYPARAMETERSHANDLER_SETVALUE_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszParamName	const char*	Name of the parameter to access.	IN. Non-NULL, NUL-terminated string; remapping applies.
pszParamValue	const char*	Symbolic name of the enumeration entry to select.	IN. Non-NULL, NUL-terminated string.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_InvalidateParameterCollection_Args

KYParametersHandler_InvalidateParameterCollection_Args

Arguments for KYParametersHandler_InvalidateParameterCollection().

Initialize version to KYPARAMETERSHANDLER_INVALIDATEPARAMETERCOLLECTION_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_InvalidateParameter_Args

KYParametersHandler_InvalidateParameter_Args

Arguments for KYParametersHandler_InvalidateParameter(). A valid, non-NULL parameter name is required. Use KYParametersHandler_InvalidateParameterCollection() to invalidate all nodes.

Initialize version to KYPARAMETERSHANDLER_INVALIDATEPARAMETER_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszParamName	const char*	Name of the parameter to access.	IN. Non-NULL, NUL-terminated string; remapping applies.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_SaveToFile_Args

Arguments for KYParametersHandler_SaveToFile(). The original resource bytes are saved, preserving their compressed or uncompressed form. This does not serialize current parameter values. An existing destination file is overwritten.

Initialize version to KYPARAMETERSHANDLER_SAVETOFILE_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszFilePath	const char*	Destination path for the original XML or ZIP resource.	IN. Non-NULL, NUL-terminated string.
piFilePathSize	size_t*	Reserved path-size pointer.	Unused by this implementation; leave NULL.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_GetParameterAttributeValueArgs

KYParametersHandler_GetParameterAttributeValueArgs

Arguments for KYParametersHandler_GetParameterAttributeValue(). Pass a NULL property buffer first to obtain its required byte count, including the terminator. Allocate at least that many bytes before retrieving the text; the caller is responsible for sufficient capacity.

Initialize version to KYPARAMETERSHANDLER_SAVETOFILE_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszParamName	const char*	Name of the parameter to access.	IN. Non-NULL, NUL-terminated string; remapping applies.
eAttributeType	KYVP_ParameterAttributeType	GenApi property to query.	IN. Currently supports Invalidator.
pszPropertyValue	char*	Buffer receiving the property text.	OUT. NULL queries the required byte count.
piPropertyValueSize	size_t*	Receives the property text size, including its terminator.	OUT. Required; query before allocating the text buffer.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYParametersHandler_IsParameterAvailableArgs

KYParametersHandler_IsParameterAvailableArgs

Arguments for KYParametersHandler_IsParameterAvailable(). This checks collection membership. Inspect the descriptor access flags for current device availability and read/write access.

Initialize version to KYPARAMETERSHANDLER_SAVETOFILE_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszParamName	const char*	Name of the parameter to access.	IN. Non-NULL, NUL-terminated string; remapping applies.
bIsAvailable	KY_BOOL	Whether the named node exists in the collection.	OUT. Does not report the node access mode.
pszErrorString	char*	Optional diagnostic text buffer.	OUT. See Optional diagnostic buffers.
piErrorStringSize	size_t*	Pointer to diagnostic buffer capacity in bytes.	IN. Optional; required when pszErrorString is supplied.

KYVP_ParameterNameRemapEntry

One application-name to XML-node-name mapping. Both names must be nonempty and NUL-terminated within their arrays. The capacity is KYVP_PARAMETER_NAME_REMAP_MAX_CHARS bytes per name, allowing at most 127 characters plus the terminator. Names are matched case-sensitively.

Structure Field	Type	Description	Remarks
pszFrom	char [KYVP_PARAMETER_NAME_REMAP_MAX_CHARS]	Application-facing source name.	IN. Nonempty, NUL-terminated string.
pszTo	char [KYVP_PARAMETER_NAME_REMAP_MAX_CHARS]	Target node name in the collection.	IN. Nonempty, NUL-terminated string.

KYParametersHandler_SetParameterNameRemapTable_Args

KYParametersHandler_SetParameterNameRemapTable_Args

Arguments for KYParametersHandler_SetParameterNameRemapTable(). Entries are copied. A zero count clears the table. Validate every entry before calling: replacement clears the old table before processing the new entries, and an invalid entry can leave a partial table.

Initialize version to KYPARAMETERSHANDLER_SETPARAMETERNAME_REMAPTABLE_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pEntries	const KYVP_ParameterNameRemapEntry*	Array of mappings to copy into the collection.	IN. Required when iNumEntries is nonzero.
iNumEntries	size_t	Number of mappings in pEntries.	IN. Zero clears the table.

KYParametersHandler_ClearParameterNameRemapTable_Args

KYParametersHandler_ClearParameterNameRemapTable_Args

Arguments for KYParametersHandler_ClearParameterNameRemapTable().

Initialize version to KYPARAMETERSHANDLER_CLEARPARAMETERNAME_REMAPTABLE_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.

KYParametersHandler_GetParameterNameRemapTableSize_Args

KYParametersHandler_GetParameterNameRemapTableSize_Args

Arguments for KYParametersHandler_GetParameterNameRemapTableSize().

Initialize version to KYPARAMETERSHANDLER_GETPARAMETERNAME_REMAPTABLESIZE_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
piSize	size_t*	Receives the number of name-remap entries.	OUT. Required; count is in elements, not bytes.

KYParametersHandler_AddParameterNameRemapEntry_Args

KYParametersHandler_AddParameterNameRemapEntry_Args

Arguments for KYParametersHandler_AddParameterNameRemapEntry(). The entry is copied into the collection.

Initialize version to KYPARAMETERSHANDLER_ADDPARAMETERNAME_REMAPENTRY_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
entry	KYVP_ParameterNameRemapEntry	Mapping to copy into the collection.	IN. Both names must be valid.
pbReplaced	KY_BOOL*	Receives KY_TRUE when an existing source mapping was replaced.	OUT. Optional.

KYParametersHandler_RemoveParameterNameRemapEntry_Args

KYParametersHandler_RemoveParameterNameRemapEntry_Args

Arguments for KYParametersHandler_RemoveParameterNameRemapEntry(). A missing source name is not an error. The optional output reports whether an entry was removed.

Initialize version to KYPARAMETERSHANDLER_REMOVEPARAMETERNAME_REMAPENTRY_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
pszFromName	const char*	Source name whose mapping should be removed.	IN. Non-NULL, NUL-terminated string; case-sensitive.
pbRemoved	KY_BOOL*	Receives KY_TRUE when a mapping was removed.	OUT. Optional.

KYParametersHandler_GetParameterNameRemapEntryByIndex_Args

KYParametersHandler_GetParameterNameRemapEntryByIndex_Args

Arguments for KYParametersHandler_GetParameterNameRemapEntryByIndex(). Query the table size first; the index must be less than the current entry count.

Initialize version to

KYPARAMETERSHANDLER_GETPARAMETERNAME_REMAPENTRYBYINDEX_STRUCT_VERSION.

Structure Field	Type	Description	Remarks
version	uint32_t	Structure version.	IN. Required.
hCollectionHandle	KYVP_COLLECTION_HANDLE	Collection to operate on.	IN. Valid collection handle required.
iIndex	size_t	Zero-based index of the mapping to retrieve.	IN. Must be smaller than the table size.
entry	KYVP_ParameterNameRemapEntry	Mapping copied from the requested table index.	OUT.